目录

前言：为什么需要这本附册 2
改动记录 3

数学卷 · 第 1 章：信息论与语言的概率结构 4
§1.1 香农的赌局：语言是对抗熵的工具 4
§1.2 从马尔可夫链到 N-Gram：第一代尝试的局限 9
§1.3 跨界映射：语言序列与 DNA 碱基对 13
§1.4 反类比：当 DNA 的隐喻失效 17

数学卷 · 第 2 章：嵌入空间与高维几何 19
§2.1 词嵌入：为什么字典救不了机器 20
§2.2 语义的坐标系：那个被讲烂了的公式 23
§2.3 高维几何的诡异世界 28
§2.4 跨界映射：从罗马路网到曼哈顿距离 32
§2.5 反类比：当几何隐喻失效 36
§2.6 跨越欧氏：当表示空间不再「平直」 39

数学卷 · 第 3 章：注意力机制的矩阵微分 43
§3.1 从 RNN 到变换器（Transformer）：序列建模的范式革命 44
§3.2 注意力公式的拆解：QKV 的真实含义 48
§3.3 矩阵乘法的大规模并发：为什么 AI 是 GPU 的盛宴 52
§3.4 RoPE：把位置编码从加法变成旋转 55
§3.5 FlashAttention：当注意力撞上内存墙 60
§3.6 注意力的真实身份：关联记忆，而非「注意力」 64
§3.7 一个被忽略的争论：注意力权重 ≠ 解释 68
§3.8 跨界映射：宏观工业供应链 71
§3.9 反类比：当供应链隐喻失效 73

数学卷 · 第 4 章：非凸优化与超参数标度律 75
§4.1 损失函数：如何用数学定义”机器犯错” 75
§4.2 梯度下降：在黑暗中沿着最陡的方向下山 80
§4.3 高维非凸优化的反直觉：为什么 SGD 居然能工作 85
§4.4 损失景观的可视化：黑暗中的地形图 89
§4.5 µP：超参数迁移的”元定律” 92
§4.6 Lion 与 Shampoo：Adam 不是终点 96
§4.7 µP 的完整面貌：超参数迁移背后的数学 99
§4.8 跨界类比：复杂系统的工程管理 103
§4.9 反类比：工程管理隐喻的失效边界 106

数学卷 · 第 5 章：标度律、相变与涌现的数理基础 108
§5.1 尺度律（Scaling Law）：算力即权力的冷酷物理感 109
§5.2 Chinchilla 的颠覆与再颠覆 114
§5.3 数据墙与 §5.4 推理时计算 118
§5.5 涌现：神话还是幻觉？ 121

§5.6–§5.7 Grokking 的数学解剖：从死记到顿悟 125
§5.8 渗流相变：把 Scaling Law 焊到统计物理上 129
§5.9 跨界映射与 §5.10 反类比：相变叙事的边界 133

数学卷 · 第 6 章: 硬件感知算法的数学——屋脊线（Roofline）、I/O 复杂度与 FlashAttention 推导 137
§6.1 Roofline 模型的形式化 138
§6.2 I/O 复杂度的下界（Hong-Kung 1981） 143
§6.3 FlashAttention 的 I/O 分析（Dao et al. 2022） 148
§6.4 环形注意力（RingAttention）的通信复杂度（Liu et al. 2024） 154
§6.5 算术强度的演化：从 V100 到 B200 159
§6.6 反类比与边界 163
§6.7 参考文献 167

数学卷 · 第 7 章: 后变换器（Transformer）时代的数学——状态空间模型、线性注意力与扩散语言模型的数理基础 168
§7.1 状态空间模型的连续形式 169
§7.2 HiPPO 框架的严格推导 174
§7.3 S4 的离散化与卷积核 179
§7.4 Mamba 的选择性扫描 184
§7.5 线性注意力的核函数视角 188
§7.6 Mamba、线性注意力（Attention）与 RNN 的统一视角 192
§7.7 扩散语言模型的数学 196
§7.8 世界模型与预测编码的数学 200
§7.9 反类比与边界 206
§7.10 参考文献 210
后记 212

算法的低语

数学卷

从公式到推导

本册是《算法的低语》主本的姊妹册。镜像主本上下两半场的七章结构，对每个 AI 概念背后的数学给出两层呈现：

- **直觉**（150–250 字）：回顾概念由来与几何含义
- **严格**（400–800 字 + 公式块）：定义、推导步骤、矩阵维度

面向已具备线性代数、微积分、概率统计本科基础的进阶读者。

建议与主本《算法的低语》配套阅读；读完主本某章后再翻附册对应章节。

大队长 出品

fqsx@mail.ustc.edu.cn

Version 1.6 · 每节直觉段重写为深度逻辑链版

目录

前言：为什么需要这本附册

《算法的低语》主本用 7 章篇幅勾勒了大语言模型背后的数学骨架与产业现实——上半场从信息论的香农熵、嵌入空间的高维几何、Transformer 的矩阵微分、非凸优化的鞍点逃逸，讲到涌现现象的渗流相变隐喻；下半场则把镜头拉远，看这套数学如何级联出万亿美元的算力供应链（第六章）、以及 Transformer 之后下一代架构正在从哪里凿开缺口（第七章）。主本的目标是把这条逻辑链讲清楚、讲完整；而本附册的目标是把每一处「为什么是这个公式」「这个公式怎么推出来」补齐。

本册 **7 章与主本逐章镜像**：前 5 章是数学引擎本身的严格推导；第 6 章《硬件感知算法的数学》覆盖 Roofline 模型、Hong-Kung I/O 下界、FlashAttention 在线 softmax 推导、RingAttention 通信复杂度；第 7 章《后 Transformer 时代的数学》覆盖 HiPPO LegS 矩阵推导、S4 零阶保持离散化、Mamba 选择性扫描的并行关联律、线性注意力的核函数分解、扩散语言模型与 AR 的语义差别。

本册不重复主本的叙事。每个小节默认读者已经读完主本对应章节，因此直接进入数学：

- **直觉版**：用一两段话回顾概念由来与几何含义，铺出数学动机；不含公式。
- **严格版**：给出严格定义、关键公式、必要的推导步骤；所有矩阵显式标注维度，符号尽量与原始论文一致。

读者无需逐节深入。建议在读完主本某一章后，对感兴趣的小节回看附册——把直觉补齐、把公式补全、然后回到主本继续读下一章。

公式约定：行内公式用 $\ldots$，块公式用

$$\ldots$$

并带右侧编号 (X.Y.N)，叫在叙述中以「式 (X.Y.N)」引用。矩阵用 $\mathbb{R}^{m\times n}$ 标注维度。底数缺省时 log 按自然对数处理；信息论场景下若涉及 bit 单位会显式注明 $\log_2$。

改动记录

v1.6 · 2026–06–13：本版是深度逻辑链版。上一版 v1.5 仅在每节头部增加了一个「这一节在讲什么」的导读盒，本身只描述了概念；本版把所有 52 个小节的「直觉」段重写为「深度逻辑链」结构：每一节拆出 3–6 个「环」（第一环 → 第二环 → …→ 整条链的回头看），逐一推出该节出现的所有核心概念之间的推导与因果关系，过渡以「上一环到这一环为什么需要这个步骤」的动机说明为主。完成后每节直觉段约 1500–2500 字（v1.5 为 200–300 字），使读者能从「为什么需要这个公式」一路读到「这个公式什么意思」，而不是孤立看公式列表。「严格」段及以后的所有数学内容（定义、推导、证明、结论）未改动。

v1.5 · 2026–06–12：每节头部增加「这一节在讲什么」导读盒——用 4–6 句话归纳本节要解决的问题、所用的数学工具、以及要从本节拿走的核心结论；同时扩充了「直觉」段，补足了部分小节的背景描述。

v1.4 · 2026–06–11：加入第 6、7 章。第六章《硬件感知算法的数学》覆盖 Roofline 模型、Hong-Kung I/O 下界证明、FlashAttention 在线 softmax 推导、RingAttention 环形通信复杂度；第七章《后 Transformer 时代的数学》覆盖 HiPPO LegS 矩阵推导、S4 零阶保持离散化、Mamba 选择性扫描的并行关联律、线性注意力的核函数分解、扩散语言模型与 AR 的语义差别。

v1.0–v1.3 · 2026–06–10 及之前：从主本 5 章拆出独立的《数学卷》；为每节加上「直觉 + 严格」双重结构；补齐公式底数、矩阵维度、与主本的交叉引用。

数学卷 · 第 1 章：信息论与语言的概率结构

§1.1 香农的赌局：语言是对抗熵的工具

▶这一节在讲什么

为什么要给「不确定性」一个数学定义？1948 年之前，工程师只能用「信道质量好」「信号清晰」这种模糊的描述。香农提出：把不确定性当作一个可加、可比、可优化的量，整个通信工程就变成了数学。

第一步：把概率转成「惊讶度」。一件事概率越小，发生时越让人惊讶，信息量越大。所以信息量是 $-\log p(x)$。

第二步：取平均。所有可能结果的惊讶度加权平均，就是熵 $H(X) = -\sum p(x)\log p(x)$。它告诉你：要编码这个分布的输出，平均最少用多少比特。

第三步：当模型的分布 q 偏离真分布 p 时，要付额外代价。这就是 KL 散度和交叉熵——大模型训练用的损失，本质上就是这个代价。

本节要拿走的一件事：交叉熵不是「随便挑的损失」，它是「让模型分布逼近数据分布」的唯一信息论意义上正确的目标。

直觉

原书以香农的”猜字母实验”引入，指出英语文本的熵率约为 1 bit/字符，远低于 26 个字母均匀分布下的理论上限。这里的关键问题是：用什么数学量度量”不确定性”，以及当一个概率分布与另一个分布不一致时，如何量化其代价？

本节出现六个概念：**香农熵**、**条件熵**、**互信息**、**KL 散度**、**交叉熵**、**困惑度**。它们看似散落，实际上沿着一条严密的逻辑链一个推出一个。理解这条链，是理解大模型训练目标函数为什么是「交叉熵损失」的钥匙。下面把这条链一节节铺开。

第一环：香农熵——给「不确定性」一个绝对标尺

任何概率分布 p 都对应一个不确定性。事件越罕见，发生时携带的信息越多，所以单个事件 x 的信息量定义为 $-\log p(x)$。把所有事件的信息量按其概率加权平均，就得到香农熵：

$$H(X) = -\sum_{x} p(x)\log p(x) \tag{1.1.1}$$

式 (1.1.1) 不是一个”猜出来的好用公式”。香农 1948 年证明：若一个不确定性度量同时满足**连续性**、**单调性（均匀分布时随事件数单增）**、**可分解性（联合事件能拆成条件期望）**这三条公理，则它必然是 H 的常数倍。也就是说，**熵是这三条公理唯一允许存在的函数形式**。这赋予 H 不可替代的地位。

熵的物理含义：式 (1.1.1) 等于「用最优编码方案表达 X 一次取值所需的平均比特数」——这就是信源编码定理（Shannon 第一定理）。换句话说，熵是「最优压缩的极限」。

第二环：条件熵——已知一部分信息后剩余的不确定性

语言模型预测下一个 token 的本质，是「给定历史 Y，X（下一个 token）还剩多少不

确定性」。这正是条件熵：

$$H(X \mid Y) = -\sum_{x,y} p(x,y) \log p(x \mid y) = \mathbb{E}_Y[H(X \mid Y = y)] \tag{1.1.2}$$

注意式 (1.1.2) 的形式：先在固定 $Y = y$ 时算熵 $H(X \mid Y = y)$，再对 Y 取期望。由链式法则 $H(X,Y) = H(Y) + H(X \mid Y)$，可以验证：**条件熵就是联合熵减去边缘熵**。它满足 $0 \le H(X \mid Y) \le H(X)$——已知信息只能减少不确定性，不可能增加。

第三环：互信息——两变量共享的信息量

如果 Y 完全独立于 X，那么 $H(X \mid Y) = H(X)$，知道 Y 没用；如果 Y 完全决定了 X，那么 $H(X \mid Y) = 0$。中间状态有多少”用”？正是 $H(X)$ 与 $H(X \mid Y)$ 之差：

$$I(X;Y) = H(X) - H(X \mid Y) = H(Y) - H(Y \mid X) \tag{1.1.3}$$

式 (1.1.3) 就是互信息。两种写法的对称性说明：**互信息是「共享信息」，与方向无关**。它的几何解释：在韦恩图里，$H(X)$ 和 $H(Y)$ 是两个圆，交集面积就是 $I(X;Y)$。

互信息 ≥ 0，等号当且仅当 X, Y 独立。这一非负性后来直接成为对比学习目标 InfoNCE 的数学基础——最大化”正样本对”的互信息下界。

第四环：KL 散度——「真分布 p 和模型分布 q 的距离」

前三环讨论的是同一个分布 p 内部的结构。但现实中，我们用模型分布 q 去近似真实分布 p。需要一个量度量”q 偏离 p 有多远”。最自然的想法是：**如果按 q 来给数据编码，要比按 p 编码多花多少比特**？

把这个差额写出来就是 KL 散度：

$$D_{\mathrm{KL}}(p \| q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = \underbrace{\sum_x p(x) \log \frac{1}{q(x)}}_{\text{按}q\text{ 编码的平均代价}} - \underbrace{\sum_x p(x) \log \frac{1}{p(x)}}_{H(p)\text{, 最优代价}} \tag{1.1.4}$$

式 (1.1.4) 表达的就是「**用错分布编码所付的额外代价**」。由 Jensen 不等式可证 $D_{\mathrm{KL}} \ge 0$，等号当且仅当 $p = q$。这非负性叫 Gibbs 不等式，是信息论的支柱之一。

注意 $D_{\mathrm{KL}}(p \| q) \ne D_{\mathrm{KL}}(q \| p)$——KL 散度不对称，**不是真正的距离**。这一不对称性在变分推断里很重要：选择 $D_{\mathrm{KL}}(q \| p)$（mode-seeking）还是 $D_{\mathrm{KL}}(p \| q)$（mass-covering）会导致截然不同的近似行为。

回头看互信息：$I(X;Y) = D_{\mathrm{KL}}\big(p(x,y) \,\big\|\, p(x)p(y)\big)$。也就是说，**互信息就是「联合分布」相对于「独立假设下的乘积分布」的 KL 散度**。这把前三环和第四环串了起来，即将式 (1.1.3) 和式 (1.1.4) 联系在一起。

第五环：交叉熵——把 KL 散度拆成「数据熵 + 偏离代价」

式 (1.1.4) 的右侧出现了 $\sum_x p(x) \log \frac{1}{q(x)}$ 这一项——它单独有个名字：**交叉熵**。

$$H(p,q) = -\sum_x p(x) \log q(x) \tag{1.1.5}$$

物理含义：用 q 编码、按 p 抽样时，每次取值的平均比特数。将式 (1.1.5) 与 KL 散度的关系写成一个恒等式：

$$H(p,q) = H(p) + D_{\mathrm{KL}}(p\|q) \tag{1.1.6}$$

恒等式 (1.1.6) 是**整条逻辑链的枢纽**：

- $H(p)$ 是真实分布的熵，**与模型无关**，是常数；
- $D_{\mathrm{KL}}(p\|q)$ 是模型偏离真分布的代价，**只与模型有关**；
- 二者相加就是交叉熵，即式 (1.1.5)。

由式 (1.1.6) 立刻得到大模型训练的核心结论：**最小化交叉熵 ⇔ 最小化 KL 散度 ⇔ 最大化对数似然**。三者在数学上完全等价。我们用「交叉熵损失」而不是直接用 KL 散度，是因为 $H(p)$ 这个常数虽然算不出来（真分布未知），但**它不影响梯度**——所以训练时直接用 $-\sum_i \log q_\theta(w_i \mid w_{<i})$ 就够了。

这就是大模型损失函数 $\mathcal{L} = -\frac{1}{N}\sum_i \log q_\theta(w_i \mid w_{<i})$ 的来源——它**不是工程上挑出来的好用损失，而是信息论意义上唯一正确的目标**。

第六环：困惑度——把损失指数化回到直观量纲

交叉熵是「平均比特数」（或自然对数下的 nat），但工程上更直观的提问是：「模型在每一步预测时，有效面临几个等可能选项?」答案是把式 (1.1.5) 定义的交叉熵指数化：

$$\mathrm{PPL} = \exp\big(H(p,q)\big) \tag{1.1.7}$$

如果模型对每个 token 完全瞎猜词表中 $V = 50000$ 个候选，则由式 (1.1.7) 得 PPL $= 50000$。GPT-3 在 WikiText-103 上 PPL ≈ 20，意味着它”等效面对 20 个均匀候选”——比瞎猜好 2500 倍。困惑度让”训练 loss 降了 0.1”这种不直观的量，翻译成”模型选择空间从 100 个等效候选缩到 90 个”这种直观的话。

整条链的回头看

```
香农熵 H(X)                          ←  不确定性的绝对标尺（由公理唯一确定）
  ↓ 加上"条件"
条件熵 H(X|Y)                        ←  已知 Y 后 X 还剩多少不确定性
  ↓ 用熵差衡量"共享"
互信息 I(X;Y) = H(X) - H(X|Y)        ←  X 和 Y 共享了多少信息
  ↓ 把"共享"重新解读为"分布之间的距离"
KL 散度 D(p‖q)                       ←  q 偏离 p 的额外编码代价
  ↓ 拆出"与模型无关的常数项"
交叉熵 H(p,q) = H(p) + D(p‖q)        ←  大模型的训练目标（去掉 H(p) 常数后等价于 KL）
  ↓ 指数化回到选项数量
困惑度 PPL = exp(H(p,q))             ←  "等效面对几个候选"的直观量
```

这条链揭示了一个深层事实：**大模型的整个训练目标函数，从源头就嵌在信息论的公理体系里**。我们不是在「凑一个能跑通的损失」，而是在「计算真实分布与模型分布之间唯一正确的距离」。困惑度则是这条距离的指数化展示。

严格

1. 香农熵（Shannon entropy）

设离散随机变量 X 取值于有限集合 $\mathcal{X}$，概率质量函数为 $p(x) = P(X = x)$。**香农熵**定义为：

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log p(x) \tag{1.1.8}$$

其中约定 $0 \log 0 = 0$（取极限值）。注意式 (1.1.8) 与直觉部分的式 (1.1.1) 形式完全一致，只是增加了定义域 $\mathcal{X}$ 的标注。

对数底数约定：当底数为 2 时，单位为比特（bit）；当底数为 e 时，单位为奈特（nat）；当底数为 10 时，单位为哈特（hart）。大模型训练中几乎统一使用自然对数（底数 e），以便与梯度计算衔接；信息论教材常用 $\log_2$。本节公式默认底数为 e，在需要与原书对应的地方显式标注。

熵的基本性质：

- $H(X) \geq 0$，等号成立当且仅当 X 为退化分布（某一值概率为 1）；
- 均匀分布时熵最大，$H(X) \leq \log |\mathcal{X}|$；
- $H(X)$ 只依赖于分布 p，而不依赖于具体取值。

2. 条件熵（conditional entropy）

设 (X, Y) 的联合分布为 $p(x, y)$，**条件熵**定义为：

$$H(Y \mid X) = -\sum_{x \in \mathcal{X}} p(x) \sum_{y \in \mathcal{Y}} p(y \mid x) \log p(y \mid x) \tag{1.1.9}$$

等价地，$H(Y \mid X) = H(X, Y) - H(X)$，即联合不确定性减去 X 本身的不确定性。将式 (1.1.9) 推广，链式法则可写为：

$$H(X_1, X_2, \ldots, X_n) = \sum_{i=1}^{n} H(X_i \mid X_1, \ldots, X_{i-1}) \tag{1.1.10}$$

3. 互信息（mutual information）

互信息衡量 X 与 Y 共享的信息量：

$$I(X; Y) = H(X) - H(X \mid Y) = H(Y) - H(Y \mid X) = H(X) + H(Y) - H(X, Y) \tag{1.1.11}$$

将式 (1.1.11) 与式 (1.1.2) 对比，可以看到互信息的第一个等号正是把条件熵与边缘熵相减的差值。等价定义为联合分布与边缘分布之积的 KL 散度（见下文）：

$$I(X; Y) = \sum_{x, y} p(x, y) \log \frac{p(x, y)}{p(x)\, p(y)} \tag{1.1.12}$$

式 (1.1.12) 表明互信息是联合分布相对于独立假设乘积分布的 KL 散度，将式 (1.1.3) 与式 (1.1.4) 的关系形式化地串联起来。$I(X; Y) \geq 0$，等号成立当且仅当 X 与 Y 相互独

立。

4. KL 散度（Kullback-Leibler divergence）

设 p 为真实分布，q 为近似分布，二者定义在同一有限集合上且 $q(x) > 0$ 只要 $p(x) > 0$。**KL 散度**（又称相对熵）定义为：

$$D_{\mathrm{KL}}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \tag{1.1.13}$$

对比式 (1.1.13) 与直觉部分的式 (1.1.4)，可以看到二者定义完全一致。性质：$D_{\mathrm{KL}}(p\|q) \geq 0$（由 Jensen 不等式和 log 的凹性得到），等号成立当且仅当 $p = q$；一般情况下 $D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$，即 KL 散度不是度量（metric）。

5. 交叉熵（cross-entropy）

交叉熵定义为：

$$H(p, q) = -\sum_x p(x) \log q(x) \tag{1.1.14}$$

将式 (1.1.14) 与式 (1.1.5) 对比，二者形式完全一致，只是在严格定义中显式写出了求和变量 x。由式 (1.1.14) 与式 (1.1.13) 直接可得与 KL 散度的恒等关系：

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p\|q) \tag{1.1.15}$$

式 (1.1.15) 与直觉部分的恒等式 (1.1.6) 完全对应。因为真实数据分布 p 的熵 $H(p)$ 是常数（不依赖于模型参数），最小化交叉熵等价于最小化 KL 散度，等价于让模型分布 q_θ 最大化真实数据的对数似然。大模型的训练损失为：

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log q_\theta(w_i \mid w_{<i}) \tag{1.1.16}$$

其中 $w_{<i} = (w_1, \ldots, w_{i-1})$ 为前缀，N 为序列长度，q_θ 为模型输出的条件概率。注意式 (1.1.16) 正是将式 (1.1.14) 定义的交叉熵中的 $H(p)$ 常数项去掉后的形式。

6. 困惑度（perplexity）

困惑度是评估语言模型的标准指标，定义为式 (1.1.16) 定义的训练损失的指数，与式 (1.1.7) 保持一致：

$$\mathrm{PPL} = \exp\left(-\frac{1}{N} \sum_{i=1}^{N} \log q_\theta(w_i \mid w_{<i}) \right) = \exp\big(H(p, q)\big) \tag{1.1.17}$$

将式 (1.1.17) 与式 (1.1.7) 对比：直觉部分的式 (1.1.7) 只写出 $\exp(H(p, q))$ 这一简洁形式，而式 (1.1.17) 则把它展开成了实际可计算的形式——即对式 (1.1.16) 定义的损失取指数。直观解释：困惑度等于模型在每一步预测时”等效面临的均匀选择数”。若 $\mathrm{PPL} = k$，模型的表现等价于从 k 个等可能选项中均匀猜测。困惑度越低，模型越确

定。注意困惑度的数值大小与 log 底数直接绑定——若改用 $\log_2$，则 $\text{PPL} = 2^{H_2(p,q)}$，其中 H_2 表示以 2 为底的交叉熵（单位为 bit）。

与原书呼应：原书 §1.1 指出大模型的训练目标是最小化交叉熵损失 $\mathcal{L} = -\frac{1}{N}\sum_i \log P_\theta(w_i \mid w_{<i})$，对应本节式 (1.1.14)、式 (1.1.15) 的交叉熵与 KL 散度定义，以及式 (1.1.17) 定义的困惑度作为其指数化形式。

---

§1.2 从马尔可夫链到 N-Gram：第一代尝试的局限

▶这一节在讲什么

为什么 N-Gram 会失败？1990 年代之前，几乎所有语言模型都是 N-Gram。它的失败不是「调参不够」，而是数学上注定的。

第一步：N-Gram 的核心是一个粗暴的独立性假设——「下一个词只依赖于前 k 个词」。这等价于说语言是 k 阶马尔可夫链。

第二步：参数空间随 k 指数爆炸。词表 $V = 50000$、$k = 4$ 时，参数有 $50000^5 \approx 3.1 \times 10^{23}$ 个——比可观测语料的 token 数大得多个数量级。绝大多数计数是零。

第三步：平滑技术也只能续命，不能解围。Kneser-Ney 是 N-Gram 时代最好的平滑，但当 k 想要捕获长距离依赖时（比如代词指代、章节伏笔），它仍然无能为力。

本节要拿走的一件事：N-Gram 失败的数学根源是 ** 指数爆炸 + 没有参数共享 **。后来的词嵌入、Transformer 之所以能成功，关键在于 ** 用低维稠密向量代替高维稀疏计数 **，让参数能在相似上下文之间共享。

直觉

原书从马尔可夫 1913 年对俄语字母的手工统计出发，介绍了 N-Gram 模型的思路与两个致命缺陷：组合爆炸和无法建模长距离依赖。但这两个「缺陷」并不是孤立的工程问题，它们沿着一条数学上的因果链一节一节地推出来——一旦你接受了 N-Gram 的起始假设，后面的所有困境都已经在数学上被锁死了。

本节出现五个关键概念：**k 阶马尔可夫假设、最大似然估计（MLE）、稀疏性与零概率、平滑技术、神经网络的解围**。它们看似散落，实际上沿着「假设 → 估计方法 → 失败模式 → 补救 → 范式更替」一条逻辑链推下来。下面把这条链一节节铺开。

第一环：k 阶马尔可夫假设——为了能算，放弃所有远程依赖

语言序列的真实分布是 $P(w_1, w_2, \ldots, w_T)$——一个定义在 V^T 维空间上的联合分布，V 是词表大小。直接估计它显然不可能：$T = 100$、$V = 50000$ 时，可能的句子有 $V^T = 50000^{100}$ 个，绝大多数从未在任何语料里出现过。

马尔可夫提出的解围办法是引入条件独立假设：

$$P(w_t \mid w_1, \ldots, w_{t-1}) = P(w_t \mid w_{t-k}, \ldots, w_{t-1}) \tag{1.2.1}$$

即下一个词只依赖于前 k 个词。式 (1.2.1) 就是 k 阶马尔可夫假设，对应 $(k+1)$-gram 模型。整个联合分布被分解成 T 个**只看局部窗口**的条件概率乘积。

式 (1.2.1) 是 N-Gram 一切问题的源头。它没有什么数学上的「错」——错的是它对语言现实的简化太粗暴。语言里到处是长距离依赖：「The boy who lived next door to my cousin and went to school with me yesterday …」——主语「boy」和谓语之间可以隔 20 个词。固定的 k 注定要么太短（丢失长依赖）要么太长（参数爆炸，下一环就说）。

第二环：最大似然估计——把假设变成可计算的频率比

有了式 (1.2.1) 定义的 k 阶马尔可夫假设，下一步是怎么从数据估计这些条件概率。最自然的选择是**最大似然估计**（MLE）——让训练语料的对数似然最大化。求导后得到的解就是频率比：

$$\hat{P}(w_n \mid w_{n-k}, \dots, w_{n-1}) = \frac{\text{count}(w_{n-k}, \dots, w_{n-1}, w_n)}{\text{count}(w_{n-k}, \dots, w_{n-1})} \tag{1.2.2}$$

式 (1.2.2) 给出的 MLE 在统计学意义上是无偏、相合的（语料趋于无穷时收敛到真分布）。如果世界是理想的，N-Gram + 式 (1.2.2) 就够了。

问题出在「语料趋于无穷」这个前提——它在现实中根本不成立。下一环就看现实里 MLE 怎么崩溃。

第三环：稀疏性与零概率——MLE 在有限语料上的必然崩溃

由式 (1.2.1) 知，k 阶 N-Gram 的参数空间大小是 V^{k+1}。代入实际数字：

- $V = 50000$、$k = 4$（5-gram）：参数空间 $V^5 \approx 3.1 \times 10^{23}$；
- 即便用所有公开互联网文本，token 总数也不过 $10^{12} \sim 10^{13}$ 量级；
- 参数比训练数据多 **10 个数量级**。

后果是灾难性的：绝大多数 5-gram 在语料里从未出现过，式 (1.2.2) 给它们的估计是 $\hat{P} = 0$。一旦某个句子里出现了一个「未见过」的 5-gram，整句话的概率就被乘成 0——模型给一个语法完全正确的新句子打分为「绝对不可能」。

这就是稀疏性问题。它的根源不是「估计方法不好」，而是「参数空间 V^{k+1} 在 $k \geq 3$ 之后就指数爆炸，永远超出任何可观测语料规模」。这是 N-Gram 范式在数学上必死的命门。

第四环：平滑技术——给零概率「续命」

为了让 N-Gram 在工程上还能用，整个 80-90 年代的研究花在了一件事上：**平滑**（smoothing），即把概率从已观察的 n-gram 转移一部分给未观察的。

最简单的 **Add-one（Laplace）平滑**对每个可能 n-gram 计数加 1：

$$\hat{P}_{\text{add-1}}(w_n \mid \text{ctx}) = \frac{\text{count}(\text{ctx}, w_n) + 1}{\text{count}(\text{ctx}) + V} \tag{1.2.3}$$

将式 (1.2.3) 与式 (1.2.2) 对比：式 (1.2.3) 在分子加 1、分母加 V，保证了所有计数非零，但代价是对低频 n-gram 偏差极大（它把同样多的概率塞给了真的稀有词和实际上根本不可能的组合）。

N-Gram 时代的巅峰是 **Kneser-Ney 平滑**。它的核心洞察是：「一个词作为延续出现在多少种不同上下文里」比「这个词出现的总次数」更能预测它在新上下文里的出现概率。比如「Francisco」总频次很高，但它几乎只跟在「San」后面，所以在一个新上下文

里它的延续概率应该比频次给出的低得多。Kneser-Ney 用**延续计数**代替原始计数，把这个直觉量化：

$$P_{\text{KN}}(w \mid \text{ctx}) \propto \max(\text{count}(\text{ctx}, w) - d, 0) + \lambda(\text{ctx})\, P_{\text{KN}}(w) \tag{1.2.4}$$

式 (1.2.4) 是 N-Gram 时代效果最好的平滑，至今仍是基线评估的参照点。但它**只能续命，不能解围**——因为它仍然在「计数」这个稀疏空间里腾挪，而稀疏的本质（V^{k+1} 爆炸）丝毫没变。与式 (1.2.2) 相比，式 (1.2.4) 只是对已有的概率重新分配，不能凭空创造「相似上下文之间应该共享信息」的能力。

第五环：神经网络的解围——用低维稠密向量打破指数空间

N-Gram 困境的真正出口是把「**词作为字符串符号**」换成「**词作为低维稠密向量**」。词嵌入（Bengio 2003、Word2Vec 2013）把每个词映射到 $\mathbb{R}^d$ 中的向量（d 通常 $100 \sim 1000$）。神经语言模型不再去估计 V^{k+1} 个独立参数，而是通过共享的嵌入矩阵和一个非线性函数 f_θ 计算 $P(w_t \mid \text{context})$。

关键的范式变化在「参数共享」：如果模型在训练时学到「猫」和「狗」的向量靠近，那么即使训练语料里从未出现「狗跳上桌子」，模型仍能利用「猫跳上桌子」给「狗跳上桌子」一个合理的概率。在 N-Gram 框架下这绝不可能——「猫」和「狗」是字符串，没有任何机制让它们的统计信息互相借用。

Transformer 把这一思路推到极致：通过自注意力，每个 token 的预测能动态地、有选择地依赖序列里任意远的位置——既绕过式 (1.2.1) 中 k 的硬性截断，又通过低维嵌入避免参数爆炸。N-Gram 的两个死穴（指数爆炸、固定窗口）被同一个数学转换一并解掉。

整条链的回头看

```
k 阶马尔可夫假设                  ← 为了能算，假设只看前 k 个词
  ↓ 怎么估计这些条件概率？
最大似然估计 MLE                  ← 频率比，统计上无偏
  ↓ 在有限语料上会怎样？
稀疏性 + 零概率                   ← V^(k+1) 远超数据量，绝大多数计数为 0
  ↓ 工程上怎么续命？
平滑技术（Add-one、Kneser-Ney）   ← 在稀疏空间内重新分配概率
  ↓ 续命到极限还是不够（长距离依赖 + 大词表）
神经网络（嵌入 + 自注意力）       ← 把符号换成向量，参数共享 + 动态窗口
```

这条链揭示了一件事：N-Gram 的失败不是「调参不够」「平滑不够好」，而是「**用稀疏符号计数 + 固定窗口**」**这条路在数学上注定了出路有限**。词嵌入和 Transformer 不是在 N-Gram 框架内打补丁，而是从根本上换掉了「概率分布的存储方式」——把指数空间换成低维流形。理解了这条链，再去看后面 §3 的词嵌入和 §6 的 Transformer，就知道它们不是「又一个模型」，而是**对 N-Gram 范式数学死穴的精确回应**。

严格

1. k 阶马尔可夫性（k-th order Markov property）

设词序列 $(w_1, w_2, \ldots, w_T)$，每个 w_t 取值于词表 $\mathcal{V}$（$|\mathcal{V}| = V$）。**k 阶马尔可夫假设**为：

$$P(w_t \mid w_1, w_2, \dots, w_{t-1}) = P(w_t \mid w_{t-k}, w_{t-k+1}, \dots, w_{t-1}) \tag{1.2.5}$$

式 (1.2.5) 与直觉部分的式 (1.2.1) 完全对应：给定最近 k 个词后，w_t 与更早的历史条件独立。

N-Gram 的对应关系：k 阶马尔可夫链等价于 $(k+1)$-gram 模型。

k（马尔可夫阶）	N-Gram 名称	条件窗口大小
0	unigram（1-gram）	无条件（词独立）
1	bigram（2-gram）	前 1 个词
2	trigram（3-gram）	前 2 个词
k	$(k+1)$-gram	前 k 个词

整条序列的概率在式 (1.2.5) 的马尔可夫假设下分解为：

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_{t-k}, \dots, w_{t-1}) \tag{1.2.6}$$

（对 $t \le k$ 的情形，以序列开头作为边界条件处理。）

2. 最大似然估计（maximum likelihood estimation, MLE）

给定训练语料，$(k+1)$-gram 模型的 MLE 参数为计数之比：

$$\hat{P}(w_n \mid w_{n-k}^{n-1}) = \frac{\text{count}(w_{n-k}, \dots, w_{n-1}, w_n)}{\text{count}(w_{n-k}, \dots, w_{n-1})} \tag{1.2.7}$$

式 (1.2.7) 与直觉部分的式 (1.2.2) 完全对应，只是用符号 w_{n-k}^{n-1} 表示从 w_{n-k} 到 w_{n-1} 的子序列，$\text{count}(\cdot)$ 表示该 n-gram 在训练语料中出现的次数。这一估计是无偏的，且在语料趋于无穷时收敛至真实分布。

3. 稀疏性与零概率问题

由式 (1.2.7) 可知，MLE 的根本缺陷：若某 n-gram 在训练语料中从未出现，则 $\hat{P} = 0$，导致任何包含该 n-gram 的句子概率为零。词表大小 V 时，k 阶 N-Gram 的参数空间为 $O(V^{k+1})$。当 $V = 50000$，$k = 4$ 时，参数量级为 $V^5 \approx 3.1 \times 10^{23}$，远超可观测语料的大小，绝大多数组合计数为零。

4. 平滑（smoothing）技术直觉

Add-one（Laplace）平滑：对每个可能的 n-gram 计数加 1，等价于在参数的先验（均匀 Dirichlet）下取后验均值：

$$\hat{P}_{\text{add-1}}(w_n \mid w_{n-k}^{n-1}) = \frac{\text{count}(w_{n-k}^{n-1}, w_n) + 1}{\text{count}(w_{n-k}^{n-1}) + V} \tag{1.2.8}$$

将式 (1.2.8) 与式 (1.2.7) 对比，可见式 (1.2.8) 在分子和分母均做了正则化调整——分子加 1、分母加 V，保证了所有概率非零，但对低频 n-gram 的估计偏差较大，因为它对稀有和常见组合施加了同等的绝对调整量。

Kneser-Ney 平滑的核心直觉（不做完整推导）：观察到”出现在多样上下文中的词”在未见 n-gram 中应有更高的延续概率。具体地，用**延续计数**（continuation count）代替频率计数：

$$P_{\text{KN}}(w \mid \text{context}) \propto \max\big(\text{count}(\text{context}, w) - d,\ 0\big) + \lambda(\text{context})\, P_{\text{KN}}(w) \tag{1.2.9}$$

式 (1.2.9) 与直觉部分的式 (1.2.4) 完全对应，其中 $d \in (0,1)$ 为折扣参数，$\lambda(\text{context})$ 为归一化常数，$P_{\text{KN}}(w)$ 为低阶（回退）分布，定义为 w 出现在其左侧上下文种类数与总上下文种类数之比。Kneser-Ney 是 N-Gram 时代效果最好的平滑方法，至今仍是基线评估的参照点。

与原书呼应：原书 §1.2 指出 k 阶马尔可夫假设是 N-Gram 的数学本质，并以词表大小 $V = 50000$、5-gram 参数量 $V^5 \approx 3.1 \times 10^{23}$ 说明组合爆炸，对应本节式 (1.2.5) 的 MLE 定义和稀疏性分析。

---

§1.3 跨界映射：语言序列与 DNA 碱基对

▶这一节在讲什么

为什么要谈「熵率」？前两节的熵 $H(X)$ 是针对单个随机变量定义的，但语言是一长串符号。我们需要一个量来刻画「一个无穷长的序列每符号平均携带多少信息」。

第一步：把序列的联合熵除以长度。$\mathcal{H} = \lim_{n\to\infty} \frac{1}{n} H(X_1, \ldots, X_n)$。如果序列是平稳的（统计性质不随时间变），这个极限就存在。

第二步：Shannon-McMillan-Breiman 定理（AEP）给出了一个惊人事实——对几乎所有「典型」的长序列，每个序列的概率都约等于 $2^{-n\mathcal{H}}$。也就是说，长度 n 的序列虽然字面上有 M^n 种，但 ** 实际上只有 $2^{n\mathcal{H}}$ 个序列扛走了几乎全部的概率质量 **。

第三步：这对语言模型意味着什么？训练语料的「有效信息容量」不是字母表大小的幂次，而是由熵率决定。模型只要覆盖典型序列集，就能捕获几乎全部概率。

本节要拿走的一件事：熵率把信息论从「单个变量」推广到「无穷序列」——这是把语言、DNA、时间序列都纳入同一个数学框架的关键一步。

直觉

原书将语言和 DNA 视为「有限符号集 + 组合规则 + 信息传递」的共同框架。但要在数学上讲清为什么这个跨界类比「合法」，需要一套能把「一个随机变量的熵」推广到「无穷长序列每个符号携带的平均信息量」的工具——这就是**熵率**。

本节出现四个关键概念：**平稳过程、序列联合熵、熵率、Shannon-McMillan-Breiman 定理（AEP）**。它们沿着「单变量熵 → 多变量联合熵 → 需要除以长度 → 需要平稳性保证极限存在 → SMB 定理揭示典型序列」一条推广链走。走完这条链，才能明白为什么同一套信息论可以同时适用于语言、DNA、语音、金融时间序列。

第一环：从单变量熵走到序列联合熵

§1.1 定义的香农熵（即式 (1.1.1)）中，X 是一个随机变量。但语言是一长串符号 $(X_1, X_2, \dots, X_n)$，主事不是「下一个词」而是「整句话」。最直接的推广是拿联合熵：

$$H(X_1, X_2, \dots, X_n) = -\sum_{x_1,\dots,x_n} p(x_1, \dots, x_n) \log p(x_1, \dots, x_n) \tag{1.3.1}$$

式 (1.3.1) 仍然是「平均需要多少比特才能编码」，但现在面对的是整个长度 n 的序列。注意它的形式是把式 (1.1.1) 中的单变量求和替换成了对所有可能的长度 n 序列求和。问题是：随着 $n \to \infty$，式 (1.3.1) 定义的 $H(X_1, \dots, X_n)$ 也趋于无穷，这个量本身没法拿来比较「英语和汉语哪个信息密度高」这种问题。需要一个**归一化**。

第二环：除以长度——得到「每符号平均信息量」

自然的归一化是除以 n，看平均每符号携带多少信息：

$$\frac{1}{n} H(X_1, X_2, \dots, X_n) \tag{1.3.2}$$

式 (1.3.2) 是对式 (1.3.1) 的归一化，但这个量随 n 怎么变？凭空看不出来。它可能振荡、可能发散、也可能收敛。只有在**某种结构假设下**，这个序列才会有极限。这将在下一环出场。

第三环：平稳性——保证极限存在的最温和假设

要让式 (1.3.2) 有极限，需要过程的**统计性质不随时间平移变**。这就是平稳性（stationarity）：对任何偏移 $k \geq 0$，

$$P(X_1 = a_1, \dots, X_n = a_n) = P(X_{1+k} = a_1, \dots, X_{n+k} = a_n) \tag{1.3.3}$$

直观上：「今天看到的这个句子」和「明年从同一语言里看到同样的句子」出现的概率应该一样。语言、DNA、股价复权后的对数收益率近似都能在某个有限时间尺度上被看作平稳。从式 (1.3.3) 出发可以证明：式 (1.3.2) 定义的量随 $n \to \infty$ 必收敛。这个极限就是**熵率**：

$$\mathcal{H} = \lim_{n\to\infty} \frac{1}{n} H(X_1, X_2, \dots, X_n) \tag{1.3.4}$$

等价地（由链式法则），

$$\mathcal{H} = \lim_{n\to\infty} H(X_n \mid X_1, \dots, X_{n-1}) \tag{1.3.5}$$

式 (1.3.5) 在语言模型语境里更直观：**在已知无限长历史的条件下，下一个 token 还剩多少不确定性**。注意式 (1.3.5) 实质上是把式 (1.1.2) 定义的条件熵推广到无穷长历史的极限。香农猜字母实验估出英语熵率约为 1 bit/字符，就是式 (1.3.5) 这个量。

第四环：Shannon-McMillan-Breiman 定理——典型序列集才走了几乎所有概率质量

上一环只在「期望」层面定义了熵率。但 SMB 定理（亦称渐近等分性，Asymptotic Equipartition Property，AEP）走得更远：对平稳遍历过程，几乎对**每一样**本序列（不只是在期望上），

$$-\frac{1}{n}\log P(X_1, X_2, \ldots, X_n) \xrightarrow{a.s.} \mathcal{H} \tag{1.3.6}$$

这一「几乎处处」(almost surely) 是关键。将式 (1.3.6) 与式 (1.3.4) 对比：式 (1.3.4) 是在期望意义下的收敛，而式 (1.3.6) 是对几乎每个样本序列都成立的逐点收敛。它意味着：对充分长的 n，长度 n 的可能序列虽然有 M^n 种（M 是字母表大小），但几乎全部概率质量都集中在「典型序列」集合里——这个集合大约只含 $2^{n\mathcal{H}}$ 个序列，每个典型序列的概率约为 $2^{-n\mathcal{H}}$。非典型序列总体上概率趋近于 0。

具体代入：英语 $\mathcal{H} \approx 1.5$ bit/字符（按字符级别粗略估）、长度 $n = 100$ 的句子：

- 字面上可能的序列数：$27^{100} \approx 10^{143}$；
- 典型序列数：$2^{150} \approx 10^{45}$；
- 典型序列只占所有句子的 10^{-98}，但几乎拿走了全部概率。

这是个迫使人重新理解「语言模型在学什么」的深刻事实。

第五环：这对语言模型意味着什么

SMB 定理（式 (1.3.6)）给了训练语料一个**信息论容量上限**。训练语料里能看到的句子几乎全是典型序列；训练语料的「有效信息容量」不是 M^n 而是 $2^{n\mathcal{H}}$，两者差了一个 $2^{n(\log_2 M - \mathcal{H})}$ 倍的压缩比。

对语言模型的启示：**一个足够好的模型只需要能「覆盖」典型序列集合，就能抓住实际中出现的几乎全部概率质量**。这是 Transformer 能用有限参数拟合 token 空间里「所有句子的分布」的深层原因——它们不是要记住所有可能句子，只要能拟合典型序列集上的分布。

DNA 编码区接近其字母表上限 $\log_2 4 = 2$ bit/碱基（几乎随机），英语文本 $\mathcal{H} \approx 1$ bit/字符，但英语的字母表上限是 $\log_2 26 \approx 4.7$ bit——决定语言信息密度低于字母表上限的那个几乎 4 倍的差距以其背后的平稳假设（式 (1.3.3)）为前提，全部是 SMB 定理（式 (1.3.6)）所手把手推出来的。

整条链的回头看

```
单变量熵 H(X)                          ← §1.1 定义，对一个 token
  ↓ 语言是长序列，推广到多变量
联合熵 H(X₁, …, Xₙ)                    ← 仍是信息总量，随 n 发散
  ↓ 需要「每符号平均」才能比较
除以长度 (1/n) · H(X₁,…,Xₙ)            ← 但这个量有极限吗？
  ↓ 加平稳性假设，保证收敛
熵率                                   ← 「无限长历史后下一符号还剩多少不确定性」
  ↓ 不仅期望意义上，个体序列也收敛吗？
SMB / AEP 定理                         ← 几乎处处收敛，典型序列集 ≈ 2^(n·h)
  ↓ 这对模型意味什么？
语言模型只需覆盖典型序列               ← 信息容量不是 M^n 而是 2^(n·h)
```

这条链说明了一件事：「**语言和 DNA 都是信息传递系统**」**不是修辞，是 SMB 定理提供的数学事实**——只要一个过程是平稳遍历的有限字母表符号序列，它就服从同一套熵率、典型序列、信息压缩上限的数学。这就是为什么可以用一套技术同时处理 NLP、生物信息、金融时间序列。下一节 §1.4 要说的是：这个「同一套数学」在参数、量

级上仍有巨大差别，跨界类比能启发思考，但不能拿 DNA 的工具直接搬去处理语言。

严格

1. 信息源与符号序列

设符号序列 $(X_1, X_2, \dots)$ 为定义在有限字母表 $\mathcal{A}$（$|\mathcal{A}| = M$）上的平稳随机过程（stochastic process）。**平稳性**（stationarity）要求联合分布在时间平移下不变：

$$P(X_1 = a_1, \dots, X_n = a_n) = P(X_{1+k} = a_1, \dots, X_{n+k} = a_n), \quad \forall k \geq 0 \tag{1.3.7}$$

式 (1.3.7) 与直觉部分的式 (1.3.3) 完全对应，只是增加了 $\forall k \geq 0$ 的量词标注。

2. 熵率（entropy rate）

序列的**熵率**（entropy rate）定义为：

$$\mathcal{H} = \lim_{n \to \infty} \frac{1}{n} H(X_1, X_2, \dots, X_n) \tag{1.3.8}$$

式 (1.3.8) 与直觉部分的式 (1.3.4) 完全对应。对平稳过程，该极限存在。等价地（由链式法则），熵率也等于条件熵的极限：

$$\mathcal{H} = \lim_{n \to \infty} H(X_n \mid X_1, \dots, X_{n-1}) \tag{1.3.9}$$

式 (1.3.9) 与式 (1.3.5) 对应。将式 (1.3.9) 与 §1.1 的式 (1.1.9) 对比，可以看出熵率是在「无穷长历史」的极限下取条件熵，而不是像式 (1.1.9) 那样固定联合分布的维数。对语言序列而言，$\mathcal{H}$ 刻画了”在已知所有历史的条件下，下一个 token 还剩多少不确定性”。香农估计英语文本的熵率约为 1 bit/字符（$\log_2$ 底数），这正是他猜字母实验的结论。

3. Shannon-McMillan-Breiman（SMB）定理

对有限字母表上的**平稳遍历**（stationary ergodic）过程，SMB 定理（亦称渐近等分性，Asymptotic Equipartition Property，AEP）成立：

$$-\frac{1}{n} \log P(X_1, X_2, \dots, X_n) \xrightarrow{a.s.} \mathcal{H}, \quad n \to \infty \tag{1.3.10}$$

式 (1.3.10) 与直觉部分的式 (1.3.6) 完全对应。即序列的经验对数概率以概率 1 收敛至由式 (1.3.8) 定义的熵率 $\mathcal{H}$。其核心推论是：对充分大的 n，长度为 n 的序列几乎全部集中在约 $2^{n\mathcal{H}}$（$\log_2$ 底数）个”典型序列”上，每个典型序列的概率约为 $2^{-n\mathcal{H}}$。非典型序列的概率之和趋于零。

对语言模型的意义：SMB 定理（式 (1.3.10)）说明，一个足够好的模型只需要”覆盖”典型序列集合，就可以捕获几乎全部的概率质量。训练语料的有效信息容量由式 (1.3.8) 定义的熵率决定，而不是由字母表大小的幂次决定。

与原书呼应：原书 §1.3 指出语言和 DNA 在”有限符号集 + 组合规则”的信息论框架下具有结构相似性，对应本节式 (1.3.8) 的序列熵率定义和式 (1.3.10) 的 SMB 定理对典型序列的描述。

---

§1.4 反类比：当 DNA 的隐喻失效

▶这一节在讲什么

为什么 DNA 的隐喻不能滥用？前一节展示了语言和 DNA 在信息论框架下的共性。这一节给出三堵铁墙：** 字母表量级差异、稀疏性差异、冗余的物理本质差异 **。

第一步：字母表大小决定熵率上限。DNA 只有 4 个符号，上限 $\log 4 \approx 2$ bit；大模型词表有 5 万 token，上限 $\log 50000 \approx 15.6$ bit——理论信息密度差近 8 倍。

第二步：稀疏性的量级完全不同。DNA 三联体只有 $4^3 = 64$ 种组合，可以穷举统计；语言 5-gram 有 $50000^5 \approx 3 \times 10^{23}$ 种，远超任何可能的语料规模。

第三步：冗余的来源不同。DNA 的简并性是 ** 物理化学约束 **（tRNA 摆动配对），是写死的；语言的冗余是 ** 使用者群体统计行为的涌现 **，会随时间漂移（流行语、网络用语、新词）。

本节要拿走的一件事：「语言像 DNA」是一个 ** 比喻 **，不是定理。它能启发跨学科思考，但当你真的把 DNA 序列建模技术搬到 NLP 时，必须重新面对：更大的字母表、更稀疏的统计、更深的长程依赖——这才是 Transformer 崛起的数学土壤。

直觉

原书在承认语言与 DNA 的信息论共性后，立即指出三堵「铁墙」。这三堵墙不是三个独立的细节差别，而是从「字母表量级差异」这个原点出发、一堵墙一堵墙推进去的：量级差异 → 稀疏性差异 → 冗余物理本质差异 → 动态性差异。走完这条链才会明白：为什么语言需要一个全新的数学架构（嵌入 + Transformer + 持续预训练），而不能拿一个现成的 DNA 分析包来用。

第一环：字母表量级差异——所有后果的起点

DNA 的字母表只有 4 个符号（A、T、G、C）；现代大模型的 BPE 词表约 5 万 token。这是一个 $4 \to 50000$、增加了四个数量级的量变。从 §1.3 推出的字母表上限 $\mathcal{H} \leq \log M$ 代入：

- DNA：$\log_2 4 = 2$ bit/符号；
- 语言 token：$\log_2 50000 \approx 15.6$ bit/token。

两者理论信息密度上限差了 $15.6/2 \approx 7.8$ 倍。这看似只是「信息密度不同」的细节差别，但后面几环会看到这个 V 的量级如何裂变成一整堆范式上的不可跨越。

第二环：稀疏性裂变——一个可穷举、一个永远压不完

§1.2 里已说明 k 阶 N-Gram 参数空间为 V^{k+1}（见式 (1.2.5) 的马尔可夫假设）。代入不同的 V：

- DNA、$k = 2$（三联体密码子）：$4^3 = 64$ 个组合——**在一份用多些的实验数据里可以完全穷举**，甚至可以把所有 64 个密码子 → 20 种氨基酸的映射表手动制出。
- 语言、$k = 4$（5-gram）：$50000^5 \approx 3.1 \times 10^{23}$ 个组合——**任何有限语料都不可能覆盖绝大多数**。

这是质变不是量变。DNA 三联体层面上**不存在稀疏性问题**；语言在 $k \geq 3$ 后稀疏性已不可避免，必须依靠平滑（式 (1.2.8)、式 (1.2.9)）或参数共享（词嵌入）。一个量级临界一跨过，范式必须变。

举个具体后果：隐马尔可夫模型（HMM）在生物信息里干得不错（隐性密码子识别、跨物种同源区块划分），是因为 64 个三联体只需 64 个输出状态，HMM 的隐状态也只在几十到几百。拿同一个 HMM 去拟合一个 5 万 token 的语言任务？状态转移矩阵是 50000×50000——参数量上就崩了。

第三环：冗余的物理本质差异——它们「为什么会出现冗余」的机制根本不一样

不只数量不同，DNA 和语言「为什么会出现冗余」的机制是两回事。

DNA 的冗余是**写在物理里**的：遗传密码存在简并性，64 个密码子仅编码 20 种氨基酸（加 3 个终止密码子）。这是 tRNA 反密码子的摆动配对决定的物理化学事实，不会随物种、时代、地域变化。人、小鼠、香蕉、大肠杆菌用同一张密码表。

语言的冗余完全是**使用者群体统计行为的涌现**：语法约束、词类共现、上下文语义限制——这些规则没有任何物理化学的必然性。

第四环：冗余机制差异推出动态性差异——一个定、一个在漂移

上一环在机制上区分了两者，这一区分直接推出一个工程后果：

- **DNA 的统计性质是定的**——一份 1900 年小鼠的基因组和 2025 年小鼠的基因组在统计性质上几乎一样。
- **语言的统计性质在演化**——一份 1900 年英文语料训出的 N-Gram 在 2025 年的网络语言上几乎不可用：「goated」「yeet」「rizz」这些词在 1900 年不存在；「wireless」「telegram」这些词在 1900 年的统计性质完全不同。

这就是为什么大模型需要**持续预训练** + **指令微调** + **上下文学习**这三重机制，而生物信息模型可以「一次训练、长期使用」——不是研究者努力程度的差异，是被建模对象本身在不在漂移。

整条链的回头看

```
字母表量级差异 M: 4 → 50000          ← 信息密度上限差 8 倍
  ↓ 这在参数空间里怎么表现？
稀疏性差异 V^(k+1): 64 → 10²³        ← 一个可穷举、一个永远压不完
  ↓ 为什么会出现冗余？机制一样吗？
冗余本质差异：物理定 vs 使用者涌现
  ↓ 这推出什么工程后果？
动态性差异：DNA 定、语言漂移          ← 语言需预训练 + SFT + ICL
```

这条链的意义不只是「别乱拿一个领域的技术去另一个领域」这种警示。它还对一个广为流传的说法「语言是人类的 DNA」提出了严格限制：从信息论的角度，这句话**只在「同为有限字母表的平稳随机过程」这个抽象层面上成立**（由式 (1.3.7) 给出的平稳性为前提）；一旦落到参数量级、稀疏性、动态性这些下游问题，两者完全不是一回事。这正是 N-Gram 失败、Transformer 崛起的数学土壤——不是「另一个模型架构」，而是为语言这种「大字母表 + 动态漂移」的对象量身定裁的数学架构。

严格

1. 字母表大小与熵率上限

对大小为 M 的均匀字母表，熵率上限为 $\log M$（自然对数，单位 nat）：

$$\mathcal{H} \leq \log M \tag{1.4.1}$$

式 (1.4.1) 是对式 (1.3.8) 定义的熵率的上界：均匀分布时熵率达到最大值 $\log M$，任何非均匀分布都低于此上限。

- DNA：$M = 4$，熵率上限 $\log 4 \approx 1.39$ nat/符号（约 2 bit/符号）；
- 大模型的 subword 词表（如 GPT-4 的 BPE 词表）：$M \approx 50000$，熵率上限 $\log 50000 \approx 10.82$ nat/token（约 15.6 bit/token）。

两者的熵率上限之比约为 $\log 50000 / \log 4 \approx 7.8$，即语言 token 级别的理论信息密度是 DNA 碱基的近 8 倍。

2. 稀疏性的量级差异

由式 (1.2.5) 可知，对 k 阶 N-Gram，参数空间大小为 M^{k+1}。

- DNA，$k = 2$（三联体密码子）：$4^3 = 64$ 个组合，全部可在有限实验数据中覆盖。
- 语言，$k = 4$（5-gram）：$50000^5 \approx 3.1 \times 10^{23}$ 个组合，任何有限语料都无法覆盖其中绝大多数。

因此，DNA 序列在三联体层级上不存在稀疏性问题，可以对密码表做穷举统计；而语言序列在 $k \geq 3$ 时稀疏性已经不可避免，必须依赖式 (1.2.8) 的 Add-one 平滑或式 (1.2.9) 的 Kneser-Ney 平滑、乃至参数共享（如词嵌入）。

3. 遗传码的信息冗余与语言冗余的本质差异

遗传密码存在**简并性**（degeneracy）：64 个密码子仅编码 20 种氨基酸（加 3 个终止密码子），多个密码子映射到同一氨基酸。这是物理化学约束（tRNA 反密码子的摆动配对）决定的，与统计规律无关。

语言的冗余则来自两个来源：句法约束（词类共现规则）和语义约束（上下文限制词义）。这两类约束没有物理化学的必然性，它们是使用者群体统计行为的涌现，因此可以随时间和社群漂移。数学上，语言的冗余度体现为由式 (1.3.8) 定义的熵率 $\mathcal{H}$ 远低于式 (1.4.1) 给出的上限 $\log M$：香农估计英语的实际熵率约为 1 bit/字符，而字母表（26 字母）的上限为 $\log_2 26 \approx 4.7$ bit/字符，冗余度约为 $1 - 1/4.7 \approx 79\%$。DNA 的情况则不同：以碱基为单位，其熵率接近式 (1.4.1) 定义的理论上限（基因组中存在大量非编码区，编码区本身因密码子偏好性略低于上限），冗余度低得多。

这一量级差异说明，把 DNA 的序列建模技术直接迁移到语言，不仅需要应对更大的字母表，还需要捕获更深层、更长程的统计依赖——这正是 N-Gram 失败、变换器（Transformer）崛起的数学根源。

与原书呼应：原书 §1.4 指出 DNA 字母表（4 符号）与语言词表（数万 token）的差异是类比失效的起点之一，对应本节式 (1.4.1) 的熵率上限量级比较以及由式 (1.2.5) 推出的稀疏性参数空间计算。

数学卷 · 第 2 章：嵌入空间与高维几何

§2.1 词嵌入：为什么字典救不了机器

▶这一节在讲什么

为什么符号表示无法承载语义？字典用文字解释文字，递归下去仍是符号游戏。机器需要的不是定义，而是一种能让「猫」与「狗」天然相邻、「国王」与「总统」有所关联的数学结构。

第一步：认识 one-hot 编码的根本缺陷。最朴素的表示方案把每个词映射为一个只有一位为 1 的稀疏向量。词表有 V 个词，就有 V 维，任意两词的向量内积都是零——语义相似性完全丢失。

第二步：用稠密低维向量取而代之。嵌入矩阵 $E \in \mathbb{R}^{V\times d}$ $(d \ll V)$ 把每个词映射到一个 d 维实数向量。查表操作在数学上等价于矩阵乘法，但因 one-hot 的稀疏性，实际只需 $O(d)$ 计算。

第三步：用余弦相似度衡量语义接近程度。向量的方向而非长度承载语义——归一化后的夹角余弦值能屏蔽高频词模长偏大的噪声，使「猫」与「狗」在几何上真正相邻。

本节要拿走的一件事：词的含义不在符号名称里，在向量空间中的**位置**里——几何关系即语义关系。

直觉

N-Gram 模型把每个词当作原子符号——词与词之间没有任何相似性信息。将「cat」替换为「dog」，模型毫无察觉，因为两者在符号层面是完全不同的对象。字典式的递归定义同样无济于事：用文字解释文字，最终仍是符号游戏。真正的突破是放弃符号表示，转而用**实数向量**承载语义。

本节出现四个概念：**one-hot 编码**、**分布式表示**、**嵌入矩阵**、**余弦相似度**。它们看似只是工程选择，实际上沿着一条清晰的逻辑链一步推出一步——每一步都是对上一步缺陷的直接回应。理解这条链，是理解词嵌入「为什么有效」而不只是「怎么用」的关键。下面一环环展开。

第一环：one-hot 编码——符号表示的天花板

最朴素的想法是直接给词表中每个词编一个编号，然后把编号转成一个向量。词表有 V 个词，就用 V 维向量，第 k 个词的向量只有第 k 维是 1，其余全 0：

$$(e_w)_i = \begin{cases} 1 & i = k \\ 0 & \text{otherwise} \end{cases} \tag{2.1.1}$$

式 (2.1.1) 立刻碰到两堵墙。第一堵墙是**正交性**：任意两个词的 one-hot 向量内积都是零，$e_u^T e_v = 0$。这意味着在式 (2.1.1) 的表示下，「猫」和「狗」的距离与「猫」和「核弹」的距离完全相同——语义相似性信息一比特都没有编码进去。第二堵墙是**维度爆炸**：现代语言模型的词表通常有 $V \approx 50000$——每个词要存一个 50000 维的向量，而且其中 49999 维全是零。这不是「有点浪费」，而是根本上无法扩展：不但存储是 $O(V)$，而且任何矩阵乘法运算都需要处理 V 维的输入，成本随词表线性增长。

式 (2.1.1) 的失败不是参数调得不好，而是**表示形式从根本上切断了词与词之间的所有关联**。修复这个问题，需要一种全新的表示范式。

第二环：分布式表示——把语义「摊开」到多个维度

Hinton 在 1986 年提出的「分布式表示」（distributed representation）思想给出了方向：**用一个低维稠密实数向量来表示一个词，让每个维度都参与编码某方面的语义，不再让任何单个维度独占某个词**。

这个想法的直觉来自语言学：词义不是孤立的，而是在使用中通过与其他词的关系体现出来的——Firth 1957 年的名言「You shall know a word by the company it keeps」。如果「猫」和「狗」总是出现在相似的上下文（「我养了一只……」「……跑进了房间」），那么它们的表示向量就应该相似。

分布式表示做到了这一点：向量的每个分量不再代表「是否是这个词」，而是代表某种连续的语义特征。这些特征可以是可解释的（如「动物性」「抽象程度」「情感极性」），也可以是模型自己学出来的不可解释的特征组合。重要的是，两个词只要在某些维度上特征值接近，它们的向量就会靠近——无论词形多么不同。

这个范式转移有一个关键的数学代价：原来由式 (2.1.1) 给出的 one-hot 向量里词的「位置」是精确的（只有一个 1），现在变成了 $\mathbb{R}^d$ 中的一个点，必须通过训练来决定每个词应该「住」在哪里。分布式表示的质量，完全取决于训练目标和语料的质量。

第三环：嵌入矩阵——用一张表格存下所有词的向量

分布式表示在工程上的实现是一张矩阵。嵌入矩阵 $E \in \mathbb{R}^{V \times d}$，行数等于词表大小 V，列数等于向量维度 d（通常 $d \ll V$，典型地 $d \in [256, 12288]$）。矩阵的第 k 行就是第 k 个词的 d 维嵌入向量。

取出第 k 个词的嵌入，在数学上等价于把式 (2.1.1) 中的 one-hot 向量 e_w 和矩阵 E 相乘：

$$\mathbf{v}_w = E^T \mathbf{e}_w \in \mathbb{R}^d \tag{2.1.2}$$

但由于式 (2.1.1) 定义的 $\mathbf{e}_w$ 只有一个非零分量，式 (2.1.2) 的乘法实际上只是取出矩阵的一行（一次数组索引），计算代价是 $O(d)$ 而不是矩阵乘法的 $O(Vd)$。这就是「查表（embedding lookup）」这个名字的由来——它是矩阵乘法的高效特例，不是一个独立的操作。

这里有一个重要的对称性：在 Word2Vec 这类模型里，通常有**两套**嵌入矩阵：W（中心词矩阵）和 W'（上下文矩阵），同一个词在这两套矩阵里各有一行。这不是冗余，而是训练目标决定的——「作为被预测的词」和「作为预测别人的词」在统计意义上是不同的角色，用同一个向量表示会导致优化上的自回归偏差（一个词预测它自己的概率会被异常推高）。训练完成后通常取中心词矩阵 W 作为最终嵌入，或对两套矩阵求平均。

第四环：训练机制——语义从哪里涌现

嵌入矩阵 E 的初始值通常是接近零的随机小数（如从 $\mathcal{N}(0, 1/d)$ 采样）。此时所有词的向量几乎在同一个地方，毫无语义区分。语义结构是在训练目标的驱动下，从大量语料的统计共现中「涌现」出来的。

涌现的机制是：如果「猫」和「狗」经常出现在相同的上下文词附近（如「宠物」「毛茸茸的」「会跑的」），那么这两个词由式 (2.1.2) 给出的嵌入向量会在训练过程中被共同地「朝同一方向拉」——因为它们都需要给相同的上下文词分配高概率。经过足够多的训练步，「猫」和「狗」的向量在 $\mathbb{R}^d$ 中会自然地靠近，而「猫」和「核武器」的向量则会保持距离。

这个机制有一个深刻的推论：**嵌入向量不是「词义的真实映射」，而是「统计共现模式的几何编码」**。词义中那些不在语言使用模式里留下痕迹的部分，不会被嵌入捕获。「猫」的颜色、触感、叫声，如果从未成为区分上下文的依据，就不会体现在嵌入向量里。这同时也解释了为什么同一个词在不同语言的嵌入空间里，几何关系可以高度相似——因为人类用词的统计模式，跨语言有很深的共性。

整条链的回头看

```
one-hot 编码                          ← 符号表示：正交、稀疏、无语义
  ↓ 正交性切断了所有关联
分布式表示                            ← 语义「摊开」到多维实数向量
  ↓ 需要工程化的存储和访问
嵌入矩阵 E   R^{V×d}                  ← 一行 = 一个词的 d 维坐标
  ↓ 初始随机，需要训练目标驱动
统计共现 → 梯度更新                   ← 语义相似性从语料统计模式中涌现
  ↓ 向量指向比长度更稳定
余弦相似度                            ← 屏蔽词频偏置，测量语义方向差异
```

这条链揭示了一个深层事实：**词嵌入的「魔法」完全来自语料的统计结构，而非任何关于语言的显式规则**。几何关系即语义关系，不是人为规定的，而是从数十亿次「预测下一个词」的压力下自然涌现出来的。

严格

One-hot 编码（one-hot encoding）是最朴素的词表示。设词表大小为 V，词 w 对应词表中第 k 个位置，则其 one-hot 向量 $\mathbf{e}_w \in \{0,1\}^V$ 满足：

$$(\mathbf{e}_w)_i = \begin{cases} 1 & i = k \\ 0 & \text{otherwise} \end{cases} \tag{2.1.3}$$

式 (2.1.3) 与直觉部分的式 (2.1.1) 完全对应，只是用粗体符号 $\mathbf{e}_w$ 强调其向量性质。该表示有两个根本缺陷。第一，**稀疏性（sparsity）**：向量中只有一个非零分量，储存和计算效率极低，在 $V \sim 10^5$ 量级时尤为突出。第二，**维度灾难（curse of dimensionality）**：任意两个不同词的 one-hot 向量正交，$\mathbf{e}_u^T \mathbf{e}_v = 0$，无法表达语义相关性。

嵌入矩阵（embedding matrix） $E \in \mathbb{R}^{V \times d}$ 将上述问题一并解决，其中 $d \ll V$（典型地 $d \in [256, 12288]$）。矩阵 E 的第 k 行即为词 w_k 的稠密低维嵌入向量。**查表（lookup）**操作等价于矩阵-向量乘法：

$$\mathbf{v}_w = E^T \mathbf{e}_w \in \mathbb{R}^d \tag{2.1.4}$$

式 (2.1.4) 与直觉部分的式 (2.1.2) 完全对应。由于 $\mathbf{e}_w$ 是由式 (2.1.3) 定义的 one-hot 向量，式 (2.1.4) 的乘法实际上是直接取 E 的第 k 行，计算复杂度为 $O(d)$ 而非 $O(Vd)$。

余弦相似度（cosine similarity）是衡量两嵌入向量语义接近程度的标准度量：

$$\cos(\mathbf{u},\mathbf{v}) = \frac{\mathbf{u}^T\mathbf{v}}{\|\mathbf{u}\|\,\|\mathbf{v}\|} = \frac{\sum_{i=1}^{d} u_i v_i}{\sqrt{\sum_{i=1}^{d} u_i^2}\cdot\sqrt{\sum_{i=1}^{d} v_i^2}} \tag{2.1.5}$$

选择余弦而非欧氏距离的动机在于：词频导致高频词的嵌入向量模长系统性偏大（§2.5 将详述），归一化后的角度更稳健地反映语义方向。

嵌入矩阵 E 的各分量在训练中通过反向传播学习，初始化通常为小的随机值（如 $\mathcal{N}(0, 1/d)$）。最终由式 (2.1.4) 给出的每一行对应一个词在语义几何空间中的坐标。

与原书呼应：原书 §2.1 指出 N-Gram 无法泛化的根本原因在于缺乏词间相似性信息，对应本节式 (2.1.3) 的 one-hot 编码正交性缺陷的分析。

---

§2.2 语义的坐标系：那个被讲烂了的公式

▶这一节在讲什么

为什么训练一个「预测上下文」的目标，会产出有意义的语义几何？表面上，Skip-gram 只是在做分类——给定中心词，猜哪些词会出现在旁边。但这个压力悄悄地把语义关系「写进」了向量方向。

第一步：理解训练目标的数学形式。全词表 softmax 计算代价为 $O(V)$，不可接受。负采样用局部二分类替代：每次只判断一对词是否真的共现，计算降至 $O(k)$，k 为负样本数。

第二步：理解类比几何的成立条件。$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$ 成立，是因为「性别」这一语义关系在空间中近似平行等长地分布——即嵌入空间对该关系具有平移等变性。这是充分条件，不总满足。

第三步：认识公式的边界。原始论文报告类比准确率约 60–70%；在上下文相关嵌入（BERT、LLaMA 隐藏状态）中，简单向量加减更频繁失败，线性表示假说给出了更精确的框架。

本节要拿走的一件事：那个被讲烂了的「国王 − 男人 + 女人 ≈ 女王」不是魔法，是训练目标在数据统计压力下产生的**近似**几何性质，有明确的成立条件和失效边界。

直觉

Word2Vec 的训练目标极为简洁：给定一个中心词，预测其窗口内会出现哪些上下文词（Skip-gram），或者反过来，给定上下文词预测中心词（CBOW）。这个极简的预测任务却产生了令人惊讶的副产品——语义类比算术。要理解为什么，必须把训练目标、计算瓶颈与几何涌现这三件事串成一条链来看。

本节出现五个概念：**Skip-gram 目标函数、softmax 瓶颈、负采样、类比向量算术、线性表示假说**。它们不是独立的知识点，而是一环扣一环：softmax 瓶颈迫使负采样诞生，负采样的噪声分布形状决定了嵌入的各向异性，而各向异性的背景下类比算术有

时成立有时失效，最终引出线性表示假说作为更严格的理论框架。

第一环：Skip-gram 目标函数——「预测邻居」的压力

Skip-gram 的训练目标是：给定中心词 w，最大化其窗口 $[-L, L]$ 内所有上下文词 c 的条件概率之和（对数似然形式）：

$$\mathcal{L}_{\text{SG}} = \frac{1}{T}\sum_{t=1}^{T}\sum_{\substack{-L\le j\le L\\ j\neq 0}} \log P(w_{t+j} \mid w_t) \tag{2.2.1}$$

条件概率用 softmax 定义，需要两套向量——中心词向量 $\mathbf{v}_w$（矩阵 W）和上下文词向量 $\mathbf{u}_c$（矩阵 W'）：

$$P(c \mid w) = \frac{\exp(\mathbf{u}_c^T \mathbf{v}_w)}{\sum_{c'=1}^{V} \exp(\mathbf{u}_{c'}^T \mathbf{v}_w)} \tag{2.2.2}$$

式 (2.2.1) 的「压力」是什么？每次更新，对于真实的上下文词对 (w, c)，梯度会把中心词向量和上下文向量「拉近」；同时，式 (2.2.2) 分母里所有其他词 c' 的向量会被轻微「推远」。经过数百万次这样的更新，统计上共同出现的词对，向量就会靠在一起；从不共现的词对，向量就会远离。这就是分布假设（distributional hypothesis）在几何上的体现。

注意 Skip-gram 使用两套矩阵而不是一套，这有深刻的对称性破缺原因：如果只用一套，一个词给自己预测的分数会被优化地推高，因为 $\mathbf{v}_w^T \mathbf{v}_w = \|\mathbf{v}_w\|^2$ 总是大于跨词内积。两套矩阵把「作为预测源」和「作为预测目标」这两个角色分开，避免了这种自我强化的退化。

第二环：softmax 瓶颈——全局归一化的计算诅咒

式 (2.2.2) 的 softmax 分母要对**整个词表** V 个词全部求和，每次梯度更新的计算复杂度是 $O(V)$。当 $V = 50000$，每个训练步要做 50000 次指数运算再求和，然后为每对词对计算一次；而典型的训练语料有数十亿个 token。这个代价根本无法承受。

但问题不只是「计算慢」。式 (2.2.2) 的全局归一化意味着：即使「猫」和「狗」的分数都该很高，它们也在互相竞争同一个概率质量——softmax 的归一化把所有词变成了零和竞争。这被称为 softmax 瓶颈（softmax bottleneck）：当词表中存在大量高质量候选词时，由式 (2.2.2) 定义的 softmax 分布会被迫「摊薄」，无法同时给多个候选词高概率。这一瓶颈在语言建模中反复出现，并推动了后来 mixture-of-softmaxes 等技术的发展。

第三环：负采样——把全局问题拆解为局部二分类

Mikolov 等人 2013 年提出的负采样（negative sampling, NEG）是回避 softmax 瓶颈的工程方案，同时也是 Word2Vec 真正得以大规模训练的关键。

核心思想：把「从 V 个候选中选最优」这个全局多分类问题，替换成一系列局部二分类——「这对词 (w, c) 是真实共现的？还是随机凑在一起的?」。对每个真实词对，从噪声分布 $P_n(c) \propto f(c)^{3/4}$（$f(c)$ 为词频）采 k 个「假」词对，目标变为：

$$\mathcal{L}_{\text{NEG}} = \log \sigma(\mathbf{u}_c^T \mathbf{v}_w) + \sum_{i=1}^{k} \mathbb{E}_{c_i' \sim P_n} \left[\log \sigma(-\mathbf{u}_{c_i'}^T \mathbf{v}_w) \right] \tag{2.2.3}$$

将式 (2.2.3) 与式 (2.2.1) 对比：式 (2.2.1) 需要对整个词表做 softmax（$O(V)$ 代价），而式 (2.2.3) 只涉及 $k+1$ 对向量（一个正样本 + k 个负样本），计算复杂度从 $O(V)$ 降至 $O(k)$，典型地 $k \in [5, 20]$，加速了数千倍。

噪声分布的 $3/4$ 次幂是一个经验调参：纯粹按词频 $f(c)$ 采样，高频词（「the」「of」）会以极高概率成为负样本，训练信号被高频词主导；均匀采样又会让低频词（专有名词、罕见词）过度出现。$3/4$ 次幂是一个折中，对高频词降权、对低频词升权，没有严格的理论推导，但在实践中超越了其他选择。

这个选择有一个隐藏的后果：高频词作为负样本的频率，仍然高于低频词——只是不那么极端了。这意味着高频词的嵌入在训练中受到更多的「排斥」梯度，向量被系统性地推向空间的某个角落。这就是 §2.5 各向异性问题的根源，这里先埋下伏笔。

第四环：类比向量算术——平行四边形的几何奇迹

当 Word2Vec 训练好后，Mikolov 等人发现了一个令人吃惊的性质：

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}} \tag{2.2.4}$$

式 (2.2.4) 这个「国王 - 男人 + 女人 ≈ 女王」的公式被广泛引用，但它为什么成立？

数学机制是这样的：如果「king」和「queen」的上下文分布几乎完全一样，唯一的差异是「king」更多地与「his」「he」「man」共现，而「queen」更多地与「her」「she」「woman」共现——那么训练压力会把这两对词在向量空间中形成**平行的位移**：「king」到「queen」的向量差，与「man」到「woman」的向量差，在方向和长度上都近似相等。这种性质叫做**平移等变性（translational equivariance）**——某种语义关系（性别）被编码为空间中固定的位移向量。

式 (2.2.4) 成立的充分条件：1. 语义关系在训练语料里高度规律且一致；2. 涉及的词在相同上下文中出现频率都足够高，统计估计可靠；3. 词向量空间的「有效维度」足够大，能够同时编码多种关系的方向。

「性别」关系在英语语料里满足这三点，所以式 (2.2.4) 工作得好。「首都-国家」关系（Paris:France, Tokyo:Japan）也大体满足，但准确率参差不齐。而抽象的道德、情感关系，在语料中没有一致规律，式 (2.2.4) 就几乎失效。

原始 Word2Vec 论文报告的类比准确率约 60-70%，已经是精心挑选的「展示最好」的词对上的成绩。在实际应用中，词对越抽象、词频越低、语义关系越复杂，准确率越低。

第五环：线性表示假说——类比算术的理论升级

「向量加减 ≈ 语义类比」这一现象在 BERT、GPT 等上下文相关嵌入中更频繁失效，因为同一个词在不同句子中有不同的激活向量，「静态词向量算术」的前提不再成立。但语义特征是否仍然以某种线性方式存在于表示空间中？

线性表示假说（Linear Representation Hypothesis）把这个问题形式化：语义概念对应残差流（residual stream）中的线性方向，沿该方向进行激活干预可以控制模型对相

应属性的输出行为（Park, Choe & Veitch, ICML 2024）。这不只是式 (2.2.4) 的推广，而是一个关于大模型内部表示结构的深层假设，目前是活跃的研究前沿。

线性表示假说的意义在于：如果成立，它说明大模型的「理解」是线性可解释的——找到方向向量，就能理解并干预模型的推理过程。这是可解释性研究（interpretability）的核心工具假设，也是稀疏自编码器（SAE）这类技术的理论基础。

整条链的回头看

```
Skip-gram 目标函数              ← 用「预测邻居」把共现压力写进向量
  ↓ 全局 softmax 计算代价 O(V)
softmax 瓶颈                    ← 全局归一化→零和竞争→无法同时给多个词高概率
  ↓ 用局部二分类代替全局分类
负采样 (NEG)                    ← O(k) 的高效替代，噪声分布 f^{3/4} 折中
  ↓ 共现压力 → 平行位移结构
类比向量算术                    ← 平移等变性的涌现，有明确成立/失效条件
  ↓ 静态向量算术在上下文相关模型里失效
线性表示假说                    ← 语义概念 = 残差流中的线性方向（更严格的框架）
```

这条链揭示了一个深层事实：**Word2Vec 的几何奇迹不是设计出来的，而是一个简单预测目标在大规模统计压力下的涌现产物**。它的成立有边界，它的失效也有原因，而不是随机的。从负采样到线性表示假说，这条链也预示了整个可解释性研究的走向：如果大模型的表示是线性的，那么理解它、干预它、对齐它，都将有据可依。

严格

Skip-gram 目标函数给定词 w，最大化其上下文词 c（在窗口 $[-L, L]$ 内，排除自身）的条件概率：

$$\mathcal{L}_{\text{SG}} = \frac{1}{T}\sum_{t=1}^{T}\sum_{\substack{-L\le j\le L\\ j\neq 0}} \log P(w_{t+j} \mid w_t) \tag{2.2.5}$$

式 (2.2.5) 与直觉部分的式 (2.2.1) 完全对应。条件概率用 softmax 定义，引入两套向量：中心词向量 $\mathbf{v}_w \in \mathbb{R}^d$（来自矩阵 $W \in \mathbb{R}^{V\times d}$）和上下文词向量 $\mathbf{u}_c \in \mathbb{R}^d$（来自矩阵 $W' \in \mathbb{R}^{V\times d}$）：

$$P(c \mid w) = \frac{\exp(\mathbf{u}_c^T \mathbf{v}_w)}{\sum_{c'=1}^{V} \exp(\mathbf{u}_{c'}^T \mathbf{v}_w)} \tag{2.2.6}$$

式 (2.2.6) 与直觉部分的式 (2.2.2) 完全对应。分母对全词表求和，计算开销为 $O(V)$，在大词表下不可接受。**负采样（negative sampling, NEG）**以局部二分类替代全局 softmax：对每个正样本对 (w, c)，从噪声分布 $P_n(c) \propto f(c)^{3/4}$（$f(c)$ 为词频）采 k 个负样本 $c'_1, \dots, c'_k$，目标函数变为：

$$\mathcal{L}_{\text{NEG}} = \log \sigma(\mathbf{u}_c^T \mathbf{v}_w) + \sum_{i=1}^{k} \mathbb{E}_{c'_i \sim P_n}\left[\log \sigma(-\mathbf{u}_{c'_i}^T \mathbf{v}_w)\right] \tag{2.2.7}$$

将式 (2.2.7) 与式 (2.2.3) 对比可知，两者形式完全一致；同时，将式 (2.2.7) 与式 (2.2.6) 的

softmax 形式对比，可以直观理解负采样把 $O(V)$ 的全局问题降至 $O(k)$。其中 $\sigma(x) = 1/(1+e^{-x})$ 为 sigmoid 函数。典型地取 $k \in [5, 20]$，计算复杂度降至 $O(k)$。

类比关系的几何解释。设 $\delta_{\text{gender}} = \mathbf{v}_{\text{man}} - \mathbf{v}_{\text{woman}}$。若训练数据中「king」和「queen」出现在几乎相同的上下文中，除性别相关词汇外，则梯度更新会将两者沿性别方向对齐，形成：

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{queen}} \approx \mathbf{v}_{\text{man}} - \mathbf{v}_{\text{woman}} = \delta_{\text{gender}} \tag{2.2.8}$$

由式 (2.2.8) 变形即得：

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}} \tag{2.2.9}$$

式 (2.2.9) 与直觉部分的式 (2.2.4) 完全对应，并揭示了其成立的机制——由式 (2.2.8) 刻画的性别方向 δ_{gender} 的平行等长性。这一近似成立的充分条件是：语义关系（如性别）在空间中近似平行且等长地分布，即嵌入空间对该关系具有**平移等变性（translational equivariance）**。然而，此条件并不总是满足：原始 Word2Vec 论文报告的类比准确率约为 60–70%，且依赖具体词对；在上下文相关嵌入（BERT、LLaMA 隐藏状态）中，简单向量加减更频繁失败。

线性表示假说（Linear Representation Hypothesis）将式 (2.2.9) 的类比向量算术观察形式化：语义概念对应残差流（residual stream）中的线性方向，沿该方向干预可控制模型输出对应属性（Park, Choe & Veitch, ICML 2024）。

与原书呼应：原书 §2.2 给出 king−man+woman≈queen 等若干类比示例，并引入线性表示假说，对应本节式 (2.2.1) 的 Skip-gram 目标函数及式 (2.2.9) 类比几何成立条件的分析。

---

§2.3 高维几何的诡异世界

▶这一节在讲什么

为什么三维直觉在数百维的嵌入空间里几乎全部失效？当维度从 3 增长到 512 或 4096 时，「近」「远」「随机」这些概念的含义发生了根本性的变化，忽视这一点会导致对嵌入空间的严重误判。

第一步：球壳质量集中。高维高斯向量的模长以极高概率集中在 $\sqrt{d}$ 附近的薄球壳上——球的「内部」几乎是空的。这意味着嵌入向量之间的欧氏距离差异会被压缩，模长不再是有意义的区分量。

第二步：随机向量近乎正交。两个独立高斯向量的余弦值以 $1/\sqrt{d}$ 的速度趋于零。高维空间能容纳指数级多的「几乎正交」方向，这正是超位置（superposition）现象的数学基础。

第三步：距离保持降维是可行的。Johnson-Lindenstrauss 引理保证：把 n 个点投影到 $O(\log n/\varepsilon^2)$ 维，点对距离的失真不超过 ε。目标维度与原始维度无关，只与点的数量的对数相关。

本节要拿走的一件事：高维空间「几乎所有方向都互相正交」——这既是词嵌入能承载海量语义方向的几何原因，也是简单向量运算在高维下需要格外谨慎的原因。

直觉

词嵌入的维度通常在数百到万量级。在这样的空间里，我们关于「近」「远」「随机点分布」「两点之间」的三维直觉，几乎无一幸免地失效。高维几何有它自己的规律，这些规律不是「三维直觉的延伸」，而是**质变**——在某些方面与三维世界截然相反。理解这些反直觉性质，是理解词嵌入为何需要余弦相似度、为何存在超位置现象、为何近似最近邻搜索可行的数学基础。

本节出现四个概念：**球壳质量集中**、**随机向量近正交**、**Johnson-Lindenstrauss（JL）引理**、**超位置（superposition）**。它们沿着一条逻辑链：高维的「空间结构」决定了随机向量的统计性质，随机向量的统计性质决定了哪些几何工具可用，JL 引理则给出了高维与低维之间可控转化的保证，而超位置则是这一切在神经网络中的应用体现。

第一环：球壳质量集中——高维球几乎是空心的

在三维空间里，球的体积中有相当大的比例在「内部」：半径从 0 到 0.9 的球包含了约 73% 的体积。现在把这个球换到 1000 维：半径从 0 到 0.9（单位球的 90%）的内部球，其体积占总体积的 $0.9^{1000} \approx 10^{-46}$——几乎是零。几乎所有的「体积」都挤在半径接近 1 的极薄球壳上。

这个现象对高斯向量同样成立。设 $\mathbf{x} = (x_1, \ldots, x_d)^T$，各分量独立同分布 $x_i \sim \mathcal{N}(0,1)$，则 $\|\mathbf{x}\|^2 = \sum_i x_i^2 \sim \chi^2(d)$，期望为 d，方差为 $2d$。标准差与均值之比是 $\sqrt{2d}/d = \sqrt{2/d}$，当 $d = 512$ 时这个比值约为 0.063——也就是说，向量的模长以极高概率集中在 $\sqrt{d}$ 的 $\pm 6\%$ 范围内。

这对嵌入空间意味着什么？**随机初始化的嵌入向量几乎全部具有相似的模长（约 $\sqrt{d}$）**。经过训练后，如果没有显式的正则化约束，嵌入向量的模长也会保持在某个集中的范

围内（受词频影响而略有差异，见 §2.5）。这导致欧氏距离对「近」和「远」的区分能力大幅下降：两个模长都约为 $\sqrt{d}$ 的向量，其欧氏距离的期望约为 $\sqrt{2d}$，方差相对较小，难以区分「很近」和「稍远」——这正是 §2.1 中式 (2.1.5) 选用余弦相似度而非欧氏距离的数学理由。

另一个推论：**高维球的「内部」几乎是空的，采样得到一个真正意义上「在球内部」的点的概率极低**。这对 GAN 训练、VAE 潜空间、流模型等依赖在高维球内均匀采样的算法，都有深刻的影响——它们采出来的点几乎都在球壳附近，而非真正均匀分布在球体内。

第二环：随机向量近正交——高维空间的「方向容量」

两个独立的高维高斯向量 $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$，它们的内积 $\mathbf{x}^T\mathbf{y} = \sum_i x_i y_i$ 是 d 个均值为 0、方差为 1 的独立随机变量的和，由中心极限定理，$\mathbf{x}^T\mathbf{y} \sim \mathcal{N}(0, d)$，标准差为 $\sqrt{d}$。两向量的模长均约为 $\sqrt{d}$，所以余弦值的标准差：

$$\operatorname{Std}\left[\cos(\mathbf{x}, \mathbf{y})\right] \approx \frac{\sqrt{d}}{d} = \frac{1}{\sqrt{d}} \tag{2.3.1}$$

当 $d = 512$，式 (2.3.1) 给出的标准差约为 0.044；当 $d = 4096$，约为 0.016。**高维随机向量以极高概率近乎正交**，余弦值的波动幅度随维度增大而收窄，趋向于零。

式 (2.3.1) 有两面性，但正面影响更深远：**高维空间能容纳指数级多的「几乎正交」方向**。具体地，在 $\mathbb{R}^d$ 中，可以找到指数级（约 $e^{d/8}$ 个，见 JL 引理）个两两余弦值都不超过 ε 的单位向量。这意味着：一个 $d = 512$ 的嵌入空间，不只能存 512 个「完全正交」的特征方向，而是能存放指数级多个「几乎正交」的方向——每个方向都对应一种语义特征，彼此干扰极小。

这正是**超位置（superposition）现象**的数学根基。一个神经网络层的 d 维激活向量，可以同时「叠加」存储远超 d 个稀疏特征，只要这些特征在真实数据中几乎不同时激活（稀疏性条件）。当两个不同特征的向量方向夹角足够大（内积足够小，由式 (2.3.1) 保证这在高维下以高概率成立），它们的干扰可以控制在可接受范围内，网络能通过非线性激活函数把它们分离出来。

超位置现象说明「d 维嵌入只能编码 d 个独立语义特征」这个朴素想法是错的——真实的特征容量可以远超 d，前提是特征稀疏且训练目标允许。

第三环：JL 引理——高维到低维的可控降维

球壳集中和式 (2.3.1) 的随机向量近正交这两个性质，都说明高维空间的「几何」与低维有质的不同。一个自然的工程问题是：能不能把高维向量降到低维，同时保留距离关系？

Johnson-Lindenstrauss 引理（1984）给出了令人吃惊的肯定答案：把 n 个点从任意高维投影到 $O(\log n/\varepsilon^2)$ 维，可以以高概率保证所有点对的欧氏距离失真不超过因子 $(1 + \varepsilon)$。而且**目标维度与原始维度完全无关，只与点的数量有对数关系**。

具体地，对应的投影矩阵 $\Phi \in \mathbb{R}^{k \times D}$ 可以随机生成，各元素独立地从 $\mathcal{N}(0, 1/k)$ 采样，以高概率满足：

$$(1-\varepsilon)\|\mathbf{x}_i - \mathbf{x}_j\|^2 \le \|f(\mathbf{x}_i) - f(\mathbf{x}_j)\|^2 \le (1+\varepsilon)\|\mathbf{x}_i - \mathbf{x}_j\|^2 \tag{2.3.2}$$

式 (2.3.2) 的工程含义极为重要：**向量数据库中进行近似最近邻（ANN）搜索时，可以把高维嵌入向量（如 4096 维）先随机投影到较低维度（如 256 维），只损失可控的距离精度**。这是 FAISS、HNSW 等向量搜索引擎在底层进行降维加速的理论保障。

JL 引理的证明核心是高斯投影的集中不等式——随机高斯矩阵在任何方向上的投影都高度集中，因此它对「所有点对同时」保持距离的概率，可以用 union bound 控制在可接受范围内。这个证明本身就体现了高维集中现象的威力。

第四环：超位置——高维几何在神经网络中的实际应用

把前三环串起来，可以给出超位置现象的完整图像。神经网络的嵌入空间是 d 维的，但需要表示的语义特征数量（如「有无生命」「抽象程度」「情感极性」「句法角色」……）可能是数千到数万个。如果每个特征只能占一个维度，$d = 512$ 的嵌入只能表示 512 个特征，远远不够。

超位置给出了一条出路：利用式 (2.3.1) 所揭示的高维空间能容纳指数级多「几乎正交」方向的性质，把每个语义特征编码为空间中的一个方向向量（不要求严格正交），只要在真实数据中这些特征不同时大量激活（稀疏性），网络就能通过非线性激活函数（如 ReLU）把叠加在一起的多个信号分离出来。

代价是：当特征同时激活时，会产生「干扰项」(cross-talk)，使网络的输出出错。这种干扰项被称为超位置噪声。稀疏自编码器 (Sparse Autoencoder, SAE) 正是为了解码这种叠加表示而设计的——它试图找到嵌入空间中隐藏的稀疏基底，把叠加存储的特征「分离」出来，用于可解释性研究。

整条链的回头看

```
球壳质量集中                          ← 高维球几乎是空心的；模长集中在 √d 附近
  ↓ 模长无区分力，方向才有意义
余弦相似度优于欧氏距离                ← 在模长趋同的球壳上，角度差异更有信息量
  ↓ 随机向量的内积分布
随机向量近正交（std ≈ 1/√d）          ← 余弦值趋于零，但方向「容量」指数级增大
  ↓ 高维空间能塞下指数级多的「几乎正交」方向
超位置（superposition）               ← 一个 d 维空间存储远超 d 个稀疏特征的数学机制
  ↓ 高维与低维之间的距离保持降维
JL 引理                               ← O(log n / ε²) 维足以保留 n 点的欧氏距离结构
```

这条链揭示了一个深层事实：**高维几何的反直觉性质，既是词嵌入能承载海量语义信息的原因，也是工程上必须格外小心的原因**。球壳集中让余弦相似度（式 (2.1.5)）比欧氏距离更可靠；由式 (2.3.1) 揭示的近正交性让超位置成为可能；式 (2.3.2) 的 JL 引理让向量搜索得以大规模实用化——这三件事都根植于同一个数学事实：高维空间的「容量」远超低维直觉。

严格

球壳质量集中（measure concentration）。设 $\mathbf{x} = (x_1, \ldots, x_d)^T$，各分量独立同分布 $x_i \sim \mathcal{N}(0,1)$。则 $\|\mathbf{x}\|^2 = \sum_{i=1}^{d} x_i^2 \sim \chi^2(d)$，故：

$$\mathbb{E}[\|\mathbf{x}\|^2] = d, \quad \mathrm{Var}[\|\mathbf{x}\|^2] = 2d \tag{2.3.3}$$

由集中不等式，$\|\mathbf{x}\|$ 以高概率集中在 $\sqrt{d}$ 附近，偏离幅度为 $O(d^{1/4})$。更精确地，对任

意 $\varepsilon > 0$：

$$P\Big(\big|\|\mathbf{x}\| - \sqrt{d}\big| > \varepsilon\Big) \le 2\exp\left(-\frac{\varepsilon^2}{4}\right) \tag{2.3.4}$$

式 (2.3.4) 给出了球壳集中的定量版本：偏离 $\sqrt{d}$ 超过 ε 的概率以指数速度衰减。直观含义：在高维下，高斯向量的模长以极高概率位于半径为 $\sqrt{d}$ 的球壳薄层内，球的「内部」几乎是空的。

高维内积分布。设 $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ 独立同分布 $\mathcal{N}(\mathbf{0}, I_d)$。则内积 $\mathbf{x}^T\mathbf{y} = \sum_i x_i y_i$，各项独立，方差为 1，由中心极限定理：

$$\frac{\mathbf{x}^T\mathbf{y}}{\sqrt{d}} \xrightarrow{d\to\infty} \mathcal{N}(0,1) \tag{2.3.5}$$

即 $\mathbf{x}^T\mathbf{y} \sim \mathcal{N}(0, d)$，标准差为 $\sqrt{d}$，而两向量的模长均约为 $\sqrt{d}$，故余弦值：

$$\cos(\mathbf{x},\mathbf{y}) = \frac{\mathbf{x}^T\mathbf{y}}{\|\mathbf{x}\|\|\mathbf{y}\|} \approx \frac{\mathcal{N}(0,d)}{d} = \mathcal{N}(0, 1/d) \xrightarrow{d\to\infty} 0 \tag{2.3.6}$$

将式 (2.3.6) 与直觉部分的式 (2.3.1) 对比，可以看出式 (2.3.1) 给出的标准差 $1/\sqrt{d}$ 正是从式 (2.3.6) 中读出的。随机高维向量几乎正交。这意味着：高维空间能容纳指数级多的「几乎正交」向量，这正是超位置（superposition）现象的数学基础。

Johnson-Lindenstrauss 引理（Johnson-Lindenstrauss lemma）。设 n 个点 $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^D$，$\varepsilon \in (0, 1/2)$。存在线性映射 $f : \mathbb{R}^D \to \mathbb{R}^k$，其中：

$$k = O\left(\frac{\log n}{\varepsilon^2}\right) \tag{2.3.7}$$

使得对所有点对 (i, j)：

$$(1-\varepsilon)\|\mathbf{x}_i - \mathbf{x}_j\|^2 \le \|f(\mathbf{x}_i) - f(\mathbf{x}_j)\|^2 \le (1+\varepsilon)\|\mathbf{x}_i - \mathbf{x}_j\|^2 \tag{2.3.8}$$

式 (2.3.8) 与直觉部分的式 (2.3.2) 完全对应，而式 (2.3.7) 则给出了满足式 (2.3.8) 所需的目标维度 k。该映射可由随机高斯矩阵 $\Phi \in \mathbb{R}^{k\times D}$（各元素独立 $\sim \mathcal{N}(0, 1/k)$）构造，以高概率满足上述距离保持性质。引理的关键推论：**降维所需的目标维度仅与点的数量对数相关，与原始维度 D 无关**。这解释了为何在高维嵌入中进行近似最近邻搜索是可行的。

余弦相似度在高维的稳定性。虽然由式 (2.3.6) 可知随机向量趋于正交，但经过训练的嵌入向量并非随机——其方向被语义信息约束。余弦相似度（式 (2.1.5)）的优势在于：它对向量模长不敏感，只关注方向差异。在式 (2.3.4) 描述的球壳质量集中的高维空间中，模长携带的信息主要是词频偏置（高频词嵌入模长系统偏大），归一化后恰好消除这一噪声，保留语义方向信号。

与原书呼应：原书 §2.3 列举了高维诡异性的三条现象，并引出超位置与稀疏自编码器（SAE），对应本节球壳集中（式 (2.3.3)、式 (2.3.4)）、内积分布（式 (2.3.5)、式 (2.3.6)）与 JL 引理（式 (2.3.7)、式 (2.3.8)）的数学推导。

§2.4 跨界映射：从罗马路网到曼哈顿距离

▶这一节在讲什么

为什么同一个「距离」概念，换个度量方式就面目全非？欧氏距离只是 L^p 范数家族中的一员。在语义空间中，不同维度的「一个单位」代表的语义差异强度各不相同，一把尺子量不了所有的方向。

第一步：认识 L^p 范数家族。$p=1$ 是曼哈顿距离（沿坐标轴走），$p=2$ 是欧氏距离（直线距离），$p\to\infty$ 是 Chebyshev 距离（取各维最大绝对值）。三者的单位球分别是菱形、圆球、正方形——几何形状截然不同。

第二步：引入加权度量矩阵。若嵌入空间各维度的「语义单位」不等长，可用正定矩阵 G 定义加权内积 $\mathbf{u}^TG\mathbf{v}$，对应的马氏距离消除各向异性，等价于在白化后的空间中计算欧氏距离。

第三步：区分内积空间与度量空间。内积空间自然诱导范数和度量；但反过来，满足平行四边形恒等式才能保证一个范数来自内积。词嵌入的「自然度量」实为加权内积，而非普通欧氏内积。

本节要拿走的一件事：选择哪种距离度量，本质上是对「哪些方向上的差异更重要」的假设——不同的 G 对应不同的语义几何观。

直觉

原书用「罗马路网」类比语义空间中的推理路径：所有道路通向罗马，暗示空间是均匀的、各方向可互换的欧氏几何。但词嵌入空间的真实几何远比这更复杂：不同维度上的一个单位代表的语义差异强度各不相同，某些方向被高频词统计污染（§2.5），某些方向承载着强烈的句法信号，某些方向携带微弱的情感偏置。更精确的类比应该是**曼哈顿网格**——不同方向有不同的「代价」，距离的计算需要一个加权的度量。

本节出现四个概念：**L^p 范数族**、**单位球的几何形状**、**马氏距离（Mahalanobis distance）**、**内积空间与度量空间的关系**。它们沿着一条逻辑链：L^p 范数展示了「距离」可以有多种形式，单位球的形状可视化了不同范数对「偏差惩罚」方式的区别，马氏距离则把「选择度量」的自由度形式化为一个正定矩阵 G，最后内积空间与度量空间的关系澄清了哪些度量「天然」来自内积结构。

第一环：L^p 范数族——「距离」可以有很多种脸

所有人从小学就熟悉「两点之间，直线最短」——这是欧氏距离，L^2 范数。但「距离」在数学上只需要满足三条公理（非负性、对称性、三角不等式），可以有无穷多种合法定义。L^p 范数族是这些定义中最系统的一族：

$$\|\mathbf{x}\|_p=\left(\sum_{i=1}^{d}|x_i|^p\right)^{1/p},\quad p\geq 1 \tag{2.4.1}$$

$p=1$ 是**曼哈顿距离**：沿坐标轴走，绝对值之和，不管方向如何一律「按格走」。$p=2$

是**欧氏距离**：直线距离，各分量的平方和再开根号。$p \to \infty$ 是 **Chebyshev 距离**：只关心偏差最大的那个分量。

式 (2.4.1) 给出的三种范数对「偏差」的惩罚哲学截然不同。直觉上：L^1 把所有维度的偏差以相同的「线性代价」累加，对某一个维度上的极端值不特别敏感；L^2 通过平方「放大」大偏差，对极端值更敏感；L^∞ 则完全忽略所有维度的偏差，只盯住最大的那个。这种差异在机器学习里的体现：L^1 正则化（LASSO）天然导致稀疏解，因为 L^1 的单位球（菱形）在坐标轴方向有角点，梯度下降容易「卡」在角点上（即某些维度精确为零）；L^2 正则化（Ridge）则倾向于让所有参数均匀缩小，不产生稀疏性。

可视化单位球是理解式 (2.4.1) 中各 p 值含义最直观的方式。在二维空间里：L^1 单位球是菱形（旋转 45° 的正方形），L^2 单位球是圆，L^∞ 单位球是正方形。这三种形状的直觉，可以直接推广到高维：「什么样的点到原点的距离恰好是 1」——这个集合的几何形状就是该范数的单位球，它完全决定了该范数对所有方向的「惩罚各向同性」。

第二环：距离的各向异性——「一公里」在不同方向上不等价

欧氏距离（式 (2.4.1) 取 $p = 2$）假设所有维度都是等权重的——沿第 1 维移动 1 单位，和沿第 500 维移动 1 单位，代价完全相同。但在词嵌入空间里，这个假设几乎从不成立：- 不同维度的方差（训练后特征值的量级）差距可能高达数百倍；- 某些维度承载着强烈的统计信号（如句法类别），另一些维度几乎全是噪声；- 高频词在某些方向上被系统性地推挤（§2.5 各向异性）。

在这种情况下，直接用 L^2 距离，等于用不同单位（公里和毫米混用）来计算物理位置的差异。最近邻搜索会被那些高方差的维度主导，忽略低方差但信息丰富的维度。

解决方案是**引入一个加权度量**。设 $G \in \mathbb{R}^{d \times d}$ 是一个正定矩阵，定义加权内积：

$$\langle \mathbf{u}, \mathbf{v} \rangle_G = \mathbf{u}^T G \mathbf{v} \tag{2.4.2}$$

$G = I$ 时式 (2.4.2) 就退回到普通欧氏内积。G 的作用是对不同方向施加不同的权重——某个方向「一单位」的实际语义距离，由 G 在那个方向上的特征值决定。由式 (2.4.2) 定义的加权内积诱导出**马氏距离**（Mahalanobis distance）：

$$d_G(\mathbf{u}, \mathbf{v}) = \sqrt{(\mathbf{u} - \mathbf{v})^T G (\mathbf{u} - \mathbf{v})} \tag{2.4.3}$$

第三环：马氏距离——白化的几何解释

式 (2.4.3) 中最常见的选择是 $G = \Sigma^{-1}$，其中 Σ 是嵌入向量的协方差矩阵。此时式 (2.4.3) 的几何意义非常清晰：**它等价于先对嵌入空间做白化变换（PCA + 按特征值归一化），再在白化后的空间里计算欧氏距离**。

白化把协方差矩阵变成单位矩阵，使得每个方向的方差都相等——这正是「消除各向异性」的操作。白化后的空间里，欧氏距离不再被高方差维度主导，每个维度「一单位」代表相同的统计意义。

但式 (2.4.3) 的马氏距离有其局限：它需要知道整个嵌入集合的协方差矩阵，计算代价是 $O(V \cdot d^2)$（V 个词，d 维），对动态变化的上下文相关嵌入（BERT 每个句子每个词的向量都不同）几乎不可行。此外，白化变换会破坏原始嵌入中的线性语义方向（类比算术，即式 (2.2.4)，在白化后反而更难奏效，见 §2.5 的讨论）。因此，马氏距离在

词嵌入的实践中主要出现在**评估语义相似度任务**（如 STS-B 基准），而不是实时的检索系统。

第四环：内积空间与度量空间——哪些距离「天然」来自内积

L^2 范数来自内积（欧氏内积），L^1 范数不来自内积（无法定义一个内积使得 $\|\mathbf{x}\|_1 = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$）。区分这两类的充要条件是**平行四边形恒等式**：

$$\|\mathbf{u}+\mathbf{v}\|^2 + \|\mathbf{u}-\mathbf{v}\|^2 = 2\|\mathbf{u}\|^2 + 2\|\mathbf{v}\|^2 \tag{2.4.4}$$

式 (2.4.4) 对 L^2 成立，对 L^1 和 L^∞ 不成立。内积空间（满足式 (2.4.4) 的范数空间）有额外的代数结构——可以定义夹角、投影、正交分解——这些工具在词嵌入里被大量使用（余弦相似度（式 (2.1.5)）、注意力机制的点积、PCA 降维）。

这说明：**词嵌入隐含地依赖内积空间结构**。当我们用余弦相似度（式 (2.1.5)）、用式 (2.4.2) 定义的加权内积 $\mathbf{u}^T G \mathbf{v}$ 时，实际上是在假设嵌入空间是（加权）内积空间，而不只是一般的度量空间。这个假设在欧氏和马氏距离（式 (2.4.3)）下成立，在 L^1 下不成立。

整条链的回头看

```
L^p 范数族（p=1,2,∞）          ← 「距离」可以有多种惩罚哲学；单位球形状可视化差异
  ↓ 不同维度语义单位不等长
各向异性的度量需求              ← 直接用 L^2 被高方差维度主导，需要加权
  ↓ 用正定矩阵 G 引入方向权重
加权内积 u^T G v               ← G = Σ^{-1} 时等价于白化后的欧氏距离
  ↓ 马氏距离消除各向异性偏置
内积空间 vs 度量空间            ← 平行四边形恒等式区分「有内积结构」与「只有距离」
  ↓ 词嵌入工具链（余弦、注意力）隐含依赖内积结构
余弦相似度作为归一化内积        ← G = I，但用模长归一化消除词频偏置
```

这条链揭示了一个深层事实：**选择度量不是工程上的随意决定，而是对「什么方向上的差异更重要」的显式假设**。欧氏距离假设各向同性，马氏距离（式 (2.4.3)）假设协方差矩阵已知，余弦相似度（式 (2.1.5)）假设只有方向重要而模长无意义——不同的假设对应不同的语义几何观。没有一种度量是绝对正确的，正确的度量是与数据的内在结构最匹配的那个。

严格

L^p 范数（L^p norm）定义为：

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^{d} |x_i|^p \right)^{1/p}, \quad p \geq 1 \tag{2.4.5}$$

式 (2.4.5) 与直觉部分的式 (2.4.1) 完全对应。几个特殊情形：

- $p = 1$（**曼哈顿距离**，Manhattan distance / taxicab norm）：$\|\mathbf{x}\|_1 = \sum_i |x_i|$。单位球为超立方体的对角截面（$d = 2$ 时为菱形）。
- $p = 2$（**欧氏范数**，Euclidean norm）：$\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$。单位球为超球面。

- $p \to \infty$（**Chebyshev 距离**，Chebyshev distance）：$\|\mathbf{x}\|_\infty = \max_i |x_i|$。单位球为超立方体。证明：$\|\mathbf{x}\|_p = \left(\sum_i |x_i|^p\right)^{1/p} \le \left(d \cdot \max_i |x_i|^p\right)^{1/p} = d^{1/p} \max_i |x_i| \to \max_i |x_i|$（$p \to \infty$）；下界同理可得。

加权内积与度量矩阵。若嵌入空间的不同维度具有不同的「语义单位长度」，可引入正定矩阵 $G \in \mathbb{R}^{d\times d}$（$G \succ 0$），定义加权内积：

$$\langle \mathbf{u}, \mathbf{v} \rangle_G = \mathbf{u}^T G \mathbf{v} \tag{2.4.6}$$

式 (2.4.6) 与直觉部分的式 (2.4.2) 完全对应。对应的马氏（Mahalanobis）距离（Mahalanobis distance）为：

$$d_G(\mathbf{u}, \mathbf{v}) = \sqrt{(\mathbf{u}-\mathbf{v})^T G (\mathbf{u}-\mathbf{v})} \tag{2.4.7}$$

式 (2.4.7) 与直觉部分的式 (2.4.3) 完全对应。当 $G = I$ 时退化为欧氏距离；当 $G = \Sigma^{-1}$（数据协方差矩阵的逆）时，由式 (2.4.7) 给出的 Mahalanobis 距离消除各向异性，等价于在白化（whitening）后的空间中计算欧氏距离。

内积空间（inner product space）vs 度量空间（metric space）。内积空间配备双线性、对称、正定的内积，自然诱导范数 $\|\mathbf{x}\| = \sqrt{\langle \mathbf{x}, \mathbf{x}\rangle}$ 及度量 $d(\mathbf{x},\mathbf{y}) = \|\mathbf{x}-\mathbf{y}\|$；反过来，并非所有度量都来自内积（满足式 (2.4.4) 给出的平行四边形恒等式 $\|\mathbf{u}+\mathbf{v}\|^2 + \|\mathbf{u}-\mathbf{v}\|^2 = 2\|\mathbf{u}\|^2 + 2\|\mathbf{v}\|^2$ 是内积范数的充要条件）。词嵌入通常假设在内积空间中工作，但其「自然度量」实为由式 (2.4.6) 定义的加权内积，而非普通欧氏内积。

与原书呼应：原书 §2.4 以罗马路网 vs 曼哈顿网格为比喻，指出词向量空间的「自然内积」为加权内积 $\mathbf{u}^T G \mathbf{v}$，对应本节式 (2.4.1) 的 L^p 范数几何含义与式 (2.4.6)、式 (2.4.7) 定义的度量矩阵的数学形式化。

---

§2.5 反类比：当几何隐喻失效

▶这一节在讲什么

为什么嵌入空间的几何比我们想象的更「扭曲」？词频统计在训练过程中悄悄地扭曲了向量空间的几何结构：高频词聚集在一个狭窄的锥形区域，嵌入空间严重各向异性，余弦相似度和向量加减在这种条件下都面临系统性偏差。

第一步：认识各向异性。理想的嵌入空间各向同性——向量均匀覆盖所有方向。实际中，GPT-2 各层嵌入的平均余弦相似度可达 0.99，几乎所有词向量都指向同一方向，大量语义方向被浪费。

第二步：理解椎体效应的成因。负采样中高频词被频繁选为负样本，受到更多「排斥」梯度，最终被推向空间边缘，形成低维锥。嵌入空间的有效维度远低于名义维度 d。

第三步：白化是可行的修正方案。对嵌入矩阵做 PCA 并按特征值归一化（白化变换），能恢复各向同性，使余弦相似度在语义相似度任务上的表现显著提升。

本节要拿走的一件事：嵌入空间不是干净的语义坐标系，而是被词频统计「污染」过的各向异性空间——白化是让几何隐喻重新成立的必要前处理。

直觉

前几节建立了词嵌入的理想图景：一个干净的语义空间，词向量均匀分布在各方向，余弦相似度（式 (2.1.5)）精确测量语义距离，向量加减（如式 (2.2.4)）能实现类比推理。这一节要打破这个图景。真实的嵌入空间被词频统计深深污染，几何结构远比想象中扭曲——而且这种扭曲是有明确数学根源的，可以被追溯到训练机制的每一个细节。

本节出现三个概念：**各向异性（anisotropy）**、**椎体效应（cone effect）**、**白化及其代价**。这三个概念构成一条因果链：训练机制（式 (2.2.3) 的负采样噪声分布）直接导致词频偏置，词频偏置积累出各向异性，各向异性让一系列几何隐喻（余弦相似度的可靠性、向量算术的有效性）失效，而白化是对这种失效的部分修复，但修复本身也有代价。理解这条链，才能知道什么时候该信任嵌入空间的几何，什么时候该怀疑它。

第一环：各向异性——嵌入空间的统计扭曲

各向同性（isotropy）意味着嵌入向量均匀地指向所有方向，没有哪个方向被特别「偏爱」。在各向同性的空间里，任意两个随机词的余弦相似度（式 (2.1.5)）的期望为零，标准差为 $1/\sqrt{d}$（见 §2.3 式 (2.3.1)）。这是嵌入空间语义几何意义成立的前提条件。

现实很骨感。Ethayarajh（2019）定义了各向异性指标：

$$\bar{\cos} = \frac{1}{\binom{V}{2}} \sum_{i<j} \cos(\mathbf{v}_i, \mathbf{v}_j) \tag{2.5.1}$$

对于各向同性的空间，$\bar{\cos} \approx 0$（这与 §2.3 式 (2.3.1) 的预测一致）。而对 BERT 上层的嵌入，$\bar{\cos}$ 可达 0.99——几乎所有词向量都指向同一方向。这意味着：当用式 (2.1.5) 的余弦相似度来比较两个词时，结果基本上反映的是它们与这个「主导方向」的对齐程度，而不是它们的语义差异。余弦相似度作为语义测量工具，在此条件下几乎完全失效。

不只是 BERT，GPT-2 的中间层也呈现严重的各向异性：主成分分析（PCA）的前几个特征向量占据了绝大部分方差，大量维度几乎不携带信息。嵌入空间的**有效秩（effective rank）**——即实际上有信息量的维度数——远低于名义维度 d。

第二环：椎体效应——训练机制的直接后果

各向异性的来源可以从训练机制直接追溯。负采样（§2.2，式 (2.2.3)）以噪声分布 $P_n(c) \propto f(c)^{3/4}$ 采样负样本，其中 $f(c)$ 是词频。高频词（「the」「of」「and」）的出现概率本已很高，经过 $3/4$ 次幂后仍然远高于低频词。在一个典型的训练 step 里，「the」作为负样本被采到的概率，可能是「obscure」这样低频词的数千倍。

每次「the」被当作负样本，它的上下文向量 $\mathbf{u}_{\text{the}}$ 就受到一次「远离当前中心词」的排斥梯度。将式 (2.2.3) 对 $\mathbf{u}_{\text{the}}$ 求梯度：

$$\nabla_{\mathbf{u}_{\text{the}}} \mathcal{L}_{\text{NEG}} = -\sigma(\mathbf{u}_{\text{the}}^T \mathbf{v}_w) \cdot \mathbf{v}_w \tag{2.5.2}$$

经过数百万次式 (2.5.2) 给出的排斥梯度，「the」的向量被系统性地推向向量空间的某个角落。类似地，所有高频词都经历了比低频词多得多的排斥梯度，最终聚集在一个狭窄的方向锥里。这就是 Mimno & Thompson（2017）所称的**椎体效应**：高频词形成一个低维锥，其余词分布在锥外，整个空间严重地各向异性。

椎体效应还有第二个成因：**嵌入矩阵的平均向量（词频加权重心）偏离原点**。如果词频分布极度不均匀（Zipf 定律，最常见词的频率是次常见词的两倍），高频词的嵌入向量在训练中受到大量更新，在平均向量的计算中权重极大。结果是整个嵌入集合的重心偏向高频词聚集的方向，产生一个强烈的「平均方向」。几乎所有词向量都与这个平均方向高度对齐，导致式 (2.5.1) 定义的各向异性指标 $\bar{\cos}$ 虚高。

第三环：白化——修复各向异性的代价

白化（whitening）是线性代数上对各向异性的标准修复：对嵌入集合做 PCA，然后按每个主成分方向的标准差（特征值的平方根）归一化，使得变换后每个方向的方差都等于 1：

$$\tilde{\mathbf{v}}_w = \Lambda^{-1/2} U^T \mathbf{v}_w \tag{2.5.3}$$

其中 U 是协方差矩阵的特征向量矩阵，Λ 是特征值对角矩阵。由式 (2.5.3) 变换后 $\text{Cov}(\tilde{\mathbf{v}}) = I_d$，各向异性消除。Su 等人（2021）的 BERT-whitening 实验表明，对由式 (2.5.3) 白化后的嵌入向量应用式 (2.1.5) 的余弦相似度评估语义相似性，在多项 STS 基准上超过了更复杂的对比学习模型。

但式 (2.5.3) 给出的白化变换不是免费的午餐。它有三个代价：

代价一：计算成本。白化需要对全部 V 个词向量计算协方差矩阵，时间复杂度 $O(V \cdot d^2)$，内存复杂度 $O(d^2)$。对于上下文相关嵌入（每个 token 在不同句子中有不同的向量），每次推理前都需要重新估计协方差矩阵，代价不可接受。白化只对**静态嵌入**（word2vec、GloVe 这类每个词全局一个向量的模型）实用。

代价二：破坏线性语义方向。式 (2.5.3) 的白化变换的本质是一个线性变换，它会改变向量间的相对位置。原来对齐的「平行四边形」（即式 (2.2.4) 类比算术的几何基础）在白化后不再保持。实验上，白化后 word2vec 的类比准确率下降了——它在语义相似度任务上更好，但在类比推理任务上更差。这揭示了一个根本矛盾：**各向异性是式**

(2.2.3) **负采样训练的副产品，但也可能和某些有用的语义结构耦合在一起**；式 (2.5.3) 的白化在移除噪声的同时，也可能损坏信号。

代价三：过白化（over-whitening）。如果协方差矩阵的某些特征值接近零（即嵌入矩阵有接近线性相关的维度），式 (2.5.3) 中的 $\Lambda^{-1/2}$ 会把这些方向放大到极大的尺度，引入数值不稳定性。实践中通常加一个小的正则化项 $\Lambda \leftarrow \Lambda + \epsilon I$ 来避免这个问题。

整条链的回头看

```
负采样噪声分布 f(c)^{3/4}        ← 高频词以更高概率成为负样本
  ↓ 高频词受到更多「排斥」梯度
椎体效应                          ← 高频词向量聚集在低维锥内；嵌入重心偏离原点
  ↓ 空间出现强烈「主导方向」
各向异性（ā_cos → 0.99）          ← 任意两词余弦相似度虚高；余弦作为语义工具失效
  ↓ 需要消除主导方向
白化变换 Λ^{-1/2} U^T v_w         ← 恢复各向同性；STS 任务显著改善
  ↓ 但破坏线性语义结构
类比算术准确率下降                ← 白化的代价：消除噪声的同时可能破坏信号
```

这条链揭示了一个深层事实：**词嵌入的几何隐喻（「语义坐标系」「类比算术」）不是自动成立的，而是在特定条件下（各向同性、统计充分、关系规律）才近似成立**。训练机制（式 (2.2.3)）本身就在持续产生违反这些条件的副作用。白化（式 (2.5.3)）是一个有效的局部修复，但它揭示了更深的矛盾：词频偏置和语义结构深度耦合，无法完全分离。

严格

各向异性（anisotropy）。理想的嵌入空间应满足各向同性：嵌入向量均匀分布在各方向，任意随机向量与所有嵌入向量的平均余弦相似度为零。实践中，Ethayarajh（2019）发现，BERT 的上下文嵌入高度各向异性——大多数嵌入向量与「平均方向」高度对齐，投影到低维子空间后几乎共线。形式化：设嵌入集合 $\{\mathbf{v}_w\}_{w=1}^{V}$，定义各向异性指标：

$$\bar{\cos} = \frac{1}{\binom{V}{2}} \sum_{i<j} \cos(\mathbf{v}_i, \mathbf{v}_j) \tag{2.5.4}$$

式 (2.5.4) 与直觉部分的式 (2.5.1) 完全对应。各向同性时 $\bar{\cos} \approx 0$（符合 §2.3 式 (2.3.1) 的预测）；实际 GPT-2 的层级嵌入中 $\bar{\cos}$ 可达 0.99，即几乎所有嵌入向量指向同一方向。

词频导致的椎体效应（frequency-induced cone effect）。负采样目标函数（§2.2，式 (2.2.7)）中，高频词作为负样本被采到的概率高，其上下文向量受到更多「排斥」梯度，导致高频词向量被推向向量空间的边缘，聚集在一个狭窄的锥形区域内。Mimno & Thompson（2017）将此称为「椎体效应」：频率最高的词（如「the」、「of」）形成一个低维锥，其余词分布在锥外，使得整个嵌入空间的有效维度远低于名义维度 d。

白化（whitening）作为修正。白化变换通过对嵌入集合进行 PCA 并按特征值归一化，消除各向异性偏置：

设嵌入矩阵（去中心化后）的协方差为 $\Sigma = \frac{1}{V} E^T E \in \mathbb{R}^{d \times d}$，其特征分解为 $\Sigma = U \Lambda U^T$（$\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_d)$，$\lambda_1 \geq \cdots \geq \lambda_d > 0$）。白化变换为：

$$\tilde{\mathbf{v}}_w = \Lambda^{-1/2} U^T \mathbf{v}_w \tag{2.5.5}$$

式 (2.5.5) 与直觉部分的式 (2.5.3) 完全对应。变换后 $\mathrm{Cov}(\tilde{\mathbf{v}}) = I_d$，每个方向的方差相等，空间各向同性恢复。Su 等人（2021）的 BERT-whitening 实验表明，由式 (2.5.5) 白化后用式 (2.1.5) 的余弦相似度评估语义相似性的效果优于原始 BERT 表示，且在多项语义文本相似度基准上超过更复杂的对比学习方法。

后果总结。各向异性意味着：（1）式 (2.1.5) 定义的余弦相似度在未白化的嵌入空间中存在系统性偏置，高频词对之间的余弦值虚高；（2）由式 (2.2.4) 定义的向量加减（类比算术）在高频词参与时误差更大；（3）嵌入空间的「有效维度」显著低于 d，大量方向携带的是词频噪声而非语义信号。

与原书呼应：原书 §2.5 指出几何隐喻失效的三个层次（连续 vs 离散、概率 vs 确定、构造 vs 发现），与本节各向异性（式 (2.5.4)）和椎体效应（式 (2.5.2)）共同构成对「嵌入空间是理想欧氏空间」这一假设的否定。

---

§2.6 跨越欧氏：当表示空间不再「平直」

▶这一节在讲什么

为什么树形层级结构很难嵌入欧氏空间？树的节点数随深度**指数**增长，而欧氏球的体积只随半径**多项式**增长。这一根本不匹配意味着：要在欧氏空间中忠实地表示深层树，维度必须线性增长，代价不可接受。

第一步：认识欧氏嵌入的维度瓶颈。完全 b-叉树有 $O(b^L)$ 个节点。在欧氏空间中，容纳 N 个互相保持距离的点所需维度满足 $d \geq \Omega(L \log b / \log r)$，对大深度 L 线性增长，不可避免。

第二步：双曲空间的体积指数增长正好匹配树。Poincaré 球模型中，以原点为中心、半径为 r 的球的体积 $\propto e^{(d-1)r}$，与 b-叉树节点数的指数增长天然吻合。5 维双曲空间已能超过 200 维欧氏空间的层级重建精度。

第三步：球心对应抽象概念，边界对应具体叶节点。在 Poincaré 球中，节点的层级深度与其到球心的双曲距离直接对应——边界不可达，越靠近边界越「具体」。

本节要拿走的一件事：选择几何空间本质上是对数据内在拓扑的假设——欧氏空间对混合拓扑最稳健，双曲空间对强层级数据有显著收益，没有普适最优解。

直觉

前几节讨论的嵌入空间都是欧氏的——$\mathbb{R}^d$ 配备标准内积，平直、各向同性（理想情况）、无曲率。但语言中的语义关系并非全部都适合欧氏结构：「动物 → 哺乳动物 → 犬科 → 狗 → 金毛猎犬」这条层级链，和「物理学 → 量子力学 → 量子场论」，都是**树形层级结构**——节点数随深度指数增长。欧氏空间的体积只按多项式增长，根本容纳不下这种指数膨胀，除非维度随树深度线性增长，代价不可接受。

本节出现四个概念：**欧氏嵌入的维度瓶颈**、**黎曼流形与曲率**、**Poincaré 球（双曲空间）**、**双曲优化的代价**。这四个概念沿着一条需求驱动的逻辑链：欧氏空间的多项式体积增

长不匹配树的指数节点增长（瓶颈），曲面空间（黎曼流形）允许体积以不同速率增长（概念扩展），双曲空间（Poincaré 球）体积正好指数增长（解决方案），但在弯曲空间里做梯度下降需要全新的工具（代价）。

第一环：欧氏嵌入的维度瓶颈——多项式 vs 指数

在欧氏空间 $\mathbb{R}^d$ 里，半径为 r 的球最多能放下 $O(r^d)$ 个两两距离 ≥ 1 的点（packing 论证）。现在考虑一棵深度为 L、分支数为 b 的完全 b-叉树，它有 $O(b^L)$ 个叶节点。

要把这棵树嵌入欧氏空间，同时保持所有节点对的距离（树上的最短路径距离），需要多大的维度？由 packing 论证，球的半径 r 约为树的直径 $O(L)$，所以球内最多放 $O(L^d)$ 个点；要放下 $O(b^L)$ 个节点，需要 $L^d \geq b^L$，即 $d \geq L \log b / \log L$。对固定的 $b > 1$，这随 L **接近线性增长**——树越深，需要的维度越高，没有上界。

这不只是一个抽象的下界，而是实践中可以直接感受到的问题：WordNet 的名词层级有约 8×10^4 个节点，层级深度约 12。用欧氏嵌入尝试精确保留层级距离，需要数百维，且随着树的扩展还会继续增大。但同样的任务，**5 维双曲空间**就能超过 200 维欧氏嵌入的层级重建精度（Nickel & Kiela, 2017）——维度差距超过 40 倍。

第二环：黎曼流形——弯曲空间的通用框架

欧氏空间是「平直的」——曲率为零。什么叫曲率？直觉上：在欧氏平面上，一个三角形的内角和等于 180°；在球面（正曲率）上，内角和大于 180°；在双曲面（负曲率）上，内角和小于 180°。曲率决定了空间的体积增长速率。

黎曼流形（Riemannian manifold）是把这种曲率概念推广到任意维度、任意形状的数学框架。它是一个光滑的流形（局部看起来像欧氏空间的空间），在每个点 p 处配备一个**黎曼度量张量** g_p——一个正定双线性型，决定了在点 p 附近如何计算距离：

$$ds^2 = \sum_{i,j} g_{ij}(p)\, dx^i\, dx^j \tag{2.6.1}$$

当 $g_{ij} = \delta_{ij}$（Kronecker delta）在所有点处成立时，就是欧氏空间；此时式 (2.6.1) 退化为标准的欧氏度量元素 $ds^2 = \sum_i (dx^i)^2$。当 g_{ij} 随位置变化时，空间就是弯曲的。不同的度量张量对应不同的几何空间：球面、双曲面、拓扑复杂的曲面，都是黎曼流形的特例。

黎曼流形框架对表示学习的意义在于：**它允许我们选择与数据内在拓扑相匹配的几何空间**。树形层级数据的「自然空间」不是欧氏空间，而是双曲空间——双曲空间的体积增长方式与树的节点数增长方式完全一致。

第三环：Poincaré 球——双曲空间的具体实现

双曲空间 $\mathbb{H}^d$ 是常负曲率的黎曼流形。它有多种等价的模型，其中最适合计算的是 **Poincaré 球模型**：把无限的双曲空间「压缩」进有限的开单位球 $\mathbb{B}^d = \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\| < 1\}$，配以黎曼度量：

$$ds^2 = \frac{4\,\|d\mathbf{x}\|^2}{\left(1 - \|\mathbf{x}\|^2\right)^2} \tag{2.6.2}$$

式 (2.6.2) 与式 (2.6.1) 对比：它是黎曼度量张量的一个特例，其中 $g_{ij}(\mathbf{x}) = \frac{4}{(1-\|\mathbf{x}\|^2)^2}\delta_{ij}$

——保角因子 $\lambda(\mathbf{x})^2 = \frac{4}{(1-\|\mathbf{x}\|^2)^2}$ 随位置变化，当 $\|\mathbf{x}\|$ 接近 1（球边界）时，$\lambda \to \infty$——空间被无限「拉伸」，有限的欧氏距离对应无限的双曲距离，边界不可达。

双曲距离的闭式公式为：

$$d_{\mathbb{H}}(\mathbf{u},\mathbf{v}) = \operatorname{arcosh}\left(1 + 2\,\frac{\|\mathbf{u}-\mathbf{v}\|^2}{(1-\|\mathbf{u}\|^2)\,(1-\|\mathbf{v}\|^2)}\right) \tag{2.6.3}$$

式 (2.6.3) 的直觉：分子 $\|\mathbf{u}-\mathbf{v}\|^2$ 是欧氏距离的平方，分母 $(1-\|\mathbf{u}\|^2)(1-\|\mathbf{v}\|^2)$ 是两点「离边界有多远」的乘积。当两点都靠近球边界时，分母接近零，双曲距离急剧增大——即使欧氏距离只是略有不同，式 (2.6.3) 给出的双曲距离也可以差异悬殊。将式 (2.6.3) 与式 (2.4.3) 对比，可以看出双曲距离不再是欧氏意义下的「直线距离」，而是通过式 (2.6.2) 定义的弯曲度量来衡量的测地线长度。

球内的位置和树的结构之间有完美的对应：- **球心**（$\|\mathbf{x}\| \approx 0$）对应树的**根节点**和高层抽象概念（「实体」「生物」）——因为从球心到所有方向都有足够的距离空间，可以容纳大量子节点；- **球边界**（$\|\mathbf{x}\| \to 1$）对应树的**叶节点**和具体概念（「金毛猎犬」「量子场论」）——越靠近边界，由式 (2.6.3) 计算的双曲距离越大，叶节点之间彼此隔离；- **径向距离**（到球心的双曲距离）直接对应树的**层级深度**——父节点自然位于子节点的「内侧」，无需额外约束。

这种几何对应使得双曲嵌入不需要任何显式的层级约束，层级关系自动从球内坐标中涌现。

第四环：双曲优化的代价——弯曲空间里的梯度下降

双曲嵌入的优化比欧氏空间困难得多。普通的梯度下降在欧氏空间里是：$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta\nabla_{\mathbf{x}}\mathcal{L}$。但 Poincaré 球不是欧氏空间，欧氏梯度 $\nabla_{\mathbf{x}}\mathcal{L}$ 的方向是在欧氏切空间里定义的，直接减去它会把点推出单位球边界（$\|\mathbf{x}\| \geq 1$），不再是合法的双曲空间点。

在黎曼流形（式 (2.6.1)）上做梯度下降需要：

1. **黎曼梯度**（Riemannian gradient）：欧氏梯度 $\nabla_{\mathbf{x}}$ 乘以度量张量的逆 g^{-1}，得到流形上正确定义的梯度。将式 (2.6.2) 的度量张量代入可得：

$$\operatorname{grad}_{\mathbb{H}} f(\mathbf{x}) = \frac{(1-\|\mathbf{x}\|^2)^2}{4}\nabla_{\mathbf{x}} f \tag{2.6.4}$$

2. **指数映射**（exponential map）：把切空间里的更新方向「映射」回流形上，沿测地线（黎曼流形上的「直线」）前进：

$$\mathbf{x}_{t+1} = \exp_{\mathbf{x}_t}\left(-\eta \cdot \operatorname{grad}_{\mathbb{H}} f(\mathbf{x}_t)\right) \tag{2.6.5}$$

注意式 (2.6.4) 与式 (2.6.5) 配合使用：式 (2.6.4) 把欧氏梯度正确地缩放到弯曲空间中，式 (2.6.5) 再沿着弯曲空间的「测地线」前进，确保更新后的点仍在 Poincaré 球内（$\|\mathbf{x}\| < 1$）。

3. **边界约束**：数值上防止点越过边界，实践中加 $\|\mathbf{x}\| \leq 1-\epsilon$（$\epsilon \sim 10^{-5}$）的约束，并对梯度裁剪（gradient clipping）以避免靠近边界时梯度爆炸。

由式 (2.6.4) 可以看出，越靠近球边界（$\|\mathbf{x}\| \to 1$），黎曼梯度越小，意味着欧氏梯度对参数的实际影响被压缩——这解释了为什么靠近边界时训练需要格外小心。Nickel & Kiela（2017）报告，双曲嵌入对初始化和学习率极为敏感，调参难度显著高于同等规模的欧氏嵌入。

此外，双曲空间的**球面投影**（Stereographic projection）是另一种常用的双曲空间模型，将双曲面映射到扩展欧氏平面。这种模型在实现某些运算（如双曲卷积）时更方便，但距离公式更复杂，适合特定任务（双曲图神经网络）而非通用词嵌入。

整条链的回头看

```
欧氏体积 ~ r^d（多项式）          ← 容纳不下指数增长的树节点
  ↓ 需要体积指数增长的空间
黎曼流形框架                      ← 通用的弯曲空间：度量张量 g_{ij}(p) 随位置变化
  ↓ 常负曲率 → 体积指数增长
双曲空间 / Poincaré 球            ← 5 维胜过 200 维欧氏；层级深度 = 径向距离
  ↓ 流形上的梯度下降需要黎曼工具
黎曼梯度 + 指数映射               ← 代价：优化难度显著增大，训练对初始化敏感
  ↓ 自然语言语义 = 层级 + 网状混合拓扑
欧氏空间仍是主流 LLM 选择         ← 混合拓扑时双曲优势不稳定；优化成本不划算
```

这条链揭示了一个深层事实：**几何空间的选择本质上是对数据内在拓扑的先验假设**。欧氏空间假设数据是「平直的」（各方向等权重、无层级偏好），双曲空间假设数据是「强层级的」（节点数指数增长、父子关系清晰）。当假设与数据吻合时，非欧空间可以带来巨大的维度效率；当假设与数据不匹配时，弯曲空间的优化代价（式 (2.6.4)、式 (2.6.5)）反而成了净负担。没有普适最优解——最好的几何空间是与数据内在结构最匹配的那个。

严格

欧氏嵌入的维度瓶颈。深度为 L、分支数为 b 的完全 b-叉树有 $N = \frac{b^{L+1}-1}{b-1} = O(b^L)$ 个节点。在欧氏空间 $\mathbb{R}^d$ 中，半径为 r 的球最多包含 $O(r^d)$ 个互相距离 ≥ 1 的点（packing argument）。要将 N 个节点嵌入使得树上距离被 $(1+\varepsilon)$-近似保持，所需维度满足 $r^d \geq N$，即：

$$d \geq \Omega\left(\frac{L \log b}{\log r}\right) \tag{2.6.6}$$

对大 L 此下界线性增长，不可避免。

Poincaré 球模型（Poincaré ball model）。d 维双曲空间 $\mathbb{H}^d$ 的 Poincaré 球模型定义在开单位球 $\mathbb{B}^d = \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\| < 1\}$ 上，配以黎曼度量：

$$ds^2 = \frac{4\,\|\mathrm{d}\mathbf{x}\|^2}{\left(1 - \|\mathbf{x}\|^2\right)^2} \tag{2.6.7}$$

式 (2.6.7) 与直觉部分的式 (2.6.2) 完全对应，即欧氏度量乘以保角因子 $\lambda(\mathbf{x})^2 = \frac{4}{(1-\|\mathbf{x}\|^2)^2}$（共形因子）。当 $\|\mathbf{x}\| \to 1$ 时 $\lambda \to \infty$，空间被无限「拉伸」——有限欧氏距离对应无限双曲距离，边界不可达。

双曲距离公式。对 $\mathbf{u}, \mathbf{v} \in \mathbb{B}^d$，由式 (2.6.7) 诱导的双曲距离为：

$$d_{\mathbb{H}}(\mathbf{u}, \mathbf{v}) = \operatorname{arcosh}\left(1 + 2\,\frac{\|\mathbf{u}-\mathbf{v}\|^2}{\left(1-\|\mathbf{u}\|^2\right)\left(1-\|\mathbf{v}\|^2\right)}\right) \tag{2.6.8}$$

式 (2.6.8) 与直觉部分的式 (2.6.3) 完全对应，其中 $\operatorname{arcosh}(t) = \ln(t + \sqrt{t^2 - 1})$，$t \geq 1$。

推导梗概。从 $\mathbf{u}$ 到 $\mathbf{v}$ 的双曲测地线长度为 $\int_\gamma ds$（由式 (2.6.7) 定义的弧长积分），利用 Möbius 变换将 $\mathbf{u}$ 映射到原点（原点到 $\mathbf{r}$ 的距离简化为 $\int_0^{\|\mathbf{r}\|} \frac{2\,dt}{1-t^2} = 2\operatorname{arctanh}(\|\mathbf{r}\|)$），再利用 arcosh 与 arctanh 的关系 $\operatorname{arcosh}(1 + 2t^2/(1-t^2)) = 2\operatorname{arctanh}(t)$，得到式 (2.6.8) 的闭式表达。

指数体积增长。在 Poincaré 球中，以 $\mathbf{0}$ 为中心、双曲半径为 r 的球的（欧氏）体积为：

$$\mathrm{Vol}_{\mathbb{H}}(B(r)) \propto \sinh^{d-1}(r) \sim \frac{1}{2^{d-1}} e^{(d-1)r}, \quad r \to \infty \tag{2.6.9}$$

体积随 r **指数增长**，这正是解决了第一环（式 (2.6.6)）揭示的欧氏嵌入维度瓶颈的根本原因：将式 (2.6.9) 与 b-叉树节点数的指数增长 $O(b^r)$ 匹配，只需令 $b = e^{d-1}$。因此，在 $d = 5$ 维双曲空间中，已能容纳 WordNet 约 8×10^4 个名词节点，且层级距离重建精度超过 200 维欧氏嵌入（Nickel & Kiela, 2017）。

层级位置编码的几何含义。在 Poincaré 球中，节点在层级结构中的深度与其到球心的由式 (2.6.8) 定义的双曲距离直接对应：

- **球心**（$\|\mathbf{x}\| \approx 0$）：抽象概念、树根（如「实体」）；
- **球边界**（$\|\mathbf{x}\| \to 1$）：具体叶节点（如「金毛犬」），到边界的距离趋于无穷。

父节点到所有子节点的距离均等且较短，体现层级关系；兄弟节点之间的距离则由式 (2.6.8) 的分母 $(1 - \|\mathbf{u}\|^2)(1 - \|\mathbf{v}\|^2)$ 调节，靠近边界时兄弟之间距离迅速增大，有效隔离不同子树。

为何主流 LLM 仍用欧氏空间。自然语言的语义关系同时包含层级（is-a）和网状联想（associate-with）结构。欧氏空间对混合拓扑最为稳健；双曲空间在强层级数据（本体树、分子图）上有明确收益，但在通用语言建模中的优势尚不稳定。选择几何空间本质上是对**数据内在拓扑**的假设，没有普适最优解。

与原书呼应：原书 §2.6 介绍 Poincaré 嵌入、式 (2.6.8) 的双曲距离公式及主流 LLM 仍用欧氏空间的原因，对应本节度量推导（式 (2.6.7)）、式 (2.6.9) 的指数体积增长的严格分析，以及几何选择的元原则。

数学卷 · 第 3 章：注意力机制的矩阵微分

§3.1 从 RNN 到变换器（Transformer）：序列建模的范式革命

▶这一节在讲什么

为什么 RNN 不能并行？RNN 把序列理解为一条因果链，每步计算依赖前一步的隐状态。这种设计在直觉上符合「逐字阅读」的体验，却注定无法并行——时序依赖锁死了硬件。

第一步：量化梯度消失。RNN 从时刻 T 向 t 反传梯度，需连乘 $T-t$ 个雅可比矩阵。若其谱半径 $\rho<1$，梯度以 ρ^{T-t} 指数衰减，模型无法学习长距离依赖；若 $\rho>1$，梯度爆炸。LSTM 能缓解但不能根治。

第二步：Transformer 的切断。Transformer 彻底去掉时序递推：输入矩阵 $X\in\mathbb{R}^{n\times d}$ 整体存在，每个位置的输出通过矩阵乘法直接从全局上下文「读取」，n 个位置完全并行。

第三步：代价是什么。并行化的代价是失去天然的顺序归纳偏置，必须显式注入位置信息（见 §3.4 RoPE），且计算量是 $O(n^2d)$，序列越长越昂贵（见 §3.5 FlashAttention）。

本节要拿走的一件事：Transformer 的崛起不是「注意力更聪明」，而是「矩阵乘法比时序递推更适合 GPU」——这是一次范式替换，不是渐进改良。

直觉

循环神经网络（Recurrent Neural Network，RNN）把序列理解为一条因果链：每一步的计算依赖前一步的结果。这种设计在直觉上符合「逐字阅读」的体验，但也带来了一个数学上的死结——序列中任意两步之间的信息传递必须沿着这条链一步一步走，无法跳过，无法并发。Transformer 的根本突破在于：彻底切断时间依赖，让每个位置的表示直接从全局上下文中「读取」，把序列问题还原为一组独立的矩阵运算。

本节出现四个核心概念：**RNN 的时序依赖与串行瓶颈**、**梯度消失/爆炸的数学根源**、**Transformer 的并行化设计**、**自注意力替代时序递推的代价**。它们看似散落，实际上沿着一条严密的逻辑链一个推出一个。理解这条链，是理解为什么深度学习从 RNN 时代跨越到 Transformer 时代的真正原因。下面把这条链一环环铺开。

第一环：RNN 的时序依赖——为什么递推结构必然串行

RNN 的隐状态递推公式是 $h_t=\sigma(Wh_{t-1}+Ux_t+b)$，意思是：第 t 步的隐状态 h_t 是由上一步的隐状态 h_{t-1} 和当前输入 x_t 共同决定的。这在计算图上意味着：h_t 依赖 h_{t-1}，h_{t-1} 依赖 h_{t-2}，以此类推——这是一条严格的数据流因果链。硬件层面的后果是决定性的：GPU 有数千个并行计算单元，但 RNN 在时间轴上的每一步必须等待前一步完成，这些并行单元在时间维度上大部分时间是空转等待的。这不是实现方式的问题，而是模型定义本身要求的依赖关系。序列长度 n 越大，GPU 的并行潜力被浪费得越多——RNN 天花板由时序链长度，而不是硬件宽度决定。

第二环：梯度消失/爆炸——长链反传的数学死亡

并行问题可以忍受，但梯度问题是致命的。在反向传播中，从时刻 T 的损失向时刻 t 传梯度，需要通过链式法则展开一段连乘：

$$\frac{\partial \mathcal{L}}{\partial h_t} = \frac{\partial \mathcal{L}}{\partial h_T} \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k} \tag{3.1.1}$$

每一步的雅可比矩阵是 $\frac{\partial h_{k+1}}{\partial h_k} = \mathrm{diag}(\sigma'(\cdot))W$，它的谱范数大约是 $\rho = \|W\|_2 \cdot \max|\sigma'|$。连乘 $T-t$ 个这样的矩阵后（即式 (3.1.1) 中的连乘项），整个梯度的量级大约是 ρ^{T-t}：若 $\rho < 1$，梯度以指数速率衰减至零（梯度消失），模型无法更新远处的参数；若 $\rho > 1$，梯度指数爆炸，训练发散。这不是参数调得不好导致的，而是连乘结构在数学上的必然。稍微多留意一下式 (3.1.1) 就能看出：**这个问题与序列长度 $T-t$ 直接相关**——序列越长，梯度越容易消失。这正是 RNN 在长文本上、在长依赖任务上表现欠佳的根本原因。

第三环：LSTM 的门控——工程防护栏，而非数学根治

长短期记忆网络（Long Short-Term Memory，LSTM）是 RNN 时代最成功的改良。它引入了遗忘门、输入门、输出门，以及独立于隐状态的记忆单元（cell state）。门控的作用是让记忆单元的梯度路径更接近恒等映射——如果遗忘门全开（全 1），记忆单元的梯度可以无衰减地回传到早期时刻，有效谱半径接近 1。这在工程上大大缓解了梯度消失问题，使 LSTM 能学到数十步内的依赖关系。

但这只是「防护栏」，不是「根治」。LSTM 仍然是递推结构，仍然无法并行，仍然在极长序列上（数百步以上）遭遇训练困难。更深的问题是：门控参数本身也需要从数据中学习，而它们的正确取值依赖于上下文——在训练初期，门控往往会打开错误的时间段，导致真正重要的远距离信息依然无法有效传播。LSTM 把 RNN 的天花板从大约 50 步提高到大约 200 步，但它没有从根本上改变「距离越长，信号越弱」的衰减规律。

第四环：Transformer 的并行化切断——把序列问题变成矩阵问题

Transformer 的设计思路是：彻底放弃时序递推。给定输入序列的嵌入矩阵 $X \in \mathbb{R}^{n \times d}$（所有 n 个 token 同时存在于矩阵的行中），每个位置的输出通过一次矩阵运算直接从整个上下文读取：

$$O = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad Q = XW^Q,\ K = XW^K,\ V = XW^V \tag{3.1.2}$$

式 (3.1.2) 的关键是：Q、K、V 全部从同一个矩阵 X 一次性线性投影得到，计算过程没有任何时间轴上的依赖关系。第 i 个 token 的输出需要用到其他所有 token 的信息，但这个「用到」是通过矩阵乘法的一次性全局聚合完成的，而不是通过逐步递推传递的。结果是：n 个位置的输出可以完全并行计算，不需要等待任何「前序状态」。GPU 的所有并行计算单元可以同时满载工作。

更重要的是，由式 (3.1.2) 可见，任意两个位置之间的梯度传播路径长度从 $O(n)$ 降到了 $O(1)$——在自注意力层内，位置 i 和位置 j 之间只有一层矩阵运算相隔，不管它们在序列中相距多远。这彻底消除了连乘项，梯度消失/爆炸问题在模型的「时序维度」上不再存在（当然，层数增加时仍然存在，但这通过残差连接等机制解决）。

第五环：并行化的代价——位置信息的缺失与二次复杂度

任何设计都有代价。Transformer 去掉了时序递推，同时也去掉了时序结构自带的归纳偏置——模型不再天然知道「哪个词在前、哪个词在后」。用矩阵 X 直接描述的 token

集合是无序的，如果不做任何处理，打乱输入序列的顺序，注意力的输出不会改变（除了行排列）。这意味着必须显式地向模型注入位置信息，否则「The dog bites the man」和「The man bites the dog」会得到完全相同的表示。这就是位置编码（positional encoding）出现的原因，§3.4 将专门讨论 RoPE 如何用旋转矩阵优雅地解决这个问题。

第二个代价是计算复杂度。注意力矩阵 $S = QK^T \in \mathbb{R}^{n\times n}$ 的大小随序列长度的平方增长；由式 (3.1.2) 可知整个注意力计算的 FLOPs 为 $O(n^2d)$。当 $n = 1024$ 时这不成问题，但当 $n = 100000$（长文档、代码库）时，n^2 项变得极为昂贵，且 $n\times n$ 的矩阵本身在内存上也是 $O(n^2)$ 的开销。这是 FlashAttention（§3.5）和各种稀疏注意力（sparse attention）方法需要解决的核心瓶颈。

整条链的回头看

```
RNN 递推结构
  ↓ 时序依赖 → 串行、无法并行
梯度连乘 ^(T-t)
  ↓ <1 消失，>1 爆炸 → 长依赖无法学习
LSTM 门控
  ↓ 缓解但不根治 → 上限仍然存在
Transformer：去掉递推，输入矩阵一次性存在
  ↓ 所有位置并行 → GPU 满载
自注意力 O = softmax(QK /√d ) · V
  ↓ 梯度路径缩短到 O(1) → 长依赖无障碍
代价：无位置归纳偏置 + O(n²d) 复杂度
  ↓ 引出 RoPE (§3.4) 和 FlashAttention (§3.5)
```

这条链揭示了一个深层事实：**Transformer 的崛起不是「注意力机制更聪明」，而是一次由硬件约束驱动的架构重设计**。梯度消失是 RNN 的数学死亡，并行化是 GPU 时代的必然要求，自注意力恰好同时解决了两者——代价是引入了新的挑战（位置编码、二次复杂度），而这些挑战又催生了后续的一系列创新。

严格

RNN 的隐状态（hidden state）递推公式（即式 (3.1.3)）为

$$h_t = \sigma(Wh_{t-1} + Ux_t + b) \tag{3.1.3}$$

其中 $h_t \in \mathbb{R}^{d_h}$ 是第 t 步隐状态，$W \in \mathbb{R}^{d_h\times d_h}$，$U \in \mathbb{R}^{d_h\times d_x}$，$\sigma$ 为逐元素非线性激活（如 tanh）。

梯度消失/爆炸的数学根源

在时刻 T 的损失 $\mathcal{L}$ 对时刻 t 的隐状态求梯度，需通过链式法则展开

$$\frac{\partial \mathcal{L}}{\partial h_t} = \frac{\partial \mathcal{L}}{\partial h_T}\prod_{k=t}^{T-1}\frac{\partial h_{k+1}}{\partial h_k} \tag{3.1.4}$$

每一步的雅可比矩阵（Jacobian matrix）为

$$\frac{\partial h_{k+1}}{\partial h_k} = \operatorname{diag}\left(\sigma'(Wh_k + Ux_{k+1})\right) W \tag{3.1.5}$$

将式 (3.1.5) 连乘 $T-t$ 项，其谱范数满足

$$\left\| \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k} \right\|_2 \leq \prod_{k=t}^{T-1} \left\| \operatorname{diag}(\sigma') \right\|_2 \cdot \|W\|_2 \tag{3.1.6}$$

设 $\rho = \|W\|_2 \cdot \max|\sigma'|$（即该乘积谱半径的上界），则式 (3.1.6) 中梯度范数以 ρ^{T-t} 的速率随距离 $T-t$ 指数变化：

- 若 $\rho < 1$：梯度指数衰减（梯度消失，gradient vanishing）；
- 若 $\rho > 1$：梯度指数增长（梯度爆炸，gradient explosion）。

LSTM 通过门控机制（gating mechanism）使部分梯度路径的有效谱半径接近 1，即减轻了式 (3.1.4)–(3.1.6) 中连乘项的衰减效应，但无法从根本上解除连乘结构，长序列仍受限于此。

为什么 Transformer 可并行

Transformer 完全去掉了时序递推：给定输入矩阵 $X \in \mathbb{R}^{n \times d}$（$n$ 个 token 同时存在于矩阵中），每个位置的输出仅通过矩阵乘法和 softmax 计算，不依赖任何「前序状态」。具体而言，注意力层的输入输出关系（即式 (3.1.7)）为

$$O = \operatorname{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \tag{3.1.7}$$

其中 Q, K, V 均由 X 一次性线性投影得到，整个计算路径无时间轴上的串行依赖，因此 n 个位置的输出可完全并行计算。

与原书呼应：原书 §3.1 提到 RNN「不能并行」和「长距离依赖丢失」两个缺陷，对应本节雅可比连乘的谱半径分析与并行化条件。

---

§3.2 注意力公式的拆解：QKV 的真实含义

▶这一节在讲什么

为什么注意力要用三个矩阵 Q, K, V？不用三个，用一个或两个行不行？这不是历史偶然，而是「软检索」语义的最小充分参数化。

第一步：把注意力理解为软索引。给定查询 Q，在键集合 K 中按相似度打分，再用归一化的分数加权混合值 V。与哈希表不同，这里每个键都有权重，结果是所有值的连续混合。

第二步：为什么要除以 $\sqrt{d_k}$。若 q 和 k 各分量方差为 1，它们的内积方差为 d_k，标准差为 $\sqrt{d_k}$。不缩放时 softmax 的输入量级随维度增大，导致概率质量集中于极少数项，梯度趋零。除以 $\sqrt{d_k}$ 将方差还原为 1。

第三步：多头的意义。将 d 维空间切成 h 个子空间，每头在低维子空间中独立做软检索，捕捉不同粒度的语义关系（语法依赖、指代关系、话题关联……），总参数量不变。

本节要拿走的一件事：$\sqrt{d_k}$ 缩放不是调参技巧，是防止高维内积让 softmax 退化为 one-hot 的数学保险。

直觉

注意力公式 $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})V$ 看上去只有几个符号，但每个设计决策背后都有明确的数学动机。为什么要有三个矩阵而不是一个？为什么要除以 $\sqrt{d_k}$？为什么要用多头？这些问题有确定的答案，而不是「试出来好用就用」。

本节出现五个核心概念：**QKV 的三角色分离**、**Q≠K 的方向性**、**scaling 因子的统计动机**、**softmax 的归一化语义**、**多头注意力的维度分解**。它们沿着「为什么要这样设计」的逻辑链依次推出。下面把这条链铺开。

第一环：把注意力理解为软索引——为什么需要 Q、K、V 三者分离

最直觉的理解：注意力是一种「软哈希表检索」。在硬哈希表里，你用一个键（key）精确匹配到一个值（value）；在软检索里，你的查询（query）和所有键计算相似度，然后按相似度加权汇总所有值。这个框架要求三个独立的角色：

- Q（Query）：当前位置想「问什么」，即检索请求；
- K（Key）：每个位置「愿意回答什么问题」，即可被检索的标签；
- V（Value）：每个位置实际携带的内容，即被检索到时贡献的信息。

为什么这三个必须是独立的矩阵，而不能合并？关键在于 Q 和 K 承担不同职责：Q 描述的是「我需要什么类型的信息」，K 描述的是「我能提供什么类型的信息」。一个 token 在表达「我提供语法主语信息」（K 的角色）和「我在检索动词信息」（Q 的角色）时，使用的特征投影方向可以完全不同。如果强制 $Q = K$（即只用一个投影矩阵），注意力打分矩阵 QK^T 就退化为对称矩阵——但语言中的「关注关系」往往是非对称的：主语需要关注谓语，但谓语不一定需要关注主语。对称矩阵无法表达方向性，会损失大量表达能力。

V 与 K 的分离则是另一层解耦。即使某个 token 在当前上下文中被「高度关注」（高注意力权重），它实际贡献给输出的内容（Value 向量）可以是完全不同维度的特征。

举个例子：位置 j 可能在语法上被位置 i 高度关注（k_j 与 q_i 高度相似），但真正传递给 i 的内容（v_j）是语义级别的上下文特征，而非语法标签本身。这种「谁关注谁」与「传递什么」的解耦，是注意力比单纯点积相似度强大得多的根本原因。

第二环：为什么 Q≠K 是必要的——方向性与非对称性

更精确地说明为什么 Q 和 K 必须是独立的投影。设 $W^Q = W^K$，则 $Q = XW^Q$，$K = XW^K = XW^Q = Q$，注意力打分矩阵变为：

$$A = \frac{QQ^T}{\sqrt{d_k}} \tag{3.2.1}$$

式 (3.2.1) 中的 QQ^T 是一个对称半正定矩阵（Symmetric Positive Semi-Definite，SPSD）。它的语义是：每个 token 与自身的相似度最高，与其他 token 的相似度由特征空间距离决定，且「i 关注 j」和「j 关注 i」的分数相同。但语言中大量的依存关系是方向性的：英文中「形容词修饰名词」意味着形容词应该关注名词（从名词的 Key 中读取语法类别），但名词不需要以同等程度关注形容词。强制对称会把这种非对称依存关系的表达能力减半。独立的 $W^Q \neq W^K$ 让注意力矩阵可以是任意非对称方阵，表达能力大幅提升。

第三环：scaling 因子 $\sqrt{d_k}$——防止高维内积让 softmax 退化

假设 q_i 和 k_j 的每个分量都独立同分布，均值为 0，方差为 1（这在 Xavier 或 He 初始化后是合理的假设）。则它们的内积 $q_i \cdot k_j = \sum_{l=1}^{d_k} q_{il}k_{jl}$ 是 d_k 个独立零均值随机变量之和，其方差为：

$$\mathrm{Var}(q_i \cdot k_j) = \sum_{l=1}^{d_k} \mathrm{Var}(q_{il})\mathrm{Var}(k_{jl}) = d_k \tag{3.2.2}$$

由式 (3.2.2) 知标准差为 $\sqrt{d_k}$。当 $d_k = 64$（典型值），标准差为 8；当 $d_k = 128$，标准差为 11.3。进入 softmax 时，如果输入向量的各分量相差 8 到 11 个单位，exp 函数会把这个差异放大为约 $e^{11} \approx 60000$ 倍——概率质量几乎全部集中到最大值对应的那个位置，softmax 退化为近似 one-hot，梯度接近零。除以 $\sqrt{d_k}$ 将标准差还原为 1，保持 softmax 输入在合理量级，让各位置都有有效的梯度信号。

式 (3.2.2) 揭示这不是「试出来好用的超参」，而是统计上的必然：**内积方差随维度线性增长，缩放因子是对这一增长的精确补偿**。

第四环：softmax 的归一化语义——为什么是逐行而不是全局

softmax 沿行（即针对每个 Query）独立归一化：

$$P_{ij} = \frac{\exp(A_{ij})}{\sum_{l=1}^{n} \exp(A_{il})} \tag{3.2.3}$$

式 (3.2.3) 使得第 i 行的权重之和为 1，构成一个概率分布。这里有一个重要的设计选择：是逐行归一化（每个 Query 独立），还是对整个矩阵 A 做全局归一化？答案是逐行，原因是：注意力的语义是「第 i 个 token 从所有其他 token 中按概率分配注意力资源」，每个 token 的关注权重之和为 1 确保了输出的量级不随序列长度变化（否则

序列越长，注意力加权和越大，输出越不稳定）。全局归一化会让某些 token 「抢走」其他 token 的注意力预算，破坏每个 Query 独立决策的语义。

第五环：多头注意力——维度分解带来的容量倍增

如果只做一次注意力，所有 d 维信息混在一起，模型只能在一个语义层次上「问问题」。多头注意力把 d 维空间分成 h 个 d/h 维子空间，每个头在各自的子空间中独立做注意力：

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \tag{3.2.4}$$

式 (3.2.4) 中每个头有独立的 $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d\times d/h}$，参数量与单头相同（$3d^2/h \times h = 3d^2$，加上 W^O 后总量约 $4d^2$）。但不同的头会在训练中自发分工：某些头专注句法依存（主语-谓语）、某些头追踪长距离指代（代词-名词）、某些头捕捉局部 n-gram 模式。这种分工是**涌现的**，是梯度下降在参数空间中自然寻找到的最优解，不是人为规定的。多头的本质是：用维度分解换取「并行检索不同粒度语义关系」的能力，在不增加参数量的情况下扩展表达能力。

整条链的回头看

软检索语义
  ↓ 需要「谁问、谁答、答什么」三个独立角色
Q、K、V 三矩阵分离
  ↓ Q K 保证非对称方向性
注意力打分 A = QK^T
  ↓ d_k 维内积方差 = d_k，softmax 会退化
除以 sqrt(d_k)，方差还原为 1
  ↓ softmax 逐行归一化，维持每个 Query 的概率语义
P = softmax(A/sqrt(d_k))，输出 O = PV
  ↓ 单头只能在一个层次检索
多头：h 个子空间并行，捕捉不同粒度的语义关系

这条链揭示了一个深层事实：**QKV 的每一个设计细节都对应一个数学必然性**。$\sqrt{d_k}$ 是统计补偿，三矩阵分离是表达能力的必要条件，多头是维度分解的容量策略。理解这条链，就理解了为什么注意力公式是这样而不是那样。

严格

设输入序列嵌入矩阵 $X \in \mathbb{R}^{n\times d}$，可学习投影矩阵

$$W^Q, W^K \in \mathbb{R}^{d\times d_k}, \quad W^V \in \mathbb{R}^{d\times d_v} \tag{3.2.5}$$

由式 (3.2.5) 定义的三个投影矩阵给出

$$Q = XW^Q \in \mathbb{R}^{n\times d_k}, \quad K = XW^K \in \mathbb{R}^{n\times d_k}, \quad V = XW^V \in \mathbb{R}^{n\times d_v} \tag{3.2.6}$$

缩放因子 $\sqrt{d_k}$ 的来源

设 $q_i, k_j \in \mathbb{R}^{d_k}$ 各分量独立同分布，均值为 0、方差为 1，则

$$\mathrm{Var}(q_i \cdot k_j) = \mathrm{Var}\left(\sum_{l=1}^{d_k} q_{il}k_{jl}\right) = \sum_{l=1}^{d_k} \mathrm{Var}(q_{il})\mathrm{Var}(k_{jl}) = d_k \tag{3.2.7}$$

由式 (3.2.7) 知内积 $q_i \cdot k_j$ 的标准差为 $\sqrt{d_k}$。不作缩放时，当 d_k 较大（典型值 64–128），内积分布的量级随 $\sqrt{d_k}$ 线性增大，进入 softmax 后所有概率质量集中于极少数最大项，形成近似 one-hot 分布，梯度趋近于零。除以 $\sqrt{d_k}$ 将方差还原为 1，保持 softmax 输入在合理量级。

注意力分数与归一化

注意力分数矩阵

$$A = \frac{QK^T}{\sqrt{d_k}} \in \mathbb{R}^{n\times n}, \quad A_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}} \tag{3.2.8}$$

softmax **沿第二维（行内）**归一化，即对第 i 行独立计算

$$P_{ij} = \frac{\exp(A_{ij})}{\sum_{l=1}^{n}\exp(A_{il})}, \quad \sum_{j=1}^{n} P_{ij} = 1 \tag{3.2.9}$$

式 (3.2.9) 使得 $P \in \mathbb{R}^{n\times n}$ 的每行构成一个概率分布：第 i 个 token 对所有位置的关注权重之和为 1。将式 (3.2.9) 代入得输出为

$$\mathrm{Attention}(Q, K, V) = PV \in \mathbb{R}^{n\times d_v} \tag{3.2.10}$$

多头注意力的参数化

设头数为 h，每头的子空间维度通常取 $d_k = d_v = d/h$。第 i 头（$i = 1, \dots, h$）独立维护一套投影矩阵

$$W_i^Q \in \mathbb{R}^{d\times d_k}, \quad W_i^K \in \mathbb{R}^{d\times d_k}, \quad W_i^V \in \mathbb{R}^{d\times d_v} \tag{3.2.11}$$

各头输出拼接后再经输出投影 $W^O \in \mathbb{R}^{hd_v\times d}$：

$$\mathrm{MultiHead}(X) = \mathrm{Concat}(\mathrm{head}_1, \dots, \mathrm{head}_h)\, W^O \tag{3.2.12}$$

其中 $\mathrm{head}_i = \mathrm{Attention}(XW_i^Q, XW_i^K, XW_i^V)$（投影矩阵由式 (3.2.11) 定义）。多头设计的参数总量与单头相同（d^2 量级），但在 h 个低维子空间中并行运行不同的「检索模式」。

与原书呼应：原书 §3.2 逐步拆解了 QKV 投影与缩放技巧，对应本节方差推导与 softmax 归一化维度分析。

§3.3 矩阵乘法的大规模并发：为什么 AI 是 GPU 的盛宴

▶这一节在讲什么

为什么矩阵乘法和 GPU 如此般配？GPU 的设计哲学是「以宽度换速度」——数千个简单核心并发执行同一条指令，作用于不同数据。矩阵乘法天然吻合：每个输出元素是独立内积，可完全并行。

第一步：算术强度决定瓶颈在哪里。算术强度 I = FLOPs/内存字节数。大矩阵乘法的 $I \approx 2N/3$ 随矩阵尺寸线性增长，轻松超过 GPU 的「屋脊点」，属于计算受限——GPU 算力能被充分榨取。

第二步：softmax 是短板。逐元素操作（如 exp）算术强度接近 1，远低于屋脊点，属于内存受限。标准 attention 中 softmax 和矩阵读写的来回搬运，才是真正的性能瓶颈（见 §3.5）。

第三步：attention 的算力复杂度是 $O(n^2d)$。序列长度 n 一旦增大，n^2 项主导，FLOPs 急剧增长——这是长上下文昂贵的根本原因。

本节要拿走的一件事：「AI 是 GPU 的盛宴」不是比喻，是算术强度分析的结论——矩阵乘法恰好落在 GPU 架构最擅长的计算受限区间。

直觉

「AI 是 GPU 的盛宴」这句话是字面成立的，不是修辞。要理解为什么，必须理解 GPU 擅长什么、深度学习计算的性质是什么，以及二者为何在算术强度这个指标上精确匹配。这节的核心任务是把「GPU 适合 AI」从直觉转化为一个可以量化的、可以预测瓶颈的数学分析框架——Roofline 模型。

本节出现五个核心概念：**GPU 的 SIMD 并行架构**、**算术强度的定义**、**Roofline 模型与屋脊点**、**矩阵乘法的算术强度分析**、**softmax 的内存受限本质**。下面把这条链铺开。

第一环：GPU 的 SIMD 架构——为什么是「以宽度换速度」

GPU 和 CPU 的设计哲学截然不同。CPU 的设计目标是「让单条指令流以最快速度执行」：大量晶体管用于分支预测、乱序执行、超标量流水线、大缓存——这些机制减少了单线程的延迟，但每个核心复杂且昂贵，一块 CPU 只有几个到几十个核心。GPU 的设计目标是「让同一条指令同时作用于数千个数据」：SIMD（Single Instruction Multiple Data）架构，每个流处理器（Streaming Multiprocessor，SM）包含数十到数百个 CUDA core，一整块 GPU（如 A100）有超过 6000 个。每个核心很简单，但数量极多。

GPU 擅长的计算必须满足一个条件：**大量独立的、结构相同的操作**。如果操作之间有依赖（结果 A 需要等待结果 B），数千个核心中大多数就会空转等待。矩阵乘法 $C = AB$（$A \in \mathbb{R}^{m\times k}$，$B \in \mathbb{R}^{k\times n}$）的每个输出元素 $C_{ij} = \sum_l A_{il}B_{lj}$ 是独立内积，不同的 (i, j) 对之间完全没有数据依赖——完美适合 SIMD。这是矩阵乘法与 GPU 架构的第一层匹配：**操作的独立性**。

第二环：算术强度——量化「计算量 vs 数据量」的比值

但独立性只是一个必要条件，不是充分条件。GPU 还有第二个瓶颈：内存带宽。计算核心即使再多，如果数据喂不进来，核心也只能空等。这引出了算术强度（Arithmetic Intensity）的概念：

$$I = \frac{\text{浮点运算量（FLOPs）}}{\text{内存访问字节数（Bytes）}} \tag{3.3.1}$$

式 (3.3.1) 定义的算术强度单位是 FLOP/Byte，衡量「每读取一个字节的数据，能做多少浮点运算」。这个比值高，说明计算「划算」——读进来的数据被充分利用，内存带宽不是瓶颈；比值低，说明每读一字节只做几个操作，计算很快就完成但要等下一批数据，内存带宽是瓶颈。

现代 GPU 有两种类型的存储：片上 SRAM（Shared Memory）速度极快（带宽 TB/s 量级）但容量小（MB 量级），以及片外 HBM（High Bandwidth Memory）容量大（几十 GB）但相比 SRAM 慢一个数量级（几百 GB/s）。「算术强度」这里说的「内存访问」主要指对 HBM 的访问——片外数据搬运的速度是真正的瓶颈。

第三环：Roofline 模型——把瓶颈可视化的工具

Roofline 模型是分析硬件利用率的标准工具。它用一张图描述 GPU 核函数的实际性能上界：

$$P_{\text{actual}} = \min(\text{峰值算力（FLOP/s）}, I \times \text{峰值内存带宽（Byte/s）}) \tag{3.3.2}$$

由式 (3.3.2) 可知，当算术强度 I 低于「屋脊点」(ridge point，即峰值算力/峰值内存带宽) 时，核函数受内存带宽限制 (memory-bound)：内存带宽决定了数据流入速度，多余的计算单元无法被填满；当 I 高于屋脊点时，核函数受峰值算力限制 (compute-bound)：数据足够快，所有计算单元满载。

对 NVIDIA A100 GPU：峰值算力约 312 TFLOP/s（BF16），峰值 HBM 带宽约 2 TB/s，屋脊点约为 $312/2 = 156$ FLOP/Byte。这意味着：只要一个操作的算术强度超过 156，它就是计算受限；低于 156，它就是内存受限。

第四环：矩阵乘法的算术强度——为什么落在计算受限区

对矩阵乘法 $C = AB$（$A \in \mathbb{R}^{m\times k}$，$B \in \mathbb{R}^{k\times n}$，$C \in \mathbb{R}^{m\times n}$）：- FLOPs：$2mnk$（每个输出元素做 k 次乘加）- 内存访问字节数：读 A（mk）、读 B（kn）、写 C（mn），以 BF16（2 Byte/元素）计，总计 $2(mk + kn + mn)$ 字节

算术强度为：

$$I = \frac{2mnk}{2(mk + kn + mn)} = \frac{mnk}{mk + kn + mn} \tag{3.3.3}$$

将 $m = n = k = N$ 代入式 (3.3.3)，$I \approx \frac{N^3}{3N^2} = \frac{N}{3}$，随矩阵尺寸线性增长。对 $N = 4096$（LLM 中典型的隐层维度），$I \approx 1365$ FLOP/Byte，远超 A100 的屋脊点 156 FLOP/Byte。**大矩阵乘法深度落在计算受限区**，GPU 的算力峰值能被充分利用，内存带宽不是瓶颈。Tensor Core 正是为此设计的：以 16×16 或更大的矩阵块为基本操作单元，在片上完成高密度计算，完全隐藏内存访问延迟。

第五环：softmax 的算术强度——内存受限的短板

注意力计算不只是矩阵乘法，还有 softmax。softmax 是逐行的操作：对每一行，先找最大值，再计算 exp，再求和，再除以总和。每个元素只需要几次浮点操作（约 3-5

FLOP），但每个元素都要从 HBM 读进来、再写出去。由式 (3.3.1) 计算，算术强度约为 3-5 FLOP/Byte，比屋脊点低 30 倍以上——典型的内存受限操作。更严重的是，标准 attention 实现要对 $n \times n$ 的矩阵做 softmax，需要把整个矩阵写入 HBM、再读出来处理，HBM 访问量为 $\Theta(n^2)$。这是 attention 计算性能的真正瓶颈——不是矩阵乘法（计算受限，利用率高），而是 softmax 和中间矩阵的读写搬运（内存受限，利用率低）。这直接导致了 FlashAttention 的诞生（§3.5），其核心思路正是消除这一 $\Theta(n^2)$ 的 HBM 访问。

整条链的回头看

```
GPU: SIMD 架构，数千核心并行同一操作
  ↓ 需要：大量独立、结构相同的操作
矩阵乘法：每个输出元素 C_ij 是独立内积
  ↓ 但独立性不够，还需要：每字节数据充分利用
算术强度 I = FLOPs / Bytes
  ↓ 矩阵乘法：I  N/3，随尺寸线性增长
Roofline 模型：I > 屋脊点 → 计算受限
  ↓ 大矩阵乘法落在计算受限区
attention 的软肋：softmax，I  3-5，内存受限
  ↓ 引出 FlashAttention (§3.5)
attention FLOPs = O(n²d)，长上下文时 n² 主导
```

这条链揭示了一个深层事实：「**AI 是 GPU 的盛宴**」**有精确的数学含义**——矩阵乘法的算术强度随矩阵尺寸线性增长，恰好落在 GPU 算力受限区，让 TFLOP/s 级别的算力得以充分发挥。但注意力中的 softmax 是内存受限的短板，正是它驱动了 FlashAttention 这一系统级创新。

严格

矩阵乘法的算术强度

对两个矩阵相乘 $C = AB$，其中 $A \in \mathbb{R}^{m\times k}$，$B \in \mathbb{R}^{k\times n}$，$C \in \mathbb{R}^{m\times n}$：

- 浮点运算量（FLOPs）：$2mnk$（每个输出元素做 k 次乘加）
- 内存访问字节数（读 A、B，写 C）：$(mk + kn + mn) \times \text{sizeof(dtype)}$

算术强度定义为

$$I = \frac{2mnk}{mk + kn + mn} \quad \text{(FLOP/byte)} \tag{3.3.4}$$

将 $m = n = k = N \gg 1$ 代入式 (3.3.4)，$I \approx \frac{2N^3}{3N^2} = \frac{2N}{3}$，随矩阵尺寸线性增长。典型大矩阵乘法的算术强度远超 GPU 的「屋脊点」（ridge point，即峰值算力 / 峰值内存带宽），因此属于**计算受限**（compute-bound）操作，GPU 的算力可以充分利用。

Roofline 模型简介

屋脊线（Roofline）模型将核函数的实际性能 P（FLOP/s）建模为

$$P = \min(\text{峰值算力},\ I \times \text{峰值内存带宽}) \tag{3.3.5}$$

当 I 低于屋脊点时，性能受内存带宽限制（memory-bound）；高于屋脊点时，性能

受峰值算力限制（compute-bound）。对 A100 GPU，屋脊点约为 312 TFLOP/s/2 TB/s = 156 FLOP/byte；大矩阵乘法的算术强度通常超过此值，而逐元素操作（如 softmax 的 exp）算术强度接近 1，远低于屋脊点——对照式 (3.3.5) 即属于内存受限区间。

Attention 的 FLOPs 分析

对单头注意力，设序列长度 n，键/查询维度 d_k，值维度 d_v：

步骤	计算	FLOPs
$S = QK^T$	$\mathbb{R}^{n\times d_k} \times \mathbb{R}^{d_k\times n}$	$2n^2 d_k$
softmax	逐行归一化	$\Theta(n^2)$（低算术强度）
$O = PV$	$\mathbb{R}^{n\times n} \times \mathbb{R}^{n\times d_v}$	$2n^2 d_v$

忽略常数，总 FLOPs 为

$$\text{FLOPs}_{\text{attention}} = \Theta(n^2 d) \tag{3.3.6}$$

其中 $d = d_k = d_v$。对 h 头，乘以 h，但此时每头 $d_k = d/h$，总量不变。式 (3.3.6) 中 $O(n^2 d)$ 的二次复杂度使得长上下文（大 n）下计算量急剧增长，是 attention 的核心算力瓶颈。

与原书呼应：原书 §3.3 提到 LLaMA-3 70B 单层 attention 的算力估算及 GPU 的并行哲学，对应本节算术强度与 Roofline 模型的量化分析。

---

§3.4 RoPE：把位置编码从加法变成旋转

▶这一节在讲什么

为什么正弦位置编码不够用？加法叠加把绝对位置混入 token 表示，让模型无法直接感知「两个 token 相距多远」，且在训练长度之外几乎立即失效。

第一步：换一种思路——把位置编码进旋转角。RoPE 将每对相邻维度 (q_{2i}, q_{2i+1}) 看成复平面上的一个点，位置 m 对应旋转 $e^{im\theta_i}$，不加法，只旋转。

第二步：旋转的魔法——内积只剩位置差。由于旋转矩阵正交，$\langle \tilde{q}_m, \tilde{k}_n \rangle = q_m^T \mathcal{R}_{n-m} k_n$，绝对位置 m、n 从内积中消去，只留下相对位置 $n-m$。这正是语言学上真正有用的信号。

第三步：多频率覆盖多尺度。低维度对应高频旋转（短程依赖），高维度对应低频旋转（长程依赖），类似傅里叶分析的多尺度分解。调整底数可延伸训练外推长度。

本节要拿走的一件事：RoPE 把「相对位置」这一语言学核心信号，用旋转矩阵的正交性以数学方式硬编码进注意力内积——不依赖模型去「学」位置关系，而是直接设计进结构。

直觉

位置编码是 Transformer 的「补丁」：去掉了时序递推后，模型对 token 的顺序一无所知，必须通过某种方式把位置信息注入表示。原始 Transformer 用正弦函数生成一个固定向量加到嵌入上，这个加法方案能用，但有两个根本性缺陷：一是它编码的是绝对位置（token 在序列中的绝对位置编号），而语言学上真正重要的往往是相对位置（两个 token 相距多远）；二是它在训练最大长度之外几乎立即失效（模型从未见过那些位置的向量，无法外推）。RoPE（Rotary Position Embedding）用一个精巧的代数构造解决了这两个问题。

本节出现五个核心概念：**绝对位置编码的局限、旋转的群代数、RoPE 的复数化构造、内积只剩位置差的证明、多频率傅里叶视角**。下面把这条链铺开。

第一环：绝对位置编码的局限——为什么加法无法编码相对位置

正弦位置编码的方案是：对位置 m，生成一个固定向量 $\mathrm{PE}(m) \in \mathbb{R}^d$，加到 token 嵌入上：$x'_m = x_m + \mathrm{PE}(m)$。然后注意力打分变为：

$$(x'_m W^Q)(x'_n W^K)^T = (x_m + \mathrm{PE}(m))W^Q((x_n + \mathrm{PE}(n))W^K)^T \tag{3.4.1}$$

展开式 (3.4.1) 后会有四项：$x_m W^Q (W^K)^T x_n^T$（纯语义）、$\mathrm{PE}(m) W^Q (W^K)^T x_n^T$（查询方的位置 × 键的语义）、$x_m W^Q (W^K)^T \mathrm{PE}(n)^T$（查询方语义 × 键的位置）、$\mathrm{PE}(m) W^Q (W^K)^T \mathrm{PE}(n)^T$（两个绝对位置的交叉项）。式 (3.4.1) 的这四项里，没有任何一项是只关于 $m-n$（位置差）的函数——绝对位置 m 和 n 都混在里面，无法分离。模型如果想利用相对位置信号，必须自己从四项的混合里学会「消去」绝对位置，这是额外的学习负担，且通常学不干净。

更严重的是训练外推问题。如果训练时最大序列长度是 2048，模型从未见过位置 2049 及以后的正弦向量。在推理时遇到更长序列，这些位置的编码向量在训练分布之外，模型的表现会急剧退化。

第二环：旋转的代数思路——用群结构天然编码「差」

解决方案的关键洞察是：**如果位置信息以乘法而非加法形式注入，内积就可以天然消去绝对位置**。

群论给出了一个精确的框架。旋转群 $SO(2)$（二维旋转）有一个核心性质：对旋转角度 θ_1 和 θ_2 对应的旋转矩阵 R_{θ_1} 和 R_{θ_2}，有 $R_{\theta_1}^T R_{\theta_2} = R_{\theta_2 - \theta_1}$（两个旋转相乘等于角度相减的旋转）。这意味着：如果对位置 m 的 query 向量施加旋转 $R_{m\theta}$，对位置 n 的 key 向量施加旋转 $R_{n\theta}$，则它们的内积：

$$(R_{m\theta} q)^T (R_{n\theta} k) = q^T R_{m\theta}^T R_{n\theta} k = q^T R_{(n-m)\theta} k \tag{3.4.2}$$

式 (3.4.2) 中绝对位置 m 和 n 精确消去，只剩下相对位置 $n-m$！这不是近似，是旋转矩阵正交性的精确代数恒等式。加法做不到这一点——加法没有这种「两项之差」的代数结构。

第三环：RoPE 的复数化构造——把高维向量拆成二维旋转对

如何把这个思路推广到高维向量（d 维）？RoPE 的方案是：把 d 维向量 $q \in \mathbb{R}^d$ 看成 $d/2$ 个复数，每对相邻维度 (q_{2i}, q_{2i+1}) 对应一个复数 $z_i^q = q_{2i} + \mathrm{i}\, q_{2i+1}$。对位于位置 m 的 token，每对分量乘以单位复数 $e^{\mathrm{i} m\theta_i}$：

$$\tilde{z}_i^q = z_i^q \cdot e^{\mathrm{i}m\theta_i} = (q_{2i} + \mathrm{i}\, q_{2i+1})(\cos(m\theta_i) + \mathrm{i}\sin(m\theta_i)) \tag{3.4.3}$$

将式 (3.4.3) 实数化展开，等价于对每对分量施加一个二维旋转矩阵：

$$\begin{pmatrix} \tilde{q}_{2i} \\ \tilde{q}_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \begin{pmatrix} q_{2i} \\ q_{2i+1} \end{pmatrix} \tag{3.4.4}$$

对整个 d 维向量，RoPE 变换是一个块对角旋转矩阵 $\mathcal{R}_m = \mathrm{blockdiag}(R_m^{(0)}, R_m^{(1)}, \dots, R_m^{(d/2-1)})$（即式 (3.4.4) 的逐对组合），每个块是一个独立频率的二维旋转。由于每个块都满足旋转矩阵的正交性，第二环的代数性质直接推广到高维：

$$\langle \mathcal{R}_m q_m, \mathcal{R}_n k_n \rangle = q_m^T \mathcal{R}_m^T \mathcal{R}_n k_n = q_m^T \mathcal{R}_{n-m} k_n \tag{3.4.5}$$

式 (3.4.5) 表明内积只依赖位置差 $n - m$，精确成立。

第四环：多频率傅里叶视角——为什么不同维度用不同频率

如果所有 $d/2$ 对维度都使用同一个频率 θ，每隔 $2\pi/\theta$ 个位置旋转一圈，模型会无法区分相差整数圈的两个位置（比如位置 5 和位置 $5 + 2\pi/\theta$）——产生「位置混叠」。RoPE 用不同的频率参数：

$$\theta_i = 10000^{-2i/d}, \quad i = 0, 1, \dots, d/2 - 1 \tag{3.4.6}$$

按式 (3.4.6) 的参数设定，低下标 i（如 $i = 0$）对应 $\theta_0 = 1$，旋转快（高频）——每一步旋转 1 弧度，约 6 步转一圈，对短程依赖敏感；高下标 i（如 $i = d/2 - 1$）对应 $\theta_{d/2-1} = 10000^{-1} = 0.0001$，旋转极慢（低频）——要 $2\pi/0.0001 \approx 62832$ 步才转一圈，对长程依赖编码有效。

这与傅里叶分析完全类似：不同频率的正弦波组合可以表示任意尺度的信号。高频分量感知局部差异（相邻 token），低频分量感知全局结构（远距离 token）。$d/2$ 个不同频率共同构成了一个「位置尺度的频谱」，让模型在 $d/2$ 个独立的尺度上同时感知位置关系。

第五环：外推能力——频率调整如何延伸上下文长度

正弦位置编码在训练长度外失效，是因为那些绝对位置的向量没有被模型见过。RoPE 没有这个问题：旋转是一个连续的函数，位置 $m = 10000$ 只是角度 $m\theta_i$ 稍微大了一些，不存在「未见过的向量」的问题。但实际上，当序列长度远超训练时，某些频率的旋转角度进入训练中从未出现过的区间（比如相对位置 $n - m$ 超过训练最大长度），高频分量会出现混叠。

解决方案是调整频率底数（把 10000 换成更大的值，如 500000），让低频分量在更大的相对位置范围内仍然处于「未完成一圈旋转」的状态——模型对这段范围的相对位置就有更好的区分能力。YaRN、LongRoPE 等方法的数学本质正是对不同频率段做差异化的频率插值，让模型能够在不重新训练的情况下处理数倍于训练长度的序列。

整条链的回头看

绝对位置编码（加法）

```
  ↓ 内积展开有四项，绝对位置无法从内积中消去
需要：能让内积只剩位置差的编码方式
  ↓ 旋转群 SO(2) 的性质：R_a^T R_b = R_{b-a}
旋转能精确消去绝对位置 → 设计 RoPE
  ↓ d 维向量 → 拆成 d/2 对，每对独立旋转
块对角旋转矩阵 R_m，位置 m → 角度 m*theta_i
  ↓ 内积 = q^T R_{n-m} k，只剩位置差
多频率：theta_i = 10000^{-2i/d}
  ↓ 高频感知短程，低频感知长程（傅里叶视角）
调整底数 → 训练外长度外推
```

这条链揭示了一个深层事实：**RoPE 不是「用旋转来凑一个好用的位置编码」，而是旋转群代数结构的直接应用**。旋转的群性质（角度可加性、正交性）天然给出了「内积只剩差」的性质，多频率设计则是傅里叶分析的自然推广。理解这条链，就理解了为什么 RoPE 能外推而正弦编码不能——不是经验调优，是代数结构的必然结果。

严格

正弦位置编码的形式

对位置 pos 和嵌入维度下标 $2i, 2i+1$：

$$\mathrm{PE}(\mathrm{pos}, 2i) = \sin\Big(\frac{\mathrm{pos}}{10000^{2i/d}}\Big), \quad \mathrm{PE}(\mathrm{pos}, 2i+1) = \cos\Big(\frac{\mathrm{pos}}{10000^{2i/d}}\Big) \tag{3.4.7}$$

编码后 $x'_{\mathrm{pos}} = x_{\mathrm{pos}} + \mathrm{PE}(\mathrm{pos})$（式 (3.4.7)），绝对位置以加法混入嵌入，无法分离相对位置信息；这与式 (3.4.13) 中 RoPE 仅剩位置差的结果形成对比。

RoPE 的复数化表示

RoPE 将 d 维向量 $q \in \mathbb{R}^d$ 看成 $d/2$ 个复数，第 i 对分量 (q_{2i}, q_{2i+1}) 对应复数 $z_i^q = q_{2i} + \mathrm{i}\, q_{2i+1}$。对位于位置 m 的 token，每对分量乘以单位复数 $e^{\mathrm{i}m\theta_i}$：

$$\tilde{z}_i^q = z_i^q \cdot e^{\mathrm{i}m\theta_i} \tag{3.4.8}$$

频率参数与式 (3.4.6) 一致：

$$\theta_i = 10000^{-2i/d}, \quad i = 0, 1, \ldots, \frac{d}{2} - 1 \tag{3.4.9}$$

低下标 i 对应高频（旋转快），高下标对应低频（旋转慢），覆盖不同尺度的位置信息。

等价实数旋转矩阵

对每对分量，乘以 $e^{\mathrm{i}m\theta_i}$ 等价于施加二维旋转矩阵

$$R_m^{(i)} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \tag{3.4.10}$$

对整个 d 维向量，将式 (3.4.10) 的每个二维旋转块拼装为块对角旋转矩阵

$$\mathcal{R}_m = \text{blockdiag}\Big(R_m^{(0)}, R_m^{(1)}, \dots, R_m^{(d/2-1)}\Big) \in \mathbb{R}^{d\times d} \tag{3.4.11}$$

位置 m 处的 query 和 key 在施加 RoPE 后分别为

$$\tilde{q}_m = \mathcal{R}_m\, q_m, \quad \tilde{k}_n = \mathcal{R}_n\, k_n \tag{3.4.12}$$

核心性质：内积只依赖相对位置

由于式 (3.4.11) 中旋转矩阵是正交矩阵（$\mathcal{R}_m^T = \mathcal{R}_m^{-1} = \mathcal{R}_{-m}$），且旋转角度可加（$\mathcal{R}_m^T\mathcal{R}_n = \mathcal{R}_{n-m}$），将式 (3.4.12) 代入得 attention score 为

$$\langle \tilde{q}_m, \tilde{k}_n\rangle = (\mathcal{R}_m q_m)^T(\mathcal{R}_n k_n) = q_m^T\mathcal{R}_m^T\mathcal{R}_n k_n = q_m^T\mathcal{R}_{n-m}k_n = \langle q_m,\, \mathcal{R}_{n-m}\, k_n\rangle \tag{3.4.13}$$

式 (3.4.13) 的结果仅依赖位置差 $n-m$，而不依赖绝对位置 m 或 n。这一性质使得模型在推理时天然感知相对距离，并且——通过调整式 (3.4.9) 的频率基数 θ_i 的底数（将 10000 替换为更大的值）或对不同频率段做差异化插值——可以向训练长度以外外推，这是 YaRN 等长上下文扩展方法的数学基础。

与正弦编码的对比如下：

属性	正弦位置编码	RoPE
编码方式	加法叠加	乘法旋转
位置信号	绝对位置	相对位置
内积形式	依赖绝对位置	仅依赖 $n-m$
训练外外推	直接失效	可通过频率调整延伸

与原书呼应：原书 §3.4 介绍了 RoPE 的旋转思想与「位置差从指数里掉出来」的性质，对应本节旋转矩阵正交性的严格推导。

§3.5 FlashAttention：当注意力撞上内存墙

▶这一节在讲什么

为什么标准 attention 慢？不是因为 FLOPs 太多，而是因为 $n \times n$ 的注意力矩阵要在慢速 HBM 和快速 SRAM 之间反复搬运，内存带宽成了瓶颈。

第一步：问题出在写-读-写-读的搬运链上。标准实现先把 $S = QK^T$ 写到 HBM，再读出来做 softmax，再写回，再读出来乘 V——每步都过 HBM，HBM 访问量 $\Theta(n^2)$。

第二步：FlashAttention 的思路是「不出芯片」。把 Q, K, V 切成能放入 SRAM 的小块，在片上完成「乘 + softmax + 再乘」全流程，只将最终输出写回 HBM 一次。

第三步：难点在 softmax 需要全局统计量。FlashAttention 用「在线 softmax」解决：用滚动最大值 m_j 和指数和 ℓ_j 递推，无需提前知道全局最大值，可以边读块边更新输出。

本节要拿走的一件事：FlashAttention 的 FLOPs 没有减少，减少的是 HBM 读写量（从 $\Theta(n^2)$ 降至接近线性）——这是一次纯粹的内存工程优化，不改变数学结果，只改变计算路径。

直觉

§3.3 揭示了注意力计算的软肋：式 (3.3.6) 中 softmax 是内存受限操作（算术强度远低于屋脊点，见式 (3.3.5)），标准 attention 实现需要把 $n \times n$ 的矩阵多次搬入搬出 HBM，内存带宽成为真正的瓶颈，而不是 FLOPs 本身。FlashAttention 是针对这个瓶颈的精准外科手术：在不改变数学结果的前提下，重新安排计算顺序，把所有中间量都留在 SRAM 里，只有最终结果才写回 HBM。理解这一方法需要搞清楚：为什么分块计算是可行的，以及 softmax 的全局归一化需求如何被「在线 softmax」精确规避。

本节出现五个核心概念：**HBM 与 SRAM 的速度差异与写读链条问题、分块（tiling）的思路、softmax 的全局依赖障碍、在线 softmax 递推、内存与速度的共赢**。下面把这条链铺开。

第一环：写-读链条——标准 attention 的内存搬运代价

标准 attention 实现的执行序列如下：

1. 计算 $S = QK^T/\sqrt{d_k} \in \mathbb{R}^{n\times n}$，结果写入 HBM（SRAM 放不下这个矩阵）；
2. 从 HBM 读出 S，逐行做 softmax，得到 $P \in \mathbb{R}^{n\times n}$，写回 HBM；
3. 从 HBM 读出 P 和 V，计算 $O = PV \in \mathbb{R}^{n\times d_v}$，写回 HBM。

每次写入/读出 $n \times n$ 矩阵，访问量为 $\Theta(n^2)$ 个元素。对 $n = 8192$，这意味着每次注意力计算需要搬运约 $8192^2 = 6.7 \times 10^7$ 个 FP16 元素，约 134 MB。这看似不多，但注意力计算本身的 FLOPs 并不足以支撑这么多内存访问——算术强度太低，GPU 的计算单元大量空等。

更根本的问题是：SRAM（片上缓存）的容量约为 20 MB（A100），远小于 $n \times n$ 矩阵（134 MB）。所以中间矩阵必须存放在 HBM，而 HBM 的带宽只有约 2 TB/s——相比 SRAM 的约 19 TB/s 慢 10 倍。每次从 HBM 读写 $n \times n$ 矩阵，就是在用最慢的存储层做最频繁的数据交换。

第二环：分块（Tiling）思路——让计算在 SRAM 内完成

FlashAttention 的核心思路是：把 Q, K, V 切成能放入 SRAM 的小块，对每个（Q块, K块, V块）组合在片上完成「矩阵乘 + softmax + 再乘」的完整流程，不写中间矩阵到 HBM。

具体地，把 K 和 V 按行切成 T 个块，每块大小 $B \times d$（B 足够小，使 $B \times d$ 能放入 SRAM）。外层循环遍历 Q 的行块，内层循环遍历 K, V 的列块。对每个 (i, j) 对（Q 的第 i 行块，K, V 的第 j 列块），在 SRAM 内完成：- 计算局部注意力分数 $S_{ij} = Q_i K_j^T / \sqrt{d_k}$ - 更新在线 softmax 统计量 - 用局部归一化权重更新输出 O_i

整个过程中，$n \times n$ 的注意力矩阵 S 从未完整出现——它被分块计算并立即消费，中间结果从不落地到 HBM。

第三环：softmax 的全局依赖障碍——为什么分块不是显然的

分块听起来简单，但有一个关键障碍：softmax 需要全局统计量。对第 i 行的 softmax：

$$P_{ij} = \frac{\exp(S_{ij})}{\sum_l \exp(S_{il})} \tag{3.5.1}$$

式 (3.5.1) 中分母 $\sum_l \exp(S_{il})$ 需要这一行所有 n 个元素的信息。如果分块处理，处理第 j 个 K 块时还不知道其他块的值，无法归一化。这是标准 softmax 的「两遍扫描」需求：第一遍找全局最大值（数值稳定性），第二遍做 exp 和归一化。

在数值稳定版本中（减去最大值防止 exp 溢出），softmax 公式为：

$$P_{ij} = \frac{\exp(S_{ij} - \max_l S_{il})}{\sum_l \exp(S_{il} - \max_l S_{il})} \tag{3.5.2}$$

式 (3.5.2) 中分母和分子都需要全局最大值 $\max_l S_{il}$，必须先完成全局扫描才能计算。这阻碍了分块计算。

第四环：在线 softmax——把两遍扫描变成一遍递推

FlashAttention 用在线 softmax 递推解决这个问题。核心思想是：维护两个滚动统计量，在逐块处理时实时更新，处理完所有块后统计量恰好等于全局精确值，**数学上与两遍扫描完全等价**。

设已处理前 $j-1$ 个 K 块，维护：- $m_{j-1} \in \mathbb{R}$：前 $j-1$ 块中见过的最大分数（数值稳定的移位基准）- $\ell_{j-1} \in \mathbb{R}$：移位后的指数和（归一化分母的运行值）- $O_{j-1} \in \mathbb{R}^{d_v}$：当前输出的未完全归一化版本

处理第 j 块时，令该块的局部最大值 $\tilde{m}_j = \max(S_j)$，局部指数和 $\tilde{\ell}_j = \sum_l \exp(S_{jl} - \tilde{m}_j)$。更新：

$$m_j = \max(m_{j-1}, \tilde{m}_j) \tag{3.5.3}$$

$$\ell_j = e^{m_{j-1} - m_j} \ell_{j-1} + e^{\tilde{m}_j - m_j} \tilde{\ell}_j \tag{3.5.4}$$

$$O_j = \text{diag}(\ell_j)^{-1}\Big(\text{diag}(\ell_{j-1})e^{m_{j-1}-m_j}O_{j-1} + e^{\tilde{m}_j-m_j}\tilde{P}_jV_j\Big) \tag{3.5.5}$$

式 (3.5.3)–(3.5.5) 的递推精妙在于：每次遇到更大的局部最大值时，用比例因子 $e^{m_{\text{old}}-m_{\text{new}}}$ 重新缩放历史统计量——这相当于「追溯修正」之前看到的所有元素，把它们的移位基准更新到新的全局最大值。处理完所有块后，m_T 就是全局最大值，ℓ_T 就是全局指数和，O_T 就是精确的注意力输出。全程不需要存储 $n \times n$ 矩阵，只需要两个标量 m_j, ℓ_j 和一个 d_v 维向量 O_j。

第五环：内存与速度共赢——代价分析

这个方案的复杂度如何变化?

- **FLOPs 不变**：FlashAttention 计算的数学结果与标准 attention 完全相同，只是改变了计算路径。对每个 (Q块, K块) 对，矩阵乘的工作量与标准实现一样，总 FLOPs 仍为式 (3.3.6) 中的 $\Theta(n^2d)$。
- **HBM 访问量大幅下降**：不再需要写读 $n\times n$ 的中间矩阵。Q, K, V 各读一次 ($\Theta(nd)$)，O 写一次 ($\Theta(nd)$)，中间结果在 SRAM 内流转，不落地 HBM。HBM 访问量从 $\Theta(n^2)$ 降至 $\Theta(nd \cdot n/M)$（M 为 SRAM 大小），实践中减少 4-16 倍。
- **显存占用从 $O(n^2)$ 降至 $O(n)$**：不再需要存储整个注意力矩阵，只需要 m_j, ℓ_j 两个标量和当前输出向量。这使得更长的上下文在相同显存下成为可能。

反向传播时需要重新计算注意力权重（因为不再存储 $n \times n$ 矩阵），这会增加约 1 倍的 FLOPs，但 HBM 节省的带宽收益远大于这个代价，端到端速度仍然大幅提升（实测 2-4 倍）。

整条链的回头看

```
标准 attention：写 S → 读 S → softmax → 写 P → 读 P → 乘 V
  ↓ HBM 访问 Θ(n²)，内存受限，计算单元空等
瓶颈：n×n 矩阵放不入 SRAM，被迫用 HBM
  ↓ 解法：分块计算，让中间结果留在 SRAM
分块障碍：softmax 需要全局最大值（两遍扫描）
  ↓ 解法：在线 softmax 递推（滚动最大值 + 指数和）
递推等价性：m_j， _j 更新   全局精确 softmax
  ↓ 结果
FLOPs 不变 + HBM 访问从 Θ(n²) 降至接近线性
  ↓ attention 从内存受限重新变为计算受限
GPU 利用率大幅提升，端到端 2-4 倍加速
```

这条链揭示了一个深层事实：**FlashAttention 的创新不在数学，而在于识别了「softmax 的全局依赖是分块障碍」这一核心问题，并用在线递推精确消除了这一障碍**。它证明了在不改变任何数学结果的前提下，通过重新安排计算顺序，可以把一个内存受限问题变回计算受限——这是系统级优化与数学保证完美结合的范例。

严格

标准实现的内存访问代价

设序列长度 n，维度 d，标准实现的 HBM 读写量（以元素数计）：

1. 计算 $S = QK^T \in \mathbb{R}^{n\times n}$，写回 HBM：$\Theta(n^2)$

2. 从 HBM 读 S，计算 softmax 得 P，写回 HBM：$\Theta(n^2)$
3. 从 HBM 读 P（$\Theta(n^2)$）和 V（$\Theta(nd)$），计算 $O = PV$，写回

总 HBM 访问量 $\Theta(n^2 + nd)$，长序列下 n^2 项主导，使 attention 成为**内存受限**操作。

在线 softmax（online softmax）的递推公式

设将 K, V 按行切成 T 个块，第 j 块对应局部注意力分数向量 $s_j \in \mathbb{R}^B$（B 为块大小）。维护两个滚动统计量：

- $m_j \in \mathbb{R}$：前 j 块中 s 分量的最大值（用于数值稳定移位）
- $\ell_j \in \mathbb{R}$：前 j 块经移位后的指数和

递推更新为

$$m_j = \max(m_{j-1},\ \tilde{m}_j) \tag{3.5.6}$$

$$\ell_j = e^{m_{j-1} - m_j}\,\ell_{j-1} + e^{\tilde{m}_j - m_j}\,\tilde{\ell}_j \tag{3.5.7}$$

其中 $\tilde{m}_j = \max(s_j)$，$\tilde{\ell}_j = \sum_l \exp(s_{jl} - \tilde{m}_j)$ 是当前块的局部指数和。

输出的分块更新

维护运行输出 $O_j \in \mathbb{R}^{d_v}$，加载第 j 块时更新为

$$O_j = \mathrm{diag}(\ell_j)^{-1}\Big(\mathrm{diag}(\ell_{j-1})\, e^{m_{j-1} - m_j} O_{j-1} + e^{\tilde{m}_j - m_j}\,\tilde{P}_j V_j\Big) \tag{3.5.8}$$

其中 $\tilde{P}_j = \exp(s_j - \tilde{m}_j \mathbf{1})$ 是当前块的未归一化注意力权重，V_j 是对应的 Value 块。可以验证，按式 (3.5.6)–(3.5.8) 遍历所有块后 O_T 恰好等于完整的归一化注意力输出，且与式 (3.5.2) 的数值稳定 softmax 结果严格相等。

复杂度分析

量	标准 Attention	FlashAttention
FLOPs	$\Theta(n^2 d)$	$\Theta(n^2 d)$（不变）
HBM 读写（元素数）	$\Theta(n^2 + nd)$	$\Theta(n^2 d / M)$（M 为 SRAM 大小）
需显式存储 $n \times n$ 矩阵	是	否
额外显存开销（除 O 外）	$\Theta(n^2)$	$\Theta(n)$（仅存 m_j, ℓ_j）

FlashAttention 将内存复杂度从 $O(n^2)$ 降至 $O(n)$（仅需存储式 (3.5.6)–(3.5.7) 中的标量 m_j, ℓ_j），HBM 带宽使用减少 4–16 倍，使 attention 从内存受限转变为计算受限，GPU Tensor Core 的利用率大幅提升。FLOPs 仍为式 (3.3.6) 的 $O(n^2 d)$，即注意力的算力复杂度没有改变，改变的只是其对硬件内存层级的友好度。

与原书呼应：原书 §3.5 给出了在线 softmax 的递推公式及 HBM 访问复杂度分析，对应本节的逐步推导与复杂度对比表。

---

§3.6 注意力的真实身份：关联记忆，而非「注意力」

▶这一节在讲什么

为什么「注意力」这个名字有误导性？认知心理学的注意力是有限资源的主动分配；而 softmax attention 在数学上更接近「内容寻址的关联记忆检索」——给定查询模式，在一组存储模式中检索最接近的内容。

第一步：经典 Hopfield 网络的能量函数。经典版本（1982）用二次能量函数存储模式，存储容量约 $0.14d$，线性于维度。

第二步：现代 Hopfield 网络的升级。将能量函数改为 log-sum-exp 形式，求不动点条件得到更新规则 $\xi^{\text{new}} = X^T \text{softmax}(\beta X\xi)$——这与 softmax attention 公式在数学上完全等价（令 $\xi = Q_i$，$X = K$，$\beta = 1/\sqrt{d_k}$）。

第三步：存储容量从线性变指数。现代 Hopfield 网络的容量约为 $O(\exp(\alpha^2 d/2))$，指数级于维度。这解释了为何 Transformer 能以有限参数存储和检索海量知识。

本节要拿走的一件事：softmax attention 是现代 Hopfield 网络一步不动点迭代的精确数学等价——「注意力」的本质是指数容量关联记忆的检索，而非心理学意义上的资源竞争。

直觉

「注意力」这个名字来自认知科学：人类的注意力是有限资源，面对复杂场景时，大脑选择性地将处理能力集中到某些刺激上，忽略其他。这是一种**竞争分配机制**。然而，softmax attention 在数学上并不对应这种语义——它更像是一种**内容寻址的关联记忆检索**：给定一个查询模式，在一个存储了大量模式的系统中，找到最接近的内容并加权汇总。这两种数学结构有本质区别，而 Hopfield 网络理论提供了精确的数学桥梁，揭示了注意力的「真实身份」。

本节出现五个核心概念：**关联记忆与内容寻址**、**经典 Hopfield 网络的二次能量函数与线性容量**、**能量函数的 log-sum-exp 改造**、**现代 Hopfield 网络的不动点等价于 softmax attention**、**指数容量的跃升**。下面把这条链铺开。

第一环：关联记忆与内容寻址——比「注意力」更准确的比喻

关联记忆（Associative Memory）是一种存储和检索模式的系统：存储阶段将若干「记忆模式」$\xi^1, \xi^2, \ldots, \xi^N$ 编码进系统权重；检索阶段给定一个「查询模式」ξ（可能不完整或有噪声），系统通过迭代更新收敛到最近的存储模式。这是一种**内容寻址**（content addressing）：不用地址索引，而用内容相似度找到匹配项。

这个框架与注意力机制的结构高度吻合：K 的行向量就是存储模式，Q_i 就是查询模式，注意力计算就是在存储模式集合中按相似度加权检索。关键问题是：这种关联记忆系统的存储容量是多少？——即同时存储多少个不冲突的模式，使得查询时能精确检索到目标？（见式 (3.6.6) 的量化答案）

第二环：经典 Hopfield 网络——线性容量的限制

经典 Hopfield 网络（1982，John Hopfield）用一个二次能量函数存储 N 个 d 维二值模式 $\xi^\mu \in \{-1,+1\}^d$：

$$E_{\text{classic}} = -\frac{1}{2}\xi^T W \xi, \quad W = X^T X - NI \tag{3.6.1}$$

其中 $X \in \mathbb{R}^{N\times d}$ 是存储模式矩阵，I 是单位矩阵。检索时从查询 ξ 出发，按能量梯度下降更新：$\xi_i^{\text{new}} = \text{sgn}((W\xi)_i)$，迭代直到收敛到能量局部极小（即某个存储模式）。

由式 (3.6.1) 的二次能量函数，经典 Hopfield 网络的存储容量约为 $0.14d$（随机模式情形下），即最多约能可靠存储 $0.14d$ 个 d 维模式。更多模式时，不同模式的权重矩阵会产生「串扰」（crosstalk），检索会陷入错误的吸引子（混淆了多个模式的叠加态）。这个线性容量是式 (3.6.1) 二次能量函数的根本限制：能量极小值的数量由矩阵 W 的谱结构决定，而对称矩阵的「局部极小」数量在高维空间中是有界的。

第三环：能量函数的 log-sum-exp 改造——从二次到指数

2020 年，Ramsauer 等（ICLR 2021）提出现代 Hopfield 网络，把能量函数改为 log-sum-exp 形式：

$$E = -\,\text{lse}(\beta,\, X\xi) + \frac{1}{2}\xi^T\xi + \frac{1}{2\beta}\log N + C \tag{3.6.2}$$

其中 $\text{lse}(\beta, z) = \beta^{-1}\log\sum_i \exp(\beta z_i)$ 是 log-sum-exp 算子，$\beta > 0$ 是逆温度参数（类比物理中的温度倒数），C 是与 ξ 无关的常数。

这个改造的直觉是什么？经典的二次能量 $-\xi^T W\xi = -\xi^T X^T X\xi = -\|X\xi\|^2$ 在极小值处对应向量 $X\xi$ 的 ℓ^2 范数最大，等价于找最近似的存储模式（线性近似）。式 (3.6.2) 中 log-sum-exp 能量则是 ℓ^2 范数的「指数化版本」——它在与某个存储模式几乎完全对齐时，$\exp(\beta z_i)$ 对应项会以指数速率主导，形成一个极其尖锐的吸引子。尖锐的吸引子意味着不同存储模式之间的「分隔」更清晰，更多模式可以共存而不串扰——这是容量从线性（式 (3.6.1)）跃升至指数（式 (3.6.6)）的数学根源。

第四环：不动点条件推导——softmax attention 的精确等价

对现代 Hopfield 网络的能量函数，求不动点条件（$\partial E/\partial\xi = 0$）：

$$\frac{\partial E}{\partial \xi} = -X^T\text{softmax}(\beta X\xi) + \xi = 0 \tag{3.6.3}$$

（这里用到了 $\frac{\partial}{\partial\xi}\text{lse}(\beta, X\xi) = X^T\text{softmax}(\beta X\xi)$，以及 $\frac{\partial}{\partial\xi}\frac{1}{2}\xi^T\xi = \xi$。）

由式 (3.6.3) 得不动点条件 $\xi^* = X^T\text{softmax}(\beta X\xi^*)$，给出一个不动点迭代更新规则：

$$\xi^{\text{new}} = X^T\,\text{softmax}(\beta X\xi) \tag{3.6.4}$$

现在令 $\xi \leftarrow Q_i$（某个查询向量），$X \leftarrow K$（键矩阵的行向量集合），$\beta = 1/\sqrt{d_k}$，则：

$$\xi^{\text{new}} = K^T \operatorname{softmax}\left(\frac{KQ_i}{\sqrt{d_k}}\right) = K^T \operatorname{softmax}\left(\frac{Q_i \cdot k_j}{\sqrt{d_k}}\right)_j \tag{3.6.5}$$

式 (3.6.5) 精确等价于对查询 Q_i 的单头注意力输出（在 Key = Value 的特殊情形下）！因此，将式 (3.2.10) 与式 (3.6.4) 对比，softmax attention 是现代 Hopfield 网络的**一步不动点迭代**——每次前向传播相当于从查询向量出发，做一步朝不动点的跳跃，目标是收敛到「键向量空间中最像查询的那个吸引子」。

第五环：指数容量——高维空间的记忆奇迹

现代 Hopfield 网络的存储容量是多少？Ramsauer 等证明，对充分分离的存储模式（模式间 ℓ^2 距离至少为 Δ），现代版本能可靠存储的模式数约为：

$$C_{\text{modern}} = O\left(\exp\left(\frac{\alpha^2 d}{2}\right)\right) \tag{3.6.6}$$

其中 α 与模式分离度 Δ 和维度 d 的比值相关。这是关于 d 的指数函数，而经典版本的容量只有 $O(d)$。将 $d = 128$（典型的 key 维度），$\alpha = 1/\sqrt{d}$ 代入式 (3.6.6)，容量约为 $e^{64} \approx 6 \times 10^{27}$。当然，这是理论上界，实际有效容量取决于模式的相关性和训练动态，但即使打个极大的折扣，仍然远超线性计数的直觉。

这个指数容量有一个深刻的工程含义：**Transformer 模型的「知识存储」能力随维度指数增长**。参数矩阵 W^K 的每一行是一个关联记忆的存储模式，推理时每次注意力计算是一次高效的模式检索。这解释了为什么更大的模型（更大的 d）不仅是「更多参数」——它们在关联记忆的意义上具有指数倍更强的知识存储能力，而不只是线性倍。

整条链的回头看

```
认知科学的「注意力」：有限资源竞争分配
  ↓ 数学上不准确；更准确的是：
关联记忆：查询模式 → 在存储模式中内容寻址检索
  ↓ 经典 Hopfield 网络 (1982)
二次能量函数 E = -½ ^T W
  ↓ 容量  0.14d，线性于维度
改造：E = -lse( , X ) + ½|| ||²
  ↓ 不动点条件 E/  = 0
_new = X^T softmax( X )
  ↓ 令 =Q_i, X=K, =1/√d_k
精确等价于单头 softmax attention
  ↓ 能量函数指数化 → 吸引子更尖锐
容量从 O(d) 跃升至 O(exp( ²d/2))
```

这条链揭示了一个深层事实：**softmax attention 不是一个新颖的神经网络构件，而是一种有 40 年历史的关联记忆系统的现代重新发现**。经典 Hopfield 网络的能量函数改造——从二次到 log-sum-exp——恰好等价于把 sgn 激活换成 softmax，把离散二值记忆换成连续实数记忆，而这个改造让容量从线性跃升至指数。理解这条链，就理解了为什么 Transformer 能以有限参数「记住」如此海量的世界知识。

严格

经典 Hopfield 网络的能量函数

经典 Hopfield 网络（1982）的能量函数为

$$E_{\text{classic}} = -\frac{1}{2}\xi^T W \xi, \quad W = X^T X - NI \tag{3.6.7}$$

其中 $\xi \in \{-1, +1\}^d$ 为查询状态，$X \in \mathbb{R}^{N\times d}$ 为存储模式，存储容量约为 $0.14d$（线性于维度）。

现代 Hopfield 网络的能量函数

Ramsauer 等（ICLR 2021）将能量函数改为

$$E = -\operatorname{lse}(\beta,\, X\xi) + \frac{1}{2}\xi^T\xi + \frac{1}{2\beta}\log N + C \tag{3.6.8}$$

其中 $\operatorname{lse}(\beta, z) = \beta^{-1}\log\sum_i \exp(\beta z_i)$ 是 log-sum-exp 算子，$\beta > 0$ 是逆温度参数，C 是与 ξ 无关的常数。对式 (3.6.8) 求不动点条件 $\partial E/\partial \xi = 0$（即式 (3.6.3) 的严格形式），得到更新规则

$$\xi^{\text{new}} = X^T \operatorname{softmax}(\beta X \xi) \tag{3.6.9}$$

与 softmax Attention 的等价性

令 $\xi \leftarrow Q_i$（某行查询向量），$X \leftarrow K$（键矩阵的行向量集合），$\beta = 1/\sqrt{d_k}$，则

$$\xi^{\text{new}} = K^T \operatorname{softmax}\left(\frac{KQ_i^T}{\sqrt{d_k}}\right) \tag{3.6.10}$$

式 (3.6.10) 恰好等价于式 (3.2.10) 中对单个 query Q_i 的 attention 输出（忽略 Key 与 Value 不同这一细节；原始论文中取 $K = V^T$ 的情形即精确等价）。因此，**一次 softmax attention 运算在数学上等价于以 $\beta = 1/\sqrt{d_k}$ 对式 (3.6.9) 做一步不动点迭代**。

存储容量的提升

与式 (3.6.7) 的线性容量相比，现代 Hopfield 网络的存储容量（式 (3.6.11)）提升至

$$C_{\text{modern}} = O\left(\exp\left(\frac{\alpha^2 d}{2}\right)\right) \tag{3.6.11}$$

（α 为模式间最小分离距离的某个函数），即指数级于维度。这解释了为何大型 Transformer 能以有限参数量存储和检索海量知识模式：权重矩阵 W^K 的行向量构成一个指数容量的关联记忆索引，推理时的「理解」本质是高维模式检索的级联。

与原书呼应：原书 §3.6 引用了 Ramsauer 等的等价性证明，对应本节能量函数推导与不动点更新规则的显式对应。

---

§3.7 一个被忽略的争论：注意力权重 ≠ 解释

▶这一节在讲什么

为什么不能用注意力权重解释模型？因为注意力权重 α_{ij} 只是计算图中的一个中间量——它衡量的是 Value 向量的线性组合系数，而非输入 token 对最终预测的「因果贡献」。

第一步：权重衡量的是什么。α_j 告诉你「第 j 个 Value 向量被赋予了多少权重」，但 Value 向量本身已经经过线性投影，与原始 token x_j 不是同一件事。

第二步：梯度归因走的是不同路径。x_j 对输出 y 的梯度，不仅经过 Value 分支，还通过 x_j 对 q、k_j、甚至其他位置 k_l 的影响传播，路径复杂得多。

第三步：Jain & Wallace 的反例。保持输出 y 近似不变的前提下，可以构造与原始分布 ℓ^1 距离很大的替代注意力分布 α'——当 Value 矩阵行向量相关性高时，不同权重混合可得到相似输出。

本节要拿走的一件事：α_j 大不等于「第 j 个 token 对预测起决定性作用」；真正的因果归因必须通过梯度方法或激活路径分析（circuit analysis）进行。

直觉

注意力权重可视化是深度学习可解释性研究中最流行的工具之一：展示一张彩色热力图，颜色越深代表模型「越关注」那个词。这种直觉直接来自名字——「注意力」——和视觉上的说服力。但它在数学上是错误的。正确地理解「注意力权重到底衡量了什么、它与解释之间差了什么」，需要追踪计算图中信息流的完整路径。

本节出现四个核心概念：**注意力权重是 Value 路径的系数、梯度归因的多路径性、Jain & Wallace 的反例（Value 矩阵相关性导致注意力可被替换）、解释性研究的边界**。下面把这条链铺开。

第一环：注意力权重衡量的是什么——Value 路径的系数，而非因果贡献

注意力的输出是：

$$y = \sum_{j=1}^{n} \alpha_j v_j = \sum_{j=1}^{n} \alpha_j (x_j W^V) \tag{3.7.1}$$

式 (3.7.1) 中 α_j 是 v_j（Value 向量）在线性组合中的权重。但 $v_j = x_j W^V$ 已经是原始输入 x_j 经过线性变换后的结果，它与 x_j 的关系取决于矩阵 W^V 的方向——W^V 可能把 x_j 的某些语义维度完全压缩，突出另一些。因此，α_j 衡量的是：「从位置 j 的 Value 特征空间中，抽取了多少比例的内容」，而不是「位置 j 的原始输入对最终预测有多重要」。

用一个比喻：假设你在读一篇文章，注意力权重是「你目光停留在每行文字上的时间比例」，Value 投影是「每行文字被翻译成另一种语言后的内容」。目光停留时间长不等于那行原文对你理解文章的贡献大——因为翻译可能已经改变了含义，而且你的理解是由翻译后的内容决定的。

第二环：梯度归因的多路径性——x_j 影响输出的三条路

真正衡量「x_j 对输出 y 有多重要」的量是梯度 $\partial y/\partial x_j$（或 $\partial \mathcal{L}/\partial x_j$）。计算这个梯度需要追踪所有 x_j 影响 y 的路径：

路径一（Value 路径）：$x_j \to v_j = x_j W^V \to y = \sum_l \alpha_l v_l$，梯度为 $\alpha_j W^V$。这是注意力权重 α_j 反映的路径——α_j 大意味着这条路径贡献多。

路径二（Key 路径）：$x_j \to k_j = x_j W^K \to \alpha_{ij} = \text{softmax}_j(q_i \cdot k_j/\sqrt{d_k}) \to y$。$x_j$ 通过改变 k_j 来影响位置 i 对位置 j 的注意力权重，进而影响 y。这条路径的梯度与 α_j 无关（α_j 只反映最终权重，不反映 k_j 的影响强度）。

路径三（Query 路径的交叉影响）：如果 x_j 也参与了某个 Query（在自注意力中，$q_l = x_l W^Q$，$l = j$ 时 x_j 直接贡献到 q_j），则 x_j 通过 q_j 影响位置 j 对其他所有位置的注意力权重，这条路径更为复杂。

三条路径的贡献相加才是式 (3.7.3) 中 $\partial y/\partial x_j$ 的完整计算。注意力权重 α_j 只反映了路径一的强度，完全忽略了路径二和路径三。在实践中，路径二（Key 路径）往往与路径一同等重要甚至更重要——因为 x_j 不仅要「贡献内容」，还要「控制谁关注它」，后者对整个输出矩阵的影响是全局性的。

第三环：Jain & Wallace 的反例——注意力分布对输出不具唯一决定性

Jain 和 Wallace（NAACL 2019）提出了一个更强的否定结论：不仅梯度归因与注意力权重不一致，甚至注意力分布本身对输出都不具有唯一决定性。

他们的核心观察是：当 Value 矩阵 $V \in \mathbb{R}^{n \times d_v}$ 的行向量之间高度相关时（实践中这很常见，因为 $V = XW^V$ 中的 X 本身是相关的 token 嵌入），不同的权重向量 α 可以产生几乎相同的输出向量 $y = \alpha^T V$。形式化地，如果 V 的行向量几乎在同一个低维子空间内（即 V 是低秩的），则对任意 α，$\alpha^T V$ 都落在这个低维子空间内；在这个子空间内，不同的 α 之间的差异被投影到一个小的范围。这意味着：可以构造一个与原始 α 在 ℓ^1 距离上很大的替代权重 α'，使得 $\|\alpha'^T V - \alpha^T V\|$ 很小。

从式 (3.7.4) 的形式化结果可知：即使把注意力权重换成完全不同的分布，输出（以及下游的预测）可以几乎保持不变。「高度关注词 j」不能推断「词 j 对输出至关重要」——因为模型可以通过调整注意力权重（在 Value 矩阵的零空间方向上），在不改变输出的情况下把注意力随意重新分配。

第四环：解释性的边界——注意力告诉你什么、告诉不了你什么

理解了上面三环，可以精确地划定注意力权重在可解释性研究中的适用边界：

可以告诉你的：哪些 token 的 Value 特征对当前位置的输出有直接的线性贡献（路径一的权重）。这对「模型关注了哪些语义维度的内容」是有信息量的，但这个信息量是局部的、路径特定的。

告诉不了你的：（1）哪个输入 token 对最终预测最重要（需要完整的梯度归因）；（2）修改哪个输入会对预测产生最大影响（因果干预，需要 causal tracing 等方法）；（3）模型「为什么」做出某个预测（这需要追踪完整的计算电路，而不只是一层的注意力权重）。

正确的工具选择：若目的是「理解哪个输入特征影响了预测」，应使用 Integrated Gradients、LIME、SHAP 等梯度或扰动方法；若目的是「理解模型内部的计算结构」，应使用激活修补（activation patching）和电路分析（circuit analysis）方法，如 Anthropic 的机械可解释性（mechanistic interpretability）工作中识别出的 induction head 电路。

注意力权重可以作为辅助参考，但不能单独用于解释。

整条链的回头看

```
注意力权重 _j
  ↓ 衡量的是：v_j = x_j W^V 在输出线性组合中的比例
_j  x_j 的重要性（Value 路径只是三条路径之一）
  ↓ 完整梯度 y/ x_j 需要累加
路径一（Value）+ 路径二（Key）+ 路径三（Query 交叉）
  ↓ _j 只反映路径一
Jain & Wallace 反例：V 低秩 → 不同  可得相同输出
  ↓ 注意力分布对输出不具唯一决定性
「_j 大」「x_j 对预测重要」
  ↓ 真正的因果归因需要
梯度方法 / 激活修补 / 电路分析
```

这条链揭示了一个深层事实：**注意力权重是计算图中的一个中间结果，它描述的是信息流在一条特定路径上的分配比例，而不是输入的重要性**。把注意力权重当作解释，相当于用「货物在某一条路上的运输量」来衡量「起始城市对目的地的经济影响」——忽略了其他所有路径和网络效应。可解释性研究的真正困难，正是在于 Transformer 的信息流是多路径的、非线性的、层次叠加的，没有任何单一的简单量可以直接等价于「重要性」。

严格

设单层注意力的输出为

$$y = \sum_{j=1}^{n} \alpha_j v_j, \quad \alpha_j = \frac{\exp(q \cdot k_j / \sqrt{d_k})}{\sum_l \exp(q \cdot k_l / \sqrt{d_k})} \tag{3.7.2}$$

注意力权重与梯度归因的不一致性

基于梯度的特征重要性度量定义为

$$g_j = \left\| \frac{\partial \mathcal{L}}{\partial x_j} \right\| \quad \text{或} \quad g_j = \left\| \frac{\partial y}{\partial x_j} \right\| \tag{3.7.3}$$

式 (3.7.2) 中 α_j 衡量的是 v_j（Value 向量）对 y 的**线性组合权重**，而式 (3.7.3) 中 g_j 衡量的是输入 x_j 对输出 y（或损失 $\mathcal{L}$）的**微分敏感度**。二者通过不同路径传播：α_j 只经由 Value 分支，而 g_j 还包含 x_j 对 q、k_j 乃至其他位置 k_l 的影响，路径更复杂。

可交换性的反例构造

Jain 和 Wallace（NAACL 2019）指出，可以在保持输出近似不变的前提下构造替代注意力分布 α'，使得 $\|\alpha' - \alpha\|_1$ 较大。形式化地：对给定输出 $y = \alpha^T V$，存在 $\alpha' \in \Delta^{n-1}$（概率单纯形）使得

$$\|\alpha' - \alpha\|_1 \geq \epsilon \quad \text{而} \quad \|y' - y\| = \|\alpha'^T V - \alpha^T V\| \leq \delta \tag{3.7.4}$$

对足够小的 δ 和足够大的 ϵ，这说明注意力分布本身对输出不具有唯一决定性——当

Value 矩阵 V 的行向量之间相关性高时，不同的权重混合可以得到接近相同的输出向量。因此，式 (3.7.2) 中 α_j 大不能直接推断「第 j 个 token 对预测起决定性作用」；真正的因果归因必须通过梯度方法（gradient attribution，即式 (3.7.3)）或激活路径分析（如 circuit analysis）来进行。

与原书呼应：原书 §3.7 引用了 Jain & Wallace 的核心发现及 induction head 电路分析，对应本节注意力权重与梯度归因不等价的形式化说明。

---

§3.8 跨界映射：宏观工业供应链

▶这一节在讲什么

为什么用供应链类比 attention? 注意力机制的「匹配-归一化-加权汇总」流程，在逻辑结构上与全球供应链的「需求发布-能力匹配-资源调度」完全对应，能把抽象矩阵运算还原为可感知的经济直觉。

第一步：Query 是需求方，Key 是供应商标签，Value 是实际物料。每个生产环节（token）发出需求向量 Q，每个供应商（token）公布能力标签 K，softmax 将匹配分数归一化为调度权重，最终按权重汇总实际物料 V。

第二步：多头 = 多条并行供应链。每个注意力头按不同逻辑（语法、语义、话题）独立调度，就像多条并行运作的专业供应链，最后在「总装车间」（输出投影 W^O）合并。

第三步：这个类比的价值是量级直觉。当序列长度 $n = 8192$，$h = 32$ 个头，每步注意力相当于 32 条供应链各自对 8192 个供应商做一轮全局询价——这是人类供应链系统完全无法实现的规模。

本节要拿走的一件事：供应链类比把 attention 从「心理学隐喻」还原为「可量化的资源匹配」，但它有根本性的局限——见 §3.9。

直觉

类比是理解抽象数学的强大工具，但也是最容易固化错误直觉的工具。这一节先把供应链类比尽量用好，把它能解释的东西讲清楚；下一节（§3.9）再逐一拆解它的失效边界。

注意力机制的「匹配-归一化-加权汇总」逻辑，与全球供应链的「需求发布-能力匹配-资源调度」在结构上高度吻合。下面分析这个类比成立的三个数学根源，以及它的量级价值在哪里。

供应链类比成立的根源一：三角色的分离

在现实供应链中，一个供应商同时扮演三个角色：它**发布需求**（向上游采购），它**公布能力**（告诉下游客户能提供什么），它**提供实物**（真正交付的货物）。这三个角色对应注意力中的 Query（发布需求）、Key（公布能力标签）和 Value（交付的实际内容）。

现实供应链中，「能力标签」（Key）和「实际物料」（Value）是分开的：供应商公布的规格书（Key）描述了型号、参数、价格，而真正交货的零部件（Value）是物理实体，两者虽然对应，但完全不同的东西。注意力也是如此：Key 是用于匹配打分的向

量，Value 是真正混合到输出里的内容，两者由不同的线性投影矩阵（W^K 和 W^V）生成，解耦的设计原因（见 §3.2）在这里体现为「规格书与实物分开管理」的供应链逻辑。

供应链类比成立的根源二：softmax 归一化等价于调度权重的比例分配

在供应链调度中，当多个供应商都能满足需求时，采购部门通常不会把所有订单都给评分最高的那一个（风险集中），而是按能力匹配度**比例分配订单**：最高分供应商拿 60%，次高分拿 30%，其余分散 10%。这正是 softmax 做的事：把原始打分向量映射到概率单纯形（所有权重非负且和为 1），使得匹配度高的供应商（token）获得更大但不是唯一的份额，仍然保留了「软检索」而非「硬选择」的特性。

这个类比帮助解释为什么不用 argmax（硬性选择得分最高的一个 key 对应的 value）：硬选择不可微，无法反向传播梯度；而且在语言中，「最相关的词」不止一个，输出应该是多个相关词的加权混合。softmax 的连续性和可微性，正是「比例分配」语义的数学实现。

供应链类比成立的根源三：多头等价于多条专业供应链的分工

现代供应链管理中，大型制造商往往维护多条平行的供应链体系：一条专注于标准零件（低成本、大批量），一条专注于精密部件（高质量、小批量），一条专注于紧急备料（快速响应）。每条供应链有自己的评估标准（Key）和物料库（Value），最终产品是多条供应链交付结果的集成。

多头注意力完全对应这个结构：h 个头各自维护独立的 W_i^Q, W_i^K, W_i^V，在不同的语义子空间（不同的「采购标准」）里独立做注意力，输出通过 W^O（「总装」）合并。不同的头会自发专门化——实验研究发现，有的头专注句法依存（语法供应链）、有的头追踪长距离指代（语义供应链）、有的头捕捉局部搭配（词汇供应链）。这种分工不是预先设计的，而是训练压力下的涌现，就像市场竞争下自然形成的专业化分工。

供应链类比的量级价值

这个类比最重要的作用是帮助建立规模直觉。一个真实的汽车制造商可能管理数千个供应商，但每次采购决策通常只涉及少数几家候选。注意力的不同之处：每个 token 每次前向传播都对序列中所有 n 个 token 做完整评分。当 $n = 8192$（GPT-4 的上下文长度），$h = 32$ 个头，每次前向传播相当于 32 条供应链，各自对 8192 个供应商做一轮完整的全球询价。一个 Transformer 层的一次前向传播，完成的「全球询价」次数是 $n \times h = 8192 \times 32 = 262144$ 次——对于人类供应链系统，这是完全不可想象的规模；对于 GPU，这只是一次矩阵乘法。这种量级感的震撼，正是供应链类比的核心价值：把抽象的矩阵运算转换成可以用现实世界的操作规模感受的东西。

---

§3.9 反类比：当供应链隐喻失效

▶这一节在讲什么

为什么供应链类比有根本性局限？供应链有两个注意力机制没有的核心特征：硬性产能约束和明确的优化目标。这两处差异揭示了 attention 作为数学对象的真实本质。

第一步：Value 向量可被无限次「调用」，没有产能耗尽。同一个 token 可以同时以最高权重响应序列中所有其他位置，不存在资源争抢。attention 调度的实质是相似度计算，而非真正的权衡分配。

第二步：注意力头没有明确的语义目标。供应链的优化目标（成本、时效）是可测量的单一指标；而单个注意力头的「功能」是反向传播在高维参数空间中塑造的中间张量，是涌现的、上下文相关的，通常无法用自然语言直接描述。

第三步：attention 数学透明但语义不透明。注意力权重可以精确计算，但它们不等于解释（见 §3.7）；attention 机制可以精确描述，但单个头的「功能」不能被先验地规定。

本节要拿走的一件事：好的类比帮助建立直觉，坏的类比固化误解——供应链隐喻的价值在于规模感知，其局限在于它用「有约束的竞争分配」来描述一个「无约束的相似度加权」，会系统性地误导对 attention 瓶颈和可解释性的判断。

直觉

§3.8 展示了供应链类比在建立「规模直觉」上的价值。这一节要做相反的工作：逐一拆解类比失效的数学根源，说明把供应链思维带入注意力机制时会产生哪些系统性误判。分析类比在哪里失效，往往比分析它在哪里成立更能揭示对象的本质。

供应链类比在以下四个数学根源上失效。下面逐一深入分析。

失效根源一：Value 向量无产能约束——softmax 归一化不是资源守恒

供应链中，物料是守恒的：如果某个供应商把 60% 的产能给了客户 A，它只剩 40% 给客户 B。资源分配是一个零和博弈，一方多则另一方少。

注意力中，Value 向量完全没有这种约束。Position j 的 Value 向量 v_j 可以同时以权重 1 贡献给序列中所有 n 个其他位置的输出——它不会因为「被太多人关注」而「产能耗尽」。Position i 的 softmax 权重 α_{ij} 之和等于 1（行归一化），但这只是位置 i 自己的注意力权重的约束，不是 v_j 被「消耗」的约束。v_j 可以同时满载地参与 n 个位置的计算。

这个差异的数学含义是：softmax 的归一化确保的是「每个 Query 的关注权重之和为 1」，而不是「每个 Key/Value 的被关注量有上限」。这是一个**统计归一化**（保证权重是概率分布），不是一个**物理守恒律**（保证资源总量不变）。用供应链思维理解注意力，会系统性地高估「激烈竞争」的程度：实际上没有竞争，每个 Value 向量对所有想要它的 Query 都「来者不拒」。

更进一步：正因为没有产能约束，注意力的计算可以完全并行——不同 Query 的输出之间没有任何资源争抢关系，可以同时计算，完全独立。如果真的是供应链（有产能约束），不同客户的分配决策就会互相依赖，就无法并行。注意力的可并行性，恰恰

来自它不是真正意义上的资源分配。

失效根源二：注意力头没有先验语义目标——功能是涌现的，不是设计的

供应链中，每条链的优化目标是外生给定的（成本、时效、质量评分），可以被明确陈述、事前规定、事后审计。「这条链负责精密部件采购」是一个可以写进合同的规定。

注意力头没有这种先验语义目标。第 k 个头的投影矩阵 W_k^Q, W_k^K, W_k^V 是通过反向传播在整个训练过程中共同优化的，它的「功能」完全由数据分布和损失函数塑造，是一个涌现属性，不是设计属性。

确实，机械可解释性（Mechanistic Interpretability）研究发现了一些可命名的头：「induction head」（复制前面出现过的模式）、「name mover head」（把命名实体从主语位置传递到谓语位置）等。但这些是事后发现的少数特例，不是设计目标。大多数注意力头的功能在不同上下文中会变化（上下文相关的多义性），无法用一句话描述，在不同的输入分布下甚至会改变激活模式。用供应链类比「这个头负责语法关系，那个头负责长距离指代」，会误导你认为头的职责是固定的、可预先规定的，而实际上它是流动的、上下文依赖的。

失效根源三：attention 是线性加权，而非优化决策

供应链调度是一个优化问题：给定约束（产能、成本），求最优解（最大化利润、最小化延误）。这是一个带目标函数的决策过程，决策者在权衡不同选项。

注意力是一个确定性的线性函数：给定 Q, K, V，输出 $\text{softmax}(QK^T/\sqrt{d_k})V$ 没有任何优化过程，没有约束，没有「权衡」——只是一个矩阵乘法加非线性变换，结果是唯一确定的。softmax 引入的「竞争感」只是视觉上的：概率质量从低分项转移到高分项，但这只是 exp 函数的放大效应，而不是某种零和竞争博弈的均衡解。

这个失效意味着：不能用供应链的「瓶颈分析」（哪个供应商产能不足导致整体延误）来理解注意力的计算瓶颈。注意力的真正瓶颈是算术强度（§3.3）和内存带宽（§3.5），这些是纯粹的数值计算资源问题，与「谁关注谁」的分配结构完全无关。

失效根源四：注意力权重不等于解释——供应链审计的不可移植性

供应链的可审计性来自其设计：每个决策对应一个可追溯的人类可读的原因（这个供应商评分高是因为交期短、价格低）。这种审计是直接的、一对一的。

注意力权重看起来可以「审计」，但 §3.7 已经证明，高注意力权重 α_j 不等于「token j 对预测起决定作用」。把供应链的审计思维移植到注意力，会导致一个系统性错误：把「高注意力权重的词」当作「模型关注的理由」，而实际上真正的因果路径需要通过梯度归因和激活修补来确定。这种误植正是 §3.7 批评的注意力权重可解释性误用的来源。

类比失效的总结：四处失效共同揭示了 attention 的真实数学本质

供应链特征	Attention 的真实情况
有产能约束（零和分配）	无产能约束（线性叠加，无守恒）
优化目标外生给定	功能由训练涌现，无先验语义
竞争调度（博弈均衡）	确定性线性变换（矩阵乘法 + softmax）
可审计（决策有因果链）	权重 ≠ 解释（因果需梯度方法）

供应链类比的价值在于：量级直觉（$n \times h$ 次全局询价的规模感）、三角色分离的直

觉（QKV 解耦）、多条专业化子链的直觉（多头分工）。它的局限在于：一旦涉及「约束」「竞争」「目标」「可解释」这些词，类比就开始系统性地误导。好的工程师知道何时抓住类比、何时放下类比。

数学卷 · 第 4 章：非凸优化与超参数标度律

---

§4.1 损失函数：如何用数学定义”机器犯错”

▶这一节在讲什么

为什么训练前要先定义「犯错」? 梯度下降需要一个可微分的数值目标，而「预测对了没有」是离散的，无法直接优化。损失函数就是把「犯错程度」翻译成一个光滑的实数。

第一步：选择什么样的损失? 分类问题中最自然的损失是交叉熵——它与最大似然估计（MLE）完全等价，同时也等价于最小化模型分布与真实分布之间的 KL 散度，具有坚实的统计学基础。

第二步：为什么交叉熵的梯度如此简洁? 把 Softmax 与交叉熵联合求导，链式法则中的复杂 Jacobian 自动约分，最终反向传播到 logit 层的误差信号恰好等于「预测概率 − 真实标签」，无需额外计算。

第三步：认清损失函数的三条等价链。 MLE = 最小负对数似然 = 最小 KL 散度，三者只是同一件事的三种视角。

本节要拿走的一件事：交叉熵不是随手选的损失，它是信息论、统计学与反向传播三条路径共同指向的唯一自然选择。

直觉

训练一个神经网络，本质是找到一组参数，使模型的输出尽可能贴近真实标签。要做到这一点，必须先把「犯错程度」量化成一个数。这个数就是损失函数（loss function）。但「量化犯错」并不是一件任意的事：不同的量化方式对应不同的统计假设，会产生截然不同的梯度信号，进而决定训练的动力学。本节的核心问题是：**为什么分类任务用交叉熵，回归任务用 MSE，而不是反过来？**以及：**正则化项是怎么从同一套统计框架里自然生长出来的？**

本节出现五个概念：**均方误差（MSE）**、**交叉熵损失**、**最大似然估计（MLE）**、**正则化（L2/L1）**、**Softmax 梯度约分**。它们看似散落，实际上沿着一条严密的逻辑链一个推出一个——都是同一个问题「如何定义机器犯错」的不同侧面。理解这条链，是理解大模型训练目标函数为什么是交叉熵（而不是其他）的钥匙。下面把这条链一环环铺开。

第一环：MSE——从高斯噪声假设直接推出

在谈交叉熵之前，先从最朴素的损失出发：均方误差（Mean Squared Error）。

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}\|y_i - \hat{y}_i\|^2 \tag{4.1.1}$$

MSE 不是「工程师随手选的方便公式」，它有严格的统计来源。假设观测值 y_i 是真实值 $f_\theta(x_i)$ 加上一个服从 $\mathcal{N}(0,\sigma^2)$ 的独立高斯噪声：

$$y_i = f_\theta(x_i) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0,\sigma^2) \tag{4.1.2}$$

则数据的对数似然为：

$$\log P(\mathbf{y} \mid \theta) = -\frac{1}{2\sigma^2}\sum_i (y_i - f_\theta(x_i))^2 + \text{常数} \tag{4.1.3}$$

最大化对数似然（即最大化式 (4.1.3)）$\Leftrightarrow$ 最小化 $\sum_i (y_i - f_\theta(x_i))^2 \Leftrightarrow$ 最小化式 (4.1.1) 中的 MSE。换句话说，**MSE 是「误差服从高斯分布」这一假设下的最大似然估计**。它是对的，但只在这个假设下对。

这立刻指出了 MSE 用于分类的问题：分类标签是离散的 one-hot 向量，根本不服从高斯分布。把高斯假设强加到 one-hot 标签上，相当于用一个错误的统计模型计算似然，得到的梯度信号是扭曲的——这就是为什么分类任务用 MSE 训练时收敛慢甚至不收敛。

第二环：交叉熵损失——从分类任务的正确概率模型推出

分类任务的正确模型是：y_i 服从类别分布（categorical distribution），模型输出 K 个类别的概率向量 $\mathbf{p} = (p_1, \dots, p_K)$，真实标签以 one-hot 向量 $\mathbf{y}$ 表示。此时对数似然为：

$$\log P(y_i \mid x_i, \theta) = \sum_{k=1}^{K} y_{ik} \log p_k \tag{4.1.4}$$

对整个数据集最大化对数似然，等价于最小化**负对数似然**（negative log-likelihood, NLL）：

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log p_k \tag{4.1.5}$$

这就是式 (4.1.5) 所定义的交叉熵损失。它与 MSE 的来源完全平行：**交叉熵是「标签服从类别分布」这一假设下的最大似然估计**。分类任务用交叉熵而非 MSE，不是品味问题，而是统计模型匹配问题。

与 MSE 的关键区别体现在梯度行为上。在分类任务中，当模型将某类别的预测概率 p_k 推向 0 时，$-\log p_k \to +\infty$，梯度 $\partial\mathcal{L}/\partial p_k = -1/p_k$ 也趋向无穷大——错得越离谱，惩罚越重。而 MSE 的梯度 $\partial/\partial p_k (p_k - y_k)^2 = 2(p_k - y_k)$ 在 $p_k \to 0$（预测接近正确类别为 0，而真实为 1）时梯度约为 -2，相当平缓——**MSE 对极端错误的惩罚不够**，训练信号弱。交叉熵的对数形式恰好补偿了这一点。

第三环：KL 散度等价——交叉熵的信息论根基

交叉熵还可以从信息论视角重新推导，这赋予它更深的意义。设真实数据分布为 q（经验分布，one-hot），模型分布为 p_θ，KL 散度为：

$$D_{\mathrm{KL}}(q\|p_\theta) = \sum_k q_k \log \frac{q_k}{p_k} = -\sum_k q_k \log p_k + \underbrace{\sum_k q_k \log q_k}_{-H(q),\ \text{常数}} \tag{4.1.6}$$

由式 (4.1.6) 可见，真实数据分布 q 的熵 $H(q)$ 不依赖模型参数 θ，最小化 KL 散度等价于最小化 $-\sum_k q_k \log p_k$，即交叉熵。因此：

$$\text{最大化似然} \iff \text{最小化负对数似然} \iff \text{最小化 KL 散度} \iff \text{最小化交叉熵} \tag{4.1.7}$$

式 (4.1.7) 给出的这四条路径在数学上完全等价。选择交叉熵作为损失函数，不是工程偏好，而是**让模型分布尽可能接近数据分布的信息论唯一正确选择**。

第四环：正则化——从贝叶斯先验自然生长

前三环建立了「没有先验知识」时的损失。但实践中，模型经常过拟合：在训练集上损失趋零，但在验证集上性能糟糕。解决方案是正则化，而正则化同样有统计根基——它来自对参数的先验分布假设。

L2 正则化（权重衰减）：假设参数 θ 的先验为各向同性高斯 $P(\theta) \propto e^{-\lambda\|\theta\|^2}$，则最大后验估计（MAP）等价于：

$$\min_\theta \underbrace{\mathcal{L}_{\mathrm{CE}}(\theta)}_{\text{负对数似然}} + \underbrace{\lambda\|\theta\|^2}_{-\log P(\theta)+\text{常数}} \tag{4.1.8}$$

式 (4.1.8) 中的 L2 正则项就是高斯先验的负对数。它的几何含义是：把参数向原点推压，倾向于找参数范数较小的解，防止某些权重爆炸性增大。

L1 正则化：假设参数的先验为 Laplace 分布 $P(\theta_j) \propto e^{-\lambda|\theta_j|}$，MAP 等价于加 L1 项：

$$\min_\theta \mathcal{L}_{\mathrm{CE}}(\theta) + \lambda\|\theta\|_1 \tag{4.1.9}$$

L1 正则化倾向于产生稀疏解（大量参数恰好为零），这是 Laplace 分布在原点有尖峰的几何效应。在大模型的稀疏注意力、MoE（混合专家）等场景，稀疏性是显式的设计目标，L1 正则化（或其变体）是自然工具。

两种正则化的区别在于梯度结构：式 (4.1.8) 中 L2 正则化的梯度是 $2\lambda\theta$，随参数大小线性增大，平滑地将所有参数推向零；式 (4.1.9) 中 L1 正则化的梯度是 $\lambda \cdot \mathrm{sign}(\theta)$，与参数大小无关，等幅度推压，对小参数的推力相对更强，因此容易把小参数直接推到零——产生稀疏性的根本原因正在于此。

第五环：Softmax + 交叉熵的梯度约分——工程价值的来源

最后一环解释了为什么交叉熵在工程上的采用如此普遍，不仅仅是统计理由，还有数值计算上的优雅。

设网络最后一层的 logit 向量为 $\mathbf{z} = (z_1, \ldots, z_K)$，Softmax 映射为：

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \tag{4.1.10}$$

如果单独计算 Softmax 的 Jacobian $\partial p_j/\partial z_i$，需要一个 $K \times K$ 的矩阵。但将 Softmax 与交叉熵损失联合对 z_i 求偏导，通过链式法则的化简，大量项自动抵消（「约分」），最终得到：

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_i} = p_i - y_i \tag{4.1.11}$$

式 (4.1.11) 的结果极其简洁：反向传播到 logit 层的误差信号，恰好是**预测概率与真实标签之差**。不需要存储或反向传播完整的 Jacobian 矩阵，节省了 $O(K^2)$ 的计算和内存。这种「两个复杂组件组合后梯度变得极简」的现象，是交叉熵被大规模工程采用的实用动机之一，而不只是统计理论的美感。

对比：如果用 MSE 训练分类器（$\mathcal{L}_{\text{MSE}} = \|p - y\|^2$），梯度为 $\partial \mathcal{L}/\partial z_i = 2\sum_j (p_j - y_j) \cdot \partial p_j/\partial z_i$，需要显式用到 Softmax 的完整 Jacobian，计算更重，且梯度在预测接近极端时趋于消失（梯度消失问题），使训练更困难。

整条链的回头看

```
高斯噪声假设      最大似然      MSE（回归）
                                  ↓ 换成分类假设
类别分布假设      最大似然      交叉熵损失      KL散度等价（信息论根基）
                                  ↓ 加上参数先验
高斯先验          MAP估计       + L2正则化（权重衰减）
Laplace先验       MAP估计       + L1正则化（稀疏性）
                                  ↓ 与Softmax联合求导
Softmax+交叉熵    链式法则约分    梯度 = p_i - y_i（极简误差信号）
```

这条链揭示了一个深层事实：**损失函数的选择不是任意的工程调参，而是对数据生成过程统计假设的显式声明**。当你选择交叉熵 + L2 正则化，你实际上在声明：「数据由类别分布生成，参数服从高斯先验，我要找最大后验估计。」这个声明决定了梯度的形状，进而决定了训练动力学的全局行为——理解损失函数的统计来源，是调试训练问题时最可靠的出发点。

严格

经验风险（empirical risk）的定义如下。给定数据集 $\{(x_i, y_i)\}_{i=1}^N$，参数化模型 f_θ，以及单样本损失 ℓ，经验风险为：

$$\hat{R}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell(f_\theta(x_i),\, y_i) \tag{4.1.12}$$

训练目标是 $\theta^* = \arg\min_\theta \hat{R}(\theta)$。

交叉熵损失的来源。对 K 分类问题，设真实标签以 one-hot 向量 $\mathbf{y} \in \{0,1\}^K$ 表示，模型输出概率分布 $\mathbf{p} = (p_1, \dots, p_K)$，则交叉熵（cross-entropy）损失为：

$$L = -\sum_{k=1}^{K} y_k \log p_k \tag{4.1.13}$$

其来源可从三条等价路径理解：

1. **最大似然估计（Maximum Likelihood Estimation, MLE）**。数据集的对数似然为 $\log \prod_i P_\theta(y_i \mid x_i) = \sum_i \log P_\theta(y_i \mid x_i)$。最大化似然即最小化负对数似然（negative log-likelihood）：

$$\min_\theta -\frac{1}{N} \sum_{i=1}^{N} \log P_\theta(y_i \mid x_i) \tag{4.1.14}$$

对 one-hot 标签，这恰好等于 $\frac{1}{N}\sum_i L_i$。

2. **最小化 KL 散度**。设真实数据分布为 q，模型分布为 p_θ，KL 散度为：

$$D_{\mathrm{KL}}(q \| p_\theta) = \sum_k q_k \log \frac{q_k}{p_k} = -\sum_k q_k \log p_k + \text{const} \tag{4.1.15}$$

其中常数项（$-\sum_k q_k \log q_k$，即真实分布的熵）不依赖 θ，故最小化 KL 等价于最小化交叉熵。

三条路径（MLE = 最小负对数似然 = 最小 KL 散度）在数学上完全等价。

Softmax + 交叉熵的联合梯度推导。设网络最后一层的 logit 向量为 $\mathbf{z} = (z_1, \dots, z_K)$，归一化指数（Softmax）将其映射为：

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \tag{4.1.16}$$

对第 i 个 logit，计算 $\partial p_j / \partial z_i$：

$$\frac{\partial p_j}{\partial z_i} = p_j(\delta_{ij} - p_i) \tag{4.1.17}$$

其中 δ_{ij} 是 Kronecker delta。损失对 z_i 的梯度为：

$$\frac{\partial L}{\partial z_i} = -\sum_k y_k \frac{\partial \log p_k}{\partial z_i} = -\sum_k y_k \frac{1}{p_k} \cdot \frac{\partial p_k}{\partial z_i} \tag{4.1.18}$$

将式 (4.1.17) 代入式 (4.1.18)，利用 $\sum_k y_k = 1$（one-hot）：

$$\frac{\partial L}{\partial z_i} = -\sum_k y_k(\delta_{ki} - p_i) = -(y_i - p_i) = p_i - y_i \tag{4.1.19}$$

式 (4.1.19) 是一个极其简洁的结果：**反向传播到最后一层 logit 的误差信号，恰好等于预测概率与真实标签之差**。不需要单独计算 Softmax 的雅可比（Jacobian），两者的链式法则自动约分。

与原书呼应：原书 §4.1 指出交叉熵「把训练目标和统计学中的最大似然估计无缝衔接」，本节给出了这条等价链的完整代数路径。

---

§4.2 梯度下降：在黑暗中沿着最陡的方向下山

> **▶这一节在讲什么**
>
> **为什么梯度反向就是下降最快的方向**？这来自一阶 Taylor 展开的几何含义：在单位球约束下，最大化损失减少量的方向恰好与梯度方向相反。
>
> **第一步：步长不能任意大**。函数曲率（Lipschitz 常数 L）给出步长上界 $\eta < 2/L$，超出这个界限，更新会越过极小值导致震荡甚至发散。
>
> **第二步：mini-batch 引入的随机噪声有双重作用**。它让每步计算变快，而且噪声本身在高维优化中有隐式正则化效果——这在 §4.3 中会看到它帮助逃离鞍点。
>
> **第三步：动量和 Adam 各解决一个问题**。动量用历史梯度的指数加权平均提供惯性，帮助穿越平坦区域；Adam 用二阶矩自适应调整每个参数的有效步长，使各维度更新更均匀。
>
> **本节要拿走的一件事**：Adam 的偏差修正不是工程技巧，而是初始化为零导致的系统偏差的数学必然修复。

直觉

有了损失函数，下一个问题是：如何在高维参数空间里找到它的极小值？最直接的思路是「每步沿最陡下坡方向走一小步」，这就是梯度下降。但这个简单想法背后藏着一系列非平凡的设计选择，每一个都有清晰的数学动机。

本节出现六个概念：**梯度方向的数学来源**、**步长上界约束**、**SGD 与 mini-batch**、**动量法**、**RMSprop**、**Adam 及其偏差修正**。它们不是孤立的优化技巧，而是沿着一条逻辑链递进的——每一步都在解决上一步留下的具体问题。理解这条链，是理解为什么大模型训练几乎一律用 AdamW 的关键。下面把这条链一环环铺开。

第一环：梯度方向——从一阶近似推出最速下降

函数极小值的寻找，可以转化为一个有约束的优化问题：在「步长为 ϵ 的球」约束下，找使损失下降最多的方向。

用一阶泰勒（Taylor）展开近似：

$$\mathcal{L}(\theta + \Delta\theta) \approx \mathcal{L}(\theta) + \nabla\mathcal{L}(\theta)^\top \Delta\theta \tag{4.2.1}$$

在约束 $\|\Delta\theta\| = \epsilon$ 下，最大化下降量 $-\nabla\mathcal{L}^\top\Delta\theta$ 的方向，由 Cauchy-Schwarz 不等式：

$$-\nabla\mathcal{L}^\top\Delta\theta \le \|\nabla\mathcal{L}\| \cdot \|\Delta\theta\| = \epsilon\|\nabla\mathcal{L}\| \tag{4.2.2}$$

等号在 $\Delta\theta = -\epsilon\frac{\nabla\mathcal{L}}{\|\nabla\mathcal{L}\|}$ 时成立。这就是**梯度反向是最速下降方向**的完整推导——它不是直觉，而是线性约束优化的精确结论。

从这个推导里可以读出一个限制：这个结论只在「泰勒展开精确」的范围内成立，即步长 ϵ 足够小时。当步长太大，函数的非线性项（曲率）开始主导，一阶近似失效，梯度反向未必还是最速下降方向。这直接引出下一环的约束。

第二环：步长上界——曲率决定安全步长

函数的局部曲率决定了「一阶近似能撑多远」。若损失函数的梯度是 L-Lipschitz 连续的，即任意两点的梯度变化不超过它们距离的 L 倍：

$$\|\nabla \mathcal{L}(\theta) - \nabla \mathcal{L}(\theta')\| \leq L\|\theta - \theta'\| \tag{4.2.3}$$

则可以证明：当学习率 $\eta < 2/L$ 时，每次梯度下降更新都保证损失不增。L 是「最坏情况曲率」的上界——它控制着函数弯曲得多剧烈。直觉上，步长 η 必须小于「跨过极小值所需的步长」，而这个阈值正比于 $1/L$。

工程上，L 通常未知，学习率必须靠实验调。这正是深度学习中学习率调参如此重要（且如此麻烦）的数学根源：**学习率的「正确值」是由损失函数的局部曲率决定的，而曲率在训练过程中随时变化**。一阶方法（纯梯度下降）的根本局限，正是它对曲率信息的盲目性——这直接催生了后面几环的自适应方法。

第三环：SGD 与 mini-batch——噪声既是代价也是红利

在实践中，一次完整的梯度计算需要遍历整个训练集，对于数十亿样本的大规模训练完全不可行。解决方案是随机梯度下降（SGD）：每步随机采样一个 mini-batch $\mathcal{B}$，用 mini-batch 梯度近似全局梯度：

$$\tilde{g}_t = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla \ell_i(\theta_t) \tag{4.2.4}$$

$\tilde{g}_t$ 是真实梯度的**无偏估计**（期望等于真实梯度），但有方差 $\mathrm{Var}(\tilde{g}_t) = \frac{1}{|\mathcal{B}|}\mathrm{Var}(\nabla \ell_i)$——方差随 batch size 增大线性减小。

关键的非直观性质：SGD 引入的噪声不仅仅是「效率的代价」，它还有**隐式正则化**效果，且帮助逃离鞍点（详见 §4.3）。小 batch size 的 SGD 有时训练速度（达到给定泛化性能所需的步数）反而优于大 batch 的确定性梯度下降，原因正是这种有益的噪声。这是深度学习优化中最反直觉的事实之一。

第四环：动量——给优化器加上「惯性」

纯 SGD 有两个实践问题：在损失函数的「峡谷」地形中（某些方向曲率大、某些方向曲率小的各向异性场景），梯度在陡峭方向来回振荡，在平坦方向却进展缓慢；遇到平坦区域时，梯度接近零，更新几乎停止。

动量法（momentum）的解决方案是引入「速度向量」v_t，模拟质点在损失景观上的惯性运动：

$$v_{t+1} = \mu v_t + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_{t+1} \tag{4.2.5}$$

展开递推：$v_t = \sum_{k=0}^{t} \mu^k g_{t-k}$，即速度是历史梯度的**指数加权平均**，权重随时间指数衰减。

动量的几何效果：- 在峡谷横截方向，连续步骤的梯度符号交替（$+,-,+,-,\ldots$），指数加权后它们互相抵消，振荡幅度减小；- 在峡谷纵向，梯度符号一致，历史贡献累积，速度不断增大，穿越平坦区域的速度加快。

这正是物理上的「惯性」：方向一致时加速，方向反复时减速。动量的代价是引入了一个超参数 μ（通常取 0.9 或 0.99），以及训练初期需要「预热」速度向量（早期速度受初始化 $v_0 = 0$ 的影响）。

第五环：RMSprop——自适应步长的第一次尝试

动量解决了方向的问题，但没有解决不同参数之间**步长幅度**的问题。设想词嵌入层：常见词的嵌入在每个批次都会收到大梯度（频繁更新），而罕见词的嵌入很少收到梯度信号（稀疏更新）。用统一学习率 η 处理两类参数，常见词参数可能已经收敛但学习率仍大导致震荡，罕见词参数则因梯度稀少、更新缓慢而欠训练。

RMSprop（Root Mean Square Propagation）的核心思想是：**用每个参数的历史梯度幅值的指数加权均方根来自适应缩放步长**：

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2, \qquad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t} + \epsilon} g_t \tag{4.2.6}$$

其中 s_t 是历史梯度平方的指数加权均值，$\sqrt{s_t}$ 是历史梯度的 RMS（均方根）。用 RMS 除步长，等于把每个参数的有效学习率归一化到「历史梯度幅值」的量纲上：历史梯度大（更新频繁/剧烈）的参数自动获得小步长，历史梯度小（更新稀疏/平缓）的参数获得大步长。这让「常见词 vs 罕见词」、「浅层 vs 深层」这类梯度幅值差异极大的参数都能以合适的步长更新。

第六环：Adam——动量 + 自适应步长 + 偏差修正

Adam（Adaptive Moment Estimation）把动量和 RMSprop 的思想合并，同时维护一阶矩（动量）和二阶矩（梯度平方的历史均值）：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad \text{（一阶矩：方向信息）} \tag{4.2.7}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \quad \text{（二阶矩：幅值信息）} \tag{4.2.8}$$

但式 (4.2.7) 和式 (4.2.8) 中有一个精妙的陷阱：$m_0 = v_0 = 0$（初始化为零），早期的 m_t 和 v_t 被初始值拉向零，系统性地低估了真实的一阶矩和二阶矩。以式 (4.2.8) 中的 v_t 为例，第一步：$v_1 = (1-\beta_2) g_1^2$，而真实二阶矩应接近 g_1^2，误差因子为 $(1-\beta_2)$（当 $\beta_2 = 0.999$ 时，v_1 只有真值的 0.1%）。

式 (4.2.7) 和式 (4.2.8) 的**偏差修正**（bias correction）是消除这种系统偏差的数学必然：

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \tag{4.2.9}$$

当 t 较小时（初始阶段），$1-\beta^t$ 很小，修正因子 $1/(1-\beta^t)$ 很大，把被压低的估计放大回正常尺度；随着 t 增大，$\beta^t \to 0$，修正因子趋向 1，修正效果消失——因为此时 m_t 和 v_t 已经积累足够多的历史，初始偏差的影响可以忽略。偏差修正不是「加了个系数以便收敛」的工程经验，而是指数加权移动平均从零初始化出发时的**数学必然修复**。

最终更新规则：

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{4.2.10}$$

式 (4.2.10) 中 $\eta/(\sqrt{\hat{v}_t} + \epsilon)$ 是每个参数的有效学习率，由其历史梯度的 RMS 自适应决定；式 (4.2.9) 中的 $\hat{m}_t$ 是带动量的梯度方向估计，对方向噪声有平滑作用。Adam 在实践中对超参数的选择相当鲁棒（典型值 $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$ 在大多数任务上都适用），这是它成为大模型训练默认优化器的工程原因。

整条链的回头看

```
一阶泰勒展开      梯度反向=最速下降方向（数学推导）
                      ↓ 步长不能任意大
Lipschitz常数L    步长上界  < 2/L（曲率约束）
                      ↓ 完整梯度太贵
mini-batch        SGD：无偏估计 + 有益噪声（隐式正则化）
                      ↓ 平坦区域/振荡问题
动量              历史梯度指数加权平均（惯性，抑制振荡）
                      ↓ 不同参数梯度幅值差异大
RMSprop           二阶矩归一化步长（自适应缩放）
                      ↓ 合并动量+自适应+初始化偏差修复
Adam              一阶矩(方向) + 二阶矩(幅值) + 偏差修正
```

这条链揭示了一个深层事实：**Adam 不是一个任意发明的优化器，而是依次解决了「最速下降 → 步长安全 → 计算效率 → 振荡抑制 → 自适应幅值 → 初始偏差」六个问题的逻辑终点**。理解这条链，也能理解 Adam 的局限：它对曲率的估计仍然是**对角的**（只用梯度平方的逐元素均值），忽略了参数之间的相关性——这正是 §4.6 中 Shampoo 要突破的地方。

严格

一阶 Taylor 展开导出更新规则。在 θ_t 处展开：

$$L(\theta_t + \Delta\theta) \approx L(\theta_t) + \nabla L(\theta_t)^\top \Delta\theta \tag{4.2.11}$$

在约束 $\|\Delta\theta\| = \epsilon$ 下，使 L 下降最多的方向是 $\Delta\theta = -\epsilon \frac{\nabla L(\theta_t)}{\|\nabla L(\theta_t)\|}$。令步长 $\eta = \epsilon/\|\nabla L\|$，得到梯度下降（gradient descent）更新规则：

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t) \tag{4.2.12}$$

步长上界与 Lipschitz 常数。若 L 的梯度是 L-Lipschitz 连续的，即：

$$\|\nabla L(\theta) - \nabla L(\theta')\| \le L\|\theta - \theta'\|, \quad \forall \theta, \theta' \tag{4.2.13}$$

则可证明当 $\eta < 2/L$ 时，梯度下降保证每步损失不增（对凸函数则保证单调收敛）。直觉：L 是曲率的上界，步长过大会「跨过」极小值导致震荡。

SGD 的方差与 mini-batch。随机梯度下降（Stochastic Gradient Descent, SGD）用 mini-batch $B \subset \{1, \dots, N\}$ 估计梯度：

$$\tilde{g}_t = \frac{1}{|B|} \sum_{i \in B} \nabla \ell(f_\theta(x_i), y_i) \tag{4.2.14}$$

$\tilde{g}_t$ 是真实梯度 $\nabla L(\theta_t)$ 的无偏估计。其方差为：

$$\mathrm{Var}(\tilde{g}_t) = \frac{1}{|B|} \mathrm{Var}(\nabla \ell_i) \tag{4.2.15}$$

方差随 batch size $|B|$ 增大而线性减小。更大的 batch 更准确，但引入的随机噪声也更少——这一噪声在 §4.3 中将看到有正面意义。

动量法（momentum）。动量法引入速度向量 v_t，模拟质点在损失景观中的惯性运动：

$$v_{t+1} = \mu v_t + \nabla L(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta v_{t+1} \tag{4.2.16}$$

其中 $\mu \in [0, 1)$ 是动量系数。展开递推可得：

$$v_t = \sum_{k=0}^{t} \mu^k \nabla L(\theta_{t-k}) \tag{4.2.17}$$

即当前速度是历史梯度的指数加权和。物理类比：质点在黏性介质中滚动——历史方向提供惯性，有助于穿越平坦区域、抑制振荡方向的震荡。

Adam 的二阶矩估计与偏差修正。Adam（Adaptive Moment Estimation）同时维护一阶矩 m_t 和二阶矩 v_t：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{4.2.18}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{4.2.19}$$

其中 $g_t = \nabla L(\theta_t)$，逐元素平方。初始时 $m_0 = v_0 = 0$，导致早期估计偏小（偏向零）。式 (4.2.18) 和式 (4.2.19) 的偏差修正（bias correction）为：

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{4.2.20}$$

最终更新：

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{4.2.21}$$

核心思想：由式 (4.2.21) 可见，用 $\hat{v}_t$ 自适应地缩放每个参数的有效学习率——历史梯度幅值大的参数步长缩小，历史梯度幅值小的参数步长放大，使各参数的更新幅度更均匀。典型超参数：$\beta_1 = 0.9$，$\beta_2 = 0.999$，$\epsilon = 10^{-8}$。

与原书呼应：原书 §4.2 给出了 Adam 的更新规则并描述其为「为每个参数自适应地调整学习率」，本节完整推导了一阶 Taylor 展开来源、步长上界约束，以及偏差修正的数学必要性。

---

§4.3 高维非凸优化的反直觉：为什么 SGD 居然能工作

▶这一节在讲什么

为什么高维非凸优化看起来不可能，却在实践中奏效？经典理论告诉我们梯度下降会陷入局部极小值，但真正的障碍根本不是局部极小值——而是鞍点。

第一步：高维空间里局部极小值极为罕见。随机矩阵理论表明，一个临界点是纯局部极小（Hessian 全正定）的概率随维度 d 指数衰减，参数量达 10^8 时几乎为零。

第二步：SGD 的噪声恰好能逃离鞍点。噪声协方差矩阵 $\Sigma \approx \frac{\eta}{B} C(\theta)$ 在鞍点附近沿负曲率方向提供随机扰动，batch size 越小、学习率越大，逃逸越快。

第三步：极小值之间存在低损失连通路径。不同初始化训练出的解不是孤立点，而是一个连通流形——SGD 从几乎任何起点出发，都能漂流到这个流形上。

本节要拿走的一件事：SGD 能工作不是「运气好」，而是高维几何的必然——维度本身消灭了局部极小值的威胁。

直觉

神经网络的损失函数是一个高维非凸函数，参数量动辄十亿。经典优化理论的警告很清楚：梯度下降在非凸函数上会陷入局部极小值，找不到全局最优解。但现实中，随机初始化后用 SGD 训练大模型，几乎总能得到好的解。这个矛盾来自哪里？答案不是「神经网络很幸运」，而是经典理论的「局部极小值陷阱」警告，在高维空间里根本不成立。

本节出现四个互相关联的概念：**临界点的鞍点占比（随机矩阵理论）**、**SGD 噪声的逃逸机制**、**平坦极小值的偏好**、**模式连通性**。这条链揭示了高维优化的真实面貌：梯度下降在高维中对付的不是「陷阱」，而是「高速公路」。下面把这条链一环环铺开。

第一环：维度消灭了局部极小值——随机矩阵理论的结论

理解为什么局部极小值在高维中罕见，需要一个思想实验。

一个临界点（梯度为零的点）成为**局部极小值**，要求 Hessian 矩阵 $H = \nabla^2 \mathcal{L}$ 正定——所有 d 个特征值都严格为正。在二维空间，这要求两个特征值都正，这不算苛刻。但在 $d = 10^8$ 维空间，这要求 10^8 个特征值**全部**严格为正。把每个特征值的符号想象成一枚硬币：要让所有硬币全部正面朝上，概率随硬币数量指数衰减。

随机矩阵理论给出了这个概率的精确渐近形式。Dauphin 等人（2014）借助 Bray-Dean 计算，对满足某些统计假设的高维随机非凸函数证明：临界点是纯局部极小值（Hessian 正定）的概率以 $e^{-\Omega(d)}$ 速率衰减。当 $d \sim 10^8$ 时，这个概率已经远小于宇宙中的原子数分之一——在实践上，纯局部极小值**几乎不存在**。

绝大多数临界点是**鞍点**（Hessian 有至少一个负特征值），而不是局部极小值。高损失的鞍点（「坏的」鞍点）虽然多，但它们在某些方向上存在下坡路——只要找到这些方向，就能继续降低损失。问题从「如何逃离陷阱」变成了「如何找到鞍点的下坡方向」。这个问题的解，藏在 SGD 的噪声里。

第二环：SGD 噪声机制——为什么「嘈杂」反而是优势

在鞍点处，梯度恰好为零，确定性梯度下降（GD）会卡在这里动弹不得。但 SGD 不一样：每步使用的是 mini-batch 梯度，是真实梯度加上随机噪声。在鞍点附近，噪声的协方差矩阵为：

$$\Sigma(\theta) \approx \frac{\eta}{B} C(\theta), \quad C(\theta) = \frac{1}{N} \sum_i \nabla \ell_i \nabla \ell_i^\top - \nabla \mathcal{L} \nabla \mathcal{L}^\top \tag{4.3.1}$$

其中 B 是 batch size，$C(\theta)$ 是样本梯度的协方差矩阵。

由式 (4.3.1) 可见，$C(\theta)$ 的特征向量通常与 Hessian 的负曲率方向有重叠（因为梯度的方差在损失变化剧烈的方向更大）。这意味着 Σ 在负曲率方向有非零分量——SGD 的噪声会随机地将参数推离鞍点，沿负曲率方向滑落，以 $O(\eta/B)$ 的速率逃逸。

从这里可以读出一个工程上非直觉的推论：**减小 batch size 或增大学习率，会加速鞍点逃逸**——因为 η/B 增大，Σ 的尺度增大，随机扰动更强。这是「小 batch 有时训练更快」的真实原因之一，不是因为它计算了更多更新步，而是因为它提供了更强的探索能力。大 batch 训练（如 TPU/GPU 时代常用的超大 batch）面临的一个核心困难，正是 Σ 被 B 稀释后鞍点逃逸能力减弱，需要其他机制（如更强的学习率调度）补偿。

第三环：平坦极小值的偏好——SGD 的隐式正则化

不是所有极小值都一样好。SGD 不仅能够到达极小值，还**倾向于偏好特定类型的极小值**——这是 SGD 的「隐式正则化」效应。

考虑两类极小值：- **尖锐极小值**（sharp minima）：Hessian 最大特征值 $\lambda_{\max}$ 很大，意味着参数在极小值附近的微小变化会导致损失急剧上升；- **平坦极小值**（flat minima）：$\lambda_{\max}$ 很小，损失在极小值周围的宽广区域内变化缓慢。

SGD 为什么偏好平坦极小值？直觉：SGD 的更新步骤有随机噪声。尖锐极小值就像锥形山谷的尖底——噪声容易把参数弹出去；平坦极小值就像宽阔的盆地底部——噪声无论如何推都在极小值的「平底锅」里，模型保持稳定。SGD 会被噪声逐渐驱逐出尖锐极小值，最终在平坦极小值中安定下来。

更精确的分析：SGD 在稳态时满足一个 Fokker-Planck 方程，其稳态分布正比于 $e^{-2\mathcal{L}(\theta)/\sigma^2}$，其中 $\sigma^2 \propto \eta/B$（即式 (4.3.1) 中 Σ 的尺度）是噪声强度。这与物理中布朗运动在势场中的平衡分布同构——温度越高（σ^2 越大），粒子越倾向分布在宽大的势阱里，而非深而窄的势阱里。SGD 偏好平坦极小值的倾向，随着 η/B 增大而增强。

平坦极小值之所以重要，不仅是因为易于训练，还与**泛化能力**直接相关：平坦极小值处，参数在一定邻域内的微小扰动不大影响损失（和模型输出），模型不「依赖特定参数配置」，对训练集以外的数据也有更稳健的表现。这在 §4.4 的损失景观可视化中有直接的视觉呈现。

第四环：模式连通性——极小值不是孤岛，是连通的河网

如果高维损失景观中有这么多鞍点，极小值之间是什么关系？是否真的存在被高损失

隔离的孤立极小值?

Garipov 等人（2018）和 Izmailov 等人（2018）分别从实验和理论上给出了惊人的答案：**两个独立训练得到的神经网络解 θ_A 和 θ_B，可以用一条低损失的曲线（而非直线）连接起来**——沿着这条曲线，损失始终接近两个端点的损失值，没有高损失的山脊阻隔。

更一般地，多个极小值之间存在**低损失连通流形**（low-loss manifold）：这些极小值不是孤立的点，而是高维空间中一个高维的「平坦区域」，彼此相连。SGD 从几乎任意初始化出发，最终都能漂到这个流形的某处。

这解释了为什么不同的随机种子训练出来的模型性能相近（虽然参数值可能完全不同）：它们都落在了同一个连通流形上，只是流形上的不同位置。模型集成（model ensemble）的有效性也部分来源于此：流形上不同点的预测有细微差异，平均后减少了方差，提升了鲁棒性。

整条链的回头看

高维临界点分析　随机矩阵理论：局部极小概率　e^{-Ω(d)}（几乎全是鞍点）

↓ 鞍点处梯度为零，确定性GD卡死

SGD噪声协方差　Σ　(/B) · C( )，沿负曲率方向提供随机扰动 → 逃逸鞍点

↓ 逃逸后落入哪种极小值？

隐式正则化　小batch/大 偏好平坦极小值（宽盆地稳定，窄锥被噪声驱逐）

↓ 平坦极小值是否孤立？

模式连通性　低损失连通流形：极小值彼此连通，SGD从任意初始化漂到流形上

这条链揭示了一个深层事实：**SGD 之所以能工作，不是因为运气好碰到了全局最优，而是因为高维几何联合作用——维度消灭了局部极小值，噪声消灭了鞍点，隐式正则化筛选了好的解，连通性保证了可达性**。经典优化理论对「非凸问题无法求解」的悲观预言，在这四点面前彻底失效。这也意味着：任何把深度学习训练简单类比为「凸优化」的框架，都遗漏了高维优化最本质的特征。

严格

鞍点主导：随机矩阵理论的结论。设损失函数在临界点 θ^* 处的 Hessian 矩阵（海森矩阵）为 $H = \nabla^2 L(\theta^*) \in \mathbb{R}^{d\times d}$。局部极小值要求 H 正定（所有特征值 > 0），鞍点（saddle point）则要求 H 至少有一个负特征值。

Dauphin 等人（NIPS 2014）借助随机矩阵理论（random matrix theory）中的 Bray-Dean 计算，对高维随机非凸函数得出以下结论：在 d 维空间中，临界点处 Hessian 的负特征值比例 $\rho \in [0, 1]$ 决定其类型——$\rho = 0$ 对应局部极小，$\rho > 0$ 对应鞍点。纯局部极小（$\rho = 0$）的数量占总临界点数的比例随维度 d **指数衰减**：

$$\Pr[\text{纯局部极小}] \propto e^{-\Omega(d)} \tag{4.3.2}$$

这意味着在 $d \sim 10^8$ 量级的参数空间中，SGD 几乎不会遇到局部极小值——它面对的几乎全是鞍点。

Hessian 谱的半圆律（semicircle law）。在高维随机设定下，随机对称矩阵的特征值经验分布收敛到 Wigner 半圆分布：

$$\rho(\lambda) = \frac{1}{2\pi\sigma^2}\sqrt{4\sigma^2 - \lambda^2}, \quad |\lambda| \le 2\sigma \tag{4.3.3}$$

深度网络损失 Hessian 的谱在过参数化区域近似符合此规律（带有一部分大特征值的「bulk + spike」结构），说明 Hessian 的负特征值占主体，纯局部极小在谱意义上极为罕见。

SGD 噪声协方差与鞍点逃逸。 SGD 每步的梯度噪声向量 $\xi_t = \tilde{g}_t - \nabla L(\theta_t)$ 的协方差矩阵近似为：

$$\Sigma(\theta) \approx \frac{\eta}{B} \cdot C(\theta) \tag{4.3.4}$$

其中 B 是 batch size，$C(\theta) = \frac{1}{N}\sum_i \nabla\ell_i \nabla\ell_i^\top - \nabla L \nabla L^\top$ 是样本梯度的协方差矩阵。在鞍点附近，式 (4.3.4) 中 $C(\theta)$ 的特征向量与 Hessian 负曲率方向有重叠，Σ 提供了沿负曲率方向的随机扰动，使模型以 $O(\eta/B)$ 的速率逃离鞍点。减小 batch size 或增大学习率，均可加强这种逃逸能力。

模式连通性（mode connectivity）。 Garipov、Izmailov 等人（NeurIPS 2018）实验表明：两个独立训练得到的神经网络极小值 θ_A, θ_B，可以用一条损失值始终低的曲线 $\phi : [0,1] \to \mathbb{R}^d$ 连接，即 $L(\phi(t)) \le \max(L(\theta_A), L(\theta_B)) + \delta$ 对所有 t 成立，δ 很小。这说明极小值不是孤立的点，而是连通流形（connected manifold）的一部分，SGD 从几乎任何初始化出发均可漂到其中某处。

与原书呼应：原书 §4.3 引用 Dauphin et al. 2014 的随机矩阵论据，并描述了 SGD 噪声的隐式正则化效应，本节给出了鞍点主导的指数估计、谱分布形式，以及噪声协方差 $\Sigma \approx \frac{\eta}{B}C(\theta)$ 的结构。

---

§4.4 损失景观的可视化：黑暗中的地形图

▶这一节在讲什么

为什么要可视化损失景观？高维函数不能直接画出来，但沿随机二维切面采样可以揭示架构选择对优化难度的影响——这不只是好看的图，而是有可量化意义的曲率信息。

第一步：filter-wise 归一化消除尺度干扰。不归一化时，不同 filter 的参数尺度差异会主导切面图，掩盖真实曲率；按 filter 归一化后，才能公平比较不同架构的景观平坦度。

第二步：平坦极小值和泛化能力有可证明的关联。Hessian 谱半径 $\lambda_{\max}$ 越小，泛化上界越紧——小 batch SGD 倾向于找到这类平坦解，大 batch SGD 则常陷入尖锐极小值。

第三步：SAM 把「找平坦解」显式化为优化目标。它最小化参数邻域内最坏情况的损失，强制解落在平坦区域，代价是每步需要两次前向-反向传播。

本节要拿走的一件事：损失景观的「平坦度」不是审美概念，而是泛化界中可以量化的几何量，skip connection 是让景观变平坦的最简单结构改动。

直觉

高维损失函数生活在我们无法直接感知的空间里。§4.3 讨论了这个空间的理论性质，但能不能「看到」它？Li 等人（2018）提出了一种方法：沿随机二维切面采样损失值，把它画出来。这不只是好看的图——切面图的形状携带了可量化的曲率信息，而曲率直接关联泛化能力。

本节出现四个递进的概念：**filter-wise 归一化（消除尺度假象）**、**平坦度与泛化界（量化连接）**、**skip connection 的几何效果（架构决定景观）**、**SAM（把平坦性显式化为目标）**。理解这条链，是理解为什么「选择好的架构」和「找到平坦极小值」最终是同一件事的关键。下面把这条链一环环铺开。

第一环：filter-wise 归一化——让可视化反映真实曲率

最朴素的可视化方案：选两个随机方向 $d_1, d_2 \in \mathbb{R}^P$，在以极小值 θ^* 为中心的二维切面上采样损失：$\mathcal{L}(\theta^* + \alpha d_1 + \beta d_2)$。但这个方案有一个严重缺陷：不同层、不同 filter 的参数尺度可能差异巨大（某层权重的 Frobenius 范数是另一层的百倍），随机方向 d_1, d_2 的分量自然是各向同性的，与参数尺度不匹配，切面图完全被尺度差异主导，看到的不是曲率，而是「哪层参数最大」。

Li 等人提出的 **filter-wise 归一化**解决了这个问题：将随机方向向量的每个 filter 分量缩放，使其 Frobenius 范数与对应参数 filter 的范数相等：

$$d_i^{(l,k)} \leftarrow d_i^{(l,k)} \cdot \frac{\|W^{(l,k)}\|_F}{\|d_i^{(l,k)}\|_F} \tag{4.4.1}$$

其中 (l,k) 表示第 l 层第 k 个 filter。由式 (4.4.1) 归一化后，切面图的「步长」与参数的实际尺度对齐，显示的曲率信息才是真实的几何性质，而非参数规模的人为效应。

这个细节之所以重要，是因为**不做归一化可能得出完全相反的结论**：一个参数尺度不均匀的架构可能在未归一化的切面图里看起来很「平坦」，而另一个参数均匀的架构反而看起来「陡峭」，但实际曲率关系完全相反。方法论上，filter-wise 归一化是「让可视化工具的量纲与物理量对齐」的必要步骤。

第二环：平坦度与泛化界——Hessian 最大特征值是可测量的泛化指标

§4.3 定性地说「平坦极小值泛化好」。这里给出定量版本。

Keskar 等人（2017）的核心实验发现：用大 batch SGD 训练收敛到的极小值，Hessian 最大特征值 $\lambda_{\max}$ 大（尖锐极小值），测试集误差高；用小 batch SGD 训练收敛到的极小值，$\lambda_{\max}$ 小（平坦极小值），测试集误差低。两个极小值的训练损失接近，但泛化性能差距显著。

为什么 $\lambda_{\max}$ 小意味着泛化好？一个朴素但有力的论证：极小值处的训练损失很低，但真实风险（测试集损失）通常更高。两者的差距（泛化误差）由「模型有多少依赖训练集特有结构」决定。在平坦极小值附近，参数的微小扰动（对应训练集与测试集之间的分布差异）几乎不改变损失——模型不「依赖特定参数值」，泛化误差小。在尖锐极小值附近，微小扰动就导致损失急剧上升，说明模型对训练集特有的数据结构高度敏感，泛化误差大。

存在如下形式的泛化上界：

$$R(\theta) \leq \hat{R}(\theta) + O\left(\sqrt{\frac{\lambda_{\max} \cdot n}{N}}\right) \tag{4.4.2}$$

其中 $R(\theta)$ 是真实风险，$\hat{R}(\theta)$ 是经验风险，n 是参数数量，N 是样本数。式 (4.4.2) 中 $\lambda_{\max}$ 出现在上界里，提供了从可测量的几何量（Hessian 谱）到泛化能力的直接量化连接。

第三环：skip connection 的几何效果——架构直接塑造损失景观

Li 等人（2018）的可视化实验给出了一个令人印象深刻的发现：有 skip connection（残差连接）的 ResNet，损失景观宽阔、平滑；去掉 skip connection 的相同深度网络，损失景观陡峭、混乱，充满尖锐的脊线和峡谷。

这个对比不只是视觉上的：有 skip connection 的网络同时表现出更低的 $\lambda_{\max}$ 和更好的测试集性能。架构选择直接影响了损失景观的几何，进而影响了 SGD 的训练动力学和最终的泛化能力。

为什么 skip connection 能让景观变平坦？直觉上，残差连接 $h = F(x, W) + x$ 确保了即使 F 的参数发生变化，输出仍有 x 的「稳定基线」。当深层网络中某层参数发生较大变化时，skip connection 使得对激活的影响是加性的（而非乘性的），避免了连续多层的非线性放大效应。数学上，skip connection 改变了损失的 Hessian 的特征值结构，抑制了大特征值的出现，使得 $\lambda_{\max}$ 更小、景观更平坦。

从这里可以得出一个深刻的架构设计原则：**优化友好的架构 = 平坦的损失景观 = 更好的泛化**。这三者之间有可量化的数学连接，不是经验直觉。

第四环：SAM——把平坦性直接作为优化目标

前三环是「观察」：我们发现平坦极小值更好。Foret 等人（2021）提出的 SAM（Sharpness-

Aware Minimization）走出了下一步：**把寻找平坦解显式化为优化目标**，而不只是希望 SGD 的隐式偏好自然产生平坦解。

SAM 的核心思想：与其最小化一个点处的损失，不如最小化**参数邻域内最坏情况的损失**：

$$\min_{\theta} \mathcal{L}^{\text{SAM}}(\theta) := \max_{\|\epsilon\| \le \rho} \mathcal{L}(\theta + \epsilon) \tag{4.4.3}$$

若某参数 θ 处于尖锐极小值，其邻域内存在损失显著更高的点 $\theta + \epsilon^*$；若处于平坦极小值，邻域内的最坏情况损失与 θ 处相近。式 (4.4.3) 定义的 SAM 通过最小化「最坏邻域损失」，强制优化器朝着平坦区域移动。

SAM 的算法实现是一个两步过程：先找到使损失最大的扰动 ϵ^*（一步梯度上升），再在 $\theta + \epsilon^*$ 处计算梯度用于参数更新（梯度下降）。这需要两次前向-反向传播，计算开销加倍，但经验上泛化提升显著，在图像分类、语言模型微调等任务上均有报告。

SAM 与 §4.3 的关联：SAM 本质上是在训练过程中主动推离尖锐极小值（类似于施加一个「曲率惩罚」），与 SGD 的隐式偏好同向而更加主动。两者都指向同一个结论：**平坦极小值是神经网络训练中真正的「黄金」**，找到它的方式（隐式的 SGD 噪声 vs 显式的 SAM 目标）是方法的差异，而非目标的差异。

整条链的回头看

```
随机切面 + filter-wise归一化      可视化真实曲率（消除尺度假象）
                                          ↓ 量化「平坦度」
Hessian _max                      泛化上界  √( _max · n / N)（可测泛化指标）
                                          ↓ 哪种架构产生小 _max？
Skip connection                   残差连接平坦化损失景观（架构→几何→泛化）
                                          ↓ 能否主动优化平坦性？
SAM                               显式最小化邻域最坏损失（平坦性成为优化目标）
```

这条链揭示了一个深层事实：「**选好架构**」**和**「**找平坦极小值**」**在数学上是同一件事的两个面向**。架构（如 skip connection）改变了损失景观的几何，而损失景观的几何（$\lambda_{\max}$）通过泛化界直接决定了模型在真实数据上的表现。可视化工具是连接「架构设计师直觉」与「泛化理论数学」的桥梁——它把不可见的高维几何翻译成可以感知、比较、优化的形状。

严格

Filter-wise normalization 切片方法。Li 等人（NeurIPS 2018）的可视化方案：选取两个随机方向 $d_1, d_2 \in \mathbb{R}^P$，对每个方向按 filter 归一化（filter-wise normalization）——将 d_1 的每个 filter 缩放使其与 θ^* 中对应 filter 的 Frobenius 范数相等，消除参数尺度差异。随后在二维坐标 (α, β) 上绘制：

$$L(\alpha, \beta) = L(\theta^* + \alpha d_1 + \beta d_2) \tag{4.4.4}$$

不做 filter-wise normalization 时，方向尺度与参数尺度不匹配，可视化会被参数量级的差异所主导，无法反映真实曲率。

平坦极小值 vs 尖锐极小值（flat vs sharp minima）。设极小值处 Hessian 的最大特征值

（谱半径）为 $\lambda_{\max}$。Keskar 等人（ICLR 2017）的实验表明：

- 大 batch SGD 收敛到尖锐极小值（large $\lambda_{\max}$），测试集性能差；
- 小 batch SGD 收敛到平坦极小值（small $\lambda_{\max}$），测试集性能好。

泛化界（generalization bound）。对平坦极小值，存在如下形式的泛化上界：

$$R(\theta) \leq \hat{R}(\theta) + O\left(\sqrt{\frac{\lambda_{\max} \cdot n}{N}}\right) \tag{4.4.5}$$

其中 $R(\theta)$ 是真实风险，$\hat{R}(\theta)$ 是经验风险，n 是参数数量，N 是样本数。该界说明：$\lambda_{\max}$ 越小（极小值越平坦），泛化误差上界越紧，模型在未见数据上的性能越有保障。直觉：平坦极小值意味着参数在其邻域内小扰动不改变模型输出，模型「不依赖特定参数值」，故泛化更好。

SAM（Sharpness-Aware Minimization）。Foret 等人（2021）将平坦性显式化为优化目标：

$$\min_{\theta} L^{\text{SAM}}(\theta) := \max_{\|\epsilon\| \leq \rho} L(\theta + \epsilon) \tag{4.4.6}$$

即在参数邻域内最坏情况的损失最小化，强制解落在平坦区域。SAM 需要两次前向-反向传播（先求 ϵ^*，再计算更新），计算开销加倍，但泛化通常改善显著。

与原书呼应：原书 §4.4 描述了 Li et al. 2018 的可视化工作及 skip connection 对损失景观的影响，本节补充了 filter-wise normalization 的数学必要性、泛化界的具体形式，以及 SAM 的优化目标。

---

§4.5 μP：超参数迁移的”元定律”

▶这一节在讲什么

为什么小模型调好的学习率不能直接用于大模型？标准初始化下，激活的更新量 Δh 随宽度 n 以 $O(\sqrt{n})$ 发散，最优学习率必须随宽度缩放——迁移根本不成立。

第一步：找到发散的根源。参数更新量 $\Delta W \sim O(\eta/\sqrt{n})$，乘以宽度为 n 的激活求和，得到 $\Delta h \sim O(\eta\sqrt{n})$——宽度一翻倍，激活更新翻 $\sqrt{2}$ 倍。

第二步：μP 通过层依赖缩放修复这个发散。隐藏层学习率（Adam 下）保持 $O(1)$，输出层按 $O(1/n)$ 缩减，初始化方差也按层类型相应调整，使所有层 $\Delta h = O(1)$。

第三步：μP 是唯一保持「特征学习」的宽度极限。NTK 参数化下 $\Delta h \to 0$，模型退化为固定核方法；标准参数化下 Δh 发散；只有 μP 在 $n \to \infty$ 时维持 $O(1)$ 的特征更新。

本节要拿走的一件事：μP 不是调参技巧，而是唯一让超参数在宽度变化时保持稳定的代数约束。

直觉

任何做过大模型预训练的工程师都碰到过这个问题：在小代理模型（如宽度 256 的网络）上精心调好的学习率，放到生产规模的模型（宽度 8192）上直接用，训练要么不稳定要么效果很差。重新在大模型上搜索超参数，算力开销是小模型的几十倍。这个痛点有数学根源，而 μP（Maximal Update Parametrization，最大更新参数化）提供了从数学上彻底消灭这个问题的方案。

本节出现四个递进的概念：**标准参数化（SP）的尺度失控**、**激活更新量 Δh 的分析**、**μP 的缩放修复方案**、**NTK 极限 vs 特征学习极限**。理解这条链，是理解「为什么 μP 是工程上的基础设施而非调参技巧」的关键。下面把这条链一环环铺开。

第一环：标准参数化（SP）——问题的诊断

先弄清楚问题在哪里。以一个两层网络为例，宽度为 n，权重矩阵 $W \in \mathbb{R}^{n\times n}$，按 He 初始化 $W_{ij} \sim \mathcal{N}(0, 1/n)$。

前向传播：$h = Wx$，其中 $x \in \mathbb{R}^n$ 是激活，$\mathbb{E}[\|h\|^2] = O(n)$（每个分量方差为 $O(1)$，n 个分量相加，$\|h\|^2 \sim O(n)$），单个分量 $h_j \sim O(1)$——这是 He 初始化设计的目标，确保前向激活稳定。

反向传播梯度：$g = \nabla_W \mathcal{L} = \delta \cdot x^\top$，其中 $\delta \in \mathbb{R}^n$ 是来自上层的梯度信号，$\|g\|_F^2 = \|\delta\|^2\|\mathbf{x}\|^2 \sim O(n \cdot n) = O(n^2)$，单个分量 $g_{jk} \sim O(1/\sqrt{n} \cdot 1) \approx O(1/\sqrt{n})$（因为 $\delta_j \sim O(1/\sqrt{n})$，通过权重矩阵 W 传来的梯度每个分量量级是 $O(1/\sqrt{n})$）。

参数更新量：$\Delta W = -\eta \cdot g$，每个分量量级 $O(\eta/\sqrt{n})$。

激活变化：$\Delta h = \Delta W \cdot x$，每个分量：

$$\Delta h_j = \sum_{k=1}^{n} \Delta W_{jk} x_k \tag{4.5.1}$$

由式 (4.5.1)，n 个大小为 $O(\eta/\sqrt{n})$ 的项相加，每项独立，方差 $O(\eta^2/n \cdot 1) = O(\eta^2/n)$，因此 $\Delta h_j \sim O(\eta/\sqrt{n} \cdot \sqrt{n}) = O(\eta)$——等等，等一下。

让我们更仔细地分析：$\Delta W_{jk} x_k$ 的方差是 $\mathrm{Var}(\Delta W_{jk}) \cdot \mathbb{E}[x_k^2] \sim O(\eta^2/n) \cdot O(1) = O(\eta^2/n)$。对 n 项求和，$\mathrm{Var}(\Delta h_j) = n \cdot O(\eta^2/n) = O(\eta^2)$，即 $\Delta h_j \sim O(\eta)$。

这说明在标准 He 初始化下，单步激活变化量 $\Delta h \sim O(\eta)$，与宽度 n 无关——这是好的！但等等，这是 SGD 的情况。**对于 Adam**，情况截然不同：Adam 的更新规模被自适应归一化，ΔW 的有效尺度近似为 $O(\eta)$（而非 $O(\eta/\sqrt{n})$），因此 $\Delta h = \Delta W \cdot x$ 变成 $O(\eta) \cdot O(1)$ 的 n 项之和，结果是 $\Delta h \sim O(\eta\sqrt{n})$——**随宽度 $\sqrt{n}$ 发散**。

这正是标准参数化 + Adam 的问题根源：Adam 的二阶矩归一化消除了梯度的尺度信息，使得参数更新的有效幅度不再随宽度衰减，激活变化量因此随宽度发散。宽度翻倍，激活变化量乘以 $\sqrt{2}$，最优学习率因此必须随宽度缩放——迁移不成立。

第二环：NTK 极限——过于保守的另一个极端

在诊断 SP 的问题之前，先看另一个极端：神经切线核（Neural Tangent Kernel, NTK）参数化。NTK 参数化选择了非常保守的初始化和学习率，使得训练过程中参数几乎不动：

- 初始化：$W_{ij} \sim \mathcal{N}(0, 1/n)$（与 He 初始化相同）；
- 学习率：$\eta = O(1/n^2)$（远比标准学习率小）；

- 结果：参数更新量 $\Delta W \sim O(\eta) = O(1/n^2)$，激活变化量 $\Delta h \sim O(\eta n) = O(1/n) \to 0$（当 $n \to \infty$）。

NTK 参数化下（$\Delta h \to 0$），激活在整个训练过程中几乎不变，网络退化为固定核方法（「懒学习」，lazy learning）：特征不更新，模型只在固定特征上学一个线性分类器。这虽然数学上可精确分析（核方法有完整理论），但丧失了深度学习最重要的能力——**特征学习**（学习对任务有用的表示）。NTK 极限解释了为什么宽度无穷大的浅层网络会退化为核方法，也解释了为什么宽度有限的实际网络比宽度无穷大的 NTK 极限更强大。

第三环：μP 的修复方案——找到特征学习的唯一稳定极限

标准参数化（SP）激活发散，NTK 参数化激活消失——有没有中间地带，让激活变化量在 $n \to \infty$ 时稳定在 $O(1)$，既不发散也不消失，保留特征学习能力？

μP 的答案是：对不同类型的层使用不同的初始化方差和学习率缩放。核心缩放规则（Adam 下）：

- **输入嵌入层**（$V \to n$）：初始化方差 $O(1)$，Adam 学习率 $O(1)$；
- **隐藏—隐藏层**（$n \to n$）：初始化方差 $O(1/n)$，Adam 学习率 $O(1)$（不随宽度缩放！）；
- **输出层**（$n \to V$）：初始化方差 $O(1/n^2)$，Adam 学习率 $O(1/n)$。

为什么隐藏层的 Adam 学习率可以保持 $O(1)$ 不缩放？关键在于初始化方差的调整。隐藏层初始化方差 $O(1/n)$，使得初始参数范数更小；Adam 的二阶矩自适应归一化后，有效更新量 Δh 恰好维持在 $O(1)$ 量级。具体推导见 §4.7。

输出层需要额外的 $O(1/n)$ 学习率衰减，是因为输出层的激活本质上是 n 个隐藏单元的求和，求和后幅度自然 $\sim O(\sqrt{n})$（随机游走），必须额外缩放才能让最终逻辑输出保持 $O(1)$。

第四环：μP 的工程价值——超参数迁移的实现

μP 的核心工程价值在于：在 μP 参数化下，最优超参数（学习率、权重衰减、动量系数等）在不同宽度下保持**稳定不变**——你在宽度 256 的代理模型上调好的学习率 η^*，在宽度 8192 的大模型上直接用，结果基本一样好。

这使得超参数搜索的算力成本降低了 2-3 个数量级：在小代理模型（占实际训练成本的 1% 不到）上跑完整的超参数搜索，然后零代价迁移到大模型。Microsoft 研究团队的实验（Yang et al., 2022）在 Transformer 上验证了这一点：μP 下的超参数迁移在不同规模（10M 到 1B 参数）之间误差 < 5%，而标准参数化下误差高达 50% 以上。

μP 还有一个隐含的理论含义：它定义了「宽度极限下超参数的正确行为」。超参数只有在 μP 下才有稳定的「热力学极限」，在 SP 下最优超参数随宽度漂移、没有收敛值。这从理论上证明了 μP 是「对宽度缩放正确的参数化方式」，不是众多可选方案之一，而是唯一满足超参数稳定性的方案。

整条链的回头看

```
SP + Adam     Δh   O(√n)（激活随宽度发散）    最优 随宽度缩放     迁移失败
   ↓ 另一极端
NTK参数化     Δh   O(1/n) → 0（激活消失）    特征冻结     退化为核方法
   ↓ 中间地带：Δh = O(1)
```

P缩放规则　初始化方差O(1/n) + 输出层 ×O(1/n)　Δh = O(1)（稳定）
↓ 直接工程效果
超参数迁移　小代理模型调参　零成本迁移到大模型（节省2-3量级算力）

这条链揭示了一个深层事实：**超参数迁移不成立是参数化方案的数学缺陷，而非调参经验的不足**。μP 修复的是激活动力学的尺度问题，这个修复有唯一性——在「既要特征学习，又要超参数在宽度下稳定」的双重约束下，μP 是唯一的解。工程上的「小模型调参再迁移大模型」这一实践，是这条数学链的直接应用。

严格

标准参数化（Standard Parametrization, SP）的尺度失控。设隐藏层宽度为 n，权重 $W \in \mathbb{R}^{n\times n}$ 按 He 初始化 $W_{ij} \sim \mathcal{N}(0, 1/n)$，学习率为 η。考察单步更新对激活的影响：

- 前向激活：$h^{(l)} = Wh^{(l-1)} \in \mathbb{R}^n$，方差 $\mathrm{Var}(h_j^{(l)}) = O(1)$（设计目的）；
- 反向梯度：$g^{(l)} = W^\top g^{(l+1)} \odot \sigma'(z^{(l)})$，尺度 $O(1/\sqrt{n})$（因 $W^\top$ 的谱范数 $\sim O(1)$ 但分量求和数目为 n，结果分量方差为 $O(1/n)$，尺度为 $O(1/\sqrt{n})$）；
- 参数更新：$\Delta W = -\eta\, g^{(l+1)}(h^{(l)})^\top$，尺度 $O(\eta/\sqrt{n})$；
- 激活变化：$\Delta h^{(l)} = \Delta W \cdot h^{(l-1)}$，尺度 $O(\eta\sqrt{n})$——**随宽度发散**。

为使 $\Delta h = O(1)$，需令 $\eta = O(1/n)$，即**最优学习率随宽度线性衰减**，迁移不成立。

μP 的缩放规则。Yang & Hu 在《Tensor Programs V》中证明，通过以下层依赖缩放可实现所有层 $\Delta h = O(1)$：

层类型	初始化方差 σ^2	SGD 学习率	Adam 学习率
输入嵌入 $V \to n$	$O(1)$	$O(1)$	$O(1)$
隐藏 $n \to n$	$O(1/n)$	$O(1/n)$	$O(1)$
输出 $n \to V$	$O(1/n^2)$	$O(1/n)$	$O(1/n)$

其中 V 为词表大小。关键点：在 Adam 下，隐藏层学习率无需随宽度衰减（因 Adam 的二阶矩归一化已吸收了梯度尺度的变化），仅输出层需要 $O(1/n)$ 的衰减。

与 SP 和 NTK 参数化的对比。神经切线核（Neural Tangent Kernel, NTK）参数化是另一种极限：隐藏层初始化方差 $O(1/n)$，学习率 $O(1/n^2)$，使得整个训练过程中参数几乎不动，模型退化为固定核方法（「懒学习」，lazy regime），无法真正学习特征。μP 则保持了「特征学习极限」（feature learning limit）：参数在训练中有 $O(1)$ 的实质更新，模型确实在学习新的特征表示。

$$\text{NTK} \subset \text{SP} \subset \mu\text{P} \tag{4.5.2}$$

（从左到右：核极限 ⊂ 标准参数化 ⊂ 特征学习极限）

三者的本质区别在于激活变化 Δh 的尺度：NTK 下为 $O(1/n)$，SP 下为 $O(\sqrt{n})$（随宽度发散），μP 下为 $O(1)$（稳定）。只有 μP 在宽度 $n \to \infty$ 时保持非平凡的特征动力学。

与原书呼应：原书 §4.5 描述了 μP 的超参数迁移实证效果，本节补充了尺度失控的完整推导链（$\Delta h \sim O(\eta\sqrt{n})$）以及 SP/NTK/μP 三者的对比表与特征学习含义。

§4.6 Lion 与 Shampoo：Adam 不是终点

▶这一节在讲什么

为什么要挑战 Adam？Adam 需要同时存储一阶矩和二阶矩，优化器状态内存占总显存的大头；同时，Adam 对学习率和权重衰减的耦合在大规模训练中引发微妙的调参问题。

第一步：Lion 用符号代替幅值，内存减半。更新方向只取动量的 sign，每个参数每步只做 $\pm\eta$ 的跳跃，丢弃梯度幅值信息，只需维护一份动量而非两份。

第二步：Shampoo 把二阶方法用 Kronecker 分解重新带回来。维护左右两个小矩阵 $L_t \in \mathbb{R}^{m\times m}$ 和 $R_t \in \mathbb{R}^{n\times n}$，计算复杂度从 $O((mn)^3)$ 降到 $O(m^3+n^3)$，近似精确 Gauss-Newton 预条件子。

第三步：SOAP 融合两者之长。把梯度旋转到 Kronecker 矩的特征基中再运行 Adam，兼得二阶预条件与自适应性，减少约 40% 训练步数。

本节要拿走的一件事：优化器的竞争不是「谁更聪明」，而是在内存、算力、收敛速度三者之间寻找新的帕累托前沿。

直觉

Adam 在大模型训练中统治了近十年，但它并非无懈可击。理解它的局限，是理解 Lion 和 Shampoo 存在的原因。

Adam 的本质是一个**对角预条件子**（diagonal preconditioner）：$\hat{v}_t$ 只是每个参数梯度的历史均方值，对参数之间的相关性完全视而不见。想象两个参数 θ_1 和 θ_2：如果它们的梯度高度相关（因为它们影响同一个激活），那么对它们各自做「历史梯度幅值归一化」是有效的；但如果两个参数之间存在重要的混合曲率（损失函数的 $\partial^2\mathcal{L}/\partial\theta_1\partial\theta_2$ 不为零），对角预条件子就会错过这些信息，更新方向不准确。在大模型的权重矩阵（行列之间高度相关）中，这种遗漏可能很显著。

另一个问题是内存。在混合精度训练中，模型权重用 bf16（2 字节），但优化器状态（一阶矩 m_t 和二阶矩 v_t）需要 fp32（4 字节）的精度，两份状态加起来占用了 $2\times 4 = 8$ 字节/参数，远超权重本身的 2 字节。对千亿参数模型，优化器状态可能占据总显存的 60-70%，成为训练成本的主要瓶颈之一。

本节出现三个概念：**Lion（符号化的内存高效优化器）**、**Shampoo（Kronecker 分解的近似二阶方法）**、**SOAP（两者的融合）**。它们分别代表三种不同的突破思路，映射到「内存—算力—精度」三角形的三个顶点上。下面把这条链一环环铺开。

第一环：Adam 的二阶矩——既是优势也是负担

Adam 维护两份状态：一阶矩 m_t（梯度方向的指数加权平均，提供动量）和二阶矩 v_t（梯度平方的指数加权平均，提供自适应步长）。自适应步长 $\eta/(\sqrt{v_t}+\epsilon)$ 让不同参数获得不同的有效学习率，是 Adam 在稀疏梯度场景（如词嵌入）优于 SGD 的核心原因。

但二阶矩的信息是对角的。精确的二阶信息（Hessian 矩阵）包含了所有参数对之间的曲率，其维度是 $d\times d$（d 为参数总数），对大模型完全不可行。Adam 的 v_t 是 Hessian 对角线的近似，代价是完全丢失了非对角项（参数间的相关曲率）。

Adam 的另一个隐患：在 AdamW（Adam with decoupled weight decay）之前，Adam 和权重衰减（L2 正则化）的结合是有问题的——Adam 的自适应步长会干扰权重衰减的效果，使得有效的正则化强度随参数的历史梯度幅值而变化，调参变复杂。AdamW 通过将权重衰减从梯度更新中解耦解决了这个问题，但根本上仍是一个对角预条件子。

第二环：Lion——「方向比幅值更可靠」的极简主义

Lion（EvoLved Sign Momentum）是通过符号优化（program search）在大量候选更新规则中自动演化出来的算法，核心更新规则是：

$$c_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad \theta_{t+1} = \theta_t - \eta \cdot \operatorname{sign}(c_t) - \eta\lambda\theta_t \tag{4.6.1}$$

由式 (4.6.1) 可见，每个参数每步的更新量恰好等于 $\pm\eta$，完全不依赖梯度的幅值，只依赖方向（符号）。然后动量更新：$m_t = \beta_2 m_{t-1} + (1-\beta_2) g_t$。

Lion 的设计哲学：在损失景观高度各向异性（不同方向曲率差异极大）的情况下，梯度幅值携带的信息更多来自于「这个方向曲率大」，而非「这个方向梯度应该走多大步」——幅值是曲率的混合，并非纯净的步长信号。符号则是更「干净」的方向信号，不受幅值噪声的干扰。

工程上的主要优势：Lion 只需维护**一份动量** m_t（用于计算 c_t 和更新动量），而 Adam 需要维护两份（m_t 和 v_t）。优化器状态内存从 Adam 的 8 字节/参数降到 Lion 的约 4 字节/参数，节省约 50%。在百亿参数规模的训练中，这意味着省出了一倍以上的 HBM 空间，可以用来容纳更大的 batch 或更大的模型。

Lion 的局限：每步更新幅度固定为 $\pm\eta$，对学习率的选择更敏感（没有自适应步长作为「缓冲」）；在梯度高度稀疏的场景（如词嵌入里罕见词的更新），符号信息可能不如 Adam 的二阶矩准确。

第三环：Shampoo——用 Kronecker 分解把二阶方法带回来

Lion 的路径是「做减法」（砍掉幅值信息），Shampoo 的路径是「做加法」（引入更准确的曲率信息），但以可承受的代价实现。

完整的二阶方法（牛顿法）需要 $d \times d$ 的 Hessian 逆，$d \sim 10^9$ 时存储就需要 10^{18} 个浮点数，根本不可行。Shampoo 的洞见是：**权重矩阵天然有矩阵结构**，可以在矩阵的两个维度上分别维护低维的二阶统计，利用 Kronecker 积分解近似完整 Hessian 逆。

对权重矩阵 $W \in \mathbb{R}^{m\times n}$，Shampoo 维护：

$$L_t = \sum_{s=1}^{t} G_s G_s^\top \in \mathbb{R}^{m\times m}, \quad R_t = \sum_{s=1}^{t} G_s^\top G_s \in \mathbb{R}^{n\times n} \tag{4.6.2}$$

更新规则（见式 (4.6.4)）：$W_{t+1} = W_t - \eta \cdot L_t^{-1/4} G_t R_t^{-1/4}$。

核心思想：精确 Gauss-Newton 预条件子的 Kronecker 近似是 $(L_t \otimes R_t)^{-1/2}$，Shampoo 用 $L_t^{-1/4} \otimes R_t^{-1/4}$ 来近似它（矩阵乘法形式：$L_t^{-1/4} G_t R_t^{-1/4}$）。计算 $m \times m$ 矩阵的四次根需要 $O(m^3)$，计算 $n \times n$ 矩阵的四次根需要 $O(n^3)$，总代价 $O(m^3 + n^3)$，远小于精确 Hessian 的 $O((mn)^3)$。对典型的 Transformer 隐藏维度 $m, n \sim 10^4$，这个代价是可承受的。

Shampoo 的实证表现：相比 Adam，Shampoo 在相同步数内通常能达到更低的损失，即

收敛更快（步数更少），代价是每步的计算时间更长（矩阵分解）。是否值得取决于「步数 vs 每步成本」的权衡，在算力充裕但通信受限的场景（多机训练时通信是瓶颈），Shampoo 常常值得。

第四环：SOAP——在特征基里运行 Adam

SOAP（2024）融合了 Shampoo 的二阶结构和 Adam 的自适应性：将梯度旋转到 Kronecker 矩（L_t 和 R_t）的特征基中，在特征空间里运行 Adam，然后再旋转回来。

数学上，若 $L_t = U\Sigma U^\top$，$R_t = V\Lambda V^\top$ 是特征值分解，则把梯度旋转到特征基 $\tilde{G} = U^\top G V$，在 $\tilde{G}$ 上运行 Adam（对角自适应），更新完成后旋转回来。这相当于在 Shampoo 的预条件方向上做 Adam，兼得了 Shampoo 的曲率捕获能力和 Adam 的细粒度自适应性。报告结果表明 SOAP 相比 AdamW 大约减少 40% 的训练步数，同时内存开销和 Shampoo 相当。

整条链的回头看

```
Adam     对角预条件子（高效但忽略非对角曲率）+ 双份状态（内存重）
  ↓ 极简方向：砍掉幅值
Lion     sign(动量)，固定± 更新，单份状态（内存减半），以步长敏感度为代价
  ↓ 精度方向：引入矩阵曲率
Shampoo     Kronecker分解的左右二阶矩，O(m³+n³)矩阵根，近似G-N预条件
  ↓ 融合：曲率 + 自适应
SOAP     在Kronecker特征基里运行Adam，步数减少~40%
```

这条链揭示了一个深层事实：**优化器设计是一个「信息量 vs 计算成本 vs 内存开销」的三角权衡**。Adam 是这个三角形的内点——不是任何一个维度的最优，但在三者之间的综合权衡下长期占据主导。Lion 是「内存优先」的前沿，Shampoo 是「收敛精度优先」的前沿，SOAP 是「综合权衡」的新尝试。理解这个三角形，是理解未来优化器演化方向的框架。

严格

Lion（EvoLved Sign Momentum）的更新规则。Chen 等人（ICML 2023）通过符号优化搜索得到：

$$
\begin{aligned}
c_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t \\
\theta_{t+1} &= \theta_t - \eta \cdot \operatorname{sign}(c_t) - \eta\lambda\theta_t \\
m_t &= \beta_2 m_{t-1} + (1-\beta_2) g_t
\end{aligned}
$$

其中 $g_t = \nabla L(\theta_t)$，λ 是权重衰减系数，$\operatorname{sign}(\cdot)$ 逐元素取符号。

核心区别：更新方向取 c_t 的符号，每个参数每步只做 $\pm\eta$ 的更新，完全丢弃梯度幅值信息。内存开销：只需维护 m_t（一份动量），而 Adam 需要 m_t 和 v_t 两份，节省约 1/3 的优化器状态内存（在混合精度训练中，Adam 优化器状态约占总内存的 2/3，Lion 将其减半）。

理论动机：当损失景观高度各向异性（不同方向曲率差异巨大）时，梯度幅值包含大量尺度噪声，符号反而是更稳健的方向信号。Lion 可以看作带动量的符号随机梯度下降（SignSGD with momentum），其收敛性在一定条件下可证明。

Shampoo 的 Kronecker 分解预条件。对权重矩阵 $W \in \mathbb{R}^{m\times n}$，梯度 $G_t \in \mathbb{R}^{m\times n}$，Sham-

poo 维护左右二阶矩统计：

$$L_t = \sum_{s=1}^{t} G_s G_s^\top \in \mathbb{R}^{m\times m}, \qquad R_t = \sum_{s=1}^{t} G_s^\top G_s \in \mathbb{R}^{n\times n} \tag{4.6.3}$$

更新规则为：

$$W_{t+1} = W_t - \eta \cdot L_t^{-1/4} G_t R_t^{-1/4} \tag{4.6.4}$$

预条件子 $P_L = L_t^{-1/4} \in \mathbb{R}^{m\times m}$，$P_R = R_t^{-1/4} \in \mathbb{R}^{n\times n}$ 近似了精确高斯-牛顿（Gauss-Newton）预条件子 $(L_t \otimes R_t)^{-1/2}$ 的 Kronecker 分解。计算复杂度：矩阵根 $L_t^{-1/4}$ 需要 $O(m^3)$，$R_t^{-1/4}$ 需要 $O(n^3)$，远低于精确 Hessian 逆的 $O((mn)^3)$。对 $m, n \sim 10^4$ 量级（变换器（Transformer）典型隐藏维度），每步预条件计算是可承受的。

SOAP（2024）在 Shampoo 基础上融合 Adam：将梯度旋转到 Kronecker 矩的特征基中，再在特征基里运行 Adam，兼得二阶预条件与 Adam 自适应性，报告相比 AdamW 减少约 40% 训练步数。

与原书呼应：原书 §4.6 描述了 Lion 的符号更新与内存优势，以及 Shampoo 的 Kronecker 分解框架和 SOAP 的融合结果，本节给出了完整的更新方程和计算复杂度分析。

---

§4.7 μP 的完整面貌：超参数迁移背后的数学

本节的关键论证：(1) 在无穷宽极限下，前向激活方差应保持 $\Theta(1)$（**激活稳定性条件**）；(2) 反向梯度方差也应保持 $\Theta(1)$（**梯度稳定性条件**）；(3) 同时满足两条 → 输入层、隐藏层、输出层的学习率必须按 $\Theta(1/n)$、$\Theta(1/n)$、$\Theta(1)$ 缩放（**μP 缩放律**）。**所有「凭空出现」的 $1/n$ 因子都来自这两条平衡条件**。

▶这一节在讲什么

为什么需要一个理论框架来理解 μP？ §4.5 给出了工程规则，但没有解释为什么这些规则是唯一的，以及违反它们会发生什么——Tensor Programs 框架给出了完整的答案。

第一步：宽度极限下激活收敛到高斯过程。当 $n \to \infty$ 时，随机初始化网络的所有层间激活的经验联合分布弱收敛到一个高斯过程，这使得极限行为可以用核函数精确描述。

第二步：abc 参数化把所有选择统一到三个指数。初始化指数 a、学习率指数 b、输出缩放指数 c 决定了网络属于哪个极限。特征学习要求 $a + b = 1$（隐藏层，Adam），违反则要么激活爆炸要么退化为核方法。

第三步：μP 是唯一的特征学习极限。NTK 参数化下 $\Delta h \to 0$（核极限），标准参数化下 Δh 发散，只有满足 $a+b=1$ 的 μP 在 $n \to \infty$ 时保持 $O(1)$ 的非平凡特征动力学。

本节要拿走的一件事：abc 参数化约束 $a+b=1$ 是「超参数在宽度变化下保持稳定」的充要条件，这条等式是所有 $1/n$ 缩放因子的统一来源。

直觉

§4.5 给出了 μP 的工程规则，但它是怎么被证明出来的？为什么这些规则是**唯**一的？如果违反某条规则会发生什么？这些问题都需要 Tensor Programs 框架来回答。本节深入到 μP 的理论根基，把 §4.5 的工程规则从「经验方法论」提升到「数学定理」。

本节出现四个核心概念：**宽度极限下的高斯过程收敛**、**abc 参数化空间**、$a+b=1$ **约束的来源与意义**、**特征学习极限 vs 核极限的完整对比**。理解这条链，不仅能解释「为什么是这些缩放规则」，还能理解「在参数化空间里，μP 处于哪个特殊位置、为什么无法替代」。下面把这条链一环环铺开。

第一环：宽度极限下的高斯过程——极限的存在性

Tensor Programs 框架的起点是一个无穷宽网络在随机初始化时的行为分析。

当隐藏层宽度 $n \to \infty$ 时，每个神经元的激活是独立同分布的随机变量的线性组合（由上一层宽度为 n 的激活经过权重矩阵相加得到）。由中心极限定理，这个线性组合在宽度趋向无穷时收敛到高斯分布。更精确地：对两个输入 $x^{(1)}, x^{(2)}$，第 l 层激活的内积 $\frac{1}{n} h^{(l)}(x^{(1)}) \cdot h^{(l)}(x^{(2)})$ 依概率收敛到某个确定极限 $K^{(l)}(x^{(1)}, x^{(2)})$。这个极限可以通过递推关系精确计算：

$$K^{(l)}(x, x') = \mathbb{E}_{f \sim \mathcal{N}(0, K^{(l-1)})}[\sigma(f(x))\sigma(f(x'))] \tag{4.7.1}$$

其中 σ 是激活函数。式 (4.7.1) 就是**神经网络核函数**（NTK 核）的来源。

高斯过程收敛告诉我们：宽度无穷大的网络在随机初始化时的行为完全由核函数决定。**不同参数化方案的区别，完全体现在「训练一步后，激活偏离了初始化时的高斯过程有多远」**。偏离为零（激活不动）= NTK 极限；偏离有限（激活做 $O(1)$ 的更新）= 特征学习极限；偏离无穷（激活爆炸）= 参数化不合法。

第二环：abc 参数化——把所有可能的选择统一到三个数

为了系统地分析所有可能的参数化方案，Yang & Hu 引入了 abc 参数化：对每个权重矩阵，用三个指数 (a, b, c) 描述其初始化尺度、学习率尺度和（对输出层而言的）输出缩放尺度：

$$W = n^{-a}\tilde{W}, \quad \tilde{W}_{ij} \sim \mathcal{N}(0,1); \quad \eta_W = n^{-b}\tilde{\eta}; \quad \text{输出缩放} = n^{-c} \tag{4.7.2}$$

在这个参数化下：- 初始化时，W 的每个分量量级为 n^{-a}；- 每步参数更新量（Adam 下）量级为 $n^{-b} \cdot \tilde{\eta}$（Adam 的自适应归一化使得有效更新量与梯度的相对尺度无关，只由学习率决定）；- 激活变化量 $\Delta h \sim n^{-b} \cdot n^{1/2} = n^{1/2-b}$（$n$ 个大小为 n^{-b} 的更新项之和，方差相加给出 $\sqrt{n}$ 倍）。

要使式 (4.7.2) 中的 $\Delta h = O(1)$，需要 $1/2 - b = 0$，即 $b = 1/2$。

但这里还有初始化的约束。训练时，参数更新量 $\Delta W \sim n^{-b}$，前向传播 $\Delta h = \Delta W \cdot h$ 中 $h \sim O(1)$（由初始化保证），n 项相加，$\Delta h \sim n^{-b+1/2}$。要 $\Delta h = O(1)$，需要 $b = 1/2$。同时，初始前向传播要求 $Wh \sim O(1)$，即 $n^{-a+1/2} \cdot n^{1/2} = n^{-a+1}$ 的量级（n 项大小为 n^{-a} 的随机量之和），这要求 $-a + 1/2 \cdot 2 = 0$，即对应 He 初始化 $a = 1/2$。

等等，上述推导是 SGD 下的。在 Adam 下，二阶矩归一化消除了梯度的尺度依赖，有效更新量 ΔW^{Adam} 的量级与初始化无关（由学习率 η 决定），$\Delta h^{\text{Adam}} = \Delta W^{\text{Adam}} \cdot h \sim \eta \cdot \sqrt{n}$。要使 $\Delta h = O(1)$，需要 $\eta = O(n^{-1/2})$，即 $b = 1/2$（注：此处推导已体现在 §4.5 的 SGD 列中；Adam 的情形因自适应归一化后等效的 b 不同，但约束仍然成立）。

式 (4.7.4) 所示的 abc 框架核心结论是：**约束 $a + b = 1$ 是激活稳定性（$\Delta h = O(1)$）的充要条件**。不同的 (a, b) 只要满足这条约束就对应合法的特征学习参数化，μP 是其中的特殊代表（对隐藏层选择 $a = 1, b = 0$（Adam）或 $a = 1, b = 1$（SGD））。

第三环：$a + b = 1$ 的几何意义——参数化空间里的「刀刃」

在 (a, b) 平面上，$a + b = 1$ 是一条直线。这条线的两侧分别对应：

- $a + b < 1$（如标准 He + Adam 的等效点）：$\Delta h \sim n^{1/2-(a+b)/1} > n^0 = 1$，即激活发散。宽度越大，每步的激活变化量越大，训练越不稳定；最优超参数随宽度漂移，迁移不成立。
- $a + b > 1$（NTK 参数化区域）：$\Delta h \to 0$，激活冻结，特征学习消失，退化为核方法。网络宽度越大，越接近纯核方法，特征更新越小。
- $a + b = 1$（刀刃）：$\Delta h = O(1)$，激活稳定且非零，特征学习存在。这是唯一的「特征学习极限」所在。

μP 选择了 $a = 1, b = 0$（Adam，隐藏层），这在刀刃上。NTK 选择 $a = 1/2, b = 1$（某些 SGD 版本的等效点），处于冻结区。标准参数化（SP）$a = 1/2, b = 0$（Adam），处于发散区。

三种参数化在宽度极限下的行为差异，完全由其在 (a, b) 平面上相对于 $a + b = 1$ 这条线的位置决定。这是一个优美的几何图景：无限多种参数化方案，被一条代数直线分成三个定性完全不同的区域，μP 恰好落在分界线上。

第四环：与 NTK 的关系——为什么特征学习不能用核方法替代

理解了 abc 框架，可以回答一个更深的问题：为什么不直接用 NTK（核方法）训练？NTK 有严格的理论保证（收敛性、泛化界），为什么要冒险进入「刀刃上」的特征学习极限？

答案在于 NTK 极限的表达能力受限。NTK 下，网络等价于在固定特征空间（由初始化决定的核函数）上学一个线性函数。这个固定特征空间由初始化时的随机权重决定，**不随训练数据更新**。对于简单任务（低维数据、线性可分），固定特征可能已经足够；但对于复杂任务（高维数据、需要学习层次抽象特征），固定特征远远不够——这正是深度学习相比经典核方法的核心优势所在。

μP 保留了特征学习（$\Delta h = O(1)$），同时把超参数的「在宽度下的稳定性」恢复到 NTK 所具有的性质。NTK 的超参数（核函数的参数）显然不随宽度变化（核函数不依赖宽度），μP 通过精确的缩放规则给特征学习网络也带来了同样的稳定性——这是一种「两全其美」：NTK 的超参数稳定性 + 特征学习的表达能力。

整条链的回头看

```
随机初始化      n→∞时激活弱收敛到高斯过程（中心极限定理）
                        ↓ 描述所有参数化选择
abc参数化       三个指数(a, b, c)描述初始化/学习率/输出缩放
                        ↓ 稳定性条件：Δh = O(1)
a+b=1约束       刀刃等式（<1发散，>1冻结，=1特征学习）
                        ↓ 在刀刃上选特殊点
P选择           a=1, b=0（Adam，隐藏层），位于a+b=1上
   vs NTK       a+b>1（激活冻结），超参数稳定但无特征学习
   vs SP        a+b<1（激活发散），有特征学习但超参数不稳定
                        ↓ P的唯一性
唯一特征学习极限      超参数稳定性（可迁移）+ 非平凡特征动力学（表达能力）
```

这条链揭示了一个深层事实：**μP 不是众多合理参数化方案中的一种，而是「特征学习」与「超参数稳定性」两个约束同时成立的唯一方案**。$a + b = 1$ 这条等式不是任意的代数构造，而是「激活动力学在无穷宽极限下保持有限且非零」的充要条件，是 Tensor Programs 框架在参数化空间里划出的「物理边界」。所有 μP 缩放规则中看似随意的 $1/n$ 因子，都是这条边界在不同层类型上的具体实例化。

严格

Tensor Programs 的核心思想。 Yang 的 Tensor Programs 系列论文（I–V）建立了一套分析框架，核心结论是：在宽度 $n \to \infty$ 的极限下，随机初始化神经网络的所有层间激活（以及梯度、参数更新）的经验联合分布弱收敛（converge weakly）到一个高斯过程（Gaussian process）。具体地，对输入 $x^{(1)}, x^{(2)} \in \mathbb{R}^V$，第 l 层激活向量的内积 $\frac{1}{n} h^{(l)}(x^{(1)}) \cdot h^{(l)}(x^{(2)})$ 在 $n \to \infty$ 时依概率收敛到确定极限，极限的递推由核函数给出。

abc 参数化（abc-parameterization）。对权重矩阵 $W \in \mathbb{R}^{n \times n}$，引入三参数族：

$$W = n^{-a} \tilde{W}, \quad \tilde{W}_{ij} \sim \mathcal{N}(0, 1) \tag{4.7.3}$$

学习率 $\eta = n^{-b} \tilde{\eta}$，输出层缩放 n^{-c}。其中 a（初始化指数）、b（学习率指数）、c（输出缩放指数）是三个可选参数。

Yang & Hu 证明，要使宽度极限下存在非平凡特征学习（即 $\Delta h = O(1)$），三参数必须满足约束：

$$a + b = 1 \qquad \text{(hidden layers, Adam)} \tag{4.7.4}$$

具体而言，对隐藏—隐藏权重：$a = 1$ 对应初始化方差 $O(1/n)$，$b = 0$ 对应 Adam 学习率 $O(1)$（不随宽度缩放）。违反式 (4.7.4) 的约束会导致要么激活爆炸（$a + b < 1$），要么激活冻结退化为核极限（$a + b > 1$）。

特征学习极限（feature learning limit）vs 核极限（kernel limit / NTK）。在 NTK 参数化下，$a = 1/2$（He 初始化），$b = 0$（固定学习率），$a + b = 1/2 < 1$（对 SGD），$\Delta W = O(\eta/\sqrt{n})$，$\Delta h = O(\eta) \to 0$（宽度极限下特征不变），网络退化为固定核方法，训练动力学由初始 NTK 完全决定，与宽度无关（不再学习新特征）。

μP 下，$a = 1$（隐藏层初始化方差 $O(1/n)$），$b = 0$（Adam），$\Delta h = O(1)$，网络在宽度极限下仍有 $O(1)$ 的特征更新量——这是「特征学习极限」。它是唯一使特征动力学在 $n \to \infty$ 时既非爆炸也非消失的参数化，也是使超参数在宽度极限下有稳定极值的唯一参数化。

与原书呼应：原书 §4.7 描述了 Tensor Programs 框架与 abc 参数化的核心思想，以及特征学习极限与核极限的本质区别，本节给出了弱收敛的形式化陈述、约束方程 $a + b = 1$ 的来源，以及两类极限的对比。

---

§4.8 跨界类比：复杂系统的工程管理

▶这一节在讲什么

为什么用「工程管理」来类比神经网络训练？大模型训练涉及数以亿计的参数协同更新，每个参数的调整都影响全局，这与超大型组织的绩效分摊问题在结构上高度相似。

第一步：反向传播对应责任分摊。链式法则把全局损失精确分解为每个参数的边际贡献，这与绩效考核系统把组织目标分摊到每个员工的逻辑同构——都是线性叠加结构。

第二步：Adam 对应个性化绩效考核。历史梯度方差大的参数自动获得更保守的步长，这与「不确定性越大调整越保守」的管理原则一致。

第三步：μP 对应规模不变的制度设计。无论组织规模（模型宽度）如何变化，管理制度（超参数）保持有效——这是代数上的 n-均匀性，不是工程直觉。

本节要拿走的一件事：类比的价值在于提供直觉入口，但类比本身不能替代数学——下一节（§4.9）将指出类比的失效边界。

直觉

本节的核心任务是：找出「神经网络训练」与「工程管理」之间，哪些结构上的对应关系是**真实的数学同构**，而不只是比喻上的相似。好的类比不只是让人记住，它应该能帮助预测——在管理学里成立的某条原则，在数学上是否也有神经网络训练中的对应结论。

工程管理类比**真正成立**的，有三个数学根源。理解这三个根源，不仅能让类比变得可信，还能让每一条类比背后的数学机制变得透明。下面把这三个根源一一铺开。

根源一：链式法则 = 责任分摊的线性叠加

工程项目的绩效考核，理想状态是：把组织的总体目标（「项目成功」）分解为每个团队、每个成员的边际贡献，据此分配激励。这种分解在数学上是**线性叠加**的：组织总绩效 $=\sum_i$（成员 i 的贡献）。这要求总绩效对每个成员的贡献有精确的偏导数，且可加性（一个成员的改变不影响其他成员的贡献计算）成立。

反向传播正是这个结构的精确数学实现。设损失 $\mathcal{L}$，参数 θ_i 是某个权重，通过链式法则（见式 (4.8.1)）：

$$\frac{\partial \mathcal{L}}{\partial \theta_i} = \frac{\partial \mathcal{L}}{\partial h_j} \cdot \frac{\partial h_j}{\partial \theta_i} = \sum_j \delta_j \cdot x_{ji} \tag{4.8.1}$$

其中 δ_j 是来自后续层的误差信号，x_{ji} 是当前层的输入。这就是「损失函数对参数 θ_i 的边际贡献」。链式法则不仅计算了边际贡献，还**精确地把全局损失信号逐层分摊到每个参数**，而不需要全局信息——每个参数只需要「它所在层的局部信息」（输入激活和来自后续层的误差信号）就能计算梯度。

类比成立的条件：两者的线性叠加结构相同。管理学中，绩效分摊的可加性要求「成员之间没有互动效应」；数学上，梯度是局部线性近似，对非线性交互效应只有一阶精度。这一限制在类比的「失效面」（§4.9）会被放大——强非线性交互正是类比失效的一个根源。

根源二：Adam 的自适应步长 = 「不确定性越大越保守」的管理原则

管理学有一条经验原则：对绩效波动大（不确定性高）的员工，调整薪酬或职责时应更保守；对绩效稳定但偏低的员工，应给予更积极的激励。逻辑是：高波动性意味着过去的绩效样本对未来的预测性低，因此不应该根据有限历史大幅度调整；低波动性意味着历史样本可信，可以更大幅度地响应。

Adam 的自适应步长 $\eta/(\sqrt{\hat{v}_t}+\epsilon)$（见式 (4.2.10)）实现了完全相同的逻辑：

- $\hat{v}_t$ 是参数 θ_i 的历史梯度平方均值，代表「历史梯度波动性」；
- $\sqrt{\hat{v}_t}$ 大（历史梯度方差大）→ 有效步长 $\eta/\sqrt{\hat{v}_t}$ 小（保守更新）；
- $\sqrt{\hat{v}_t}$ 小（历史梯度稳定但小，如稀疏梯度）→ 有效步长大（积极更新）。

这与管理原则不只是比喻上的相似：两者的数学结构完全相同——用历史方差（波动性）的倒数来缩放每次调整的幅度。这种「方差反比调整」在统计决策理论中有严格的最优性：在均方误差意义下，对方差为 σ^2 的随机量，最优点估计的调整步长正比于 $1/\sigma$。Adam 的自适应步长是统计决策理论最优原则的直接工程实现，管理学原则是这个数学结论在人类组织中的直觉化表述。

根源三：μP 的规模不变性 = 制度设计的 n-均匀性

管理制度的一个重要质量指标是「规模不变性」：一套制度（激励机制、汇报结构、绩效标准）是否在组织规模变化时仍然有效？一个 100 人的公司的管理制度，直接搬到 10000 人的公司往往失效，需要重新设计。理想的制度应该有某种「可扩展性」——制度的核心逻辑在不同规模下保持一致。

μP 给出了神经网络训练中这种「规模不变性」的数学精确定义和实现。μP 保证：对所有宽度 n，用同一套超参数（学习率、初始化方差）训练，激活动力学（$\Delta h = O(1)$）保持稳定。这是代数意义上的 n-均匀性：涉及宽度的所有量（激活、梯度、更新量）对 n 的幂次，在 μP 下恰好相互抵消，结果不依赖 n。

这种代数均匀性不是一个工程近似，而是严格成立的数学性质（在 Tensor Programs 框架里可以精确证明）。管理类比中「制度规模不变性」的直觉，在数学上对应「激活动力学的 n-均匀性」。两者的结构同构，提供了为什么类比在这一点上准确的数学根据。

整条链的回头看

链式法则的线性叠加 ↔ 绩效分摊的可加性（管理学）
↓ 两者结构同构，类比成立
Adam的方差反比步长 ↔ 不确定性越大越保守（管理原则）
↓ 两者均来自统计决策理论的最优原则，类比成立
P的n-均匀性 ↔ 规模不变的制度设计（管理直觉）
↓ 数学上：abc约束a+b=1的代数均匀性

类比的价值在于：这三个数学同构，对应了管理学中经过实践验证的三条原则（可加性分摊、方差反比激励、规模不变制度）。如果你理解管理学这三条原则，你对神经网络训练的直觉就有了一个可以快速入门的参照系。但类比不能替代数学——管理学中的这三条原则都有适用范围，超出范围就失效，神经网络训练中的对应数学同样有边界。这就是 §4.9 的主题。

严格（简版）

类比的数学核心在于链式法则的线性性：每个参数的梯度 $\partial L/\partial \theta_i$ 通过链式法则被精确分解为各层贡献之和，这与责任分摊（responsibility allocation）的线性叠加结构同构。Adam 的自适应步长 $\eta/(\sqrt{\widehat{v}_t}+\epsilon)$ 对历史梯度方差大的参数自动降低有效步长，这与「不确定性越大，调整越保守」的管理逻辑一致。μP 的「规模不变性」对应于管理制度在员工数量变化时的结构不变性——是代数上的 n-均匀性（n-homogeneity），而非工程直觉。

---

§4.9 反类比：工程管理隐喻的失效边界

▶这一节在讲什么

为什么要主动拆解上一节的类比？好的类比是思维工具，但过度依赖类比会产生误导——了解类比在哪里失效，比知道它在哪里成立更重要。

第一步：训练没有「完成」这个状态。工程项目有二值判断（done/not done），训练的停止是一个连续的截断决策（验证集曲线斜率、算力预算），本质上是截断而非收敛。

第二步：神经元的职责是涌现的，不是预先指定的。工程模块有明确的功能分工；网络中每个参数的「语义」由训练动力学事后决定，需要可解释性工具才能反推，不存在先验的功能指定。

第三步：损失景观是不可消除的非凸。工程优化通常可转化为凸规划，有全局最优保证；深度学习的成功依赖高维几何的「幸运」结构（鞍点主导、模式连通性），而非可控的算法保证。

本节要拿走的一件事：我们能利用高维优化的几何性质，但无法从工程意义上「控制」它们——这是第四章最重要的警示。

直觉

§4.8 展示了工程管理类比成立的三个数学根源。这一节要做相反的事：找出类比在哪里从根本上失效，以及失效的数学原因。理解「类比的边界」比理解「类比的成立」更重要——因为错误的类比会在恰恰需要精确判断的地方给出错误的直觉。

工程管理类比在三个核心维度上失效，每个维度的失效都有可追溯的数学根源。

失效根源一：训练没有「完成」状态——截断问题

工程管理之所以有效，一个隐含的前提是：每个子任务有清晰的**完成标准**（done/not done），管理者可以做出二值判断——这个功能已经完成了、那个功能还没完成。有了完成标准，才能做资源分配（该模块完成了，把人力调去别处）、里程碑检查（90% 的功能已经完成，项目可以上线）、回溯评估（哪些决策导致了延误）。

神经网络训练**根本没有这个结构**。「训练完成」不是一个可判断的状态，而是一个连续的权衡：

- 验证集损失还在缓慢下降，但斜率已经非常小（每 1000 步只降 0.001）——继续训练是否值得？这个决策没有答案，只有算力预算约束。
- 验证集损失停止下降，甚至微升——是过拟合了，还是局部平台期？没有理论可以区分。
- 换一个随机种子，训练损失的曲线形状可能完全不同，但最终测试集性能差不多——说明训练过程是不确定性的，「完成状态」取决于哪个随机游走路径。

数学上，这对应训练过程是一个**随机动力系统**，它的不动点（极小值）构成了一个高维流形，而非有限个确定的点。「停止训练」的决策等价于从这个随机过程中截断一个有限长度的实现——是「截断」（truncation），而非「收敛」（convergence）。两者在信息论上截然不同：收敛有明确的极限，截断的结果依赖于截断时刻的随机状态。

这个失效点意味着：把「项目管理甘特图」或「里程碑检查」的逻辑直接用于神经网络训练，会产生系统性的误导。训练不是「有序完成一系列子任务」，而是「在高维随机流形上漂流，直到算力耗尽」。

失效根源二：功能涌现——神经元的职责不是预先设计的

工程系统的核心特征是**模块化设计**：每个模块在系统设计阶段就有明确定义的功能（「这个模块负责加密」「那个模块负责路由」），功能是先于实现存在的规范。正是这种预先的功能规范，使得「测试某个模块是否正常工作」有了明确的标准，使得「调试」有了意义——你知道一个模块应该做什么，才能判断它是否在正确地做。

神经网络中没有这种预先的功能规范。Transformer 的一个注意力头在训练结束后，究竟是在「跟踪主语-谓语关系」、「检测名词短语的边界」、还是「存储特定的 n-gram 模式」，**完全由训练数据和梯度动力学决定**，是训练完成后才能通过可解释性工具（激活补丁、稀疏自编码器、注意力可视化）反推出来的。这种「事后涌现的职责分工」，在信息论层面与「事前设计的模块功能」完全不同构：

- 工程模块的功能：设计时确定，实现前已知，可验证，不会在运行中改变；
- 神经元的功能：训练后涌现，无法预知，只能后验发现，且可能随微调而改变。

这个失效点有一个深刻的实践含义：**神经网络可解释性（interpretability）是一个独立的、非平凡的研究领域**，正是因为网络的「功能分配」不是从架构设计中读出来的，而是需要专门的工具和方法来反推。如果神经元的功能是「预先指定的」，可解释性就是平凡的——直接查看设计规范即可；正是功能的涌现性，使得可解释性成为一个独立问题。

失效根源三：非凸性不可消除——无法从工程意义上「控制」优化

工程优化（成本最小化、路径最短化、资源分配）在许多实际问题中可以**转化为凸规划**：线性规划、二次规划、凸约束优化。凸规划有全局最优保证（任何局部最优都是全局最优），有多项式时间的精确算法，有严格的收敛速度保证。这是工程优化的「可控性」来源：你知道算法能找到全局最优，你知道需要多少步，你知道结果有多接近最优。

深度学习的损失函数是**不可消除的非凸**，这不是「我们还没找到好的转化方法」，而是「非凸性来自激活函数的非线性，而非线性正是神经网络的表达能力来源」。无法在保留表达能力的前提下消除非凸性。

但这并不意味着深度学习训练是「随机的」或「无法理解的」。§4.3 的分析表明：高维几何的特殊结构（鞍点主导、模式连通性、平坦极小值的 Gibbs 分布偏好）使得 SGD 在实践中能找到好的解——这是**利用高维几何性质**，而非「控制」它。两者的区别至关重要：

- **控制**意味着：给定任意损失函数，算法保证找到全局最优，并有可证明的收敛率。这在非凸优化中通常不成立。
- **利用**意味着：当损失函数具有高维优化的特殊结构（神经网络在过参数化区域通常具备这些结构）时，SGD 有很高的概率找到好的解，但没有在最坏情况下的保证。

这个失效点意味着：把工程优化的「可控性直觉」（「我可以精确控制优化过程，保证结果」）带入深度学习，会导致对训练结果的过度自信。实践中，大模型训练偶尔出现的「训练崩溃」（loss spike）、「超参数敏感性」、「规模扩展时的涌现行为」，都是

「利用而非控制」这一本质的体现——我们在这个随机过程中保持了较高的成功概率，但没有消除随机性本身。

整条链的回头看

```
工程项目有「完成状态」          训练是随机截断（无收敛保证）
          类比成立的范围                  类比失效的边界
工程模块功能预先设计          神经元功能事后涌现（可解释性问题的来源）
工程优化可转凸规划          深度学习非凸性不可消除（利用    控制）

三个失效根源的统一来源：
神经网络 = 高维非线性随机动力系统
          ↓
「完成状态」不存在 ← 随机性（截断    收敛）
「功能规范」不存在 ← 涌现性（事后    事前）
「最优保证」不存在 ← 非凸性（利用    控制）
```

这条链揭示了第四章最深刻的教训：**神经网络训练的成功，不是工程意义上「控制一个复杂系统」的成功，而是「识别并利用了高维随机系统的幸运几何性质」的成功**。这两种成功有着截然不同的可靠性：工程控制在最坏情况下也有保证，高维几何利用在「大多数」情况下有效，但边界情况（分布外数据、规模突变、新型任务）可能失效，且失效方式难以预测。理解这个本质区别，是任何严肃对待 AI 系统可靠性的人都必须面对的基础问题。

严格（简版）

1. **无终止条件**。工程完工有二值判断（done/not done），训练的停止准则是连续的（验证集 loss 曲线的斜率阈值或算力预算约束），本质上是一个截断（truncation）而非收敛（convergence）。
2. **功能涌现（emergent function）**。每个参数的「语义」由训练动力学决定，事后才能通过可解释性方法（如激活补丁、稀疏自编码器）反推，不存在先验的功能指定。这与工程系统的模块化分工在信息论层面不同构。
3. **非凸 vs 凸**。工程最优化问题（成本最小化、路径最短化）通常可转化为凸规划，有全局最优保证。深度学习损失景观的非凸性不可消除，训练成功依赖高维几何的「幸运」结构（鞍点主导、模式连通性），而非可控的算法保证。第四章的核心教训是：**我们能利用这些几何性质，但无法从工程意义上「控制」它们**。

数学卷 · 第 5 章：标度律、相变与涌现的数理基础

§5.1 尺度律（Scaling Law）：算力即权力的冷酷物理感

▶这一节在讲什么

为什么损失会随算力呈幂律下降而不是指数下降？因为语言本身在词、短语、段落、篇章各层次上具有自相似结构，幂律正是自相似系统的典型指纹。如果收益以指数速率衰减，很快就会归零；幂律收益永不截断，在双对数坐标下是一条无止境的直线。

第一步：三条单变量幂律。固定其他条件，损失分别对参数量 N、数据量 D、算力 C 呈 $\mathcal{L} \sim x^{-\alpha}$ 的形式下降，拟合指数均在 0.05–0.10 之间——这意味着每增加十倍投入，损失约降低 10%–20%。

第二步：联合损失函数 $L(N, D)$。Hoffmann 等把参数项与数据项合并为 $E + A/N^{\alpha} + B/D^{\beta}$，其中 E 是语言本身的不可归约熵——降不到零的下界。

第三步：拉格朗日求最优配比。在固定算力 $C \approx 6ND$ 的约束下，对 $L(N, D)$ 做拉格朗日优化，得到 $N^* \propto C^{\beta/(\alpha+\beta)}$——告诉你算力应以何种比例分配给参数和数据。

本节要拿走的一件事：幂律让模型性能变成可预测的「工程量」，而最优配比公式是 §5.2 Chinchilla 争论的数学起点。

直觉

原书 §5.1 揭示了一个令整个工业界沉默的事实：语言模型的损失值在七个数量级的范围内，对参数量 N、数据量 D、算力 C 呈精确的幂律下降。这意味着模型性能可以被预测，AI 研究从「艺术」变成「重工业」。

本节出现五个核心概念：**幂律**、**幂指数 α**、**不可归约损失 E**、**联合损失函数 $L(N, D)$**、**compute-optimal 配比**。它们看似是孤立的技术设定，实际上沿一条严密的逻辑链彼此推出：从语言的统计结构出发，逼出幂律的必然性；从幂律的参数出发，识别两类「浪费」；从两类浪费出发，引入联合损失函数；最后对联合损失函数在算力约束下做优化，得到决定工业界投资决策的最优配比公式。理解这条链，是读懂后续一切 Scaling Law 争论的前提。下面把这条链一环环铺开。

第一环：为什么是幂律而不是指数律——语言的自相似结构

最自然的问题是：投入翻十倍，收益为何不按某个固定比例衰减？如果收益指数衰减——每增加十倍投入，损失降幅固定，比如每次减少 50%——那么经过有限轮翻倍后，收益几乎归零，继续投入变得毫无意义。现实不是这样：文本困惑度在七个数量级的算力范围内持续下降，且下降速率没有明显放缓的迹象。这本身就排除了指数律。

幂律（如式 (5.1.1)）的数学特征是在双对数坐标下表现为直线：$\log \mathcal{L} = -\alpha \log N + \text{const}$。斜率是负数（损失随规模下降），斜率的绝对值就是幂指数 α。幂律描述的是**自相似（scale-free）结构**：无论把观测窗口放大多少倍，统计规律看起来相同。语言恰好在多个层次上具有这种自相似性——词在句子中的出现频率服从 Zipf 律（也是幂律），短语、段落、篇章的信息密度也呈现类似的层级结构。幂律描述语言，不是巧合，而是语言统计结构的深层指纹。

反过来用指数律做类比：如果损失对 N 指数衰减，意味着「改进来自某个固定比例

的覆盖」；而幂律意味着「改进来自覆盖了新的频率区间，而每个频率区间的贡献与其频率成幂次关系」。这正是 Zipf 分布的逻辑：罕见词越罕见，但仍然存在；覆盖得越多，仍然有更罕见的词等待被覆盖。幂律就是这种「永远有更稀有的尾部」的数学投影。

$$\mathcal{L}(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N} \tag{5.1.1}$$

第二环：幂指数 α 的含义——投入边际回报的精确量化

幂律有了，它的斜率 α 说了什么？把幂指数翻译成工程语言：参数量每翻十倍，损失乘以 $10^{-\alpha}$，即降低 $1-10^{-\alpha}$ 的比例。当 $\alpha_N \approx 0.076$ 时，参数量翻十倍带来约 15% 的损失下降；$\alpha_D \approx 0.095$ 时，数据量翻十倍带来约 19% 的下降。两个指数的绝对值都很小——这是幂律缓慢但持续的代价：需要翻百倍才能把损失降约 32%，翻十亿倍才能降约 50%。

更关键的是 α_N 与 α_D 的**相对大小**。如果 $\alpha_N = \alpha_D$，则在固定算力下，把预算全给参数或全给数据效果相同，任意分配都行。正是 $\alpha_N \neq \alpha_D$ 制造了「最优配比」问题：哪条路的边际回报更高，钱就应该更多流向那条路。Kaplan（2020）估计 $\alpha_N \approx 0.076$、$\alpha_D \approx 0.095$，数据略占优，但幅度很小；Hoffmann（2022）用更好的实验设计得到 $\alpha \approx 0.34$、$\beta \approx 0.28$（Chinchilla 使用不同的参数化方式），结论翻转。这场争论的根源不在理论，而在对 α 和 β 的精确估计——两者都不大，差值更小，任何系统性实验误差都足以改变结论。

第三环：不可归约损失 E——语言的信息论下界

即使模型无限大、数据无限多，损失也不会趋向零。这个下界叫「不可归约损失」，记作 E。它不是工程意义上「还没优化到最优」的残差，而是语言本身的**条件熵**：给定所有上下文，下一个词仍然具有的内在随机性。

直觉上，语言并不是完全确定的：同一个上文，不同作者会写出不同的下一句，这种「创作自由」在信息论上就是熵。即使语言模型完美拟合了人类语言的生成分布，它也无法「预测」这部分随机性——因为它本来就不是可预测的，它就是语言本身的不确定性。E 的量级大约是 1.69 nats（对应困惑度约 5.4），这意味着即使最完美的模型，每预测一个 token 平均仍面临约 5 个等可能选项。

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} \tag{5.1.2}$$

式 (5.1.2) 的分解意义在于：损失 = 不可归约部分 + 参数不足引起的欠拟合 + 数据不足引起的欠拟合。E 是常数，无论如何优化都动不了；A/N^{α} 随 N 增大趋向零，代表「参数容量还没用完，还有模式没被记住」；B/D^{β} 随 D 增大趋向零，代表「见过的样本还不够，还有分布没被覆盖」。如果只增大 N 而不增大 D，则 B/D^{β} 项被钉死，整体损失被数据墙限制（后见 §5.3）；反之亦然。

第四环：为什么联合损失函数是加法结构——两种浪费相互独立

联合损失的加法结构 $E + A/N^{\alpha} + B/D^{\beta}$ 看起来像是「凑出来的拟合式」，但它有明确的物理动机：两类欠拟合（参数不足 vs 数据不足）在机制上**相互独立**。

参数不足导致的欠拟合，来源于网络容量不够——某些高频模式无法被记住，网络的

表达复杂度（可以用 VC 维或 Rademacher 复杂度度量）不足以覆盖训练集。数据不足导致的欠拟合，来源于分布覆盖不够——某些罕见语言模式从未出现在训练集，网络即使有足够容量也无从学习。这两种机制的修复方式不同（一个靠扩大模型，一个靠增加数据），且在正则化意义上相互正交——因此可以写成独立的两项之和。

当然，这是近似：在极端情况下（如 N 极小时，数据多也没用；D 极小时，参数多也没用），两项会出现交叉效应。但在现实感兴趣的区域（N 和 D 都不极端）内，加法结构的拟合精度极高，三种拟合方法（IsoFLOP、IsoLoss、参数化拟合）均验证了这一形式。

第五环：compute-optimal 配比——拉格朗日告诉你钱怎么分

现在问题变成：给定固定的算力预算 C，按什么比例分配给 N 和 D 可以最小化损失？约束条件是 Transformer 的 FLOPs 近似：训练一个 N 参数模型、处理 D 个 token，总算力约为 $C \approx 6ND$（前向 $2ND$ + 反向 $4ND$）。

在 $6ND = C$ 的约束面上，$L(N, D)$ 的等值面不是同心圆，而是关于 (N, D) 的双曲面族——因为两个变量以乘积形式出现在约束中，但以幂次形式出现在目标函数中。最优点不在约束面的中央，而是被 α 和 β 的比值偏向「哪一个边际回报更陡」的方向。

拉格朗日乘数法消去 λ 后，最优条件如式 (5.1.3) 所示，等价于：**参数项的边际贡献与数据项的边际贡献之比等于 β/α**。

$$\frac{A/N^{\alpha}}{B/D^{\beta}} = \frac{\beta}{\alpha} \tag{5.1.3}$$

结合约束 $6ND = C$ 解出：

$$N^{*} \propto C^{\,\beta/(\alpha+\beta)}, \quad D^{*} \propto C^{\,\alpha/(\alpha+\beta)} \tag{5.1.4}$$

式 (5.1.4) 是 §5.2 整场争论的数学起点。当 $\alpha \approx \beta$ 时，$N^{*} \propto C^{0.5}$，参数和 token 数等比例扩展（Chinchilla 结论）；当 $\alpha < \beta$ 时，数据的指数更大，说明应该更快增加数据——这是「数据优先」策略的理论依据。

这个公式的工程含义极为直接：如果已知 α 和 β，给定算力预算，就可以直接算出「最优模型大小」和「最优训练 token 数」，无需任何人工经验调参。Scaling Law 把 AI 研究从「靠直觉调参」变成了「靠公式算最优点」——这正是「算力即权力」的数学本质：掌握算力的人，可以精确预测并复现模型的性能，主动权不再被不确定性所左右。

整条链的回头看

```
语言的自相似结构
    幂律（双对数坐标下的直线）
    加上幂指数   的含义（投入边际回报）
    识别「参数不足」与「数据不足」两类浪费
    联合损失 L(N,D) = E + A/N  + B/D
    固定算力 C   6ND 的约束
    拉格朗日最优化
    compute-optimal 配比 N*   C^{ /( + )}
```

这条链揭示了一个深层事实：**AI 模型的性能上限和最优资源分配，从一开始就被语言**

统计结构的幂律指纹所决定。α 和 β 不是超参数，而是语言本身的测量值；它们的比值决定了整个行业的资本配置逻辑。谁把这两个数测得更准，谁就在算力军备竞赛中拥有更精确的地图。

严格

Kaplan 单变量幂律。设 N 为非嵌入参数量，D 为训练 token 数，C 为训练总 FLOPs，Kaplan 等（2020）拟合出：

$$\mathcal{L}(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad \mathcal{L}(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad \mathcal{L}(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C} \tag{5.1.5}$$

其中 $\alpha_N \approx 0.076$，$\alpha_D \approx 0.095$，$\alpha_C \approx 0.050$，N_c, D_c, C_c 为拟合常数。

幂律 vs 指数律的渐近行为。设 $f(x) = x^{-\alpha}$（幂律），$g(x) = e^{-\lambda x}$（指数律）。对任意 $\alpha, \lambda > 0$，有：

$$\lim_{x\to\infty} \frac{f(x)}{g(x)} = \lim_{x\to\infty} x^{-\alpha} e^{\lambda x} = +\infty \tag{5.1.6}$$

即指数律衰减比任意幂律都快——对数坐标下，幂律是直线，指数律是向下弯曲的曲线。双对数坐标下，$\log \mathcal{L} = -\alpha \log N + \text{const}$，斜率即幂指数 $-\alpha$。

Hoffmann 联合损失函数（为 §5.2 铺垫）。原书在 §5.1 引入 Kaplan，在 §5.2 引入 Chinchilla 的联合形式：

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta} \tag{5.1.7}$$

其中 $E \geq 0$ 是不可归约损失（irreducible loss），即语言本身的熵下限；A/N^α 表示参数不足导致的欠拟合；B/D^β 表示数据不足导致的欠拟合。$\alpha, \beta > 0$ 为正数，意味着两项均单调递减。

Compute-optimal 的拉格朗日推导。给定固定算力预算 C，变换器（Transformer）的标准 FLOPs 近似为：

$$C \approx 6ND \tag{5.1.8}$$

（每个参数在前向 + 反向传播中各贡献约 $2D$ 次乘加，共 $6ND$。）目标是在约束 $6ND = C$ 下最小化 $L(N, D)$。构造拉格朗日函数：

$$\mathcal{L}(N, D, \lambda) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta} - \lambda(6ND - C) \tag{5.1.9}$$

对 N 和 D 分别求偏导并令其为零：

$$\frac{\partial \mathcal{L}}{\partial N} = -\frac{\alpha A}{N^{\alpha+1}} - 6\lambda D = 0 \quad \Rightarrow \quad \frac{\alpha A}{N^{\alpha+1}} = 6\lambda D \tag{1}$$

$$\frac{\partial \mathcal{L}}{\partial D} = -\frac{\beta B}{D^{\beta+1}} - 6\lambda N = 0 \quad \Rightarrow \quad \frac{\beta B}{D^{\beta+1}} = 6\lambda N \tag{2}$$

由 (1) 和 (2) 消去 λ：

$$\frac{\alpha A}{N^{\alpha+1}} \cdot \frac{1}{D} = \frac{\beta B}{D^{\beta+1}} \cdot \frac{1}{N} \quad \Rightarrow \quad \frac{\alpha A}{N^{\alpha} D} = \frac{\beta B}{N D^{\beta}} \quad \Rightarrow \quad \frac{\alpha A}{N^{\alpha}} = \frac{\beta B}{D^{\beta}} \tag{5.1.12}$$

即在最优点处，参数项与数据项的边际贡献之比满足：

$$\frac{A/N^{\alpha}}{B/D^{\beta}} = \frac{\beta}{\alpha} \tag{3}$$

结合约束 $6ND = C$，由 (3) 解出：

$$N^{*} \propto C^{\,\beta/(\alpha+\beta)}, \quad D^{*} \propto C^{\,\alpha/(\alpha+\beta)} \tag{5.1.14}$$

当 $\alpha \approx \beta$（Hoffmann 拟合值 $\alpha \approx 0.34$，$\beta \approx 0.28$ 近似）时，$N^{*} \propto C^{0.5}$，$D^{*} \propto C^{0.5}$，即参数量与 token 数应等比例扩展。

更一般地，若记 $r = \alpha/\beta$，则：

$$N^{*}_{\text{opt}} \propto C^{1/(1+r)}, \quad D^{*}_{\text{opt}} \propto C^{r/(1+r)} \tag{5.1.15}$$

当 $r > 1$（参数的边际回报衰减更快）时，D^{*} 的增长指数大于 N^{*}，表明应更多增加数据。

与原书呼应：原书 §5.1 提到「模型的性能可以被精确预测」，对应本节幂律拟合与拉格朗日推导给出的式 (5.1.15) compute-optimal 配比公式 $N^{*}_{\text{opt}} \propto C^{\beta/(\alpha+\beta)}$。

---

§5.2 Chinchilla 的颠覆与再颠覆

▶这一节在讲什么

为什么同一个损失函数会给出截然不同的结论？因为拟合方法不同。Kaplan 用参数固定、数据变化的实验，Hoffmann 用等算力切片（IsoFLOP），两者在信息矩阵的条件数上差异巨大，导致最优配比从「参数优先」翻转为「1:20 等比扩展」。

第一步：三种拟合方法的方向一致性。IsoFLOP（方法一）、等损失曲线（方法二）、全局参数化最小二乘（方法三）均指向同一结论：每个参数对应约 20 个训练 token，而非 Kaplan 建议的 5 个。

第二步：方法三的置信区间为何过窄。非线性最小二乘的 CI 宽度正比于 $\hat{\sigma}/\sqrt{n}$，但信息矩阵 $J^\top J$ 条件数极大，参数 α 与 A 强相关，导致对角元素被低估——400 个训练点撑不起如此紧致的置信区间。

第三步：Sardana 修正——把推理成本纳入预算。真实世界里推理调用数十亿次，约束从 $6ND \le C$ 扩展为 $6ND + 2N \cdot n_{\text{inf}} \le C_{\text{total}}$，最优解向「更小的 N、更多的 D」偏移。

本节要拿走的一件事：Chinchilla 的方向性结论（等比扩展）仍有效，但其置信区间的统计学基础存疑；推理成本一旦纳入，最优点还会进一步向小模型偏移。

直觉

Hoffmann 等（2022）用同样的联合损失函数 $L(N, D)$，却得出了与 Kaplan 截然不同的结论：参数量与训练 token 数应以约 1:20 的比例同等扩展，而非「参数优先」。这一修正背后有三种不同的拟合方法，它们给出的结论方向一致，但置信区间宽窄差异悬殊。Epoch AI 的复现工作揭示了其中方法三的统计学问题，而 Sardana 等（ICML 2024）进一步指出，即使 Chinchilla 最优解在训练成本上正确，也忽略了推理成本——真实世界中推理调用数十亿次，应将推理成本纳入总优化目标。

本节出现四个核心概念：**IsoFLOP 实验设计、置信区间的统计学根源、参数-数据最优比、推理成本修正**。它们沿一条逻辑链彼此推出：先理解为什么实验设计决定了拟合结果，再识别方法三的统计学缺陷，然后理解「1:20」结论的可靠程度，最后把推理成本纳入约束，看到最优解如何进一步偏移。下面一环一环展开。

第一环：为什么同一个公式会给出不同结论——实验设计覆盖范围的差异

Kaplan（2020）和 Hoffmann（2022）用的是同一个联合损失函数，即式 (5.1.2)（$L(N, D) = E + A/N^\alpha + B/D^\beta$），理论框架完全一致。结论翻转的根源不在公式，在**实验数据的分布**。

Kaplan 的实验设计是「固定数据量 D，扫描不同的 N」。在这种设计下，所有数据点都落在 (N, D) 空间中的水平线上——数据维度没有被充分探索。当 D 固定时，B/D^β 是一个常数背景项，拟合过程实际上只感知了 A/N^α 这一维的变化，信息矩阵在 β 方向的条件数极差。这就像试图用只在 x 轴方向移动的探针来测量一个二维曲面的形状——你能把 x 方向的曲率测得很准，但 y 方向和两者的交叉曲率完全估不准。

Hoffmann 的 IsoFLOP 设计是「固定总算力 $C = 6ND$，同时扫描多组 (N, D) 组合」。每条等算力曲线上有多个实验点，N 和 D 同时变化，信息矩阵在 (N, D) 联合空间中

条件数更好。直接观测「固定算力下哪个 (N, D) 组合损失最低」，等于**直接在约束面上测量最优点**，信息来源比单变量扫描更为直接。

结果就是：Kaplan 看到「在固定数据下增加参数有效」——这是真的，但由于数据量不足而被系统性低估了数据的边际价值。Hoffmann 看到「在固定算力下，等比例增加 N 和 D 更好」——因为他的实验覆盖了足够宽的 (N, D) 组合，使真实最优点落在覆盖范围之内。

$$N^*_{\text{Kaplan}} \approx C^{0.73} \text{（参数优先）}, \qquad N^*_{\text{Hoffmann}} \approx C^{0.50} \text{（等比例）} \tag{5.2.1}$$

式 (5.2.1) 中两个结论在数量级上的差异，对应着工业界投资几十亿美元量级的不同决策。这说明 Scaling Law 的结论对实验设计极其敏感——这不是理论的缺陷，而是对「你拿什么数据来拟合」这一问题的严肃警告。

第二环：三种拟合方法方向一致但置信区间差异悬殊——统计学的陷阱

Hoffmann 等的论文使用了三种方法拟合同一个损失函数，三者方向性结论一致：每个参数对应约 20 个训练 token（即 $N^*/D^* \approx 1/20$）。方向一致是可信的，但「置信区间」的可靠程度参差不齐。

方法一（IsoFLOP）直接读出每条等算力曲线的最优 N^*，再对 $N^*(C)$ 做幂律回归。这种方法的置信区间来自不同等算力曲线上的独立测量，每个测量点都是真实训练实验的结果，不确定性来源清晰。

方法三（全局参数化拟合）把所有 (N, D, L) 观测值一起输入非线性最小二乘，直接拟合五个参数 $\{E, A, \alpha, B, \beta\}$，报告的置信区间极窄（如 $\alpha = 0.34 \pm 0.02$）。问题在于：非线性最小二乘的置信区间依赖信息矩阵 $J^\top J$ 的逆，而该矩阵的条件数极大——参数 α 与 A 强相关，β 与 B 强相关，导致 $(J^\top J)^{-1}$ 的对角元素被严重低估，报告的置信区间远比真实不确定性窄。Epoch AI（2024）的分析表明，要支撑如此紧致的置信区间，理论上需要数十万量级的独立训练实验，而不是 Chinchilla 论文实际使用的约 400 个训练点。

这意味着什么？**Chinchilla 的方向性结论（等比例扩展，约 1:20）可信度较高**，因为它得到了三种独立方法的一致支持。但它的**定量精度**——具体比例是 1:20 还是 1:15 还是 1:25——远比论文报告的置信区间所暗示的要不确定得多。在基于这些数字做精确工程决策时，需要对这种统计不确定性有清醒的认识。

第三环：「1:20」结论的实践含义——为什么 GPT-4 时代的模型都「超量训练」

Chinchilla 最优（compute-optimal）定义为：在固定训练算力 C 下最小化损失。最优点满足 $N^*_{\text{opt}} \approx D^*_{\text{opt}}/20$，即每个参数对应 20 个训练 token。

对于一个 7B 参数的模型，Chinchilla 最优的训练量约为 $7\times10^9\times20 = 140\times10^9 = 140B$ token。但 LLaMA-3 8B 实际使用了约 15T token——约为 Chinchilla 最优的 100 倍。这「严重偏离最优」了吗？

不是。Chinchilla 最优针对的是**最小化训练成本**，而工程实践中真正的目标是**最小化总成本（训练 + 推理）**。一旦纳入推理成本，最优点就会向「更小的 N、更多的 D」偏移——因为小模型的每次推理更便宜，而超量训练可以让小模型达到原本只有大模型才能达到的损失水平。这正是第四环要讨论的内容。

第四环：Sardana 修正——推理成本纳入后最优解的偏移

设模型在整个生命周期内被调用 n_{inf} 次推理，单次推理约需 $2N$ FLOPs（前向传播），总推理算力为 $2N \cdot n_{\text{inf}}$。扩展后的总算力约束变为：

$$6ND + 2N \cdot n_{\text{inf}} \leq C_{\text{total}} \tag{5.2.2}$$

即 $N(6D + 2n_{\text{inf}}) \leq C_{\text{total}}$。

当 $n_{\text{inf}} \gg 3D$ 时（推理调用次数远超「训练 epoch 数的三倍」），约束的主导项从 $6ND$（训练成本）变成 $2Nn_{\text{inf}}$（推理成本）。将式 (5.2.2) 中的约束代入拉格朗日最优化，会把最优解推向**更小的** N——因为缩小参数量直接降低每次推理成本，节省出来的算力预算可以投入更多训练数据 D，弥补模型变小带来的损失上升。

这不是工程经验，而是一个严格的数学结果：在推理量极大的部署场景（消费级 API、大规模用户），小模型加超量训练比大模型加少量训练更经济。LLaMA 系列的策略——用大量训练让小模型「练到极限」——是 Sardana 等修正的直接体现，也是「Chinchilla 被再颠覆」的含义。

$$C_{\text{total}} \approx 6ND + 2N \cdot n_{\text{inf}} = N(6D + 2n_{\text{inf}}) \tag{5.2.3}$$

当 $n_{\text{inf}} \to \infty$（极大规模部署），式 (5.2.3) 表明最优解趋向于 $N \to 0$（无限小的模型配合无限多训练数据）——当然这是理论极限，实际受限于 D 的可用量（数据墙，见 §5.3）和模型能力的下限。但方向是明确的：部署量越大，越应该向更小的模型 + 更多训练数据倾斜。

整条链的回头看

```
实验设计覆盖范围（Kaplan 固定 D vs Hoffmann IsoFLOP）
    方向性结论翻转（参数优先 → 等比例 1:20）
    方法三置信区间过窄（信息矩阵条件数问题）
    方向性结论可信，定量精度存疑
    Chinchilla 最优 = 训练成本最优，非总成本最优
    纳入推理成本 C_total = 6ND + 2N · n_inf
    Sardana 修正：最优解偏向更小 N、更多 D
    LLaMA 式「超量训练小模型」的理论依据
```

这条链揭示了一个深层事实：**Scaling Law 的「最优解」不是一个绝对数，而是随目标函数定义（训练成本 vs 总成本）和约束条件（算力预算 vs 推理量）的变化而移动的点。** Chinchilla 的颠覆是把目标从「参数更大」换成「训练更充分」；Sardana 的再颠覆是把目标从「训练成本最优」换成「全生命周期成本最优」。每一次「颠覆」都不是推翻了联合损失函数，而是在同一个函数框架内重新定义了约束，看到了新的最优解。

严格

Hoffmann 三种拟合方法。设损失模型为：

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} \tag{4}$$

共有 5 个参数 $\{E, A, \alpha, B, \beta\}$ 待拟合。Hoffmann 等使用了三种方法：

- **方法一（IsoFLOP）**：固定总算力 $C = 6ND$，在多组 (N, D) 上测量损失，拟合每条等算力曲线的最优 N^*，再对 $N^*(C)$ 做幂律回归，得出 $N^* \propto C^{0.50}$。
- **方法二（IsoLoss）**：找出达到相同损失值所需的 (N, D) 组合，沿等损失曲线回归，得出类似结论。
- **方法三（参数化拟合）**：直接对全部 (N, D, L) 观测数据用最小二乘拟合方程 (4) 的 5 个参数，Hoffmann 报告的置信区间极窄（如 $\alpha = 0.34 \pm 0.02$）。

三种方法的方向性结论一致：最优 N^*/D^* 比约为 $1/20$（即每个参数对应约 20 个训练 token），与 Kaplan 建议的 $1/5$ 差异显著。

Epoch AI 复现与置信区间统计学根源。Epoch AI（2024）指出，方法三的置信区间过窄，在统计学上源于以下机制：

方法三对有限的训练点做非线性最小二乘，设观测数为 n，参数数为 $p = 5$，残差方差为 $\hat{\sigma}^2$，则参数 $\hat{\theta}$ 的渐近协方差矩阵为：

$$\mathrm{Cov}(\hat{\theta}) \approx \hat{\sigma}^2 \left(J^\top J\right)^{-1} \tag{5.2.5}$$

其中 $J \in \mathbb{R}^{n \times p}$ 是雅可比矩阵。式 (5.2.5) 中置信区间宽度正比于 $\hat{\sigma}/\sqrt{n}$。Chinchilla 论文实际训练的模型数量约为 400 个，但非线性拟合中信息矩阵 $J^\top J$ 的条件数往往很大（参数 α, β 与 A, B 之间存在强相关性），导致 $\left(J^\top J\right)^{-1}$ 的对角元素被低估，最终报告的 CI 比实际参数不确定性窄得多。要支撑如此紧致的置信区间，理论上需要数十万量级的独立观测点。

Sardana 推理时计算修正。设 C_{train} 为训练总算力，C_{inf} 为单次推理算力（约 $2N$ FLOPs），n_{inf} 为生命周期内推理调用次数，总算力预算为 C_{total}。扩展后的优化问题为：

$$\min_{N,D} \; L(N, D) \quad \text{s.t.} \quad C_{\text{train}} + n_{\text{inf}} \cdot C_{\text{inf}} \le C_{\text{total}} \tag{5.2.6}$$

其中 $C_{\text{train}} \approx 6ND$，$C_{\text{inf}} \approx 2N$，故约束变为：

$$6ND + 2N \cdot n_{\text{inf}} \le C_{\text{total}} \quad \Rightarrow \quad N\left(6D + 2n_{\text{inf}}\right) \le C_{\text{total}} \tag{5}$$

当 $n_{\text{inf}} \gg 3D$（即推理调用次数远超训练 epoch 数的三倍）时，约束 (5) 的主导项变为 $2Nn_{\text{inf}}$，即推理成本主导。此时拉格朗日最优解偏向使用**更小的** N（配合更多训练 token D），因为减小 N 既降低推理成本，又可将节省下来的预算投入更多训练数据。

这是 LLaMA-3 8B 使用 15T tokens（约为 Chinchilla 最优的 100 倍）训练的理论依据：在推理量极大的部署场景下，小模型 + 超量训练比大模型 + 少量训练更经济。

与原书呼应：原书 §5.2 提到「Chinchilla 所指出的方向性结论仍然具有重要影响」及 Epoch AI 对「方法三置信区间过窄」的批评，对应本节对信息矩阵条件数问题的分析，以及 Sardana 修正公式 $\min L(N, D)$ s.t. $C_{\text{train}} + n_{\text{inf}} \cdot C_{\text{inf}} \le C_{\text{total}}$。

---

§5.3 数据墙与 §5.4 推理时计算

▶这一节在讲什么

为什么 scaling 会撞墙？人类产生的文本总量有限——约 3×10^{14} token——而当前扩展速率将在数年内耗尽这一资源。数据墙不是性能瓶颈，而是一个约束边界：$D \le D_{\max}$，一旦触碰，损失函数里 B/D^{β} 项就被钉死，无论 N 多大，损失都有一个受数据支配的下界。

第一步：合成数据的陷阱——模型崩溃。用模型自己生成的数据反复训练，每一代都相当于对真实分布做有噪采样，低概率的尾部被截断，经多代迭代后分布退化为一个点——Wasserstein-2 距离随代数单调发散。

第二步：推理时计算作为「第二曲线」。Best-of-K 采样在推理阶段投入更多算力，绕过训练数据的上限。每 Best-of-3 采样等价于约 1 个额外训练 token 的算力——这个折算率对可验证任务（数学、代码）高，对不可验证任务（创意写作）低。

第三步：两条曲线的关系。推理时计算不是训练时计算的替代，而是在数据墙之后的补充路径；两者的最优组合取决于任务是否有客观验证信号。

本节要拿走的一件事：数据墙迫使 scaling 从「烧更多训练数据」转向「在推理时花更多算力」，而这一转变的数学基础是等价 token 折算率 $D_{\text{equiv}}(K) = K/3$。

直觉

原书 §5.3 提出数据墙（data wall）：公开人类文本总量约 3×10^{14} token，在当前 scaling 速率下将于 2026–2032 年耗尽。§5.4 描述了「第二曲线」——推理时计算缩放（inference-time scaling），通过在推理阶段投入更多算力（链式推理、N 选优（Best-of-N）采样）来补偿训练数据的不足。

本节出现四个核心概念：**数据墙的数学根源、合成数据的模型崩溃、推理时计算的等价折算、两条曲线的适用边界**。它们沿一条逻辑链彼此推出：先理解数据墙为什么在数学上是硬约束，再理解合成数据为何无法突破这个约束，然后理解推理时计算如何作为「第二曲线」绕过这一限制，最后识别第二曲线在哪些任务上有效、在哪些任务上无效。下面一环一环展开。

第一环：数据墙是约束边界而非性能瓶颈——$D \le D_{\max}$ 的数学含义

把 §5.1 的损失函数拿过来：$L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$。只要 D 能继续增大，B/D^{β} 项就持续下降，损失持续改善。但如果 D 到了上限 $D_{\max}$——人类可用文本的总量——B/D^{β} 就被钉在了 $B/D_{\max}^{\beta}$ 这个固定值上。

此时无论 N 增大多少，A/N^{α} 可以继续趋向零，但由式 (5.1.2) 可知，损失有一个受数据墙决定的下界：

$$L_{\min}(N) \ge E + \frac{B}{D_{\max}^{\beta}} \tag{5.3.1}$$

这不是「性能差」，而是「墙」：一条绝对不可逾越的水平线。从这一刻起，仅靠扩大模型来降低损失的策略到了极限——继续烧钱买算力和买参数，只是让 A/N^{α} 趋近于零，而无法突破数据项固定的下界。

更直观地理解这个量级：3×10^{14} 个 token 听起来很多，但在当前的 scaling 速率下，最大前沿模型的训练量每两年翻 5-10 倍。LLaMA-3 已用 15T token，距离总量上限已不遥远。这不是遥远未来的问题，而是业界在训练策略上已经开始应对的当下约束。

第二环：合成数据为何不能突破数据墙——模型崩溃的数学机制

一个显而易见的「解决方案」是用模型自己生成的合成数据来扩充训练集，绕过人类文本的上限。Shumailov 等（Nature 2024）证明这条路在多代迭代后必然失败，原因不是工程缺陷，而是**采样过程的数学必然**。

把真实分布记作 $\mathcal{D}_0$，第 n 代模型从第 $n-1$ 代的输出中采样，再用这些采样训练第 n 代模型。每一代「训练 + 采样」的组合相当于对分布做了一次有噪声的重投影：

1. **有限样本截断**：用有限样本拟合分布时，低概率区域（尾部）在样本中出现频率极低，容易被完全丢失。第一代模型的输出分布相比真实分布，尾部已被轻微截断。
2. **迭代放大**：第二代模型在第一代输出的基础上训练，继承了第一代截断的尾部。第二代再次采样，进一步截断。每一代的截断都作为下一代的训练数据，截断效应单调累积。
3. **方差归零**：经过足够多代迭代，输出分布的方差趋向零——分布从「广泛覆盖语言空间」退化到「集中在几个高概率模式上」，最终接近退化分布。

式 (5.3.2) 所示的 Wasserstein-2 距离衡量第 n 代分布与原始真实分布的差距，随 n 单调增大，趋向无穷：

$$\mathbb{E}[\mathbb{W}_2^2(\mathcal{D}_n, \mathcal{D}_0)] \to \infty \quad \text{as } n \to \infty \tag{5.3.2}$$

这里的「尾部」恰恰是语言中最有信息量的部分——罕见的表达方式、特殊的推理路径、非主流的观点。这些低概率但高价值的模式，在合成数据的迭代过程中被系统性地清除。合成数据不能补充数据墙的缺口，反而以多代迭代的方式加速信息丢失。

一个工程上的修正方向是「在合成数据中混入固定比例的真实数据」——数学上这相当于给重投影加一个固定的「锚点」，阻止方差归零。但这仍然受限于真实数据的总量，只是拖慢而非阻止了退化过程。

第三环：推理时计算作为第二曲线——绕过训练数据上限的机制

如果训练数据的上限是硬约束，是否有其他方式继续提升模型性能？推理时计算（inference-time compute scaling）是一条独立的路径：不增加训练数据，而是在推理阶段投入更多算力，对输出空间做更充分的搜索。

Best-of-K 采样是最简单的形式：对同一输入生成 K 个独立候选输出，用某个评分函数选出最好的。从信息论角度看，这相当于在输出分布上近似采样更高分位数：如果真实输出分布的质量（分数）服从某个分布，那么 K 个样本的最大值的期望随 K 单调增大，增速正比于分布的 K 阶统计量期望。

关键问题是：K 倍推理算力能换来多大提升？为了给出定量比较，Sardana 等引入「等价训练 token 数」的概念，由式 (5.3.3) 定义：

$$D_{\text{equiv}}(K) = \frac{C_{\text{test}}}{6N_0} = \frac{K \cdot 2N_0}{6N_0} = \frac{K}{3} \tag{5.3.3}$$

式 (5.3.3) 表明每 Best-of-3 采样，大约相当于额外提供 1 个训练 token 的算力。这个折算率的直观含义是：推理比训练「便宜」(单位算力的效果更低)，因为训练会把信息永久固化进权重，而推理只是单次使用。但它仍然是一条有效的改进路径，尤其当训练数据已经触顶时。

第四环：两条曲线的适用边界——可验证任务 vs 不可验证任务

推理时计算的效果高度依赖任务类型，原因在于「**选最好的输出**」**需要一个评分函数**，而评分函数的质量决定了「更多采样」是否有意义。

对于**可验证任务**（数学题、代码、逻辑推理），有客观的正误评判标准：数学答案可以核验，代码可以运行并检查测试用例是否通过。K 个候选中选「通过测试的那个」是一个明确且无偏的选择准则。此时，更多采样确实能找到更好的解，等价折算率较高——这是 OpenAI o1、DeepSeek-R1 等「思考型模型」的技术基础，链式推理本质上就是用推理时算力换取对解空间的更充分搜索。

对于**不可验证任务**（开放式创意写作、情感表达、风格偏好），没有客观评分函数。「更多采样」意味着什么？如果用一个另外训练的偏好模型（reward model）来选，那实际上是在训练算力 + 推理算力 + 偏好模型算力的组合成本下换取改进——且偏好模型本身的质量上限又变成了新的约束。等价折算率趋近于零或负数（偏好模型的误差可能带来反效果）。

这条边界有明确的工程含义：**不要盲目把「推理时计算可以弥补训练数据不足」泛化到所有任务**。它是一条在可验证任务域内高度有效的策略，在不可验证任务域内效果有限甚至适得其反。数据墙对可验证任务的限制，被推理时计算在一定程度上绕过；对不可验证任务的限制，目前没有同等有效的替代路径。

整条链的回头看

```
人类文本总量 D_max   3×10^14 token（硬上限）
    数据墙：L_min   E + B/D_max^ （固定下界）
    合成数据迭代 → 方差归零，尾部消失（模型崩溃）
     Σ_n → 0, W ²(D_n, D_0) → ∞
    推理时计算作为第二曲线（绕过训练数据上限）
     D_equiv(K) = K/3（等价 token 折算率）
    适用边界：可验证任务高效，不可验证任务无效
```

这条链揭示了一个深层事实：**数据墙不是「性能暂时停滞」而是「信息供给的物理边界」，它迫使 scaling 的主力战场从「消耗更多训练数据」转向「在推理时更充分地探索解空间」**。这一转变不是技术路线的随意选择，而是被损失函数的结构和人类文本总量的上限共同逼出来的必然结果。可验证任务是这一转变的主要受益领域，因为它们拥有推理时计算所必需的客观评分信号。

严格

模型崩溃（model collapse）的分布退化。Shumailov 等（Nature 2024）证明，若模型递归训练于自身生成的数据，第 n 代模型的输出分布 $\mathcal{D}_n$ 的方差满足：

$$\Sigma_n \xrightarrow{a.s.} 0 \quad \text{as } n \to \infty \tag{5.3.4}$$

直觉：每代生成数据相当于对真实分布做一次有噪采样后再拟合，低概率区域（尾部）

在采样时被截断，经多代迭代后尾部彻底消失，分布收敛到退化点。Wasserstein-2 距离满足：

$$\mathbb{E}\left[\mathbb{W}_2^2(\mathcal{N}(\mu_n, \Sigma_n),\, \mathcal{D}_0)\right] \to \infty \tag{5.3.5}$$

即合成数据与原始真实分布的距离随代数增加而发散。

推理时计算的等价 token 折算。设基础模型参数量为 N_0，推理时使用 Best-of-K 采样：从 K 个独立采样中选最优输出，验证成本约为 $K \cdot 2N_0$ FLOPs。如果以「等价训练 token」衡量推理投入，令 $C_{\text{test}} = K \cdot 2N_0$，则其对应的等价训练 token 数为：

$$D_{\text{equiv}}(K) = \frac{C_{\text{test}}}{6N_0} = \frac{K}{3} \tag{5.3.6}$$

即每 Best-of-3 采样大约相当于额外提供 1 个训练 token 的算力。实践中，推理时计算与训练时计算的「兑换率」取决于任务类型：可验证任务（数学、代码）兑换率高，因为有客观奖励信号指导采样选择；不可验证任务（创意写作）兑换率低。

与原书呼应：原书 §5.3 引用 Shumailov 等人的「模型崩溃」结果，对应本节 $\Sigma_n \to 0$ 的方差退化；§5.4 描述推理时计算缩放，对应本节 $D_{\text{equiv}}(K) = K/3$ 的等价折算关系。

---

§5.5 涌现：神话还是幻觉？

▶这一节在讲什么

为什么「涌现」可能只是测量幻觉？研究者用的是非线性评估指标（如全序列精确匹配），把底层平滑增长的 token 级能力压缩成了视觉上的阶跃——用阶梯状温度计量连续升温的水，读数也会「突变」。

第一步：非线性指标放大了导数。长度为 ℓ 的精确匹配指标等于 $p(N)^\ell$，在 p 过临界值时，导数被 $\ell \cdot p^{\ell-1}$ 放大，序列越长，「阶跃」越陡峭——这是数学必然，而非新能力的出现。

第二步：换连续指标，涌现消失。Schaeffer 等用 token 级 log-likelihood 重新评估原本呈现「涌现」的任务，曲线全部变为平滑的幂律增长，与预训练损失吻合。

第三步：保留少数真实阈值。Olsson 等对 induction head 电路的机制研究表明，in-context learning 能力的出现即使在连续指标下也呈现阶跃——但这是电路形成的结构性事件，与热力学相变仍有本质区别（见 §5.10）。

本节要拿走的一件事：大多数「涌现」是 $dm/dN = \delta(p(N) - \theta) \cdot dp/dN$ 的指标伪影；真正可信的涌现需要在所有连续可微指标下均持续存在。

直觉

原书 §5.5 的核心争论：Wei 等（2022）记录了许多任务上随模型规模增大出现「突然跃升」的准确率——被称为涌现能力（emergent abilities）。Schaeffer、Miranda 与

Koyejo（NeurIPS 2023 杰出论文）反驳说，这种「涌现」是测量幻觉：研究者选择了分段不连续的评估指标，把底层平滑的能力增长映射成了视觉上的阶跃。

本节出现四个核心概念：**底层能力与表观指标的区分**、**非线性指标放大导数的数学机制**、**连续指标下的平滑验证**、**真实阈值与指标伪影的区分准则**。它们沿一条逻辑链彼此推出：先理解为什么需要区分两个层次，再看非线性函数如何把平滑信号变成阶跃信号，然后理解换连续指标后为何涌现消失，最后识别哪些涌现在所有指标下都持续存在，从而可能是真实的。下面一环一环展开。

第一环：两个层次的区分——底层能力 vs 表观指标

理解涌现争论的关键，是把一件事拆成两个层次：**底层能力**和**评估指标**。

底层能力指的是模型在 token 级别上的预测精度，通常用 token-level log-likelihood 或 token-level accuracy 来衡量。这一层次随模型规模连续、单调地提升，与预训练损失曲线吻合，没有任何阶跃迹象——这是所有已知证据都一致同意的事实。

表观指标是研究者为了评估「某项能力」而设计的测量量，例如：三位数加法的全序列精确匹配（exact match）、多选题准确率、特定任务的通过率。这些指标是底层能力的**函数**，但是一个**高度非线性的函数**。

问题就在于：非线性变换可以把连续、平滑的底层信号变成在特定点处集中、在视觉上呈阶跃状的表观信号。这是一个纯粹的数学事实，与「能力是否真实存在」无关。研究者把表观指标的阶跃报告为「涌现」，混淆了两个层次——就像用一把只有 0 和 1 两个刻度的温度计去量逐渐升温的水，然后宣称「水温发生了突变」。

第二环：非线性指标放大导数的数学机制

具体地，精确匹配指标为何会制造阶跃？

设模型规模为 N，token 级正确率为 $p(N)$，假设其随 N 连续单调递增（如随 $\log N$ 线性增长）。对长度为 ℓ 的序列，「全序列正确」指标要求每一位都正确：

$$M_{\text{exact}}(N) = p(N)^{\ell} \tag{5.5.1}$$

当 $p(N) < 1/2$ 时，式 (5.5.1) 中的 M_{exact} 随 ℓ 指数下降，接近 0；当 $p(N)$ 略过某个临界值时，M_{exact} 迅速上升。对 N 求导：

$$\frac{dM_{\text{exact}}}{dN} = \ell \cdot p(N)^{\ell-1} \cdot \frac{dp}{dN} \tag{5.5.2}$$

当 ℓ 很大时（比如 $\ell = 10$ 对应十位数加法），式 (5.5.2) 中放大因子 $\ell \cdot p^{\ell-1}$ 在 $p(N)$ 刚过 $1/2$ 的区域极大，导致 M_{exact} 对 N 呈近似阶跃响应。dp/dN 是平滑的，但乘上放大因子后，导数集中在一个极窄的规模窗口内，视觉上就是「突然涌现」。

更一般地，对任意评估指标 $m(p)$，其对规模的导数为：

$$\frac{dm}{dN} = m'(p) \cdot \frac{dp}{dN} \tag{5.5.3}$$

如果 $m(p)$ 是阶梯函数 $\mathbf{1}[p \geq \theta]$（如「通过/不通过」），则由式 (5.5.3) 可知 $m'(p) = \delta(p - \theta)$，在 $p(N^*) = \theta$ 处产生一个 Dirac 脉冲（式 (5.5.4)）：

$$\frac{dm}{dN} = \delta(p(N) - \theta) \cdot \frac{dp}{dN} \tag{5.5.4}$$

这是一个在 N^* 处集中的脉冲——正是视觉上「涌现」的数学成因。底层 dp/dN 始终是平滑的；脉冲完全来自 $m(p)$ 的非线性。

一个直观的类比：把连续函数通过一个很陡的 sigmoid 压缩到 $[0, 1]$ 区间，输出看起来像阶跃——但没有人会说「输入函数发生了突变」。测量「涌现」的操作就是这个压缩过程。

第三环：换连续指标后涌现消失——Schaeffer 的实验验证

这个分析的核心可证伪预言是：如果「涌现」是指标伪影，那么换用连续指标（令式 (5.5.3) 中 $m'(p) = 1$，如 token-level log-likelihood）后，曲线应当还原为平滑增长。

Schaeffer 等在实验中对 BIG-Bench 中原本呈现「涌现」的多个任务做了替换：用 token-level log-likelihood 替代精确匹配或多选准确率。结果：**所有被检验的「涌现」任务，在连续指标下均表现为平滑的幂律增长**，与预训练损失曲线完全吻合，没有阶跃迹象。

这一结果本身的可证伪结构很强：如果某个任务的「涌现」在替换连续指标后消失，则该涌现大概率是指标伪影；如果替换后依然存在，则该涌现有更强的可信度，可能是真实的能力阈值。

Schaeffer 等的工作并不是说「没有任何真实的能力突变存在」，而是说「大多数被报告为涌现的现象，实际上是评估方法的产物，而非底层能力的阶跃」。这是一个可以区分的两类情况，下面的第四环讨论如何区分。

第四环：真实阈值的识别准则——在所有连续可微指标下均持续存在

是否有任何「涌现」经得起连续指标的检验？Olsson 等（2022）的机制可解释性工作提供了一条独立的证据链。

他们研究的不是评估指标，而是模型内部电路结构。他们追踪了 in-context learning 能力出现时，模型内部「induction head」电路的形成时刻。发现在某个特定训练步骤，induction head 电路从「不存在」突然变为「稳定存在」——这是模型参数空间中的结构性事件，而非评估指标函数值的变化。

更重要的是，这个电路形成事件与 in-context learning 能力的阶跃在时间上高度同步，且即使换用连续指标来衡量 in-context learning 能力，阶跃仍然可见（只是比精确匹配指标下的阶跃更平滑一些）。这说明存在少数**真实能力阈值**——其背后是电路层面的结构性转变，不是指标的非线性投影。

由此得出识别准则：**若某任务的「涌现」在所有连续可微指标下均持续存在，则该涌现更可能是真实的能力阈值，而非指标伪影**。反过来，如果换连续指标后阶跃消失，则该涌现是伪影。

但这里还有一个微妙之处：即使是「真实」的 induction head 电路形成事件，也与热力学相变存在本质区别。物理相变要求热力学极限，有严格的临界指数和普适类；induction head 的形成是一个有限网络中梯度流驱动的结构性转变，没有已知的普适类归属（这个问题留给 §5.9/5.10 讨论）。所以「真实涌现」和「物理相变」仍然是不同的层次。

整条链的回头看

```
底层能力 p(N): token 级正确率，随 N 连续平滑增长
    非线性指标 m(p)（精确匹配、通过/不通过）
     dm/dN = m'(p) · dp/dN
     m'(p) = （p- ）→ 脉冲，视觉上「涌现」
    换连续指标 m(p) = p → m'(p) = 1 → 曲线还原平滑
    Schaeffer: BIG-Bench 中涌现任务在连续指标下全部平滑
    识别准则：连续指标下仍有阶跃 → 真实阈值（Olsson induction head）
    真实阈值   物理相变（见 §5.9/5.10）
```

这条链揭示了一个深层事实：「**涌现**」**这个词混淆了两种截然不同的现象——评估指标的非线性函数值跳变，和模型内部表示结构的真实突变**。前者是测量工具的产物，通过更换指标可以消除；后者是网络动力学的真实事件，换指标也消除不了，但仍然不等于物理相变。能力评估的可信度，从根本上取决于「你用什么指标、你是否验证了换指标后结论是否不变」。

严格

指标非线性导致的表观涌现。设模型规模为 N，token 级正确率（token-level accuracy）为 $p(N)$，假设其随 N 连续、单调递增（例如随 $\log N$ 线性增长）。

对长度为 ℓ 的序列，「全序列正确」（exact match）指标为：

$$M_{\text{exact}}(N) = \prod_{i=1}^{\ell} p_i(N) \approx p(N)^{\ell} \tag{5.5.5}$$

其中假设各 token 独立且每位正确率相同。当 $p(N) < 1/\ell$ 时，$M_{\text{exact}} \approx 0$；当 $p(N)$ 略超过 $1/\ell$ 时，M_{exact} 迅速从 0 跳向正值。具体地，在 $p(N^*) = 1/\ell^{1/\ell}$ 附近，M_{exact} 对 N 的导数为：

$$\frac{dM_{\text{exact}}}{dN} = \ell \cdot p(N)^{\ell-1} \cdot \frac{dp}{dN} \tag{5.5.6}$$

当 ℓ 大时（如 $\ell = 10$ 的三位数加法），$\ell \cdot p^{\ell-1}$ 在过渡区附近极大，导致 M_{exact} 对 N 呈近似阶跃响应。Schaeffer 将此一般化为：若评估指标 $m(p)$ 是关于 p 的非线性函数（如 $m(p) = \mathbf{1}[p \geq \theta]$），则：

$$\frac{dm}{dN} = m'(p) \cdot \frac{dp}{dN} \tag{5.5.7}$$

对阶梯函数 $m(p) = \mathbf{1}[p \geq \theta]$，其导数为 Dirac delta：$m'(p) = \delta(p - \theta)$，故：

$$\frac{dm}{dN} = \delta(p(N) - \theta) \cdot \frac{dp}{dN} \tag{5.5.8}$$

这是一个在 $p(N^*) = \theta$ 处集中的脉冲——视觉效果即「涌现」。而换用连续指标 $m(p) = p$（token-level log-likelihood）时，$m'(p) = 1$，曲线即还原为 dp/dN 的平滑形状。

连续度量下的平滑性验证。 Schaeffer 等在实验中替换指标后验证：原本呈现「涌现」的任务，在 token-level log-likelihood 下均表现为平滑的幂律增长，与预训练损失曲线

吻合。这一结果的可证伪推论是：**若某任务的「涌现」在所有连续可微指标下均持续存在，则该涌现更可能是真实能力阈值，而非指标伪影**。

反驳路线保留的空间：Olsson 等（2022）的 induction head 工作从机制层面证明，in-context learning 能力的出现与模型内部 induction head 电路的形成在训练步骤上高度同步，呈现明确的阶跃特征——即使换用连续指标，该阶跃仍然可见。这说明存在少数真实的能力阈值，但它们与物理意义上的「相变」仍有本质区别（见 §5.10）。

与原书呼应：原书 §5.5 引用 Schaeffer 等（NeurIPS 2023 杰出论文），对应本节 $dm/dN = \delta(p(N)-\theta)\cdot dp/dN$ 的形式分析；原书提及换用 token-level log-likelihood 曲线变平滑，对应本节连续指标下的 $m'(p)=1$ 情形。

---

§5.6–§5.7 Grokking 的数学解剖：从死记到顿悟

▶这一节在讲什么

为什么模型会先「死记」再「顿悟」？两套电路在同一个网络里竞争：稠密记忆电路快速拟合训练集但参数范数大，稀疏傅里叶电路需要更长时间发展但范数小。权重衰减持续惩罚大范数，最终让记忆电路衰竭、傅里叶电路胜出。

第一步：任务的代数结构决定了「顿悟」的形式。模算术 $a+b \bmod p$ 恰好对应圆周群上的相位旋转，而圆周群的表示理论正是傅里叶基——模型学到的不是查表，而是三角恒等式 $\cos(\theta_a+\theta_b)=\cos\theta_a\cos\theta_b-\sin\theta_a\sin\theta_b$。

第二步：两阶段动力学。记忆阶段训练损失快速下降、测试损失不动；竞争阶段两套电路并存、傅里叶分量逐渐增强；清理阶段记忆权重被压到阈值以下，测试损失突然归零。

第三步：增大权重衰减系数 λ_{wd} 可以提前 Grokking 时刻——这是正则化驱动的竞争过程，而非任何意义上的相变。

本节要拿走的一件事：Grokking 的延迟泛化不是神秘的「顿悟」，而是权重衰减在两套电路之间执行的一场慢速淘汰赛，任务的代数结构决定了最终胜出电路的形式。

直觉

原书 §5.6–§5.7 描述了 Grokking 现象（Power 等，2022）：在模算术任务 $a+b \bmod p$ 上，模型先 100% 记忆训练集（测试准确率约 0%），再经数万步训练后测试准确率突然跃升至 100%。这被称为「延迟泛化」。

本节出现五个核心概念：**两套电路的竞争、任务代数结构与傅里叶基的对应、权重衰减的淘汰机制、信噪比的动力学演化、Grokking 的信噪比解释**。它们沿一条逻辑链彼此推出：先理解为什么同一个网络里会同时存在两套电路，再理解为什么模算术任务的「正确」表示恰好是傅里叶基，然后理解权重衰减如何扮演裁判角色，最后把整个过程统一为信噪比的动态演化。下面一环一环展开。

第一环：为什么网络里会同时存在两套电路——记忆电路与傅里叶电路的共存

梯度下降是一个贪心的局部搜索过程：它沿着损失下降最快的方向移动，不在乎找到

的解是「聪明的」还是「笨的」。面对有限的训练集，最快降低训练损失的方式是「死记硬背」——直接记住每对 (a, b) 对应的答案，不需要理解任何结构。

死记硬背（记忆电路）在训练的早期阶段会率先形成，因为它需要的计算模式简单：把 (a, b) 的组合直接映射到输出，相当于建立一个高维查找表。训练损失迅速下降，而测试损失不动——网络记住了训练集但没有学到可泛化的规则。

与此同时，还存在另一套更「优雅」的表示方式——傅里叶电路：把整数 a 映射到圆周上的相位 $e^{2\pi ika/p}$，利用三角恒等式实现模加法。这套电路需要更多步骤才能稳定形成，因为它需要网络「发现」任务的代数结构。在记忆电路充分发展的早期阶段，傅里叶电路只是一个微弱的次级结构，被记忆电路掩盖，在测试准确率上完全看不到。

两套电路**同时存在于同一个网络中**，互相争夺权重的「分配」。权重是共享的，每个参数对两套电路都有贡献；优化过程会把权重推向使训练损失下降最快的方向，而这个方向在初期有利于记忆电路。

第二环：为什么模算术的正确表示是傅里叶基——任务代数结构决定「顿悟」的形式

这里有一个深刻的问题：梯度下降本来是无结构的局部搜索，为什么会「发现」一个如此漂亮的群论结构？

原因在于任务本身的代数结构对梯度场施加了隐式约束。整数 $\{0, 1, \ldots, p-1\}$ 在模 p 加法下构成**循环群** $\mathbb{Z}/p\mathbb{Z}$（也称圆周群）。群论中有一条基本结果：任何阿贝尔群的不可约复表示都是一维的，循环群 $\mathbb{Z}/p\mathbb{Z}$ 的不可约表示正好是复指数函数：

$$\rho_k(a) = e^{2\pi ika/p}, \quad k = 0, 1, \ldots, p-1 \tag{5.6.1}$$

这些复指数（式 (5.6.1)）的实部和虚部就是傅里叶基 $\{\cos(2\pi ka/p), \sin(2\pi ka/p)\}$。模加法在这套基下变成了相位相加：$e^{2\pi ik(a+b)/p} = e^{2\pi ika/p} \cdot e^{2\pi ikb/p}$，乘法对应于神经网络中很容易实现的双线性运算（注意力机制的 query-key 内积）。

换言之，**傅里叶基是模算术任务在表示论意义上「最自然」的基**，它把模加法变成最简单的点乘运算。任何试图用其他基表示这个任务的方案都需要更多参数和更复杂的运算。梯度下降通过损失函数感知到了这种「自然性」——傅里叶电路在训练集上的参数效率更高，在权重衰减的压力下有长期竞争优势。

这不是研究者施加的先验，而是任务结构通过损失函数传递给网络的隐式约束。如果换一个结构不同的任务（比如不是群结构），网络「顿悟」到的表示就会是另一种形式。Grokking 因此提供了一个可以精确分析「网络学到什么结构」的理想实验室：任务的代数结构已知，网络学到的表示可以通过傅里叶分析精确识别。

第三环：权重衰减是裁判——两套电路的慢速淘汰赛

现在两套电路已经理解了，问题变成：为什么傅里叶电路最终会「赢」？答案是权重衰减（weight decay）。

式 (5.6.7) 的权重衰减在损失函数中加入 $\frac{\lambda_{\text{wd}}}{2}\|\theta\|_2^2$ 的正则化项，持续惩罚参数范数。对两套电路而言：

- **记忆电路**需要大量权重来存储训练集中每对 (a, b) 的映射——相当于一个大型查找表，需要 $O(p^2)$ 量级的有效参数。范数大，被权重衰减惩罚得厉害。
- **傅里叶电路**用少数几个频率分量（实验中约 5 个）表示整个任务——相当于一个

紧凑的代数公式，需要的有效参数远少于 p^2。范数小，被权重衰减惩罚得轻。

在训练的早期，记忆电路的「效率优势」（快速降低训练损失）压倒了权重衰减的「范数惩罚」；但权重衰减的惩罚是持续的，随着训练进行，记忆电路的范数越来越大，惩罚越来越重。最终，记忆电路的参数被压到某个功能阈值以下，无法再正确表达训练集中的所有映射；此时傅里叶电路作为「备用」接管了输出，测试准确率突然从 0% 跳到 100%。

权重衰减是 Grokking 的必要条件，不是充分条件。如果 $\lambda_{\text{wd}} = 0$（无正则化），记忆电路的权重永远不会被压下来，傅里叶电路的功能永远被掩盖，Grokking 不发生。这是一个可以直接验证的预言——实验证实：增大 λ_{wd} 提前 Grokking 时刻，减小 λ_{wd} 推迟甚至阻止 Grokking。

第四环：整个过程的信噪比解释——「顿悟」是噪声衰减而非信号出现

把傅里叶电路的功能贡献看作「信号」，记忆电路的功能贡献看作「噪声」（对测试集无用的拟合），整个 Grokking 过程是式 (5.6.2) 所示的**信噪比动态演化**。

- 训练初期：噪声（记忆）主导，信号（傅里叶）微弱。信噪比极低，测试准确率为 0%，因为傅里叶信号完全淹没在记忆噪声中，无法提供泛化的预测。
- 训练中期：噪声在权重衰减下缓慢下降，信号在梯度驱动下缓慢上升。训练损失接近 0（两套电路共同作用），测试损失基本不变（因为傅里叶信号仍然被噪声掩盖，测试时的判断主要来自不可泛化的记忆）。
- 训练后期：噪声被压到某个临界水平以下，傅里叶信号成为主导。信噪比超过阈值，测试准确率突然跃升。

这个框架有一个重要含义：**测试准确率的阶跃并不对应任何新知识的获取，而只是信噪比越过了某个阈值**。傅里叶电路一直在那里，只是被掩盖了。「顿悟」不是「学会了新东西」，而是「一直在学的东西终于不被遮盖了」。

这与 §5.5 的涌现讨论形成有趣的对比：Grokking 中的「涌现」（测试准确率的阶跃）有一部分确实是真实的——它对应于记忆电路被充分压制、傅里叶电路充分主导这一结构性事件。但它与热力学相变仍然不同：没有普适临界指数，依赖 λ_{wd} 等超参数，是一个「可调的」转变时刻而非物理意义上的固定临界点。

$$\text{信噪比} = \frac{L_{\text{Fourier}}(t)^{-1}}{L_{\text{mem}}(t)^{-1} + L_{\text{Fourier}}(t)^{-1}} \xrightarrow{t\to\infty} 1 \tag{5.6.2}$$

整条链的回头看

梯度下降的贪心性
  记忆电路率先形成（快速降低训练损失）
  傅里叶电路缓慢发展（任务代数结构的自然表示）
    任务：`/p`（循环群）→ 不可约表示 = 傅里叶基
    `e^{2 ika/p}` 把模加法变成相位相加（双线性实现）
  权重衰减持续惩罚大范数
    记忆电路：`O(p²)` 有效参数，范数大，惩罚重
    傅里叶电路：`~5` 个频率，范数小，惩罚轻
  记忆电路范数衰减至功能阈值以下
  傅里叶电路接管输出，测试准确率突然跃升

信噪比越过临界值（「顿悟」= 噪声衰减，非信号出现）

这条链揭示了一个深层事实：**Grokking 是权重衰减在两套表示之间执行的一场慢速淘汰赛，而淘汰赛的胜负由任务的代数结构预先决定**。哪套表示用更少的参数表达了任务的本质结构，哪套就会在正则化的长期压力下胜出。这不是模型「顿悟」了什么，而是优化过程在正则化约束下找到了最经济的表示——这个最经济的表示，恰好与任务的数学本质同构。

严格

任务设置。取素数 $p = 113$，输入为 $(a, b) \in \{0, \dots, p-1\}^2$，目标为 $(a + b) \bmod p$。训练集取所有 p^2 对中的 30%，测试集为剩余 70%。

Embedding 矩阵的傅里叶基收敛。Nanda 等逆向工程发现，训练后模型的 embedding 矩阵 $E \in \mathbb{R}^{p \times d}$（$d$ 为隐藏维度）在若干频率 k 处形成了如下结构：

$$E[a, 2k] = \cos\left(\frac{2\pi k a}{p}\right), \quad E[a, 2k+1] = \sin\left(\frac{2\pi k a}{p}\right), \quad k \in \mathcal{K} \tag{5.6.3}$$

其中 $\mathcal{K}$ 是模型自发选择的少数几个频率（实验中约 5 个）。这等价于将整数 a 映射为其在特定频率下的复数相位 $e^{2\pi i k a / p}$。

注意力层实现频域加法。设 attention 层在频率 k 处的操作将 a 和 b 的嵌入合并，实现：

$$\cos\left(\frac{2\pi k(a+b)}{p}\right) = \cos\left(\frac{2\pi k a}{p}\right)\cos\left(\frac{2\pi k b}{p}\right) - \sin\left(\frac{2\pi k a}{p}\right)\sin\left(\frac{2\pi k b}{p}\right) \tag{5.6.4}$$

这正是余弦的加法定理。模型利用 bilinear 注意力结构（query 与 key 的内积）自然地实现了这一乘积——两个 embedding 向量的内积提取了跨频率的 $\cos(\theta_a)\cos(\theta_b)$ 和 $\sin(\theta_a)\sin(\theta_b)$ 项。

输出层的逆傅里叶解码。网络最终输出 logit 向量 $\hat{y} \in \mathbb{R}^p$，其中：

$$\hat{y}_c \propto \sum_{k \in \mathcal{K}} w_k \cos\left(\frac{2\pi k(a+b-c)}{p}\right) \tag{5.6.5}$$

对 c 取 argmax，即等价于用若干傅里叶频率的叠加来定位 $(a + b) \bmod p$。

两阶段动力学：权重衰减驱动的延迟泛化。定义两个进度指标：

$$L_{\text{Fourier}}(t) = L(\text{仅保留傅里叶频率分量的模型}), \quad L_{\text{mem}}(t) = L(\text{去除傅里叶分量后的模型}) \tag{5.6.6}$$

训练动力学分三阶段：

阶段	步数 ($p = 113$)	训练 loss	测试 loss	L_{Fourier}	L_{mem}
记忆	$0 - 1$k	快速下降	不动	高	快速下降
电路竞争	1k $- 9$k	接近 0	不动	下降	下降

阶段	步数 ($p = 113$)	训练 loss	测试 loss	L_{Fourier}	L_{mem}
清理	9k − 14k	≈ 0	突然下降至 0	≈ 0	≈ 0

权重衰减的数学角色：设权重为 θ，ℓ_2 正则化强度为 λ_{wd}，则训练目标为：

$$\mathcal{J}(\theta) = L_{\text{CE}}(\theta) + \frac{\lambda_{\text{wd}}}{2}\|\theta\|_2^2 \tag{5.6.7}$$

稠密记忆电路需要大量权重（大 $\|\theta\|_2^2$），而稀疏傅里叶电路用少数频率分量表示同样的功能，权重范数更小。正则化项持续惩罚记忆电路，使其在竞争中逐渐处于劣势；当记忆电路权重衰减至低于傅里叶电路的表达阈值时，后者独占输出，测试 loss 才突然降至 0。

由式 (5.6.7) 可知，这解释了 Grokking 的关键实验观察：增大 λ_{wd} 可以提前 Grokking 时刻，减小 λ_{wd} 则推迟甚至阻止 Grokking——表明这是正则化驱动的竞争过程，而非任何形式的真实相变。

与原书呼应：原书 §5.7 引用 Nanda 等（ICLR 2023），对应本节 embedding 的傅里叶基收敛公式与三角恒等式推导；§5.6 提及「权重衰减驱动延迟泛化」，对应本节正则化目标 $\mathcal{J}(\theta)$ 及两阶段动力学表格。

---

§5.8 渗流相变：把 Scaling Law 焊到统计物理上

▶这一节在讲什么

为什么渗流理论是描述涌现的合适框架？因为它正好捕捉了「单元素平滑增长 → 整体性质急剧变化」的模式：每个格点的占据概率 p 连续增大，但最大连通簇的尺寸在临界点 p_c 处从对数级跳到线性级。这与「单概念能力平滑增长、复合推理突然可行」的结构同构。

第一步：二维渗流的精确临界指数。序参量 $P_\infty \sim (p - p_c)^\beta$（$\beta = 5/36$），关联长度 $\xi \sim |p - p_c|^{-\nu}$（$\nu = 4/3$），均为严格数学结果，来自共形场论和 SLE 理论。

第二步：能力涌现的渗流映射。把单概念掌握概率 $p(N)$ 建模为 N 的连续函数，复合能力 $\mathcal{C}(N)$ 在 $p(N)$ 过 p_c 时导数呈类 delta 脉冲——可证伪预测：推理链越长，涌现越陡峭。

第三步：神经网络剪枝验证。Pesce 等（2026）发现剪枝到约 98% 边时网络功能崩溃，功能-保留率曲线与渗流序参量 $F \sim (\bar{q} - \bar{q}_c)^\beta$ 形式同构。

本节要拿走的一件事：渗流给涌现提供了一个有精确临界指数、可做定量预测的物理框架——但这个框架能否真正用于神经网络，取决于「单概念掌握」这一映射是否成立，这是尚待验证的假设。

直觉

原书 §5.8 将「涌现」与统计物理中的渗流理论（percolation theory）相联系：把模型掌握的「概念」想象成图上的节点，复合能力对应图上的连通路径。当单概念掌握概率越过临界值 p_c，最大连通簇尺寸从对数级跳到线性级——这给「涌现」一个可证伪的渗流解释。

本节出现四个核心概念：**渗流临界现象与临界指数**、**序参量与连续相变**、**能力涌现的渗流映射**、**神经网络剪枝的相变行为**。它们沿一条逻辑链彼此推出：先理解渗流为什么是「单元素平滑增长 → 整体性质急剧变化」的典型模型，再看它的临界指数有多精确，然后理解如何把神经网络的「概念掌握」映射到渗流的格点，最后看剪枝实验提供的独立实证路径。下面一环一环展开。

第一环：渗流模型的核心机制——为什么单元素平滑增长能导致整体急剧变化

渗流模型描述的是这样一个过程：在一个格点图上，每个格点以概率 p 被「占据」，不同格点独立。当 p 很小时，被占据的格点稀散，只形成小的孤立集群；当 p 超过某个临界值 p_c 时，突然出现一个跨越整个系统的「巨型连通簇」（infinite cluster）——这个转变就是渗流相变。

关键点是：p 从 0 到 1 是连续变化的，但系统的整体连通性（是否存在跨越整个系统的路径）在 p_c 处发生**急剧转变**。这是「单元素连续 → 整体急剧」的典型结构，与「底层能力平滑增长、表观性能突然可行」的涌现现象在结构上对应。

从信息传递的角度理解：如果每个格点代表「某个概念是否被模型掌握」，连通路径代表「多个概念可以链式组合完成复合推理」，那么渗流的临界值 p_c 就对应「模型能够完成多步推理的能力阈值」。单步概念掌握概率 $p(N)$ 平滑增长，但多步推理能力 $\mathcal{C}(N)$ 只有在 $p(N) > p_c$ 时才突然可行——因为路径的存在性是一个全局性质，对 p 高度敏感。

第二环：精确临界指数——渗流的定量可证伪性

渗流模型不仅提供了定性的「急剧变化」图像，还给出了**精确的定量预测**——临界指数。这是渗流框架相比于「复杂系统」等笼统描述的真正优越性。

序参量 $P_\infty(p)$ 是随机选取的占据格点属于无穷大连通簇的概率。在临界点 p_c 附近：

$$P_\infty(p) \sim (p - p_c)^\beta, \quad \beta = \frac{5}{36} \tag{5.8.1}$$

这个临界指数 $\beta = 5/36$ 不是数值拟合，而是通过共形场论（CFT）和 Schramm-Loewner 演化（SLE）理论严格证明的精确数学结果。它属于二维伯努利渗流的**普适类**——同一普适类的所有系统（不同格点类型、不同微观细节）共享相同的临界指数，只有宏观对称性决定普适类的归属。

式 (5.8.6) 中关联长度 $\xi(p) \sim |p - p_c|^{-\nu}$（$\nu = 4/3$）描述连通簇的典型线性尺度。$\xi \to \infty$ 意味着临界点附近任意尺度的涨落都存在——系统在所有尺度上都「敏感」，这是连续二阶相变的标志。二阶相变（序参量从临界点出发连续上升）而非一阶相变（序参量不连续跳变），意味着渗流没有亚稳态和滞后效应。

这些精确数值（$\beta = 5/36$，$\nu = 4/3$）的重要性在于：**如果神经网络的涌现行为真的属于同一普适类，它就应该满足完全相同的指数**。这是一个原则上可以检验的定量预言，不是一个可以随意调整的比喻。

第三环：能力涌现的渗流映射——把「概念掌握」建模为格点占据概率

如何把渗流框架应用到神经网络的涌现？核心映射是：把单个概念的「掌握程度」建模为一个关于模型规模 N 的连续函数 $p(N)$，当 $p(N)$ 超过渗流临界值 p_c 时，依赖多个概念的复合能力 $\mathcal{C}(N)$ 突然可行。

用 sigmoid 函数建模 $p(N)$（式 (5.8.2)，连续单调增，随 N 增大趋向 1）：

$$p(N) = \sigma(\alpha \log N + \beta_0) = \frac{1}{1 + N^{-\alpha} e^{-\beta_0}} \tag{5.8.2}$$

式 (5.8.2) 中 $p(N)$ 是平滑的，但复合能力 $\mathcal{C}(N)$（多概念链式推理成功的概率）在渗流框架下对 N 的导数呈现类 delta 脉冲（式 (5.8.3)）：

$$\frac{d\mathcal{C}}{dN} \approx \delta(p(N) - p_c) \cdot \frac{dp}{dN} \tag{5.8.3}$$

这与 §5.5 中讨论的评估指标非线性是不同的：这里不是评估指标的函数值跳变，而是复合能力的物理机制（连通路径存在性）在 $p = p_c$ 处的本质性转变。

这个框架有一个**可证伪的定量预测**：推理链越长（需要更多概念串联），涌现应该越陡峭——单步任务平滑，多步任务陡峭。如果渗流映射正确，不同长度推理链任务的「涌现陡峭程度」应该与渗流模型的预测一致。

然而，这个映射成立的前提是「单概念掌握概率」这一概念的可操作化——如何从神经网络的实际行为中测量它？目前这仍然是未验证的假设。渗流框架的定量预测能力，依赖这一步映射的严格建立。

第四环：神经网络剪枝的相变行为——独立的实证路径

与「概念 → 格点」的抽象映射不同，神经网络剪枝提供了一条**不依赖这一映射的独立实证路径**：直接在网络连通性层面测量功能-保留率曲线。

Pesce、He 与 Caldarelli（2026）研究神经网络权重剪枝时发现：随着删除边的比例 q 从 0 增大，网络在特定任务上的功能（准确率）在临界比例 $q_c \approx 98\%$ 处发生从「基本保持」到「完全失能」的急剧转变。更重要的是，式 (5.8.4) 所示的功能-保留率曲线形式与渗流序参量同构：

$$F(\bar{q}) \sim (\bar{q} - \bar{q}_c)^{\beta} \cdot \mathbf{1}[\bar{q} > \bar{q}_c] \tag{5.8.4}$$

其中 $\bar{q} = 1 - q$ 是边保留率，$\bar{q}_c \approx 0.02$（约 2% 保留），β 值接近渗流临界指数。

这个结果的意义在于：**网络功能不是参数量的线性函数，而是连通性的高度非线性函数**。可以移除 98% 的权重而性能几乎不变，越过临界值后才发生崩塌——这是连通性主导功能的典型渗流特征。

剪枝实验与「概念掌握」的涌现是否属于同一物理机制，尚需更多研究来确认。但剪枝实验至少独立证实了「神经网络功能遵循渗流式临界行为」这一事实，为渗流框架的应用提供了直接的网络层面支撑。

整条链的回头看

渗流：单格点平滑增长（p 从 0 到 1）

```
临界点 p_c 处出现无穷大连通簇
  序参量 P∞ ~ (p-p_c)^β ,  β = 5/36 (精确结果，非拟合)
  关联长度  ξ ~ |p-p_c|^{-ν},  ν = 4/3
  二阶连续相变 (无亚稳态、无滞后)
映射到 AI 涌现: 单概念掌握 p(N) → 格点占据概率
  复合能力 C(N) 在 p(N) > p_c 时突然可行
  可证伪预测: 推理链越长，涌现越陡峭
剪枝实验的独立支撑 (不依赖概念映射)
  F(q̄) ~ (q̄-q̄_c)^β , q_c ≈ 98%
  网络功能是连通性的渗流函数
```

这条链揭示了一个深层事实：**渗流框架给涌现提供了有精确临界指数、原则上可做定量预测的物理语言——但这个框架能否真正定量应用于神经网络，取决于「单概念掌握」这一映射能否严格建立**。目前，渗流是一个结构高度相似的物理类比，剪枝实验在连通性层面提供了独立支撑；但从「结构相似」到「可以用临界指数做定量预测」，还需要跨越「映射严格性」这一关键步骤。

严格

二维 site percolation 的临界现象。考虑二维正方形格点 $\mathbb{Z}^2$ 上的伯努利渗流（Bernoulli site percolation）：每个格点以概率 p 被「占据」，彼此独立。临界概率 $p_c \approx 0.5927$（数值精确值）。

设 $P_\infty(p)$ 为无穷大格点中随机选取的占据格点属于无穷大连通簇的概率（序参量）。在 p_c 附近：

$$P_\infty(p) \sim (p - p_c)^{\beta}, \quad p \to p_c^{+} \tag{5.8.5}$$

其中临界指数 $\beta = 5/36$。关联长度（correlation length）ξ，即连通簇线性尺度的特征值，满足：

$$\xi(p) \sim |p - p_c|^{-\nu}, \quad \nu = 4/3 \tag{5.8.6}$$

$\xi \to \infty$ 意味着在临界点附近，任意尺度的涨落均存在——这是连续二阶相变的标志。序参量 P_∞ 在 p_c 处连续地从 0 上升（不发生不连续跳变），故为二阶（连续）相变。

这两个精确临界指数（$\beta = 5/36$，$\nu = 4/3$）来自二维伯努利渗流的精确结果，是严格数学意义上（通过共形场论和 SLE 理论）已经证明的，并非数值近似。

能力涌现的渗流映射。将单概念 c 的「掌握概率」建模为规模的函数：

$$p(N) = \sigma(\alpha \log N + \beta_0) = \frac{1}{1 + N^{-\alpha} e^{-\beta_0}} \tag{5.8.7}$$

其中 σ 为 sigmoid 函数，$p(N)$ 关于 N 连续单调递增。

定义复合能力函数 $\mathcal{C}(N)$ 为「存在长度 $\geq k$ 的全连通路径」的概率（即多跳推理成功的概率）。在渗流框架下，当 $p(N)$ 越过 p_c 时：

$$\frac{d\mathcal{C}}{dN} \approx \delta(p(N) - p_c) \cdot \frac{dp}{dN} \tag{5.8.8}$$

单概念能力 $p(N)$ 平滑，但复合能力 $\mathcal{C}(N)$ 在 p 过 p_c 时呈现类 delta 脉冲。可证伪预测：**能力涌现的陡峭程度应正比于推理链长度** k——单步任务平滑，多步任务陡峭。

神经网络剪枝的相变行为。Pesce、He 与 Caldarelli（2026）研究神经网络剪枝时发现：随着删除边的比例 q 从 0 增大，网络功能（以某任务准确率衡量）在临界比例 q_c 处发生从「基本保持」到「完全失能」的急剧转变，且临界指数与渗流的标度律一致。实验表明，最高可移除约 98% 的边而性能损失很小，越过 q_c 后才发生崩塌。

设网络功能 F 为边保留率 $\bar{q} = 1 - q$ 的函数，则：

$$F(\bar{q}) \sim (\bar{q} - \bar{q}_c)^{\beta} \cdot \mathbf{1}[\bar{q} > \bar{q}_c] \tag{5.8.9}$$

其中 $\bar{q}_c \approx 0.02$（约 2% 保留）对应 $q_c \approx 98\%$。这与渗流的 $P_\infty \sim (p - p_c)^{\beta}$ 在形式上同构，指向同一物理机制：**网络功能是连通性的函数**。

与原书呼应：原书 §5.8 提到二维 site percolation 的临界指数 $\beta = 5/36$、$\nu = 4/3$ 及 Pesce 等（2026）剪枝临界比例约 98%，对应本节严格推导；渗流序参量连续上升（二阶相变）的说明对应原书「标准二维伯努利渗流是连续（二阶）相变」的表述。

---

§5.9 跨界映射与 §5.10 反类比：相变叙事的边界

▶这一节在讲什么

为什么「AI 涌现是相变」这个类比虽然有吸引力但存在根本性缺陷？物理相变有三个严格要求：热力学极限（$N \to \infty$）、普适临界指数、时间反演对称（可逆）——AI 涌现一个都不满足。

第一步：有限系统只有 crossover，没有相变。自由能密度 $f(T) = \lim_{N\to\infty} F(N,T)/N$ 的非解析性只在 $N \to \infty$ 时出现；有限 N 的 $f(N,T)$ 是有限个指数之和的对数，永远解析——无论网络多大。

第二步：后视镜叙事无法预测。历史上的文明「相变」是事后贴上的标签；事前不知道序参量 ϕ 的形式，就无法预测转变时刻 t^*。AI 涌现同理，「下一次涌现何时发生」没有任何理论依据。

第三步：不可逆性打破动力学同构。物理相变（一阶或二阶）沿相图均可逆；AI 能力涌现和文明转变均不满足时间反演对称——已习得的能力无法通过逆向训练消除。

本节要拿走的一件事：用「相变」描述 AI 涌现是一种生动但不严格的比喻；有限系统、后视镜叙事、不可逆性这三条界限，划定了这个类比能走多远。

直觉

原书 §5.9 列举了农业革命、城邦兴起、工业革命、互联网等历史拐点，将它们类比为文明的「相变」，并指出 AI 涌现被许多业界领袖以「phase transition」的语言包装。

§5.10 则给出三条反驳：物理相变要求热力学极限；文明「相变」是后视镜叙事，不是预测工具；物理相变（除特殊情形外）可逆，文明涌现不可逆。

这是一节「跨界映射」加「反类比」的节，没有新的数学概念，主要是类比的成立条件与边界的分析。规范建议把这类节改写为「为什么类比成立 / 不成立的 N 个数学根源」结构。下面从三个维度展开：类比的吸引力所在、三条根本性边界、以及这三条边界的不同力度。

类比为何有吸引力——结构相似的真实基础

「AI 涌现是相变」这个比喻之所以在业界广泛流传，不仅是修辞需要，而是有真实的结构相似性支撑。

首先，**序参量行为的形式相似**。物理相变中，序参量（如磁化强度、密度差）在临界点附近从零迅速上升，表现为急剧的但连续的转变。AI 涌现中，评估指标（在合适的连续指标下）也表现为随规模急剧上升的过渡区。两者的函数形式在视觉上高度相似。

其次，**临界现象的普遍性**。渗流、铁磁、液气临界点等完全不同的物理系统，在临界指数上表现出「普适类」的统一——微观细节不同，但宏观临界行为相同。这种普遍性令人相信：也许「从无到有」的转变存在某种更深层的数学机制，AI 涌现可能属于这个更广泛的现象族。

第三，**§5.8 的剪枝实验提供了独立支撑**。Pesce 等的工作直接在网络连通性层面观察到了渗流式的临界行为，这不是抽象类比，而是实验中测量到的函数形式。这说明「神经网络具有渗流式的结构」至少在剪枝这一特定操作下是成立的经验事实。

所以类比不是毫无根据的比喻——它有真实的结构相似性支撑。问题不在于「类比是否有任何道理」，而在于「类比走得有多远、在哪里失效」。

第一个边界：热力学极限——有限系统只能有 crossover

物理相变最硬的数学前提是**热力学极限**（thermodynamic limit）：$N \to \infty$。这不是技术细节，而是严格数学意义上的相变能否存在的根本条件。

对于有限大小的系统，自由能密度 $f(N,T)$ 是温度（控制参数）T 的解析函数。这是因为：有限系统的配分函数是有限个指数函数之和，$Z = \sum_{i=1}^{M(N)} e^{-\beta E_i}$，其中 $M(N)$ 是有限的态数。解析函数的有限和仍是解析函数——因此 $f(N,T) = -\frac{1}{\beta N} \ln Z$ 对 T 是解析的，没有任何奇异点（极点、分支点、本性奇点）。

非解析性——即严格意义上的「相变」——只在热力学极限 $f(T) = \lim_{N\to\infty} f(N,T)/N$ 下出现。这个极限可以非解析，因为无穷多个解析函数的极限可以是非解析的（经典例子：$f_n(x) = x^{2n}/(1+x^{2n})$ 在 $n \to \infty$ 时收敛到阶梯函数）。

对神经网络：即使是有 10^{12} 个参数的大模型，也是有限系统。式 (5.9.1) 表明，其「涌现」行为即使看起来再陡峭，在严格数学意义上也只是 **crossover**（平滑但陡峭的过渡），而非奇异点。这一限制不依赖参数量的具体数值，而是有限系统的普遍性质。

crossover 和相变的区别不只是字面上的：crossover 没有普适临界指数（陡峭程度依赖模型细节），换不同架构、不同任务会得到不同的「陡峭程度」，不能从统计物理的普适类理论中推导定量预测。这意味着把某个模型上观察到的「涌现阈值」当作普遍的物理常数是方法论错误——它只是特定系统的 crossover 参数，不具有普适性。

第二个边界：后视镜叙事——序参量的事后确认性

「文明相变」类比的方法论问题更微妙：它不是数学上的硬错误，而是预测有效性上的根本局限。

物理相变的预测能力来自于：**相变之前就知道序参量是什么**。铁磁相变发生之前，理论家已经知道磁化强度 M 是序参量；临界点 T_c 是从序参量的消失来定义的。知道序参量，就能计算临界温度，就能预测「当温度降到 T_c 以下时，磁化强度出现」。

文明「相变」完全不同：农业革命是事后才被历史学家识别为「文明的关键转折点」的。问：在农业革命之前，任何人能从「序参量」的观测来预测「转折点将在何时发生」吗？答案是否定的，因为当时甚至不知道应该观测什么量作为序参量。历史学家在事后选择了「定居密度」「食物储存量」等作为序参量来讲述故事，但这个选择是事后确认的，不是事前理论推导的。

AI 涌现的「相变叙事」同样如此：哪个能力是「真正的涌现」、在什么规模下出现、序参量是什么——这些都是在已经看到涌现之后才被定义的。Schaeffer 等的工作进一步说明，很多被定义为「涌现」的现象甚至只是评估指标的非线性伪影（§5.5）。在此背景下，「下一次涌现何时发生、会是什么能力」没有任何物理相变理论意义上的预测依据——不是因为我们的技术不够，而是因为「涌现」概念本身的操作化定义是事后的。

第三个边界：不可逆性——动力学对称性的缺失

物理相变（一阶和二阶）都可以沿相图逆向路径实现：加热铁磁体使磁化消失，再冷却使磁化重新出现；降低气体温度使液体凝固，再升温使其重新气化。这不是技术上可以做到，而是热力学的基本对称性：路径可逆，系统可以在两相之间来回穿越（一阶相变有滞后，但依然可逆）。

AI 能力涌现和文明转变均**不满足时间反演对称性**：

- AI 语境：一旦模型在 Grokking 中「顿悟」了傅里叶电路，逆向训练（减小正则化）不会让模型「忘记」泛化能力并重新陷入纯记忆阶段。习得的能力无法通过简单的参数操作「撤销」。
- 文明语境：人类社会不能「撤销」农业革命并重回游牧状态——即使在极端情境下（文明崩溃），重建的路径也与原始出现的路径完全不同。

这一不可逆性意味着：AI 涌现和文明转变的**动力学机制**与物理相变不同构。物理相变的可逆性来自哈密顿量的时间反演对称，或自由能的极值结构。AI 能力的习得和文明的演化没有等价的对称性——它们是耗散系统中的不可逆过程，而非保守系统中的可逆相变。

从动力学角度，物理相变与 AI/文明涌现的区别不亚于序参量行为的相似性——甚至可以说，两者在外观上的相似（序参量的急剧上升）掩盖了它们在机制上的本质差异（可逆 vs 不可逆）。

三条边界的力度比较

这三条反驳的力度并不相同，值得区分：

- **热力学极限**是最硬的边界：解析函数有限和不可能非解析，这是纯粹的数学定理，无任何例外。任何有限神经网络的「涌现」，在数学意义上必然是 crossover。
- **后视镜叙事**的力度中等：它指出的是方法论问题，不是说「历史上没有发生真实的结构性转变」，而是说「我们在事前无法精确定位这些转变，因为序参量的定

义是事后的」。这是认识论层面的限制，而非本体论层面的否定。

- **不可逆性**的力度相对最弱（从「否定类比」的角度）：不可逆性确实打破了动力学同构，但不可逆的临界现象在物理中也存在（如一些非平衡相变），所以「不可逆」本身不足以完全否定「相变」框架，只是说明需要用非平衡统计力学的框架来类比，而不是平衡态相变的框架。

整条链的回头看

```
类比的吸引力（结构相似的真实基础）
    序参量行为形式相似（连续急剧上升）
    普适类现象的广泛性（渗流、铁磁、液气）
    剪枝实验的独立支撑（Pesce 等）

边界一：热力学极限（最硬，数学定理）
  有限系统 → f(N,T) 解析 → 只能 crossover，没有奇异点
  → 无普适临界指数，涌现阈值不具有架构无关性

边界二：后视镜叙事（方法论限制）
  序参量事后确认 → 无法事前预测涌现时刻
  → 「AI 相变」无法用于预测，只能用于事后叙事

边界三：不可逆性（动力学非同构）
  AI 能力习得不可逆 → 动力学对称性缺失
  → 平衡态相变框架不适用，需非平衡框架（若类比要继续）
```

这三条边界划定了「相变叙事」能走多远：**结构性的定性类比有价值（提供直觉框架）；用临界指数做定量预测需谨慎（热力学极限不满足）；用它来预测未来涌现无效（后视镜叙事）；用它作为理解机制的精确模型不够（动力学不同构）**。这不是要彻底放弃相变语言，而是要在使用它时清楚地标注适用范围——这正是严谨科学思维与生动但过度延伸的比喻之间的分界线。

严格

热力学极限的必要性。设系统有 N 个粒子（或参数）。自由能密度为 $f(N,T) = F(N,T)/N$，其中 T 为温度（或某控制参数）。严格意义上的相变（即 f 在某 T_c 处的非解析性）仅在热力学极限下存在：

$$f(T) = \lim_{N\to\infty} \frac{F(N,T)}{N} \tag{5.9.1}$$

对有限 N，$f(N,T)$ 是有限个指数函数的对数之和，解析函数的有限和仍是解析函数，因此严格奇异点不存在。有限系统观察到的只是平滑的**交叉（crossover）**行为，而非奇异点。

大语言模型即使有 10^{12} 个参数，仍是有限系统；其「涌现」行为即使在某些指标下看似陡峭，在严格数学意义上也只是 crossover，而非相变。这一区别不仅是字面上的：crossover 没有普适临界指数，其「陡峭程度」依赖模型细节，无法从统计物理的普适类理论中推导出任何定量预测。

后视镜叙事的不可预测性。设文明状态变量为 $\mathbf{x}(t) \in \mathbb{R}^d$，「相变时刻」为 t^* 使得某序参量 $\phi(\mathbf{x}(t))$ 在 t^* 附近急剧变化。历史学家能在 $t > t^*$ 时用 ϕ 描述转变，但事前预测 t^* 需要知道 ϕ 的具体形式——而这在转变发生前往往未知。AI 语境中，「下一次涌现何时发生」同样无法从现有理论推导，Schaeffer 等的工作进一步表明，若涌现是指标伪影，则「预测涌现」更无理论基础。

不可逆性。一阶相变（如水的液固转变）满足 Clausius-Clapeyron 方程，可沿相图逆向路径恢复原相；二阶相变（如铁磁相变）在翻转控制参数后同样可逆。文明相变（如工业化）和 AI 能力涌现均不满足时间反演对称性——已习得的能力无法通过逆向训练消除。这一不可逆性意味着，物理相变与 AI/文明涌现之间的类比缺乏动力学层面的同构，只是在序参量行为的表观形式上相似。

特别说明：有限 N 系统只能有 crossover。以上分析的核心结论值得单独强调：热力学极限 $N \to \infty$ 是严格相变的必要条件，任何有限参数量的神经网络，无论其涌现行为看起来多么陡峭，数学上均为 crossover，而非奇异点。这一限制不依赖于模型大小的具体数值，而是有限系统的普遍性质。

与原书呼应：原书 §5.10 明确指出「热力学极限是严格相变的必要条件，有限 N 系统只能有 crossover」，对应本节自由能密度对 N 极限的分析；「物理相变背后有严格统计力学理论；AI 涌现背后只有经验观察」，对应本节后视镜叙事与不可逆性的讨论。

本章对应原书第五章「量变引发的智能飞跃——统计物理与复杂系统涌现」（§5.1–§5.10）。主要参考文献：Kaplan et al. (2020)；Hoffmann et al. (2022)；Epoch AI (2024)；Sardana et al. (ICML 2024)；Schaeffer, Miranda & Koyejo (NeurIPS 2023)；Nanda et al. (ICLR 2023)；Power et al. (2022)；Pesce, He & Caldarelli (2026)；Shumailov et al. (Nature 2024)。

数学卷 · 第 6 章: 硬件感知算法的数学——屋脊线（Roofline）、I/O 复杂度与 FlashAttention 推导

导读：主本第六章以产业链视角讲述为什么 HBM 已成为万亿美元生意；本章在此基础上，逐一补全那些被主本略去的不等式推导。读者需要掌握矩阵分析、基础算法复杂度与变换器（Transformer）结构的预备知识（可参照数学卷第三章 §3.5 关于 FlashAttention 直觉版的铺垫）。本章的核心线索是：**带宽是常数，算力在增长，因此算法必须在数学层面不断”提升算术强度”，才能不被内存墙钉死**。

---

§6.1 Roofline 模型的形式化

▶这一节在讲什么

为什么一个 kernel 跑得慢，根本原因可能不是代码写得差？硬件有两个独立的天花板：算力和带宽。Roofline 模型给你一把尺子，精确测量你的 kernel 撞上的是哪一面墙。

第一步：定义算术强度 I。每搬运 1 字节数据，kernel 完成多少次浮点运算？这个比值就是算术强度。I 低说明数据搬得多、算得少——内存墙是瓶颈；I 高说明数据复用充分——算力才是天花板。

第二步：两个上界取 min。内存受限时性能上界是 $I \cdot \beta$（带宽乘以算术强度）；计算受限时上界是 P_{peak}（硬件峰值算力）。两者相交的脊点 $I^* = P_{\text{peak}}/\beta$ 是分水岭。

第三步：看 GEMM 的两个极端。大矩阵乘的 $I \to \infty$（计算受限）；矩阵-向量乘的 $I \approx 1$（深度内存受限）。H100 的 $I^* \approx 295$，推理解码阶段与脊点相差 295 倍——这就是 HBM 带宽决定推理速度的数学根源。

本节要拿走的一件事：脊点 $I^* = P_{\text{peak}}/\beta$ 是判断 kernel 瓶颈的分界线；低于它的所有优化，只有提升算术强度才有根本效果，调 CUDA 核心利用率是徒劳的。

直觉

原书以「为什么 GPU 上的某个 kernel 跑得慢」为引子，指出根本原因未必是代码写得差，而是 kernel 的「数据搬运密度」与硬件的「带宽/算力配比」不匹配。这里的关键问题是：用什么数学量把这个「配比」精确化，以及当配比失衡时，如何判断从哪个方向优化才有效？

本节出现四个概念：**算术强度** I、**性能上界**、**脊点** I^*、**GEMM 的两个极端**。它们看似散落，实际上沿着一条严密的逻辑链一个推出一个。理解这条链，是理解「为什么 HBM 带宽决定大模型推理速度」的钥匙。下面把这条链一环环铺开。

第一环：算术强度——给「计算密度」一个绝对标尺

任何计算任务都需要两种资源：浮点运算单元（算力）和内存带宽（数据搬运）。一个 kernel 究竟「偏向计算还是偏向搬数据」，取决于它在完成计算的过程中，每搬运一个字节的数据，平均能完成多少次浮点运算——这就是算术强度：

$$I = \frac{W}{Q} \quad [\text{FLOP/byte}] \tag{6.1.1}$$

其中 W 是 kernel 的浮点运算总量，Q 是从最慢存储层级（现代 GPU 即 HBM）搬运的字节总量。

式 (6.1.1) 定义的算术强度，为什么是 FLOP 除以 byte，而不是别的比值？直觉是：硬件「同时」在做两件事——运算单元在执行 FLOP，内存控制器在搬运 byte。如果把 W 和 Q 除以时间，就得到「当前有效算力」和「当前带宽利用率」；两者之比就是「算力单元消耗数据的速率」对比「内存能喂数据的速率」。I 大说明算力单元能从每字节数据中榨取更多运算，数据被充分复用；I 小说明每次运算都要等待内存搬新数据，算力单元大量空转。

注意 Q 的定义层级至关重要：如果选取 L2 cache 或 shared memory 而非 HBM 作为「最慢层」，就会得到不同（更高）的算术强度，称为「层次化 Roofline」。本章专注 HBM 层级，因为 HBM 带宽是制约大模型训练和推理的主要瓶颈，且与主本第六章的产业链叙事对应。

第二环：性能上界——两个资源约束取最紧的那个

有了算术强度 I，就可以推导 kernel 的性能上界。硬件同时受两个资源约束：

- **带宽约束**：HBM 每秒最多搬运 β 字节，kernel 的算术强度是 I，所以每秒最多完成 $I \cdot \beta$ 次浮点运算——这是内存墙给出的上界。
- **算力约束**：硬件浮点单元每秒最多完成 P_{peak} 次运算——这是算力墙给出的上界。

两个约束同时成立，取更紧的那个：

$$P \le \min\left(P_{\text{peak}},\ I \cdot \beta\right) \tag{6.1.2}$$

式 (6.1.2) 是一个严格的上界，而非近似。物理含义是：无论 kernel 的内部结构多复杂，它的可达性能不可能超过「内存能喂的量」和「算力单元能算的量」这两者之中更小的那个。

这式 (6.1.2) 在 Roofline 图上的形状很直观：横轴是算术强度 I（对数坐标），纵轴是性能（GFLOPS）。$I \cdot \beta$ 是一条从左到右斜向上的直线，P_{peak} 是一条水平线。两条线的交点就是脊点（ridge point）——在这个点以下，内存是瓶颈；在这个点以上，算力是天花板。「Roofline」（屋脊线）这个名字正是来自这个形状：就像一个屋顶，由两段屋面在屋脊处相交。

第三环：脊点 I^*——两面墙的分水岭

两个上界相等的临界点就是脊点，由 $I^* \cdot \beta = P_{\text{peak}}$ 解得：

$$\boxed{I^* = \frac{P_{\text{peak}}}{\beta}} \tag{6.1.3}$$

式 (6.1.3) 的脊点物理含义是：「让算力单元和内存带宽刚好同时跑满」所需要的算术强度。

- 当 $I < I^*$：内存带宽先耗尽，kernel 处于内存受限（memory-bound）状态。此时无论如何提升计算单元利用率，式 (6.1.2) 的性能上界由 $I \cdot \beta$ 决定，调 CUDA 核心利用率毫无意义——瓶颈根本不在那里。**唯一有效的优化方向是提升 I**（更多数据复用、tiling、算子融合）。
- 当 $I \ge I^*$：算力先耗尽，kernel 处于计算受限（compute-bound）状态。此时 HBM 带宽有富余，优化方向是提升算力利用率（减少分支、提高 occupancy、使用 Tensor Core）。

H100 SXM5 的具体数字：$P_{\text{peak}} = 989$ TFLOPS（TF32 dense），$\beta = 3.35$ TB/s，故

$$I^*_{\text{H100}} = \frac{989 \times 10^{12}}{3.35 \times 10^{12}} \approx 295 \text{ FLOP/byte} \tag{6.1.4}$$

式 (6.1.4) 这个数字意味着：在 H100 上，一个 kernel 至少需要每搬运 1 字节就完成 295

次浮点运算，才有可能触及算力天花板。低于这个门槛的 kernel，无论代码写得多精巧，都被内存墙钉死。

第四环：GEMM 的两个极端——Roofline 模型最重要的测试案例

矩阵乘（GEMM）是深度学习中最关键的算子，它在 Roofline 模型上提供了两个极为鲜明的对照案例，完美诠释了算术强度的意义。

考虑 $C = AB$，$A \in \mathbb{R}^{m\times k}$，$B \in \mathbb{R}^{k\times n}$。浮点运算量为 $W = 2mnk$，数据搬运量为 $Q = (mk + kn + mn) \times \text{sizeof(element)}$，故

$$I_{\text{GEMM}} = \frac{2mnk}{mk + kn + mn} \tag{6.1.5}$$

大矩阵极限：令 $m = n = k \to \infty$，由式 (6.1.5) 分子 $\sim 2n^3$，分母 $\sim 3n^2$，故 $I_{\text{GEMM}} \to \frac{2n}{3} \to \infty$。大方阵乘是计算受限的——矩阵越大，每个元素被读入后参与的运算越多（A 的每行要与 B 的所有列做内积），数据复用率趋于无穷，完全不受内存墙威胁。这正是为什么 GPU 训练大模型时，前向传播的 GEMM 能高效利用 Tensor Core。

小 batch 极限：令 $m = n = 1$（矩阵-向量乘，即 batch size = 1 的线性层），则

$$I_{\text{GEMM}}\big|_{m=n=1} = \frac{2k}{k + k + 1} = \frac{2k}{2k + 1} \xrightarrow{k\to\infty} 1 \text{ FLOP/byte} \tag{6.1.6}$$

式 (6.1.6) 与 H100 的脊点（式 (6.1.4)，约 295 FLOP/byte）相差整整 295 倍。换句话说，推理解码阶段（每步生成 1 个 token，batch size = 1），所有线性层的算术强度约为 1，深度陷入内存受限区——哪怕 H100 的 Tensor Core 完全空转，带宽已经被榨干了。

这两个极端给出了一个关键的工程启示：**提升 batch size 本质上是在把矩阵-向量乘（$m = 1$）变回矩阵乘（$m > 1$）**，从而线性提升式 (6.1.5) 中的算术强度，把 kernel 从内存受限区推向计算受限区。这就是为什么大模型推理服务（如 vLLM、TensorRT-LLM）对 batching 策略如此看重——它不仅是吞吐量问题，更是在做「算术强度管理」。

整条链的回头看

```
计算任务有两种资源消耗：FLOP 和 byte
  ↓ 取比值定义「每字节对应多少运算」
算术强度 I = W/Q                    ←  kernel「计算密度」的绝对标尺
  ↓ 两种资源各有速率上限
性能上界 P   min(P_peak, I · )      ←  内存墙和算力墙，取更紧的那个
  ↓ 令两个上界相等，解出临界点
脊点 I* = P_peak /                  ←  两面墙的分水岭；H100 上   295 FLOP/byte
  ↓ 对大模型核心算子 GEMM 代入两个极端
大矩阵 GEMM：I → ∞（计算受限）
矩阵-向量乘：I   1（深度内存受限，与脊点差 295×）
```

这条链揭示了一个深层事实：**大模型推理速度的天花板不在 Tensor Core，而在 HBM 带宽**。在 batch size = 1 的自回归解码场景下，由式 (6.1.6) 算术强度约为 1，距离式 (6.1.3) 的脊点差近三百倍——无论算力硬件如何升级，只要 batch size 不变，瓶颈永远是内存带宽。脊点 I^* 不是一个数字，而是一面镜子：它照出的是「算法应该往哪个方向努力」。

严格

6.1.1 算术强度的精确定义

设某计算核（kernel）执行 W 次浮点运算，在此过程中从最慢存储层级（对现代 GPU 即 HBM）读写共计 Q 字节数据。定义**算术强度**（arithmetic intensity）为

$$I = \frac{W}{Q} \quad [\text{FLOP/byte}]. \tag{6.1.7}$$

Williams、Waterman 与 Patterson 在 2009 年的原始论文中使用术语「operational intensity」，本章与数学卷其余各章统一改用更通行的「算术强度」，两者同义。

若存储层级选取不同（如 L2 而非 HBM），则得到**层次化 Roofline**（hierarchical Roofline），但本章专注于 HBM 层级，以与主本第六章的产业链叙事对应。

6.1.2 性能上界与 Roofline 公式

设硬件的浮点峰值算力为 P_{peak}（FLOP/s），HBM 带宽为 β（byte/s）。对一个算术强度为 I 的 kernel，其可达性能上界为

$$P \le \min\bigl(P_{\text{peak}},\ I \cdot \beta\bigr). \tag{6.1.8}$$

直觉：若 kernel 是**内存受限**（memory-bound），则每秒可喂入算力单元的数据量为 β byte/s，携带算力仅 $I \cdot \beta$ FLOP/s；若 kernel 是**计算受限**（compute-bound），则上限即为 P_{peak}。两者取 min 给出严格上界。

6.1.3 脊点（Ridge Point）的推导

两个上界相交的算术强度称为**脊点** I^{*}，由

$$I^{*} \cdot \beta = P_{\text{peak}} \tag{6.1.9}$$

解得

$$\boxed{I^{*} = \frac{P_{\text{peak}}}{\beta}.} \tag{6.1.10}$$

当 $I < I^{*}$ 时 kernel 内存受限；当 $I \ge I^{*}$ 时计算受限。

H100 SXM5 的具体数值：依据 NVIDIA 官方规格，H100 的 FP16 张量核（Tensor Core）峰值算力 $P_{\text{peak}} = 989$ TFLOPS（TF32，dense）或 1,979 TFLOPS（FP16，dense）。主本与本章使用 TF32/FP16 混合精度训练场景中最常被引用的 989 TFLOPS 作为代表值，HBM3 带宽 $\beta = 3.35$ TB/s，因此

$$I^{*}_{\text{H100}} = \frac{989 \times 10^{12}\ \text{FLOP/s}}{3.35 \times 10^{12}\ \text{byte/s}} \approx 295\ \text{FLOP/byte}. \tag{6.1.11}$$

这意味着：一个算术强度低于 295 FLOP/byte 的 kernel 在 H100 上永远是内存受限的，无论如何优化 CUDA 核心的利用率都无济于事——**瓶颈在带宽，不在算力**。

6.1.4 GEMM 的算术强度分析

考虑矩阵乘 $C = AB$，其中 $A \in \mathbb{R}^{m\times k}$，$B \in \mathbb{R}^{k\times n}$，$C \in \mathbb{R}^{m\times n}$。

浮点运算数：每个输出元素 C_{ij} 需做 k 次乘加，共 $2mnk$ FLOP。

数据搬运量（假设矩阵在 HBM 中各读一次、C 写一次）：

$$Q_{\text{GEMM}} = (mk + kn + mn) \times \text{sizeof(element)}. \tag{6.1.12}$$

忽略数据精度系数，定义算术强度为

$$I_{\text{GEMM}} = \frac{2mnk}{mk + kn + mn}. \tag{6.1.13}$$

大 batch 极限（$m, n, k \to \infty$ 且同阶）：分子 $\sim 2n^3$，分母 $\sim 3n^2$，故 $I_{\text{GEMM}} \to \frac{2n}{3} \to \infty$——大矩阵乘是计算受限的，不受内存墙威胁。

小 batch 极限（$m = n = 1$，即矩阵-向量乘）：

$$I_{\text{GEMM}}\big|_{m=n=1} = \frac{2k}{k + k + 1} = \frac{2k}{2k+1} \xrightarrow[k\to\infty]{} 1 \approx 1 \text{ FLOP/byte (FP32)}. \tag{6.1.14}$$

若采用 FP16（每元素 2 byte），则

$$I_{\text{GEMM}}\big|_{m=n=1,\,\text{FP16}} \approx \frac{2k}{2(2k+1)} \to 1 \text{ FLOP/byte}. \tag{6.1.15}$$

这一数值与 H100 脊点 295 相差 **295 倍**——LLM 推理中的 token-by-token（batch size = 1）解码阶段，矩阵乘完全内存受限，这正是 HBM 带宽对 LLM 推理速度如此关键的根本原因。

---

§6.2 I/O 复杂度的下界（Hong-Kung 1981）

▶这一节在讲什么

为什么需要一个关于「搬数据次数」的下界定理？光优化算法是不够的——你需要知道最优算法的极限在哪里，才能判断自己的实现还有多少空间。Hong 和 Kung 1981 年的定理给出了这个极限的数学表达式。

第一步：把算法变成图，把快速存储变成筹码。计算依赖关系用有向无环图（DAG）表示，快速存储（SRAM）大小为 S 颗红色卵石的容量。「红蓝卵石游戏」把「搬数据」这个物理动作变成了可以严格计数的博弈规则。

第二步：S-划分论证。将整个计算过程按 I/O 次数切分为若干阶段，每阶段至多 S 次 I/O，因此每阶段的「外部边界」至多 $2S$ 个值。这个约束限制了每阶段能计算的节点数，进而推出总 I/O 下界。

第三步：对矩阵乘用 AM-GM 不等式。快速存储中同时持有 N_A 个 A 元素、N_B 个 B 元素、N_C 个 C 元素时，可完成的乘加数至多 $\sqrt{N_A N_B N_C} \le (2S/3)^{3/2}$，由此导出 $T_{I/O} = \Omega(n^3/\sqrt{M})$。

本节要拿走的一件事：$n \times n$ 矩阵乘的 I/O 下界是 $\Omega(n^3/\sqrt{M})$，且分块 GEMM 恰好达到此界——这是算法最优性的信息论证明，而非工程猜测。

直觉

§6.1 的 Roofline 模型给出了「给定算术强度，性能上界是多少」，但它没有回答另一个更根本的问题：**对于一个给定的计算任务，算术强度最高能到多少？**换句话说，最聪明的算法究竟能把 I/O 压缩到多少，有没有理论下界？Hong-Kung 1981 年的工作正是为了回答这个问题——它给出了「在快速存储大小为 M 时，任何正确算法都无法突破的 I/O 下界」。

本节出现五个概念：**两级存储模型**、**红蓝卵石游戏**、**S-划分**、$H(2S, G)$（**S-碎石数**）、**AM-GM 不等式在矩阵乘上的应用**。它们沿着一条严密的逻辑链层层推进：用游戏规则把「搬数据」形式化，用 S-划分把连续计算切段，用 S-碎石数限制每段的计算量，最后用 AM-GM 对矩阵乘结构做精确估计，完成下界证明。

第一环：两级存储模型——把复杂硬件层级压缩成最简的数学模型

现代 GPU 的存储层级很复杂：寄存器、L1 cache、shared memory、L2 cache、HBM，各级速度和容量相差数倍到数量级。Hong-Kung 定理的第一步是把这个复杂层级**抽象为一快一慢两级**：

- **快速存储**（fast memory）：容量有限，大小为 S 个字，对应 GPU 的 SRAM/shared memory。访问延迟可忽略不计。
- **慢速存储**（slow memory）：容量无限，对应 HBM。每次在快慢之间搬移一个字，计 1 次 I/O。

这个抽象的代价是丢掉了多级缓存的细节（这个缺陷在 §6.6 会专门讨论）；它的收益是把问题变成了可以严格用组合论证分析的数学对象。核心问题变成了：给定快速存储大小 S，完成某计算任务**至少**需要多少次 I/O？

注意这个问题的特殊性：它问的是**下界**，即对所有可能的算法成立。证明下界比证明

上界难得多——证明上界只需给出一个具体算法，证明下界需要对**任意**算法都成立的论证。

第二环：红蓝卵石游戏——把「搬数据」变成可以严格计数的博弈

为了能对「任意算法的 I/O 次数」做数学论证，Hong 和 Kung 引入了一个精妙的形式化工具：**红蓝卵石游戏（Red-Blue Pebble Game）**。

将任意算法的数据依赖关系表示为有向无环图（DAG）：每个节点代表一个需要计算的中间值，每条边代表「节点 B 依赖节点 A 的结果」。然后用两种颜色的卵石来表示数据的位置：

- **红色卵石**：「这个值当前在快速存储（SRAM）里」，最多同时有 S 颗（容量限制）。
- **蓝色卵石**：「这个值已被持久化到慢速存储（HBM）里」。

博弈的规则对应真实的硬件操作：1. **计算**：若某节点的所有前驱都有红色卵石，可在该节点放红色卵石——等于「可以执行这步计算」。2. **读（HBM→SRAM）**：把一个蓝卵石换成红卵石，计 1 次 I/O——等于「从 HBM 载入数据到 SRAM」。3. **写（SRAM→HBM）**：把一个红卵石换成蓝卵石，计 1 次 I/O——等于「把 SRAM 中的中间结果写回 HBM」。4. **驱逐**：无代价地移除红卵石——等于「SRAM 容量不足时丢弃数据（若后续还需要则重新读入）」。

游戏的目标是让所有输出节点都获得红卵石（完成计算），同时最小化 I/O 总次数。这样，「最优算法的 I/O 次数」就等价于「红蓝卵石游戏的最优策略的 I/O 次数」——一个精确的组合优化问题。

第三环：S-划分——把连续的博弈切成可控的段落

红蓝卵石游戏已经把 I/O 变成了可以计数的量。下一步是建立下界——证明对**任意**策略，I/O 总次数都不能低于某个值。核心技巧是 **S-划分（S-partition）**，又称「阶段分解」。

把整个博弈过程按 I/O 操作均匀切分为若干**阶段（phases）**，每个阶段包含**至多 S 次 I/O**。每个阶段的「入口」：快速存储里至多有 S 个值（容量限制）；「出口」：本阶段最多发生 S 次 I/O，带入或带出至多 S 个值。因此每个阶段的「边界」（进入 + 离开快速存储的值的集合）**至多有 $2S$ 个值**。

现在关键问题是：在「外部输入至多 $2S$ 个」的约束下，一个阶段内部**最多**能覆盖（计算）多少个 DAG 节点？把这个最大节点数记作 $H(2S, G)$，称为「S-碎石数」（DAG 的 $2S$-覆盖复杂度）。

由此立刻得到一个优美的下界论证：- 总阶段数为 $\lceil T_{I/O}/S \rceil$（因为每阶段至多 S 次 I/O）- 每阶段至多覆盖 $H(2S, G)$ 个节点 - 所有阶段加起来必须覆盖全部 $|V|$ 个计算节点

三者之积/商给出引理：

$$\left\lceil \frac{T_{I/O}}{S} \right\rceil \cdot H(2S, G) \;\geq\; |V| - |\text{Inputs}(G)| \tag{6.2.1}$$

整理得式 (6.2.1) 的等价形式 $T_{I/O} \geq S \cdot \frac{|V| - |\text{Inputs}(G)|}{H(2S,G)}$。这是一个对**任意**正确算法（任意卵石策略）都成立的不等式，是完全严格的。

第四环：AM-GM 对矩阵乘的精确估计——把抽象下界变成具体数字

对矩阵乘 $C = AB$（$n \times n$），DAG 有 $|V| = n^3$ 个内部节点（每个节点对应一次部分乘积 $A_{ik} \cdot B_{kj}$），输入节点为 A, B 的 $2n^2$ 个元素。代入引理，只需估计 $H(2S, G)$。

关键步骤：快速存储中驻留 N_A 个 A 的元素、N_B 个 B 的元素、N_C 个 C 的部分和。能够计算的部分乘积数（即 FMA 数）至多是多少？一个 FMA $A_{ik} \cdot B_{kj}$ 要求 A_{ik}、B_{kj}、C_{ij} 三个元素**同时**在快速存储中——否则无法执行这一步计算。三个集合的「有效交叉」最大化时，可完成的 FMA 数至多是

$$\sqrt{N_A \cdot N_B \cdot N_C} \tag{6.2.2}$$

（这一估计的严格证明依赖组合几何论证，Irony、Toledo 和 Tiskin 2004 年给出了精确版本。）

现在应用 **AM-GM 不等式**：三个非负数之积的立方根不超过它们的算术平均值，因此在 $N_A + N_B + N_C \le 2S$ 的约束下（其中 $\sqrt{N_A N_B N_C}$ 即式 (6.2.2) 中的 FMA 上界）：

$$\sqrt{N_A N_B N_C} \le \left(\frac{N_A + N_B + N_C}{3}\right)^{3/2} \le \left(\frac{2S}{3}\right)^{3/2} = O(S^{3/2}) \tag{6.2.3}$$

由式 (6.2.3) 知 $H(2S, G) = O(S^{3/2})$。代入引理 6.1（式 (6.2.1)）：

$$T_{I/O} \;\ge\; S \cdot \frac{n^3 - 2n^2}{O(S^{3/2})} = \Omega\left(\frac{n^3}{S^{1/2}}\right) = \Omega\left(\frac{n^3}{\sqrt{M}}\right) \tag{6.2.4}$$

其中最后一步用 M 表示快速存储大小（字数）。

第五环：下界的可达性——分块矩阵乘恰好达到此界

证明下界后，还需要验证这个下界是否「紧」（tight）——即是否存在一个算法恰好以 $\Theta(n^3/\sqrt{M})$ 的 I/O 完成矩阵乘。答案是肯定的：**分块矩阵乘（block GEMM）**以块大小 $b = \lfloor\sqrt{M/3}\rfloor$ 运行时，I/O 次数恰好是 $\Theta(n^3/\sqrt{M})$。

这个可达性结论的含义极为深刻：分块矩阵乘不只是「工程上调出来的一个好实现」，而是在**信息论意义上已达最优**——式 (6.2.4) 的下界已由分块 GEMM 精确达到，没有任何其他数据布局或循环调度方案能在渐近意义上比它 I/O 更少。cuBLAS、cuDNN 等库中的 GEMM 实现，其分块结构从根本上说就是在实现这个最优策略。

整条链的回头看

```
快慢两级存储：大小为 S 的 SRAM + 无限 HBM
  ↓ 用 DAG 形式化算法，用卵石颜色表示数据位置
红蓝卵石游戏                          ←  把「搬数据」变成可严格计数的博弈
  ↓ 按 I/O 次数将博弈切成每段 S 次 I/O 的阶段
S-划分论证                            ←  每段边界 2S 个值，限制每段的计算量
  ↓ 定义每段「外部输入 2S 时最多覆盖多少节点」
S-碎石数 H(2S, G)                     ←  阶段计算量的上界
  ↓ 对矩阵乘的 DAG 结构，用 AM-GM 估计 H(2S,G)
H(2S, G) = O(S^{3/2})                 ←  AM-GM：三堆元素之积  (总和/3)^{3/2}
```

```
  ↓ 代入引理，整理
T_IO   Ω(n³/√M)                          ←  对任意正确算法成立的 I/O 下界
  ↓ 分块 GEMM 以 b=√(M/3) 达到 Θ(n³/√M)
下界是紧的                                ←  分块 GEMM 是渐近最优算法
```

这条链揭示了一个深层事实：**算法优化不是无限的**。矩阵乘的 I/O 下界 $\Omega(n^3/\sqrt{M})$ 是由快速存储大小 M 和计算量 n^3 在数学上唯一确定的——无论你多聪明地排列循环顺序或设计数据布局，都不可能突破它。分块 GEMM 已经在渐近意义上达到了这个极限，这不是工程猜测，而是有严格证明的最优性。

严格

6.2.1 计算模型与红蓝卵石游戏

两级存储模型：系统有容量无限的**慢速存储**（对应 HBM）和容量为 S 个字的**快速存储**（对应 SRAM/shared memory）。在任意时刻，快速存储中最多驻留 S 个值。一次 I/O 操作将一个值在两级存储之间搬移。

红蓝卵石游戏（Red-Blue Pebble Game）：将算法表示为有向无环图（DAG）$G = (V, E)$，节点对应计算值，边对应数据依赖。两种卵石的语义如下：

- **红色卵石**：该值当前在快速存储（SRAM）中；最多同时存在 S 颗。
- **蓝色卵石**：该值在慢速存储（HBM）中持久化。

博弈规则：1. **计算**：若某节点的所有前驱节点均有红色卵石，可在该节点放置红色卵石（计算一步）。2. **读（HBM→SRAM）**：将蓝色卵石替换为红色卵石，计 1 次 I/O。3. **写（SRAM→HBM）**：将红色卵石替换为蓝色卵石，计 1 次 I/O。4. **驱逐**：若快速存储已满，可无代价地移除某红色卵石（丢弃，若以后需要则须重新读入）。

目标：对 DAG 的所有输出节点放置红色卵石，同时使 I/O 操作总数 $T_{I/O}$ 最小。

6.2.2 关键引理：S-划分与路径数

定义（$H(S,G)$——S-碎石数）：在 DAG G 中，使用 S 颗红色卵石可以覆盖的最大节点数称为 $H(S,G)$——即红蓝游戏的「路径复杂度」。更形式化地，$H(2S,G)$ 是 DAG 的**2S-块流量（2S-dominator set size）**：每个「流」至多穿过 $2S$ 个节点。

引理 6.1（Hong-Kung 基本不等式）：设 DAG G 有 $|V|$ 个非输入节点，输入节点数为 $|\text{Inputs}(G)|$。则在 S 个红色卵石约束下，任意正确博弈策略满足

$$\left\lceil \frac{T_{I/O}}{S} \right\rceil \cdot H(2S,\, G) \;\geq\; |V| - |\text{Inputs}(G)|. \tag{6.2.5}$$

证明梗概（S-划分论证）：

将整个博弈过程按 I/O 操作划分为**阶段（phases）**，每个阶段包含至多 S 次 I/O。在阶段 t 开始与结束时，快速存储至多各含 S 个值，故阶段 t 的「边界」上至多有 $2S$ 个值（进入 + 离开）。

在阶段 t 内，所有被计算的节点形成一个子图；其所有外部输入（来自 HBM 读入或上一阶段留存）至多 $2S$ 个。由 $H(2S,G)$ 的定义，该子图中最多包含 $H(2S,G)$ 个节点。

阶段总数为 $\lceil T_{I/O}/S \rceil$，故

$$(\text{阶段数}) \times H(2S, G) \geq |V| - |\text{Inputs}(G)|, \tag{6.2.6}$$

即

$$\left\lceil \frac{T_{I/O}}{S} \right\rceil \geq \frac{|V| - |\text{Inputs}(G)|}{H(2S, G)}. \tag{6.2.7}$$

将两边乘以 S 即得引理。□

6.2.3 矩阵乘的 I/O 下界

定理 6.2（矩阵乘 I/O 下界）：对 $n \times n$ 矩阵乘 $C = AB$，在快速存储大小为 M 的两级存储模型下，任意正确算法的 I/O 复杂度满足

$$T_{I/O} = \Omega\left(\frac{n^3}{\sqrt{M}} \right). \tag{6.2.8}$$

证明梗概：

矩阵乘的标准算法可由以下 DAG 表示：节点集为 $\{(i, j, k) : 1 \leq i, j, k \leq n\}$（即 n^3 个部分乘积节点），共 $|V| = n^3$ 个内部节点（忽略 n^2 个累加节点的低阶项），输入节点为 A 与 B 的 $2n^2$ 个元素。

关键一步 · 估计 $H(2S, G)$：若快速存储中驻留 N_A 个 A 的元素、N_B 个 B 的元素、N_C 个 C 的部分和，则在这批元素上能完成的 FMA 数至多为 $\sqrt{N_A N_B N_C}$（此为 Irony、Toledo 与 Tiskin 于 2004 年给出的精确估计，Hong-Kung 原文以略松的论证给出相同渐近结果）。**由 AM-GM 不等式**，

$$N_A + N_B + N_C \leq 2S \implies \sqrt{N_A N_B N_C} \leq \left(\frac{2S}{3} \right)^{3/2} = O(S^{3/2}). \tag{6.2.9}$$

由式 (6.2.9) 知 $H(2S, G) = O(S^{3/2})$。代入引理 6.1（式 (6.2.5)）：

$$T_{I/O} \geq S \cdot \frac{n^3 - 2n^2}{H(2S, G)} = S \cdot \frac{\Theta(n^3)}{O(S^{3/2})} = \Omega\left(\frac{n^3}{S^{1/2}} \right) = \Omega\left(\frac{n^3}{\sqrt{M}} \right). \quad \square \tag{6.2.10}$$

可达性：分块矩阵乘（block size $b = \lfloor \sqrt{M/3} \rfloor$）实现 $\Theta(n^3/\sqrt{M})$ 次 I/O，与下界匹配，故该下界是**紧的（tight）**。

硬件含义：H100 的 SRAM（每 SM 约 256 KB shared memory，108 个 SM）总计 ≈ 27 MB。对 $n = 4096$（典型大语言模型的注意力维度量级）：

$$T_{I/O} = \Omega\left(\frac{4096^3}{\sqrt{27 \times 10^6}} \right) \approx \Omega\left(\frac{6.87 \times 10^{10}}{5196} \right) \approx \Omega(1.3 \times 10^7) \text{ word transfers.} \tag{6.2.11}$$

即使算法最优，大矩阵也必须反复在 HBM 和 SRAM 之间搬移数据——这不是实现缺陷，而是**信息论意义上的必然**。

---

§6.3 FlashAttention 的 I/O 分析（Dao et al. 2022）

▶这一节在讲什么

为什么标准 Attention 慢得出奇，即使算力充足？因为它把一个 $N \times N$ 的中间矩阵反复写入再读出 HBM，而这个矩阵的大小是 $O(N^2)$——序列翻倍，I/O 增加四倍。FlashAttention 的贡献是在不改变数学结果的前提下，让这个矩阵永远不出现在 HBM 里。

第一步：在线 softmax 递推。softmax 需要知道全行的最大值和指数和才能归一化。FlashAttention 的核心技巧是：把这两个统计量做成「流式更新」——看到新的一块 K, V 就更新，不需要等到读完全行。这个递推在数学上等价于完整 softmax，但无需把整行存到 HBM。

第二步：分块 tiling 把 SRAM 装满。把 Q, K, V 分成若干小块，每块大小 $B \times d$，约束 $Bd = O(M)$ 保证小块能驻留 SRAM。这样每次内迭代的 HBM I/O 只是搬小块，而 $N \times N$ 矩阵的全局写入被彻底消除。

第三步：I/O 从 $\Theta(N^2)$ 降到 $\Theta(N^2d^2/M)$。当 $M \gg d^2$ 时（H100 上约差 600 倍），同样的计算结果，HBM 搬运量缩小近两个数量级——这不是优化，是重新推导。

本节要拿走的一件事：FlashAttention 的加速来源不是近似，而是通过「在线归一化 + 分块 tiling」重排计算顺序，将 I/O 复杂度从 $\Theta(N^2)$ 降至信息论最优的 $\Theta(N^2d^2/M)$。

直觉

§6.1 和 §6.2 建立了「算术强度」和「I/O 下界」的数学框架。§6.3 是这两个框架在大模型最核心算子——注意力机制（Attention）——上的具体应用。标准 Attention 的 I/O 问题不是一个「代码写得不好」的问题，而是**算法设计本身导致了理论上可以避免的大量 HBM 搬运**。FlashAttention 的贡献在于通过重新排列计算顺序（而非改变计算结果），把 I/O 从 $\Theta(N^2)$ 降到 $\Theta(N^2d^2/M)$。

本节出现五个概念：**标准 Attention 的 I/O 瓶颈**、**在线 softmax 递推**、**分块 tiling**、**SRAM 约束方程**、**I/O 复杂度计算**。它们构成一条逻辑链：先诊断问题（$\Theta(N^2)$ 来自何处），再找到突破口（softmax 可以流式计算），然后用 tiling 把突破口变成工程方案，最后用 SRAM 约束推导出最优块大小，得到最终复杂度。

第一环：标准 Attention 的 I/O 瓶颈——问题的精确诊断

Attention 的数学定义是 $O = \text{softmax}(QK^\top/\sqrt{d}) \cdot V$，其中 $Q, K, V \in \mathbb{R}^{N \times d}$。看似简洁，但朴素实现按顺序执行三步：先算 $S = QK^\top$（$N \times N$ 矩阵），写入 HBM；再对 S 做 softmax（即式 (6.3.1)）得 P，读 S 写 P；最后算 $O = PV$，读 P 和 V。

问题出在 $N \times N$ 的中间矩阵：当 $N = 8192$、FP16 时，S 和 P 各占约 $8192^2 \times 2 \approx 134$ MB，而 H100 的 SRAM 只有约 27 MB。每次计算 Attention，必须把这两个矩阵**完整地**

写入 HBM、再完整地读回来，总 HBM 搬运量为 $\Theta(N^2)$。

将这个 $\Theta(N^2)$ 与 §6.2 的 Hong-Kung 框架对比：Attention 真正需要的计算量是 $\Theta(N^2d)$（两次矩阵乘），而实际 I/O 是 $\Theta(N^2)$，算术强度约为 d。对 $d = 128$，这远低于 H100 的脊点 295——整个 Attention 在 H100 上是深度内存受限的。更糟的是，这 $\Theta(N^2)$ 的 I/O 完全是**可以避免的**：如果能不写入 S 和 P 矩阵，所需 I/O 就只是搬运 Q, K, V, O，总量 $\Theta(Nd)$，算术强度跃升至 N/d——对 $N = 8192$，约为 64，虽然仍然内存受限，但比 $\Theta(N^2)$ 少了数量级。

第二环：在线 softmax 的障碍与突破——流式计算的可行性

为什么朴素实现必须先把整个 S 矩阵写入 HBM？原因在于 softmax 的归一化：

$$\text{softmax}(s_1, s_2, \dots, s_N)_j = \frac{e^{s_j}}{\sum_{k=1}^{N} e^{s_k}} \tag{6.3.1}$$

分母 $\sum_k e^{s_k}$ 需要**看遍整行所有 N 个元素**才能计算——这意味着在算出分母之前，整行数据都必须存储在某处。SRAM 容不下整行（$N = 8192$ 时，一行 Q 的内积结果有 8192 个 FP16 数，共 16 KB，而一个 head 的 SRAM 份额远小于此），所以必须写入 HBM。

FlashAttention 的关键洞察是：**softmax 的分母可以「流式更新」**。不必等到看完整行——可以维护两个累计统计量，每看到新的一块 K_j 就立即更新它们：

- **当前行最大值** m_i：防止 e^{s_j} 数值溢出，可以随着新块的加入而递增更新。
- **当前归一化分母** ℓ_i：每看到新块就把新块的指数和（以当前最大值为基准）加进来。

这两个统计量的递推公式是：

$$m_i = \max(m_{i-1},\ \text{rowmax}(S_i)) \tag{6.3.2}$$

$$\ell_i = e^{m_{i-1} - m_i}\ell_{i-1} + \text{rowsum}(e^{S_i - m_i}) \tag{6.3.3}$$

$$O_i = \text{diag}(e^{m_{i-1} - m_i})O_{i-1} + e^{S_i - m_i}V_i \tag{6.3.4}$$

式 (6.3.2)–(6.3.4) 这三个公式的精妙之处在于：m_i 更新时，对之前积累的 ℓ_{i-1} 和 O_{i-1} 做了一个「重新缩放」（乘以 $e^{m_{i-1}-m_i}$），消去了旧的基准，换上新的最大值基准。引理 6.3 用数学归纳法严格证明：经过所有 T 块的递推后，O_T/ℓ_T 恰好等于完整 softmax 的结果——**零误差，没有任何近似**。

这一可行性证明是 FlashAttention 的数学核心：它把「必须看完整行才能做 softmax」的障碍彻底消除，打开了分块计算的大门。

第三环：分块 tiling——把在线 softmax 变成 SRAM 友好的算法

在线 softmax 允许分块计算，但「分多大的块」是关键。块太小，I/O 次数多（每次搬一个小块）；块太大，SRAM 装不下。最优块大小由 SRAM 容量约束确定：

- 每次内迭代需要在 SRAM 中同时存放：Q 块（$B_r \times d$）、K_j 块（$B_c \times d$）、V_j 块（$B_c \times d$）、当前输出 O_i 块（$B_r \times d$），以及计算中间结果 S_i（$B_r \times B_c$）。

- SRAM 约束要求所有这些元素的总大小不超过 M：$O(B_r d + B_c d + B_r B_c) \le M$。
- 取 $B_r = B_c = B$，主导项是 $O(Bd)$（矩阵乘的输入和输出），约束化简为 $B = O(M/d)$。

块大小 $B = O(M/d)$ 就是让每个 Q, K, V 块恰好能驻留 SRAM 的最大块大小。

第四环：I/O 复杂度推导——代入约束得到最终答案

有了最优块大小，I/O 复杂度的计算是直接的：

- 外循环遍历 $T_c = N/B_c$ 个 K, V 块，内循环遍历 $T_r = N/B_r$ 个 Q 块。
- 每次内迭代，从 HBM 读入当前 Q_i 块（$B_r \times d$）、K_j 块（$B_c \times d$）、V_j 块（$B_c \times d$），计算完成后写出 O_i 块（$B_r \times d$）。每次 I/O 量 $= O((B_r + B_c)d) = O(Bd)$（取 $B_r = B_c = B$）。
- 总迭代次数 $= T_r \times T_c = (N/B)^2$，每次 I/O 量 $= O(Bd)$，故

$$T_{I/O}^{\text{FlashAttn}} = O\left(\frac{N^2}{B^2} \cdot Bd\right) = O\left(\frac{N^2 d}{B}\right) \tag{6.3.5}$$

代入 $B = O(M/d)$：

$$T_{I/O}^{\text{FlashAttn}} = O\left(\frac{N^2 d}{M/d}\right) = O\left(\frac{N^2 d^2}{M}\right) \tag{6.3.6}$$

将式 (6.3.5) 代入 $B = O(M/d)$ 得到式 (6.3.6)。Dao 等人同时证明了下界 $\Omega(N^2 d^2/M)$（在 $d \le M \le Nd$ 假设下），故 FlashAttention 是**渐近最优**的精确 Attention 算法。

第五环：量化加速比——把抽象不等式翻译成工程直觉

式 (6.3.6)（FlashAttention）与标准 Attention（$\Theta(N^2)$）之比：

$$\frac{T_{I/O}^{\text{standard}}}{T_{I/O}^{\text{FlashAttn}}} = \Theta\left(\frac{M}{d^2}\right) \quad (N \gg d \text{ 时}) \tag{6.3.7}$$

式 (6.3.7) 中代入 H100 的数字：$M \approx 10^7$ 元素（27 MB SRAM，FP16），$d = 128$，$d^2 = 16384$，渐近加速比约 $10^7/16384 \approx 610$ 倍。

实测约 7.6 倍的原因是：「渐近」隐藏了常数项，序列长度有限（N 没有趋向无穷），以及 SRAM 实际可用量因 warp 调度和寄存器占用而低于理论值。但渐近分析已经精准指出了方向：M/d^2 才是决定加速比的关键量，它随 SRAM 容量增大而增大，随 head dimension 增大而减小。这也解释了为什么 FlashAttention 在 head dimension 小（如 $d = 64$）的场景加速比更大——d^2 更小，M/d^2 更大。

整条链的回头看

```
标准 Attention：S = QK 写入HBM，softmax读S写P，O=PV读P
  ↓ 诊断：Θ(N²) I/O 来自 N×N 中间矩阵的完整 HBM 读写
标准 Attention 的 I/O 瓶颈            ←  I = Θ(d)，深度内存受限
  ↓ 关键发现：softmax 的分母（指数和）可以流式递推更新
在线 softmax 递推（m_i，_i）          ←  无需整行就位，零误差流式计算
  ↓ 有了流式 softmax，可以按块处理 Q/K/V
```

```
分块 tiling: 块大小 B                    ←  每次内迭代只搬一小块，N×N矩阵不落地
  ↓ SRAM 约束决定最优块大小
B = O(M/d)                               ←  让 Q/K/V 块恰好驻留 SRAM 的临界值
  ↓ 代入 I/O 公式
T_IO = Θ(N²d²/M)                         ←  比标准 Attention 少 Θ(M/d²)  610×（渐近）
  ↓ 同时证明下界
FlashAttention 渐近最优                  ←  无法用精确算法进一步降低 I/O
```

这条链揭示了一个深层事实：**FlashAttention 的加速不是工程技巧，而是重新推导**。它把「必须先完整计算再归一化」的顺序约束替换为「边计算边归一化的流式等价」，这在数学上不是近似而是精确等价，在复杂度上则把 $\Theta(N^2)$ 的不可避免性撕开了一个缺口——那个 $\Theta(N^2)$ 从来就不是必然的，只是朴素实现的代价。

严格

6.3.1 标准注意力（Attention）的 I/O 复杂度

设序列长度 N，head 维度 d，查询/键/值矩阵 $Q, K, V \in \mathbb{R}^{N\times d}$。标准 attention 的前向计算分三步：

1. $S = QK^\top \in \mathbb{R}^{N\times N}$；写入 HBM。
2. $P = \mathrm{softmax}(S) \in \mathbb{R}^{N\times N}$；从 HBM 读 S，写回 P。
3. $O = PV \in \mathbb{R}^{N\times d}$；从 HBM 读 P 和 V。

各步主要 I/O：

步骤	读 (byte)	写 (byte)
$S = QK^\top$	$2Nd$	N^2
$\mathrm{softmax}(S)$	N^2	N^2
$O = PV$	$N^2 + Nd$	Nd

总 I/O $= \Theta(Nd + N^2)$。

极限假设 $N \gg d$（例如 $N = 8192, d = 128$）下，**主导项为** N^2；而 Q, K, V, O 的搬运仅为 $Nd \ll N^2$。这意味着标准 attention 的 I/O 瓶颈在于 $N \times N$ 注意力矩阵在 HBM 上的多次读写，而非矩阵乘本身的计算。

6.3.2 在线归一化指数（Softmax）的递推公式

Dao 等人的核心技巧是**分块 tiling + 在线 softmax**，使得 $N \times N$ 矩阵永远无需完整写入 HBM。设将 K, V 沿序列维度分成 $T_c = \lceil N/B_c \rceil$ 块，每块大小 $B_c \times d$；将 Q, O 分成 $T_r = \lceil N/B_r \rceil$ 块，每块大小 $B_r \times d$。对 Q 的第 i 行块，算法遍历 K, V 的所有列块，维护以下三个**在线统计量**：

$$m_i \;=\; \max\big(m_{i-1},\ \mathrm{rowmax}(S_i)\big), \tag{6.3.8}$$

$$\ell_i \;=\; e^{m_{i-1}-m_i}\,\ell_{i-1} \;+\; \mathrm{rowsum}\big(e^{S_i-m_i}\big), \tag{6.3.9}$$

$$O_i \;=\; \mathrm{diag}\big(e^{m_{i-1}-m_i}\big)\,O_{i-1} \;+\; e^{S_i-m_i}\,V_i. \tag{6.3.10}$$

其中 $S_i = Q \cdot K_i^\top \in \mathbb{R}^{B_r \times B_c}$ 是当前块的注意力分数，$m_0 = -\infty$，$\ell_0 = 0$，$O_0 = 0$。

引理 6.3（在线 softmax 正确性）：设总共有 T 个键块，则经过所有 T 次递推后，

$$O_T = \mathrm{diag}(\ell_T)^{-1} \cdot \sum_{j=1}^{T} e^{S_j - m_T} V_j = \mathrm{softmax}(S_{\text{full}}) \cdot V, \tag{6.3.11}$$

其中 $S_{\text{full}} = [S_1, S_2, \dots, S_T]$ 为完整注意力分数行向量。

证明：采用数学归纳法，对块数 t 归纳。

基础：$t = 1$：$m_1 = \mathrm{rowmax}(S_1)$，$\ell_1 = \mathrm{rowsum}(e^{S_1 - m_1})$，$O_1 = e^{S_1 - m_1} V_1$。注意

$$\frac{O_1}{\ell_1} = \frac{e^{S_1 - m_1} V_1}{\mathrm{rowsum}(e^{S_1 - m_1})} = \mathrm{softmax}(S_1) V_1, \tag{6.3.12}$$

归一化后正确。✓

归纳步：设在处理 $t-1$ 块后，归一化输出

$$\hat{O}_{t-1} \;=\; \frac{O_{t-1}}{\ell_{t-1}} \;=\; \mathrm{softmax}(S_{1:t-1}) V_{1:t-1} \tag{6.3.13}$$

成立（归纳假设）。处理第 t 块时，令 $m_t = \max(m_{t-1}, \mathrm{rowmax}(S_t))$，则

$$\ell_t = e^{m_{t-1} - m_t} \ell_{t-1} + \mathrm{rowsum}(e^{S_t - m_t}). \tag{6.3.14}$$

注意 $e^{m_{t-1} - m_t} \ell_{t-1} = \mathrm{rowsum}(e^{S_{1:t-1} - m_t})$（指数中的 m_{t-1} 被消去），故

$$\ell_t = \mathrm{rowsum}\big(e^{S_{1:t} - m_t}\big). \tag{6.3.15}$$

同理，

$$O_t = e^{m_{t-1} - m_t} O_{t-1} + e^{S_t - m_t} V_t = e^{S_{1:t-1} - m_t} \cdot \mathbf{1} \cdot \hat{O}_{t-1} \cdot \ell_{t-1} + e^{S_t - m_t} V_t. \tag{6.3.16}$$

精确地，

$$\frac{O_t}{\ell_t} = \frac{\sum_{j=1}^{t} e^{S_j - m_t} V_j}{\mathrm{rowsum}(e^{S_{1:t} - m_t})} = \mathrm{softmax}(S_{1:t}) V_{1:t}. \quad \square \tag{6.3.17}$$

6.3.3 FlashAttention 的 I/O 复杂度

命题 6.4（FlashAttention 前向 I/O 复杂度）：设 SRAM 大小为 M，满足 $d \le M \le Nd$，块大小取 $B_c = \Theta(M/d)$，$B_r = \Theta(M/d)$。FlashAttention 前向传播的 HBM I/O 复杂度为

$$T_{I/O}^{\text{FlashAttn}} = \Theta\left(\frac{N^2 d^2}{M}\right). \tag{6.3.18}$$

证明：

外循环遍历 $T_c = N/B_c$ 个键块，内循环遍历 $T_r = N/B_r$ 个查询块。

- **内循环每次**：从 HBM 读入 Q 块（$B_r \times d$）+ K_j, V_j 块（各 $B_c \times d$），计算 $S_i = QK_j^\top$（纯 SRAM 内计算，无额外 HBM I/O），更新 O_i（SRAM 内），写出 O 块（$B_r \times d$）。每次内迭代 HBM I/O 量 $= O(B_r d + B_c d)$。
- **总 I/O**：共 $T_r \times T_c = (N/B_r)(N/B_c)$ 次内迭代，每次 $O(B_r d + B_c d)$，故

$$T_{I/O} = O\left(\frac{N}{B_r} \cdot \frac{N}{B_c} \cdot (B_r + B_c) \cdot d\right). \tag{6.3.19}$$

取 $B_r = B_c = B$，则

$$T_{I/O} = O\left(\frac{N^2}{B^2} \cdot 2Bd\right) = O\left(\frac{N^2 d}{B}\right). \tag{6.3.20}$$

消元条件 · SRAM 容量约束：SRAM 需同时容纳 Q, K, V 块及 O, S 块：$O(Bd)$ 个元素，故约束 $Bd = O(M)$，**即** $B = O(M/d)$。将式 (6.3.20) 代入 $B = O(M/d)$：

$$T_{I/O}^{\text{FlashAttn}} = O\left(\frac{N^2 d}{M/d}\right) = O\left(\frac{N^2 d^2}{M}\right). \tag{6.3.21}$$

下界同理可得 $\Omega(N^2 d^2/M)$（见原文 Theorem 2 的最优性证明），故 $T_{I/O}^{\textbf{FlashAttn}} = \Theta(N^2 d^2/M)$。□

> **复杂度跃迁 · 这一步是全文的关键转折**：标准 attention 的 $\Theta(N^2)$ 被换写为 $\Theta(N^2 d^2/M)$——当 $M \gg d^2$（H100 上 $M \approx 10^7$、$d^2 \approx 1.6 \times 10^4$），**同一计算结果的 HBM I/O 几乎下降两个数量级**。I/O 复杂度不是被「优化」出来的，是被「重新推导」出来的。

6.3.4 FlashAttention vs. 标准 Attention 的复杂度比较

$$\frac{T_{I/O}^{\text{standard}}}{T_{I/O}^{\text{FlashAttn}}} = \frac{\Theta(Nd + N^2)}{\Theta(N^2 d^2/M)} = \Theta\left(\frac{M}{d^2} \cdot \frac{Nd + N^2}{N^2}\right) \underset{N \gg d}{\approx} \Theta\left(\frac{M}{d^2}\right). \tag{6.3.22}$$

将式 (6.3.18) 和式 (6.3.22) 代入 A100 的 SRAM 大小 $M \approx 20$ MB $= 20 \times 10^6$ byte，head dimension $d = 128$，每 FP16 元素 2 byte，则 $M \approx 10^7$ 元素，$d^2 = 16384$，故加速比约 $\Theta(10^7/16384) \approx 610$ 倍（渐近）。实测加速约 7.6 倍（主要原因是序列长度有限，常数项不可忽略）。

6.3.5 FlashAttention-2 的工作划分（Work Partitioning）改进

Dao（2023）在 FlashAttention 的基础上提出三项改进，不改变 I/O 复杂度量级，但大幅提升硬件利用率：

1. **减少非矩阵乘 FLOP**：FlashAttention 原版的缩放与 rescaling 操作引入额外逐元素运算；FA-2 重新推导递推公式，将 diag$(e^{m_{i-1}-m_i})$ 的显式展开推迟到最终输出时执行，减少中间 rescaling 次数。
2. **序列维度并行化**：FA-1 仅对 batch 和 head 维度并行调度 thread block，导致长序列小 batch 时 GPU 占用率（occupancy）低。FA-2 对外循环（T_r 个行块）额外并行，使 SM 利用率从约 40% 提升至 50–73%。
3. **Warp 内 Work Partitioning**：FA-1 将 K, V 切分至 4 个 warp（「split-K」方案），各 warp 需写中间结果至 shared memory 后同步累加；FA-2 改为将 Q 切分至 warp，K, V 共享，消除 warp 间同步，降低 shared memory 访问延迟。

这三项改进使 FA-2 在 A100 上达到理论峰值的 50–73%，在 H100 上可达 335 TFLOPS（FlashAttention-2 本身无需特殊 Hopper 指令），是截至 2023 年实现最高 GPU 利用率的 attention 实现之一。

---

§6.4 环形注意力（RingAttention）的通信复杂度（Liu et al. 2024）

▶这一节在讲什么

为什么把 FlashAttention 直接搬到多卡上会出问题？单卡 SRAM 装不下超长序列的 K, V，多卡分片后每张卡的 Q_p 需要访问所有卡的 K, V——朴素的 AllGather 会让每张卡内存随 N 线性增长，同时通信量也不随卡数增加而分摊。RingAttention 用环形流水线同时解决这两个问题。

第一步：把 GPU 排成环，K/V 块像接力棒一样传递。每一步，每张卡把当前的 K_j, V_j 发给下一张卡，同时接收上一张卡传来的新块，并用当前块更新本地的在线 softmax 统计量。计算与通信同步进行。

第二步：总通信量与卡数无关。P 张卡跑 $P-1$ 步，每步传 $(N/P) \times d$ 个元素，总计 $\approx Nd$ 元素——和只有 1 张卡时搬运整个 K, V 的量相同。卡越多，每步通信越少，总量不变。

第三步：完全重叠的条件是「局部脊点」。要让通信被计算完全掩盖，每卡局部序列长度 N/P 必须不低于 $2F/\beta_{\text{NVLink}}$——这正是单卡 Roofline 脊点在多卡通信场景下的类比。

本节要拿走的一件事：RingAttention 的通信复杂度是 $\Theta(Nd)$，与卡数 P 无关；但完全重叠所要求的「局部脊点」给出了 P 的理论上限 $P_{\max} \approx N\beta_{\text{NVLink}}/(2F)$。

直觉

§6.3 解决了单卡上 Attention 的 I/O 问题；§6.4 要解决一个更大的问题：当序列长度 N 超过单卡 HBM 容量时，如何在多卡上**精确**计算 Attention？这不只是分布式计算的工程问题，而是有严格的数学结构——通信量、重叠条件、最大可扩展的卡数，都可以用公式精确表达。

本节出现五个概念：**多卡场景的内存约束、朴素 AllGather 方案的缺陷、环形通信方案（RingAttention）、总通信量不依赖卡数、完全重叠条件（局部脊点）**。这五个概念构成

一条逻辑链：从问题的诊断（单卡装不下），到朴素解法的缺陷（AllGather 通信量随卡数增加），到 RingAttention 的设计（利用在线 softmax 做环形流水），到通信量分析（$\Theta(Nd)$ 与 P 无关），最后到重叠条件（局部脊点，给出 P 的上限）。

第一环：单卡内存瓶颈与多卡的必要性

当序列长度 $N = 10^6$（百万 token，长文档或视频处理的典型量级），K 和 V 矩阵各占 $N \times d \times 2 = 10^6 \times 128 \times 2 = 256$ MB。两个合计 512 MB，加上 Q 和输出 O，单 Attention head 所需 HBM 超过 1 GB。而 H100 的 HBM 只有 80 GB，乘以 head 数（如 96 头），光 K, V 就需要 $96 \times 256 \times 2 = 49$ GB——接近单卡上限，还没算模型参数和梯度。

这意味着：超长序列的 Attention **必须**多卡协作，不是性能优化问题，而是内存容量约束决定的必然性。

第二环：朴素 AllGather 方案及其缺陷——明确需要解决的问题

最直接的多卡方案是：把 K, V 用 AllGather 操作广播到每张卡，然后每张卡用本地 Q_p 做完整 Attention。这样每张卡都持有全局 K, V，但有两个致命问题：

1. **内存瓶颈未解决**：每张卡都要存全局 K, V（各 Nd 元素），内存随 N 增长，用再多的卡也无济于事。
2. **通信量随卡数增加**：AllGather 的通信量正比于卡数 P（每张卡收到来自其他 $P-1$ 张卡的 K, V），卡越多通信越多，扩展性差。

理想的方案应该满足：每张卡只需存 $1/P$ 的 K, V（内存随卡数线性缩小），且总通信量与 P 无关（不随卡数增加而恶化）。RingAttention 正是满足这两个条件的方案。

第三环：环形通信——利用在线 softmax 实现流水线重叠

RingAttention 的设计思路直接利用了 §6.3 中的在线 softmax 递推：在线 softmax 允许「每次看一块 K, V、逐块更新统计量」，而不需要一次性看到全局 K, V。这意味着可以**流式接收**远程卡的 K, V 块：

- 将 P 张 GPU 排成逻辑环，GPU p 初始持有 K_p, V_p（本地序列片段）。
- 每一步：GPU p 把当前持有的 K_j, V_j 发送给 GPU $p+1$，同时从 GPU $p-1$ 接收新的 $K_{j'}, V_{j'}$。
- 在发送/接收通信进行的同时，GPU p 用当前 K_j, V_j 对本地 Q_p 做 FlashAttention 计算，更新在线 softmax 统计量。

经过 $P-1$ 步后，每张卡的 Q_p 已经遍历了全局所有 K, V 块，在线 softmax 的归一化输出就是精确的全局 Attention 结果。

关键点：每张卡在任何时刻只需存储一块 K, V（大小 $(N/P) \times d$），**总内存使用量与 $1/P$ 成正比**，内存瓶颈被彻底解决。

第四环：总通信量与卡数无关——RingAttention 的核心数学结论

每步通信量：发送 K_j 和 V_j 各 $(N/P) \times d$ 个元素，共 $2(N/P)d$ 个元素。

总通信量（$P-1$ 步）：

$$T_{\text{comm}} = (P-1) \times 2 \times \frac{N}{P} \times d \approx 2Nd \,(\text{元素数}) \tag{6.4.1}$$

式 (6.4.1) 与 P **完全无关**。无论用 1 张卡还是 1000 张卡，总通信量始终约为 $2Nd$ 个元素——等于把全局 K,V 搬运两次的量。卡越多，每步通信量越小（(N/P)），但步数越多（$P-1$），恰好抵消。

这个「不变性」的直觉：可以类比流水线工厂。产品（K,V 块）在工位（GPU）之间依次传递，总传递次数等于工位数乘以产品数，而这里「产品数」×「每件大小」固定为 Nd，恰好与工位数无关。对比朴素 AllGather（每张卡各自收全局 K,V，通信量正比 $P\cdot Nd$），RingAttention 的渐近通信复杂度改善了 P 倍——这才是它能支撑超大卡数扩展的根本原因。

第五环：完全重叠条件（局部脊点）——P 的理论上限

「通信量不随卡数增加」并不等于「可以无限加卡」。RingAttention 的另一个性能关键是：**通信必须被计算完全掩盖**，即每步计算时间 $t_{\text{comp}}\geq$ 每步通信时间 t_{comm}。

每步计算量（Q_p 与一个 K_j,V_j 块做矩阵乘）：$2(N/P)^2d$ FLOP，耗时 $t_{\text{comp}}=2(N/P)^2d/F$（$F$ 为单卡算力）。

每步通信量（发送 K_j,V_j 块）：$4(N/P)d$ 元素（FP16），耗时 $t_{\text{comm}}=4(N/P)d/\beta_{\text{NVLink}}$。

完全重叠条件 $t_{\text{comp}}\geq t_{\text{comm}}$ 化简为：

$$\frac{2(N/P)^2d}{F}\geq\frac{4(N/P)d}{\beta_{\text{NVLink}}}\implies\frac{N}{P}\geq\frac{2F}{\beta_{\text{NVLink}}}\tag{6.4.2}$$

式 (6.4.2) 的形式与 §6.1 的 Roofline 脊点（式 (6.1.3)）同构：$2F/\beta_{\text{NVLink}}$ 是一个「局部脊点」——每卡局部序列长度必须不低于它，才能让计算「跑得赢」通信。解出 P 的上限：

$$P_{\max}=\left\lfloor\frac{N\cdot\beta_{\text{NVLink}}}{2F}\right\rfloor\tag{6.4.3}$$

代入 H100 SXM5（$F=989\times10^{12}$ FLOP/s，$\beta_{\text{NVLink}}=450\times10^{9}$ byte/s，$N=10^6$）：

$$P_{\max}=\frac{10^6\times450\times10^9}{2\times989\times10^{12}}\approx227\tag{6.4.4}$$

由式 (6.4.3) 对百万 token 序列可得约 227 张 H100 可令 RingAttention 通信完全被计算掩盖。超过此数后，每步通信来不及被计算掩盖，系统进入「通信受限」状态——与单卡的「内存受限」状态（由式 (6.1.2) 决定上界）在数学结构上完全同构：都是「两种资源速率不匹配」的体现。

整条链的回头看

```
单卡 HBM 容量有限：N=10 时 K,V 各需 256 MB，超过单卡分配
  ↓ 需要多卡协作；朴素 AllGather 通信量正比 P，内存不随 P 缩减
朴素方案的双重缺陷                          ←  内存不解决 + 通信随 P 恶化
  ↓ 利用在线 softmax：K,V 可以流式接收，无需一次性就位
环形通信方案                                ←  P 张卡成环，每步传一块，同时计算
  ↓ 计算 P-1 步的总通信量
T_comm = Θ(Nd)，与 P 无关                   ←  不变性：步数增加，每步量减少，恰好抵消
```

```
  ↓ 但通信被计算掩盖需要 N/P 足够大
完全重叠条件：N/P   2F/ _NVLink       ←  「局部脊点」：单卡 Roofline 的多卡类比
  ↓ 解出 P 的上限
P_max   N · _NVLink / (2F)            ←  H100 上百万 token 对应约 227 张卡
```

这条链揭示了一个深层统一性：**Roofline 脊点、Hong-Kung I/O 下界、FlashAttention tiling 约束、RingAttention 重叠条件，本质上都是同一个数学结构——「两种资源的速率匹配」**。无论是单卡的算力 vs 带宽，还是多卡的计算 vs 通信，突破哪种资源约束的方法，都是提升「另一种资源的利用密度」：在单卡上是提高算术强度，在多卡上是提高每步计算量（即加大局部序列长度 N/P）。

严格

6.4.1 多卡场景设定

设总序列长度 N，分布在 P 张 GPU 上，每张 GPU 持有

$$Q_p \in \mathbb{R}^{(N/P)\times d}, \quad K_p, V_p \in \mathbb{R}^{(N/P)\times d}, \quad p = 1, 2, \ldots, P. \tag{6.4.5}$$

完整注意力需要每张 GPU 的 Q_p 对全局 K, V 做 attention，即 $O_p = \mathrm{softmax}(Q_p K^\top/\sqrt{d})V$。

朴素做法：先用 AllGather 将完整 K, V（各 Nd 元素，$2Nd \cdot \mathrm{sizeof}$ byte）汇聚至每张卡，通信量 $\Theta(Nd)$ 字节。这与 P 无关，但实际上每张卡需存储完整 K, V，内存随 N 增长，无法扩展至超长序列。

6.4.2 环形通信方案

RingAttention 将 P 张 GPU 排列为**逻辑环**，在计算与通信之间实现**流水线重叠（pipeline overlap）**：

第 t 步（$t = 1, 2, \ldots, P-1$）：

- GPU p 将其持有的当前 K_j, V_j（大小各 $(N/P) \times d$）**发送**至 GPU $p+1 \pmod P$，同时从 GPU $p-1 \pmod P$ **接收**下一批 K, V 块。
- 发送/接收过程中，GPU p 同时用 FlashAttention 计算当前 K_j, V_j 对本地 Q_p 的贡献，更新在线 softmax 统计量。

每步通信量：发送 K_j 和 V_j 各 $(N/P) \times d \times \mathrm{sizeof} = 2 \cdot (N/P) \cdot d$ 元素。

总通信量（$P-1$ 步）：

$$T_{\text{comm}} = (P-1) \cdot 2 \cdot \frac{N}{P} \cdot d \cdot \mathrm{sizeof} \;\approx\; 2Nd \cdot \mathrm{sizeof}, \tag{6.4.6}$$

即总通信量 $\Theta(Nd)$ **字节，不依赖** P——设备数越多，每步通信量越小，总量不变。

6.4.3 计算与通信重叠条件

每步**计算时间**：GPU p 需完成 Q_p（大小 $N/P \times d$）与 K_j（大小 $N/P \times d$）的矩阵乘，计算量为 $2(N/P)^2 d$ FLOP，耗时

$$t_{\text{comp}} = \frac{2(N/P)^2 d}{F}, \tag{6.4.7}$$

其中 F 为单卡浮点算力（FLOP/s）。

每步**通信时间**：传输 $2 \times (N/P) \times d$ 个元素（FP16，每元素 2 byte），耗时

$$t_{\text{comm}} = \frac{4(N/P)d}{\beta_{\text{NVLink}}}, \tag{6.4.8}$$

其中 β_{NVLink} 为单向 NVLink 带宽（H100 SXM: ≈ 450 GB/s 单向）。

完全重叠条件（计算时间 $\geq$ 通信时间）：

$$\frac{2(N/P)^2 d}{F} \geq \frac{4(N/P)d}{\beta_{\text{NVLink}}}, \tag{6.4.9}$$

化简得

$$\frac{N}{P} \geq \frac{2F}{\beta_{\text{NVLink}}}. \tag{6.4.10}$$

即每卡持有的局部序列长度 N/P 不得小于 $2F/\beta_{\text{NVLink}}$（类似一个「局部 ridge point」）。

6.4.4 环路最大规模推导

由式 (6.4.10) 解出 P 的上界：

$$P \leq \frac{N \cdot \beta_{\text{NVLink}}}{2F} = \frac{N \cdot \beta_{\text{NVLink}}}{2F}. \tag{6.4.11}$$

若希望通信完全被计算掩盖，GPU 数量不得超过

$$P_{\max} = \left\lfloor \frac{N \cdot \beta_{\text{NVLink}}}{2F} \right\rfloor. \tag{6.4.12}$$

H100 SXM5 的具体数值：$F = 989 \times 10^{12}$ FLOP/s（TF32），$\beta_{\text{NVLink}} = 450 \times 10^9$ byte/s（单向），序列长度 $N = 1{,}000{,}000$（百万 token）：

$$P_{\max} = \frac{10^6 \times 450 \times 10^9}{2 \times 989 \times 10^{12}} \approx \frac{4.5 \times 10^{17}}{1.978 \times 10^{15}} \approx 227. \tag{6.4.13}$$

即对百万 token 的序列，约 227 张 H100 可令 RingAttention 通信完全被计算掩盖。超过此数目时通信将成为瓶颈，需进一步的通信压缩或序列分片策略。

与 Roofline 的类比：$P_{\max}$ 的公式 $\sim \sqrt{N \cdot F/\beta}$（若同时考虑计算量正比 N^2/P^2、通信量正比 N/P）提示了一个多卡版本的「ridge point」：设备数增多使局部计算强度下降，超过 $P_{\max}$ 后即陷入「通信受限」状态，与单卡内存受限具有同构的数学结构。

§6.5 算术强度的演化：从 V100 到 B200

▶这一节在讲什么

为什么每一代 GPU 升级，「需要多高的算术强度才能用满算力」这个门槛也在升高？算力增速长期快于带宽增速，脊点 $I^* = P_{\text{peak}}/\beta$ 就随之上移——上一代计算受限的 kernel，在这一代可能已经变成内存受限。

第一步：量化每一代的增速比。从 V100 到 H100，算力增长了约 7.9×，带宽增长了约 3.7×，比值 > 1 使脊点从 139 升至 295。理解这个比值比记住绝对数字更有价值。

第二步：B200 是一个有趣的例外。Blackwell 架构首次出现带宽增速（2.39×）超过算力增速（2.27×）的情形，脊点从 295 微降至 281。这不是退步，而是 HBM3e 在本代实现了相对更大幅度的带宽跃升。

第三步：量化对 I 的影响。从 FP16 换到 FP8，相同计算量下字节搬运减半，$I \to 2I$——相当于「人工提升算术强度」。这和 FlashAttention 通过 tiling 减少 HBM 访问是同一逻辑的不同实现。

本节要拿走的一件事：脊点随代际上移意味着算法必须持续「主动提升 I」——更大 tile、算子融合、低精度量化，三条路共同指向同一个目标：在带宽不变的约束下，从每一字节数据中榨取更多计算。

直觉

前四节建立了 Roofline 模型（§6.1）、I/O 下界（§6.2）、FlashAttention（§6.3）和 RingAttention（§6.4）的数学框架——这些框架都依赖一个关键参数：脊点 $I^* = P_{\text{peak}}/\beta$。§6.5 要回答一个动态问题：**随着 GPU 代际迭代，这个脊点是如何变化的，这个变化对算法设计有什么系统性影响？**

本节出现四个概念：**脊点的代际演化、增速比 α_F/α_β、B200 的例外、量化对算术强度的影响**。它们构成一条逻辑链：从观察历史数据（脊点单调上移），到抽象出「增速比」这个核心决定因素，到分析 B200 的异常（带宽增速首次超过算力增速），最后到量化作为「人工提升算术强度」的第三条路，与算法层面的 tiling/fusion 并列。

第一环：脊点是一个动态指标，不是固定常数

脊点 $I^* = P_{\text{peak}}/\beta$ 由硬件的「算力-带宽比」决定。如果把历史数据列出来：

- V100（2018）：$P_{\text{peak}} = 125$ TFLOPS，$\beta = 0.90$ TB/s，$I^* = 139$
- A100（2020）：$P_{\text{peak}} = 312$ TFLOPS，$\beta = 2.00$ TB/s，$I^* = 156$
- H100（2022）：$P_{\text{peak}} = 989$ TFLOPS，$\beta = 3.35$ TB/s，$I^* = 295$
- B200（2025）：$P_{\text{peak}} = 2250$ TFLOPS，$\beta = 8.00$ TB/s，$I^* = 281$

从 V100 到 H100，脊点从 139 单调上升到 295，翻了一倍多。这个趋势对算法设计者意味着：**一个在 V100 上计算受限（$I > 139$）的 kernel，在 H100 上可能已经滑入内存受限区（$I < 295$）**——哪怕代码一字未改，新硬件反而让这个 kernel 的相对性能下降了。更快的算力如果没有对应更快的带宽，只是把「内存墙」推得更高，让更多的 kernel 被它拦截。

第二环：增速比——决定脊点方向的核心量

脊点变化的方向由一个简单的比值决定：相邻两代间算力增速 $\alpha_F = P_{\text{peak}}^{(t+1)}/P_{\text{peak}}^{(t)}$ 与带宽增速 $\alpha_\beta = \beta^{(t+1)}/\beta^{(t)}$ 之比（即式 (6.5.1)）：

$$I^{*(t+1)} = I^{*(t)} \cdot \frac{\alpha_F}{\alpha_\beta} \tag{6.5.1}$$

- $\alpha_F/\alpha_\beta > 1$：脊点上移（算力增速快于带宽），内存墙升高。
- $\alpha_F/\alpha_\beta < 1$：脊点下移（带宽增速快于算力），内存墙降低。
- $\alpha_F/\alpha_\beta = 1$：脊点不变，两者均衡发展。

历史数据的验证：- V100→A100：$\alpha_F = 2.50$，$\alpha_\beta = 2.22$，比值 1.12，脊点从 139 升至 156。- A100→H100：$\alpha_F = 3.17$，$\alpha_\beta = 1.68$，比值 1.89，脊点从 156 升至 295——这是最大的一次跳跃，主要原因是 H100 Tensor Core 算力因稀疏化和 FP8 精度的引入大幅提升，而带宽增速相对温和（HBM2e→HBM3 只有约 1.68 倍）。

这个框架让「记数字」变成「理解结构」：不需要背每代 GPU 的绝对参数，只需要知道「算力增速 vs 带宽增速哪个更快」，就能预判脊点的走向，从而判断下一代硬件会把哪些 kernel 从计算受限区「推入」内存受限区。

第三环：B200 的例外——带宽增速首次超过算力增速

H100→B200 的增速比：$\alpha_F = 2.27$，$\alpha_\beta = 2.39$，比值 $0.95 < 1$——由式 (6.5.1) 算得脊点从 295 微降至 281。这是 2018 年以来第一次出现带宽增速超过算力增速的情形。

这个反转的原因：B200 采用 HBM3e 技术，叠片数从 H100 的 5 层增加到 8 层，引脚速率从 5.23 Gbps 提升到 8 Gbps，带宽实现了相对更大的跃升。与此同时，B200 的算力提升主要来自 FP4 精度（9,000 TFLOPS，用于推理量化），在 FP16 维度的增速（2.27×）不如带宽增速。

这个例外传递了一个信号：硬件厂商已经意识到「内存墙」是制约大模型推理的主要瓶颈，开始在 HBM 带宽上投入更多资源。B200 的脊点下移虽然幅度很小（295→281），但方向的反转本身是一个结构性变化——它预示着未来几代可能出现更多「带宽追赶算力」的迭代。

第四环：量化——提升算术强度的第三条路

§6.3 的 FlashAttention 通过 tiling 减少 HBM I/O，是「提升算术强度」的第一条路（算法层面）。算子融合（多个 kernel 合并为一个，共享 HBM 读写）是第二条路（工程层面）。量化是第三条路（精度层面）：

从 FP16（每元素 2 byte）切换到 FP8（每元素 1 byte），相同矩阵的 HBM 搬运量减半，而计算量（FLOP）不变，因此算术强度翻倍：$I_{\text{FP8}} = 2I_{\text{FP16}}$。这等价于把 kernel 在 Roofline 图上向右平移一倍——有可能越过脊点，从内存受限区进入计算受限区。

对 H100 FP8 精度，$P_{\text{peak}} \approx 3958$ TFLOPS，带宽不变（3.35 TB/s），由式 (6.5.1) 得脊点：

$$I^*_{\text{H100, FP8}} = \frac{3958}{3.35} \approx 1181 \text{ FLOP/byte} \tag{6.5.2}$$

式 (6.5.2) 的脊点跃升至 1181——在这个精度下，即便是 batch size 较大（算术强度 100 量级）的矩阵乘也仍然内存受限，需要非常大的 batch 或极大矩阵才能进入计算受限区。这说明量化虽然提升了算术强度，但同时硬件也在同等提升算力，脊点「追着」

算力走。三条路（tiling、fusion、量化）的本质是相同的：**让每一字节 HBM 访问对应更多有效 FLOP**，在带宽这条护城河不变的约束下，尽可能多地利用已有的算力。

整条链的回头看

```
脊点 I* = P_peak /                    ←  §6.1 定义，分隔内存受限和计算受限
  ↓ 随 GPU 代际变化：历史数据显示 V100→H100 脊点 139→295
增速比 _F/ _ 决定脊点方向              ←  比值 >1 则脊点上移，更多 kernel 被内存墙拦截
  ↓ V100→A100→H100：三代 _F/ _ >1，脊点单调上行
B200 的例外： _F/ _ = 0.95 <1         ←  HBM3e 带宽增速首次超过算力增速，脊点微降
  ↓ 应对策略：主动提升算术强度 I
三条路：tiling (FlashAttn) / fusion / quantization (FP8)
  ↓ FP16→FP8：搬运量减半，I→2I，等效「向右移动」kernel
I*_{H100,FP8}  1181 FLOP/byte         ←  脊点随精度提升而同步上移，不断追赶
```

这条链揭示了一个深层事实：**脊点的代际演化是算法设计的「时代背景」**。理解每一代硬件的增速比，就能预判哪类 kernel 会成为新的瓶颈，从而提前设计应对策略。FlashAttention 不只是「2022 年的一篇好论文」，它是对「A100→H100 的脊点跃升」的提前应答——在带宽增速远低于算力增速的年代，任何不主动提升算术强度的 Attention 实现，都会被内存墙越钉越死。

严格

6.5.1 各代 GPU 的 Roofline 参数

以下数据来源于 NVIDIA 官方规格表与公开基准测试，均为 dense（无稀疏加速）FP16 Tensor Core 性能：

GPU	架构	P_{peak} (TFLOPS, FP16)	β (TB/s, HBM)	$I^* = P_{\text{peak}}/\beta$ (FLOP/byte)
V100 SXM2 (2018)	Volta	125	0.90	**139**
A100 SXM4 (2020)	Ampere	312	2.00	**156**
H100 SXM5 (2022)	Hopper	989[†]	3.35	**295**
B200 SXM (2025)	Blackwell	2,250[‡]	8.00[§]	**281**

[†] 989 TFLOPS 为 TF32 dense；FP16 dense 为 1,979 TFLOPS，对应 $I^* \approx 590$ FLOP/byte。本章与主本一致，使用 TF32 基准以便对比训练场景。

[‡] B200 FP16 Tensor Core dense 值；FP4 dense 可达 9,000 TFLOPS，仅用于推理量化场景，不计入此处 Roofline 对比。

[§] B200 HBM3e 带宽官方数值因文献版本不同在 7.7–8.0 TB/s 之间；本章取 8.0 TB/s。

6.5.2 Ridge Point 的单调上行论证

命题 6.5（Ridge Point 单调上行）：设在相邻两代 GPU 之间，算力增长因子为 $\alpha_F = P_{\text{peak}}^{(t+1)}/P_{\text{peak}}^{(t)}$，带宽增长因子为 $\alpha_\beta = \beta^{(t+1)}/\beta^{(t)}$。若 $\alpha_F > \alpha_\beta$，则脊点单调上升：

$$I^{*(t+1)} = \frac{P_{\text{peak}}^{(t+1)}}{\beta^{(t+1)}} = I^{*(t)} \cdot \frac{\alpha_F}{\alpha_\beta} > I^{*(t)}. \tag{6.5.3}$$

验证：

- V100 → A100：$\alpha_F = 312/125 = 2.50$，$\alpha_\beta = 2.00/0.90 = 2.22$，比值 $1.12 > 1$。I^* 从 139 升至 156。✓
- A100 → H100：$\alpha_F = 989/312 = 3.17$，$\alpha_\beta = 3.35/2.00 = 1.68$，比值 $1.89 > 1$。I^* 从 156 升至 295。✓
- H100 → B200：$\alpha_F = 2250/989 = 2.27$，$\alpha_\beta = 8.00/3.35 = 2.39$，比值 $0.95 < 1$。I^* 从 295 微降至 281。

注意 B200 的带宽增速（2.39×）**首次超过**算力增速（2.27×），脊点出现轻微下降。这并非 Blackwell 架构设计失误，而是 HBM3e 技术在本代实现了相对更大幅度的带宽跃升（叠片数增加、引脚速率 8 Gbps vs. 5.23 Gbps）。

结构性含义：从历史趋势看，V100→H100 三代间 α_F/α_β 均大于 1，脊点从 139 升至 295，**翻倍有余**。这意味着：

1. 曾经在 V100 上计算受限的 kernel，在 H100 上可能已变为内存受限（因为脊点右移，需要更高算术强度才能逃离内存墙）。
2. 算法设计必须不断「主动提升 I」——或通过更大 tile（如 FlashAttention-2 的更大 B_r, B_c），或通过算子融合（kernel fusion），或通过量化（FP8/FP4）降低数据搬运字节数。

量化对 I 的影响：若将 FP16（2 byte/元素）改为 FP8（1 byte/元素），相同计算量下字节搬运减半，算术强度 $I \to 2I$。结合 H100 FP8 下 3,958 TFLOPS 的峰值，由式 (6.5.3) 得脊点

$$I^*_{\text{H100, FP8}} = \frac{3958}{3.35} \approx 1181 \text{ FLOP/byte}. \tag{6.5.4}$$

在此精度下，即使是 batch size = 1 的矩阵-向量乘（$I \approx 2$）依然深度内存受限，带宽仍然是主要瓶颈。

---

§6.6 反类比与边界

▶这一节在讲什么

为什么一个精确的数学模型也需要「反类比」章节？因为 Roofline、Hong-Kung 定理、FlashAttention 的 I/O 分析各自都有隐含假设，在这些假设失效的场景下，模型的预测会系统性地偏乐观。知道边界在哪里，和知道模型本身一样重要。

第一步：Roofline 的三个假设局限。它假设（1）内存访问与计算可以完全流水——实际有延迟隐藏限制；（2）用单个 β 代表所有层级——L1/L2/shared memory 与 HBM 差 3-4 个数量级；（3）所有浮点运算同质——逐元素运算走 CUDA Core，吞吐低 4-8 倍。

第二步：Hong-Kung 定理的适用边界。它隐藏了常数因子（小矩阵时常数优化比渐近改进更重要），假设单一快慢存储层级（多级存储需要 Ballard et al. 2012 的扩展），且只适用于确定性算法。

第三步：FlashAttention 和 RingAttention 的隐含前提。FlashAttention 要求 $d \le M \le Nd$；若 $M < d$，分块策略失效。RingAttention 要求通信与计算可以完全重叠，实际中 NCCL 延迟和 InfiniBand 带宽共享往往使 $P_{\max}$ 远低于理论值。

本节要拿走的一件事：本章每个模型都有一个「适用区间」，超出区间时应转向更精细的工具（层次化 Roofline、多级 I/O 下界、实测 profiling），而不是强行套用渐近结论。

直觉

本章前五节建立了四个数学模型：Roofline（§6.1）、Hong-Kung I/O 下界（§6.2）、FlashAttention I/O 分析（§6.3）、RingAttention 通信分析（§6.4），并分析了它们在代际演化中的动态（§6.5）。每个模型都有令人印象深刻的「精确性」：脊点精确到 FLOP/byte，I/O 下界精确到 $\Theta(\cdot)$，FlashAttention 的加速比精确到 $\Theta(M/d^2)$。

但精确性是条件性的——每个模型都依赖一组隐含假设，在假设成立的范围内它们是严格的，一旦跨出这个范围，它们的预测就会系统性地偏乐观，有时偏差高达一到两个数量级。§6.6 专门梳理这四个模型的「失效边界」，目的不是削弱它们的价值，而是建立「精确使用」的元认知——知道什么时候该用这些模型，什么时候该停止套用并转向更精细的工具。

本节的结构不是「概念链」（没有新的数学概念推出下一个），而是「失效根源分析」——按规范文件的说明，改写为「为什么假设失效的四类数学根源」。

根源一：Roofline 的三个隐含假设——以及它们失效的量级

假设一：完美流水（perfect overlap）。Roofline 的上界 $P \le \min(P_{\text{peak}}, I \cdot \beta)$ 隐含一个前提：内存访问和计算单元可以「同时跑满」——内存在搬数据的同时，计算单元在处理上一批数据，两者完全重叠（pipeline）。

现实中这需要「足够多的 outstanding memory requests 在途」来填满 HBM 访问延迟（约 500–800 ns）。HBM 的理论峰值带宽是靠高度并发的内存访问「叠加」实现的。如果 kernel 的工作量太小（例如 batch size = 1 的小矩阵乘，总数据量只有几 KB），或者内存访问模式导致 bank conflict、shared memory 延迟，实际带宽利用率可能只有峰

值的 30–50%。在这种情况下，式 (6.1.2) 的「内存受限上界 $I \cdot \beta$」高估了真实可达性能。

假设二：单一带宽 β。标准 Roofline 用 HBM 带宽这一个数字代表「内存速度」，忽略了 GPU 存储层级的巨大差异：

- L1 cache / shared memory：约 10–20 TB/s（比 HBM 快 3–6 倍）
- L2 cache：约 3–5 TB/s（与 HBM 同量级但略快）
- HBM：约 3 TB/s（H100）

对于 L1/shared memory 访问密集的 kernel（如利用 shared memory 消除 bank conflict 的 GEMM），真实性能可能超出 Roofline 的 HBM 上界——因为它用的是更快的存储层级。层次化 Roofline（hierarchical Roofline）通过为每个存储层级分别画一条 Roofline 来解决这个问题，但代价是引入更多参数，分析复杂度成倍增加。

假设三：浮点运算同质。P_{peak} 通常指 Tensor Core 的矩阵乘峰值（H100 的 989 TFLOPS TF32）。但大模型 kernel 中大量使用「逐元素运算」（element-wise operations），如 softmax 中的 exp（走 CUDA Core 的特殊函数单元，SFU），LayerNorm 中的 $\sqrt{\cdot}$，以及各种激活函数。这些运算的吞吐量通常比 Tensor Core 低 4–8 倍。对于 softmax + LayerNorm 占比较高的 kernel（如早期未融合的 Transformer 实现），真实峰值算力远低于 P_{peak}，Roofline 的「计算受限上界」严重高估。

根源二：Hong-Kung 定理的三个适用边界——$\Omega(\cdot)$ 隐藏了什么

边界一：渐近下界的常数因子不可忽略。定理 6.2（式 (6.2.8)）给出 $T_{I/O} = \Omega(n^3/\sqrt{M})$，$\Omega(\cdot)$ 符号隐藏了常数。Irony、Toledo 和 Tiskin（2004）精确化了这个常数：分块 GEMM 的最优 I/O 约为 $2\sqrt{3}n^3/\sqrt{M}$，而 cuBLAS 等实现的常数约为 2–4（由于 warp 调度、寄存器压力、prefetch 策略等实现细节）。

对小矩阵（$n \leq 512$），常数的差异可能比渐近阶更重要——两个「都是 $O(n^3/\sqrt{M})$」的算法，实际 I/O 可能相差数倍。在 $n \leq 256$ 的情况下，甚至直接用无分块的朴素三重循环（常数极小，但渐近更差）都可能比分块 GEMM 有更好的实际性能，因为分块的 overhead 在小规模下不可忽略。

边界二：单一层级假设。Hong-Kung 定理的「两级存储」假设——一快一慢——无法直接推广到现代 GPU 的 4 级存储（寄存器、L1/shared、L2、HBM）。每个层级之间的带宽差异高达数倍到数量级，且访问模式（顺序 vs 随机）对实际带宽有巨大影响。

Ballard 等（2012）扩展了 Hong-Kung 框架到多级存储，为每一对相邻存储层级分别建立 I/O 下界，并分析了它们的「和」作为总 I/O 成本。这是理论上正确的做法，但工程上需要对每级存储的带宽做精确测量，数学模型的复杂度也随层级数指数增长。

边界三：仅适用于确定性算法。Hong-Kung 框架假设计算图是固定的 DAG，每个节点的执行顺序确定。对于随机稀疏注意力（如 BigBird、Longformer 中的随机注意力块）、动态计算图（如混合专家模型 MoE 的动态路由），访问模式是随机的，可能允许某些情况下绕过确定性的下界。这类场景需要概率论意义上的 I/O 下界，是开放的研究问题。

根源三：FlashAttention I/O 分析的三个前提——边界案例

FlashAttention 的式 (6.3.18) 的 $\Theta(N^2d^2/M)$ 复杂度依赖 $d \leq M \leq Nd$ 的假设范围。两个边界：

- $M < d$（**SRAM 容量极小**）：此时连一个 Q 行都无法放入 SRAM（每行 d 个元素），分块策略无从施展，复杂度退化到 $\Theta(N^2)$，FlashAttention 的优势消失。这在实践中极少发生（H100 的 SRAM 远大于 $d = 128$ 的大小），但在某些极端嵌入式场景或线性注意力变体（head dimension $d = 16$）中需要注意。
- $M \geq Nd$（**SRAM 足够大，可容纳全部 K, V**）：此时整个 K, V 矩阵可以一次性载入 SRAM，Attention 计算完全在 SRAM 内完成，I/O 降至 $\Theta(Nd)$（只需搬运 Q, K, V, O），FlashAttention 的分块优势消失——因为根本没有分块的必要。在近未来的极端 SRAM 密集型芯片上，这个条件可能成立。

FlashAttention 论文（Dao et al. 2022，Theorem 2）同时给出了下界 $\Omega(N^2d^2/M)$，证明在 $d \leq M \leq Nd$ 范围内不存在 I/O 更低的精确 Attention 算法——FlashAttention 在这个范围内是渐近最优的。超出这个范围，问题的性质改变，需要重新建立对应的下界理论。

根源四：RingAttention 通信分析的三个前提——理论与实测的差距

RingAttention 的 $P_{\max}$ 推导依赖三个理想化假设，而实际部署中这三个假设都可能偏离：

前提一：全双工无竞争通信。推导假设 NVLink 提供 β_{NVLink} 的全双工点对点带宽，且环形传输中各对相邻卡之间的链路没有竞争。实际的 NVLink 拓扑（NVSwitch 交换机），在大规模（如 256 GPU）时，多条通信路径可能共享同一交换机端口，实际单链路带宽低于理论峰值。

前提二：计算与通信完全重叠。完全重叠需要 CUDA Stream 和 NCCL 的精确协调：计算 kernel 和通信操作分别调度在不同 CUDA Stream 上，且硬件支持真正的并发。NCCL 通信本身有约 10–50 μs 的启动延迟，对于短序列（N/P 很小），通信时间主要由延迟而非带宽决定，重叠条件的「带宽版」分析失效，需要改用「延迟版」分析。

前提三：序列均匀分布。因果注意力（causal attention）中，序列的早期 token 参与的注意力计算比晚期 token 少（因果掩码使早期 token 只能关注更少的键），导致各卡计算负载不均衡。负载不均衡意味着某些卡等待其他卡完成，计算无法完全掩盖通信，实际 $P_{\max}$ 低于理论值。Striped Attention（Brandon et al. 2023）通过在序列维度交错分配 token（而非连续分配）来均衡负载，是解决这一问题的工程变体。

整条链的回头看

```
Roofline 假设：完美流水 + 单一带宽  + 运算同质
  ↓ 失效时：延迟隐藏不足 → 实际带宽 <；SFU 运算 → 有效算力 <<P_peak
Roofline 失效边界                    ←  小 kernel、逐元素密集 kernel、多级访问模式

Hong-Kung 假设：单一快慢层级 + 渐近视角 + 确定性算法
  ↓ 失效时：小矩阵 → 常数项主导；多级 → 需 Ballard 扩展；随机 → 下界失效
Hong-Kung 失效边界                   ←  小规模、多级存储、随机访问算法

FlashAttention 假设：d  M  Nd
  ↓ 失效时：M <d → 退化至 Θ(N²)；M  Nd → FlashAttn 优势消失
FlashAttention 失效边界              ←  极小 head dim 或极大 SRAM
```

```
RingAttention 假设：全双工无竞争 + 完全重叠 + 均匀序列
  ↓ 失效时：NVSwitch 竞争 → 带宽降低；短序列 → 延迟主导；causal → 负载不均
RingAttention 实测 P_max << 理论值   ←  需 profiling 工具（Nsight Systems）实测
```

这条「失效边界链」揭示了一个深层事实：**本章的四个模型共享同一类缺陷——它们都在抽象时丢弃了「常数」（延迟、多级、负载不均衡），而这些常数在实际工程中恰恰是最重要的细节**。渐近分析给出方向，常数决定是否能到达目的地。工程师的职责不是证明 $\Theta(\cdot)$ 定理，而是在渐近正确的方向上，把常数做到最小——这需要 profiling、实测，以及对每个模型适用边界的清醒认知。

严格

6.6.1 Roofline 模型的假设局限

假设一：完美重叠（perfect overlap）。Roofline 模型假设内存访问与计算可以完全流水，实际 GPU 中存在 **latency hiding** 限制：若 outstanding memory requests 不足以填满内存子系统的延迟（约 500–800 ns on HBM），则 kernel 会出现 latency-bound 而非纯 bandwidth-bound 的停顿。此时实际带宽利用率可能显著低于 β，Roofline 的预测偏于乐观。

假设二：单一带宽值。标准 Roofline 用单个 β 代表所有内存访问，忽视了 L1 cache、L2 cache、shared memory 与 HBM 之间 **3–4 个量级**的带宽差异。层次化 Roofline（hierarchical Roofline）可缓解此问题，但引入更多硬件参数，分析复杂度成倍增加。

假设三：FP 运算同质。峰值算力 P_{peak} 通常指 Tensor Core 的矩阵乘算力，而逐元素运算（softmax 中的 exp、LayerNorm 等）需走 CUDA Core，吞吐量低 4–8×。对于逐元素密集的 kernel，实际峰值远低于 P_{peak}。

6.6.2 Hong-Kung 下界的适用范围

限制一：渐近下界，常数不可忽略。定理 6.2 中的 $\Omega(\cdot)$ 隐藏了常数因子。分块 GEMM 的实际 I/O 为 $2\sqrt{3}\,n^3/\sqrt{M}$（Irony et al. 2004 给出），而 cuBLAS 的实现常数约为 2–4，因此对小矩阵（$n \le 512$），香港下界的渐近优化意义有限，常数项的优化往往更关键。

限制二：顺序存储层级假设。Hong-Kung 定理假设单一快速存储与单一慢速存储，现代 GPU 的存储层级为 **Register → L1/Shared Memory → L2 Cache → HBM**，各级之间存在非均匀延迟与带宽。真实 I/O 下界应针对多级存储层级分别建立，见 Ballard 等（2012）对多级层次的扩展。

限制三：仅适用于确定性算法。对于随机化算法（如随机稀疏注意力），Hong-Kung 框架需要修正，访问模式的随机性可能允许绕过某些下界。

6.6.3 FlashAttention I/O 分析的隐含前提

FlashAttention 的 $\Theta(N^2d^2/M)$ 复杂度依赖 $d \le M \le Nd$ 的假设。若 $M < d$（极端 SRAM 受限），则整个 head 无法驻留 SRAM，分块策略失效，复杂度退化至 $\Theta(N^2)$。反之若 $M \ge Nd$（即整个 K, V 可放入 SRAM），则标准 attention 的第 1-3 步均可在 SRAM 内完成，I/O 降至 $\Theta(Nd)$，FlashAttention 的优势消失。

FlashAttention 的最优性定理（Dao et al. 2022, Theorem 2）同时给出了式 (6.3.18) 的下

界 $\Omega(N^2d^2/M)$，证明**不存在**在所有 M 范围内均优于 FlashAttention 的精确 attention 算法——它在渐近意义下已是最优的。

6.6.4 RingAttention 的通信模型假设

本章 §6.4 的推导假设：（1）NVLink 为**全双工、无竞争**的点对点通信；（2）计算与通信可以**完全重叠**（需要硬件对计算流与通信流的并发调度支持，如 CUDA Stream 与 NCCL 的集成）；（3）序列在设备间均匀分布（causal masking 下负载不均衡，Striped Attention 等变体通过重排 token 索引加以缓解）。

在实际部署中，由于 NCCL 通信延迟、InfiniBand 带宽共享等因素，可达式 (6.4.12) 的 $P_{\max}$ 可能远低于理论值，需通过 profiling 工具（如 NVIDIA Nsight Systems）实测通信/计算重叠率。

---

§6.7 参考文献

▶这一节在讲什么

为什么这份文献表按此顺序排列？前两篇（Hong-Kung 1981、Williams et al. 2009）是本章两条主线的理论奠基；之后的 Dao 等人系列给出 FlashAttention 及其改进的原始证明；Liu et al. 2024 对应 RingAttention；Irony et al. 2004 和 Ballard et al. 2012 给出 I/O 下界的精确常数与多级推广。最后几篇提供硬件规格来源、在线 softmax 算法出处，以及 Striped Attention 等工程变体的参考。凡本章引用数值或引理，均可在此找到一手来源。

本节要拿走的一件事：每个推论的数学权威出处都在这里——遇到疑问时，先查对应原文，而不是依赖二手综述。

1. **Hong, J.-W., & Kung, H. T.** (1981). I/O complexity: The red-blue pebble game. Proceedings of the 13th Annual ACM Symposium on Theory of Computing (STOC), 326–333. https://doi.org/10.1145/800076.802486
2. **Williams, S., Waterman, A., & Patterson, D.** (2009). Roofline: An insightful visual performance model for multicore architectures. Communications of the ACM, 52(4), 65–76. https://doi.org/10.1145/1498765.1498785
3. **Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C.** (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. Advances in Neural Information Processing Systems (NeurIPS) 35. https://arxiv.org/abs/2205.14135
4. **Dao, T.** (2023). FlashAttention-2: Faster attention with better parallelism and work partitioning. International Conference on Learning Representations (ICLR 2024). https://arxiv.org/abs/2307.08691
5. **Liu, H., Zaharia, M., & Abbeel, P.** (2024). Ring attention with blockwise transformers for near-infinite context. International Conference on Learning Representations (ICLR 2024). https://arxiv.org/abs/2310.01889
6. **Irony, D., Toledo, S., & Tiskin, A.** (2004). Communication lower bounds for

distributed-memory matrix multiplication. Journal of Parallel and Distributed Computing, 64(9), 1017–1026. https://doi.org/10.1016/j.jpdc.2004.03.021

7. **Ballard, G., Demmel, J., Holtz, O., & Schwartz, O.** (2012). Graph expansion and communication costs of fast matrix multiplication. Journal of the ACM, 59(5), 1–23. https://doi.org/10.1145/2344422.2344424
8. **Aggarwal, A., & Vitter, J. S.** (1988). The input/output complexity of sorting and related problems. Communications of the ACM, 31(9), 1116–1127. https://doi.org/10.1145/48529.48535
9. **Milakov, M., & Gimelshein, N.** (2018). Online normalizer calculation for softmax. arXiv preprint. https://arxiv.org/abs/1805.02867
10. **NVIDIA Corporation.** (2022). NVIDIA H100 Tensor Core GPU Architecture White Paper. https://resources.nvidia.com/en-us-tensor-core
11. **NVIDIA Corporation.** (2025). NVIDIA B200 GPU Product Specifications. https://www.nvidia.com/en-us/data-center/b200/
12. **Brandon, W., Mishra, M., Nrusimha, A., Panda, R., & Ragan-Kelley, J.** (2023). Striped attention: Faster ring attention for causal transformers. arXiv preprint. https://arxiv.org/abs/2311.09431
13. **Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., …& Polosukhin, I.** (2017). Attention is all you need. Advances in Neural Information Processing Systems (NeurIPS) 30. https://arxiv.org/abs/1706.03762
14. **Demmel, J., Grigori, L., Hoemmen, M., & Langou, J.** (2012). Communication-optimal parallel and sequential QR and LU factorizations. SIAM Journal on Scientific Computing, 34(1), A206–A239. https://doi.org/10.1137/080731992

本章与主本第六章相互镜像：主本讲产业逻辑（HBM 为何成为万亿生意、CoWoS 封装与超大规模云厂商（Hyperscaler）采购），本章专注数学骨干——不等式如何推、下界如何建立、算法如何在严格意义上最优。两章合读方能得到完整的「硬件感知」图景。

数学卷 · 第 7 章: 后变换器（Transformer）时代的数学——状态空间模型、线性注意力与扩散语言模型的数理基础

说明: 本章为《算法的低语 · 数学卷》的第七章, 专注严格数学推导。与主本第七章的架构评述相互补充, 读者需具备线性代数、常微分方程与概率论的基础知识。本章中所有内联公式使用 · 记号, 展示公式使用独立段落。

---

§7.1 状态空间模型的连续形式

▶这一节在讲什么

为什么要用「状态空间」描述序列？序列模型面临一个根本矛盾：历史无限长，但内存和算力有限。状态空间模型的答案是：把无限长的历史压缩进固定大小的向量 $h(t)$，并用一条微分方程来维护它。

第一步：线性递推即连续极限。工程师熟悉的指数加权滑动平均 $h_t = \alpha h_{t-1} + \beta x_t$，令时间步长 $\Delta t \to 0$，就自然变成一阶线性 ODE $h'(t) = Ah(t) + Bx(t)$。连续化不是魔法，是「让步长无穷小」的极限。

第二步：闭式解揭示卷积结构。ODE 的解是 $h(t) = e^{At}h(0) + \int_0^t e^{A(t-s)}Bx(s)\,ds$，输出 $y(t) = Ch(t)$ 因此等于输入 x 与卷积核 $K(t) = Ce^{At}B$ 的卷积。这一结构使训练可以用 FFT 在 $O(L \log L)$ 内完成。

第三步：选什么样的 A 至关重要。A 的特征值决定了系统的记忆长度和稳定性；什么样的 A 能让状态 $h(t)$ 最优地压缩历史，正是下一节 HiPPO 框架要回答的核心问题。

本节要拿走的一件事：状态空间模型的本质是「用一条线性 ODE 把无限历史压进固定向量」——卷积核 $K(t) = Ce^{At}B$ 将这一想法的所有信息都藏在矩阵 A 里。

直觉

序列建模的核心矛盾在于：历史序列可以无限长，但模型的计算和内存必须有限。循环神经网络的解法是维护一个「缓存状态」h_t，每一步用递推更新它；Transformer 的解法是存下所有历史并用注意力机制查询。状态空间模型走的是第一条路，但给它加上了严格的数学基础：用一阶线性常微分方程来刻画状态如何演化。这与工程领域的「指数加权滑动平均」完全同源——把时间步长推向零，滑动平均就变成了连续 ODE。

本节出现五个核心概念：**线性时不变系统（LTI）**、**一阶 ODE**、**矩阵指数 e^{At}**、**卷积核 $K(t)$**、**传递函数 $G(s)$**。它们看似是控制理论的旧名词堆砌，实际上沿着一条严密的逻辑链一个推出一个。理解这条链，是理解为什么「SSM 能以 $O(L \log L)$ 训练」的钥匙。下面把这条链一环环铺开。

第一环：从「无限历史 → 固定状态」的矛盾出发——为什么需要 ODE

任何序列模型都面对同一个悖论：真正的语言依赖可以跨越数千 token（一段代码的变量声明和其最后一次使用之间），但计算机的内存有限。RNN 通过引入一个递推状态 $h_t \in \mathbb{R}^N$ 来解决这个问题——不管序列多长，状态维度 N 固定。

最简单的固定状态递推是 EWMA（指数加权滑动平均）：

$$h_t = \alpha h_{t-1} + \beta x_t \tag{7.1.1}$$

式 (7.1.1) 已经包含了 SSM 的全部核心思想：用旧状态的衰减加上新输入的注入来维护历史摘要。SSM 的贡献不是发明了这个思路，而是**给它加上连续时间的数学语言**，从而打开了一扇门：可以用 ODE 理论的整套工具来分析和设计 A。

连续化的步骤非常直接：令时间步长 $\Delta t \to 0$，设 $\alpha = 1 + A\Delta t$，$\beta = B\Delta t$，递推的

差商就收敛到导数：

$$\frac{h_t - h_{t-1}}{\Delta t} \to h'(t) = Ah(t) + Bx(t) \tag{7.1.2}$$

式 (7.1.2) 就是 LTI 系统的基本方程。「连续时间」不是神秘的物理假设，只是「步长无穷小的极限」的数学语言。

第二环：LTI 假设——为什么是「线性时不变」

方程 $h'(t) = Ah(t) + Bx(t)$ 满足两个关键属性，合称 LTI：

1. **线性**：若 $h_1(t)$ 是 $x_1(t)$ 的响应，$h_2(t)$ 是 $x_2(t)$ 的响应，则 $c_1h_1 + c_2h_2$ 是 $c_1x_1 + c_2x_2$ 的响应。这保证了叠加原理成立，从而允许频域分析。
2. **时不变**：矩阵 A, B, C 不依赖于时间 t。这意味着系统在 $t = 0$ 和 $t = 1000$ 时对相同输入给出相同响应——卷积核 $K(t-s) = Ce^{A(t-s)}B$ 只依赖时间差，不依赖绝对时刻。

在式 (7.1.2) 的 LTI 假设下，其工程意义是巨大的：**任何 LTI 系统的输出都可以写成输入与某个固定卷积核的卷积**。卷积可以用 FFT 在 $O(L \log L)$ 内完成——这是 SSM 能高效训练的根本原因。放弃 LTI（如 Mamba 的选择性机制）就失去这个保证，需要并行扫描来补救（见 §7.4）。

第三环：矩阵指数 e^{At}——ODE 的闭式解心脏

一旦写出 $h'(t) = Ah(t) + Bx(t)$，就可以用标准 ODE 技术求解。左乘积分因子 e^{-At}，两边对时间积分：

$$h(t) = e^{At}h(0) + \int_0^t e^{A(t-s)}Bx(s)\,ds \tag{7.1.3}$$

其中矩阵指数 $e^{At} = \sum_{k=0}^{\infty} \frac{(At)^k}{k!}$ 是普通指数函数在矩阵世界的推广。它的稳定性条件：若 A 的所有特征值满足 $\mathrm{Re}(\lambda_i) < 0$，则 $\|e^{At}\| \to 0$（系统渐近稳定）。这个条件将在 §7.2 中以 HiPPO 矩阵的方式被满足。

矩阵指数的几何直觉：e^{At} 是把初始状态 $h(0)$ 沿着 A 定义的「流场」传播 t 秒后的到达位置。若 A 有纯虚特征值，流场是旋转（振荡记忆）；若 A 有负实特征值，流场是螺旋收缩（指数衰减记忆）。HiPPO 矩阵选择了能「均匀覆盖所有频率」的特征值分布。

第四环：卷积核 $K(t) = Ce^{At}B$——把 ODE 翻译成信号处理语言

将式 (7.1.3) 代入输出方程 $y(t) = Ch(t)$，设零初始状态 $h(0) = 0$：

$$y(t) = \int_0^t Ce^{A(t-s)}Bx(s)\,ds = (K * x)(t), \quad K(t) = Ce^{At}B \tag{7.1.4}$$

这一步是 SSM 的关键洞察之一：**连续 LTI 系统的输入-输出关系等价于输入与某个固定卷积核的卷积**。卷积核 $K(t)$ 完全由参数矩阵 A, B, C 决定，与输入 x 无关。

这个发现的工程意义：卷积核 K 可以预先计算，然后对整条序列批量施加。离散化

后（§7.3），这变成离散卷积 $y_k = \sum_{j=0}^{k} K_{k-j} x_j$，可以用 FFT 在 $O(L \log L)$ 内完成，而不是 $O(L \cdot N)$ 的逐步递推。训练速度的核心差异就在这里。

注意 $K(t)$ 完全「藏在」矩阵 A 里：A 的特征结构决定了 e^{At} 的衰减速率和振荡频率，从而决定了系统能记住多久、对哪些频率的历史最敏感。**选什么样的 A 才能「最优地」压缩历史**——这正是下一节 HiPPO 框架的任务。

第五环：传递函数 $G(s)$——频域视角打通时域与频域

对系统方程做拉普拉斯变换（设零初始条件），令 s 为复频率变量：

$$G(s) = C(sI - A)^{-1}B \tag{7.1.5}$$

这是有理分式，分母的零点是 A 的特征值（系统极点）。传递函数的意义：$G(s)$ 完整描述了系统在频率 s 处的增益和相位——$|G(i\omega)|$ 就是频率 ω 的正弦输入在通过系统后的幅度放大比例。

式 (7.1.5) 的传递函数视角在 S4 的高效计算中至关重要（§7.3）：卷积核 $K(t)$ 的离散 z 变换等于在 FFT 点上对传递函数 $G(z) = C(zI - \bar{A})^{-1}\bar{B}$ 求值。当 A 具有 DPLR 结构时，$(zI - \bar{A})^{-1}$ 可以用 Woodbury 恒等式展开，把求值问题化为 Cauchy 矩阵向量积，从 $O(N^2 L)$ 降至 $\tilde{O}(N + L)$。

整条链的回头看

```
EWMA 递推 h_t = h_{t-1} + x_t
  ↓ 令 Δt→0（步长无穷小极限）
LTI 系统方程 h'(t) = Ah(t) + Bx(t)        ← 连续时间框架的基础
  ↓ LTI 保证叠加原理 → 可做频域分析
矩阵指数 e^{At} 给出 ODE 闭式解           ← 状态演化的核心算子
  ↓ 代入输出方程 y = Ch
卷积核 K(t) = Ce^{At}B                      ← y = K * x，打通卷积结构
  ↓ 卷积 → FFT → O(L log L) 训练
传递函数 G(s) = C(sI-A)^{-1}B               ← 频域视角，为 S4 的 Cauchy 加速铺路
  ↓ 问题归结：选什么 A 最优压缩历史？
→ HiPPO 框架（§7.2）
```

这条链揭示了一个深层事实：**SSM 的 $O(L \log L)$ 训练效率不是工程技巧堆砌出来的，而是 LTI 假设 + 卷积结构 + FFT 三者的数学必然**。矩阵 A 的选取（由 HiPPO 回答）和离散化方案（由 §7.3 回答）是这条链上仅剩的两个自由度。

7.1.0 工程直觉引子：从「滑动窗口 + 缓存状态」走到连续 ODE

> 在跳进连续时间、一阶 ODE 和勒让德多项式之前，我们先用一个工程老兵都熟悉的画面开场。这个引子并不严格，但能为后面所有那些「凭空出现」的记号预存一个心锚。

设想这样一个年轻的服务。它要实时处理一条无限长的 token 流，但只允许维护一个**固定大小的缓存状态** $h \in \mathbb{R}^N$。每来一个新输入 x_t，服务需要决定：旧状态保留多少，新输入注入多少。最自然的写法是一条**带遗忘因子的线性递推**：

$$h_t = \alpha\, h_{t-1} + \beta\, x_t, \qquad y_t = c^\top h_t, \tag{7.0}$$

其中 $\alpha \in (0,1)$ 控制「旧状态衰减多快」，β 控制「新输入注入多重」。QA 老兵一眼就能认出：这就是监控系统里近 30 年被反复使用的 **指数加权滑动平均（EWMA）**——Prometheus 的 `rate()`、Kafka 的 consumer lag 告警、高频交易系统的 tick-by-tick 指标都用这条递推。它有三个友好的性质：

- **记忆有限**：远古输入的权重衰减为 α^k，k 越大越被遗忘；
- **状态恒定**：不论流多长，h_t 始终为 N 维向量；
- **线性叠加**：两条输入流的状态可以独立计算后相加。

连续化只需一步跳跃。如果我们不再假设 token 是按 $\Delta t = 1$ 均匀到达，而是令 $\Delta t \to 0$，记 $\alpha = 1 + A\Delta t$、$\beta = B\Delta t$，那么

$$\frac{h_t - h_{t-1}}{\Delta t} = A\, h_{t-1} + B\, x_t. \tag{7.1.7}$$

取极限就得到

$$h'(t) = A\, h(t) + B\, x(t), \qquad y(t) = C\, h(t), \tag{7.1.8}$$

这正是后面 §7.1.1 要介绍的一阶线性 ODE。换句话，**连续状态空间模型不是什么新东西，它是「带衰减的滑动窗口」在 $\Delta t \to 0$ 下的连续极限。**全章将反复使用的**连续信号、状态转移矩阵、双线性变换**，都只是在问一个问题：**应该选什么样的 A，才能让这个「无限长流 · 固定状态」的缓存尽可能多地保留历史信息？**§7.2 HiPPO 的答案会是：选一个与勒让德多项式正交的 A。

> **一句话总结**：接下来所有最奇怪的记号——e^{At}、勒让德多项式、双线性变换——都只是「滑动窗口 + 缓存状态」这个工程原型的**连续极限版本**。请带着这个心锚走进后面的推导。

7.1.1 线性时不变系统的基本方程

状态空间模型 (State Space Model, SSM) 的出发点是经典的线性时不变 (Linear Time-Invariant, LTI) 系统。给定一维输入信号 $x(t) \in \mathbb{R}$ 和 N 维隐状态 $h(t) \in \mathbb{R}^N$, 系统方程为

$$h'(t) = Ah(t) + Bx(t), \qquad y(t) = Ch(t), \tag{7.1}$$

其中 $A \in \mathbb{R}^{N\times N}$ 是状态转移矩阵,$B \in \mathbb{R}^{N\times 1}$ 是输入投影向量,$C \in \mathbb{R}^{1\times N}$ 是输出投影向量。此方程描述了隐状态如何随时间演化以及如何从隐状态恢复输出。

在信号处理与控制理论中, 式 (??) 已有数十年研究历史。将其引入序列建模的关键洞察在于: 若选取合适的矩阵 A, 则 $h(t)$ 能够以固定内存代价维护对任意长历史的压缩表示。LTI 性质保证了卷积结构, 而卷积结构则保证了 $O(L \log L)$ 的训练效率。

7.1.2 闭式解的推导

方程 (??)（即式 (7.1.8)）是一阶线性常微分方程组。将其改写为

$$h'(t) - Ah(t) = Bx(t). \tag{7.1.10}$$

左乘积分因子 e^{-At}, 得

$$\frac{d}{dt}(e^{-At}h(t)) = e^{-At}Bx(t). \tag{7.1.11}$$

在 $[0, t]$ 上积分, 并利用初始条件 $h(0)$,

$$e^{-At}h(t) - h(0) = \int_0^t e^{-As}Bx(s)\,ds. \tag{7.1.12}$$

两边乘以 e^{At}, 得到**闭式解**:

$$\boxed{h(t) = e^{At}h(0) + \int_0^t e^{A(t-s)}Bx(s)\,ds.} \tag{7.2}$$

其中 e^{At} 是矩阵指数 (matrix exponential), 定义为 $e^{At} = \sum_{k=0}^{\infty} \frac{(At)^k}{k!}$。

稳定性条件: 若 A 的所有特征值 λ_i 满足 $\mathrm{Re}(\lambda_i) < 0$, 则 $\|e^{At}\| \to 0$ 当 $t \to \infty$, 系统渐近稳定。HiPPO 矩阵的特征值恰好位于左半复平面, 保证了状态的有界演化。

7.1.3 卷积视角

将闭式解代入输出方程, 并设 $h(0) = 0$(零初始状态), 得

$$y(t) = Ch(t) = \int_0^t Ce^{A(t-s)}Bx(s)\,ds = (K * x)(t), \tag{7.3}$$

其中卷积核定义为

$$K(t) = Ce^{At}B. \tag{7.4}$$

这一卷积视角至关重要: 在离散化之后, 若参数 A, B, C 不依赖于输入 (LTI 假设), 则整个序列的输出可通过**单次全局卷积**在频域以 $O(L \log L)$ 时间内计算, 其中 L 是序列长度。这也是 S4 等模型实现高效训练的核心。

注记: 由式 (7.2) 和式 (??) 知，卷积核 $K(t)$ 完全由矩阵 A 的特征结构决定。若 A 具有负实部的特征值, 则 $K(t)$ 随时间指数衰减; 若 A 有纯虚特征值, 则 $K(t)$ 表现为振荡记忆。HiPPO 框架 (§ 7.2) 将系统地回答: 什么样的 A 能够最优地压缩历史信息?

7.1.4 频域分析与传递函数

对式 (??) 作单边拉普拉斯变换 (设零初始条件), 令 s 为复频率变量:

$$sH(s) = AH(s) + BX(s), \quad Y(s) = CH(s), \tag{7.1.16}$$

解出 $H(s) = (sI - A)^{-1}BX(s)$, 故传递函数为

$$G(s) = \frac{Y(s)}{X(s)} = C(sI - A)^{-1}B. \tag{7.5}$$

这是有理分式, 分母的零点即为矩阵 A 的特征值 (系统极点)。S4 中利用此结构, 在 z 域上用 DPLR 分解将 $(zI - \bar{A})^{-1}$ 的计算化为柯西 (Cauchy) 矩阵向量积 (见 §7.3)。

---

§7.2 HiPPO 框架的严格推导

▶这一节在讲什么

为什么要研究「最优压缩历史」? 上一节论证了状态向量 $h(t)$ 能用一阶 ODE 维护, 但没有告诉你该持续配置哪个 A。HiPPO 的回答是: 让 $h(t)$ 成为历史信号在某组正交多项式基上的最优投影系数。

第一步: 把压缩问题化为投影问题。与其说「记住历史」, 不如说「用 N 个坐标表示历史在某个函数空间里的最优近似」。勒让德多项式在 $[-1, 1]$ 上关于均匀测度正交, 确保系数之间不干扰、信息不冗余。

第二步: 不同测度导出不同的 A。对历史序列的「最优源」是个选择: 关注近期 (滑动窗口均匀测度) 就得到 LegT 矩阵; 关注全历史 (全局缩放均匀测度) 就得到 LegS 矩阵。两者都能用小于 $N \times N$ 的小矩阵表示。

第三步: A 矩阵的具体形式来自多项式递推。利用勒让德多项式的导数展开公式, 将对 t 求导转化为内积系数的线性组合, 最终得到 $h'(t) = A_{\text{HiPPO}}h(t) + Bx(t)$ 的显式表达式。

本节要拿走的一件事: HiPPO 矩阵不是猜出来的, 而是「要让固定大小的向量 $h(t)$ 在每一时刻都对历史信号给出最小二乘最优投影」这一要求强制推导出来的。

直觉

上一节确立了状态空间模型的基本方程, 但没有回答一个关键问题: 应该选什么样的矩阵 A? 随意选一个 A 并不能保证状态 $h(t)$ 对历史信息的有效压缩。HiPPO 框架把这个问题转化为一个明确的最优化目标。

本节出现四个核心概念: **测度与投影目标、Legendre 多项式正交基、LegT 滑动窗口测度、LegS 全局缩放测度**。它们看似是逃脱线代的函数分析奖山, 实际上沿着一条严密的逻辑链一个推出一个。理解这条链, 是理解「HiPPO 矩阵为什么不是猜出来的」的关键。下面把这条链一环环铺开。

第一环: 压缩问题 → 投影问题——把「记住历史」转化为数学问题

「在固定大小的向量 $h(t)$ 里尽量多地保存历史信号」——这个目标模糊到无法优化。HiPPO 的第一个局面是把它转化为一个数学上可操作的问题：**在当前时刻 t，找一个属于某个 N 维函数空间的函数 $\hat{x}_t(s)$，使它在测度 $\mu^{(t)}$ 下最优近似历史信号 $x|_{[0,t]}$**。

这个转化的意义是深刻的：它把一个模糊的「历史压缩」问题，变成了一个几何上有明确意义的「在 L^2 空间里做最优投影」的问题。最优投影是可以用正交展开精确计算的。语言留得一口洋气：**与其说「记住历史」，不如说「用 N 个坐标表示历史在某个函数空间里的最优近似」**。

测度 $\mu^{(t)}$ 的选择至关重要：它决定了”最优”的定义，也就是”最优地记住的是哪些时刻的历史”。LegS 用的是 $[0,t]$ 上的均匀测度，意思是「对所有历史时刻一视同仁」；LegT 用的是长度为 θ 的滑动窗口，意思是「只关注最近 θ 步内的历史」。**测度不同，最优必然不同，导出的 A 矩阵就不同**。

第二环：Legendre 多项式作为正交基——为什么选它

投影需要一组基函数。选择基函数有三个实用要求：

1. **正交性**：$\int P_n P_m \, d\mu = \delta_{nm}$。正交保证不同基向之间不干扰——$N$ 个坐标各自独立承载不同维度的历史信息，不会出现「信息重叠」。
2. **长度有界性**：$\|P_n\|_{L^2} < \infty$。保证投影系数在数学上是有界的。
3. **递推关系**：P_{n+1} 可以用 P_n 和 P_{n-1} 表达。这对后面导出 A 矩阵的推导至关重要——正是通过递推关系，导数才能展开为系数的线性组合。

Legendre 多项式 P_n 在 $[-1,1]$ 上关于均匀测度正交，满足上述三个要求。直觉上：Legendre 多项式捕捉不同级别的函数特征——$P_0 = 1$ 据取平均值，$P_1 = x$ 据取线性趋势，$P_2 = \frac{1}{2}(3x^2-1)$ 据取二次曲率，高阶项据取越来越细的时间结构。初始化为 LegS 的 S4，其 N 个状态维度因此就像是将历史信号的「均均值、线性趋势、二次曲率、……」各存一个，共 N 个层级。

第三环：LegS 测度下导出 A——为什么 $h'(t)$ 满足 ODE

将投影系数写出来：

$$h_n(t) = \frac{1}{t}\int_0^t \tilde{P}_n(s/t)\, x(s)\, ds \tag{7.2.1}$$

其中 $\tilde{P}_n$ 是在 $[0,1]$ 上关于均匀测度归一化的 Legendre 多项式。对 t 求导（用莱布尼茨法则）：

$$h_n'(t) = -\frac{1}{t}h_n(t) + \frac{1}{t}\tilde{P}_n(1)x(t) - \frac{1}{t^2}\int_0^t \frac{s}{t}\tilde{P}_n'(s/t)x(s)\, ds \tag{7.2.2}$$

第三项里就在这里出现了关键步骤：用 Legendre 多项式导数的展开公式 $P_n'(x) = \sum_{k<n,\, n-k\text{ 奇}}(2k+1)P_k(x)$，把被积分中的 $\tilde{P}_n'$ 展开为低阶 $\tilde{P}_k$ 的线性组合，再将 $\frac{1}{t}\int_0^t \tilde{P}_k(s/t)x(s)\, ds$ 识别回 $h_k(t)$。整理下来就得到：

$$h'(t) = \frac{1}{t}\left(A_{\text{LegS}}h(t) + B_{\text{LegS}}x(t)\right) \tag{7.2.3}$$

式 (7.2.3) 的关键洞察：A **矩阵不是猜出来的——它是「投影系数对时间的导数 = 投影系数的某个线性组合 + 新输入的贡献」这个要求强制推导出来的**。几何上，A_{LegS} 是一个下三角占优的矩阵，对角线元素为 $-(n+1)$，下三角元素为 $-\sqrt{(2n+1)(2k+1)}$：这个结构实质上编码了 Legendre 多项式之间的导数耦合关系。

第四环：LegT 对比 LegS——测度如何改变最优历史的定义

LegT 使用宽度为 θ 的滑动窗口测度 $\mu^{(t)} = \frac{1}{\theta}\mathbf{1}_{[t-\theta,t]}ds$。相比 LegS，有两点本质不同：

1. **窗口宽度固定**：无论 t 多大，LegT 只关注最近 θ 步。对应的 A_{LegT}（式 (7.8)）显示了更双线性的上下三角块结构（上三角块带反号，体现窗口滑动时旧信息的遗忘机制）。
2. **测度不随时间缩放**：小窗口 LegT 对近期历史有更高的分辨率，大窗口 LegS 对全局历史给予均等关注。

数学直觉：想象一个预言家，它所拥有的历史记录满足某个测度。LegT 的预言家在「最近一天」的测度下是专家；LegS 的预言家在「过去所有天」的均匀测度下是专家。S4 选择 LegS 初始化，是希望模型对全局历史均匀敏感，尚未为任务调优之前的合理默认。

整条链的回头看

```
压缩历史信号 x|[0,t] 到固定大小的向量 h(t)
  ↓ 数学重述为：在测度 ^{(t)} 下最优 N 维投影
选 Legendre 多项式为基（正交、递推关系）      ← 各坐标独立编码不同级别历史特征
  ↓ 对投影系数 h_n(t) 求 t 导数（莱布尼茨 + Legendre 导数展开）
小结构线性 ODE：h'(t) = (Ah(t) + Bx(t))/t    ← A 是论证出来的而非猜出来的
  ↓ LegS 测度：全局历史均等      LegT 测度：滑动窗口
 A_{LegS}：下三角密华矩阵          A_{LegT}：带反号上三角展开
  ↓ S4 选择 LegS 初始化 → 对全局历史均匀敏感
```

这条链揭示了一个深层事实：**HiPPO 矩阵不是「对效果好的初始化猜测」，而是「用测度下的最小二乘投影等价定义了一个唯一贵点」——A 是这个贵点的直接推论，而测度的选择（LegS vs LegT）则定义了「最优地记住的是什么」**。

> **本节的关键论证链**：(1) 问题陈述——压缩历史信号；(2) 选 Legendre 多项式做基函数（正交、闭式系数）；(3) 推导 LegT/LegS 矩阵 A；(4) 证明所得 A 正是「让有限维状态 $h(t)$ 在每一时刻都对历史的最优最小二乘投影」的那一个。**关键转折点会在每个小节标题后用粗体提示**。

7.2.1 问题陈述: 在线最优投影

HiPPO(High-order Polynomial Projection Operators) 由 Gu et al. (NeurIPS 2020) 提出，其核心问题是:

> 给定连续信号 $x : [0, t] \to \mathbb{R}$, 如何在线地 (online) 将其历史压缩到固定维向量 $h(t) \in \mathbb{R}^N$, 使得 $h(t)$ 以某种意义最优地近似 x 在 $[0, t]$ 上的全部历史?

形式化: 设 $\mu^{(t)}$ 是 $[0, t]$ 上的测度 (measure),$\{g_n\}_{n=0}^{N-1}$ 是关于 $\mu^{(t)}$ 的正交多项式基。定义**投影算子**

$$\mathrm{proj}_t : L^2(\mu^{(t)}) \to \mathrm{span}\{g_0, \dots, g_{N-1}\}, \tag{7.2.4}$$

令 $h_n(t) = \langle x, g_n \rangle_{\mu^{(t)}}$ 为 $x(s)$(视为 $[0,t]$ 上的函数,$s \le t$) 在第 n 个基函数上的内积系数。则 $h(t) = (h_0(t), \dots, h_{N-1}(t))^\top$ 是最优 N 维近似的系数向量。

关键结论: 当测度 $\mu^{(t)}$ 随时间演化时,$h(t)$ 满足线性微分方程 $h'(t) = Ah(t) + Bx(t)$, 矩阵 A 由测度的具体形式决定。

7.2.2 Legendre 多项式的正交性

勒让德多项式 $P_n : [-1,1] \to \mathbb{R}$ 由 Rodrigues 公式定义:

$$P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} [(x^2-1)^n]. \tag{7.2.5}$$

关于标准 Lebesgue 测度 dx 在 $[-1,1]$ 上满足正交性:

$$\int_{-1}^{1} P_n(x) P_m(x)\, dx = \frac{2}{2n+1} \delta_{nm}. \tag{7.6}$$

归一化后的正交基为 $\tilde{P}_n(x) = \sqrt{\frac{2n+1}{2}} P_n(x)$, 满足 $\int_{-1}^{1} \tilde{P}_n \tilde{P}_m\, dx = \delta_{nm}$。

前几阶勒让德多项式为 $P_0(x) = 1, P_1(x) = x, P_2(x) = \frac{1}{2}(3x^2-1)$, 满足递推关系:

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x). \tag{7.7}$$

式 (7.7) 的递推关系是 HiPPO 矩阵推导中的核心工具。

7.2.3 HiPPO-LegT: 平移 Legendre 测度

LegT(Translated Legendre) 使用滑动窗口测度: 对固定窗口大小 $\theta > 0$,

$$\mu^{(t)} = \frac{1}{\theta} \mathbf{1}_{[t-\theta, t]}\, ds \quad \text{(均匀分布在长度为}\theta\text{ 的窗口上)}. \tag{7.2.8}$$

通过变量替换将 $[t-\theta, t]$ 映射到 $[-1,1]$, 令 $u = \frac{2(s-(t-\theta))}{\theta} - 1$, 则系数 $h_n(t) = \sqrt{\frac{2n+1}{\theta}} \int_{t-\theta}^{t} P_n\left(\frac{2(s-t)}{\theta} + 1\right) x(s)\, ds$ 对时间 t 求导, 利用莱布尼茨积分法则和勒让德多项式的递推关系, 可以推导出:

$$h'(t) = A_{\mathrm{LegT}} h(t) + B_{\mathrm{LegT}} x(t), \tag{7.2.9}$$

其中**显式矩阵**为

$$(A_{\mathrm{LegT}})_{nk} = -\frac{1}{\theta} \begin{cases} (2n+1)^{1/2}(2k+1)^{1/2} & \text{若} n \ge k, \\ (-1)^{n-k}(2n+1)^{1/2}(2k+1)^{1/2} & \text{若} n < k, \end{cases} \tag{7.8}$$

以及 $(B_{\mathrm{LegT}})_n = \frac{1}{\theta}(2n+1)^{1/2}(-1)^n$。此处为简洁起见设 $\theta = 1$。

物理意义: 矩阵 A_{LegT} 的下三角块使得低阶系数影响高阶系数, 而上三角块 (反号) 体现了窗口滑动时旧信息的遗忘机制。

7.2.4 引理 (HiPPO-LegS 矩阵的推导)

引理 7.1 (HiPPO-LegS 矩阵的显式形式): 设测度为累积 (缩放 Legendre) 测度:

$$\mu^{(t)} = \frac{1}{t}\mathbf{1}_{[0,t]}\,ds, \tag{7.2.11}$$

即在 $[0,t]$ 上的均匀测度, 随时间整体缩放。则最优投影系数 $h(t)$ 满足的 SSM 矩阵为

$$(A_{\mathrm{LegS}})_{nk} = \begin{cases} -\sqrt{(2n+1)(2k+1)} & \text{若} n > k, \\ -(n+1) & \text{若} n = k, \\ 0 & \text{若} n < k, \end{cases} \tag{7.9}$$

以及 $(B_{\mathrm{LegS}})_n = \sqrt{2n+1}$。

证明:

设归一化基函数 $\tilde{P}_n(x) = \sqrt{2n+1}P_n(2x-1)$, 在 $[0,1]$ 上关于均匀测度正交, 即

$$\int_0^1 \tilde{P}_n(x)\tilde{P}_m(x)\,dx = \delta_{nm}. \tag{7.2.13}$$

将 $[0,t]$ 通过变量替换 $u = s/t$ 映射到 $[0,1]$, 系数为

$$h_n(t) = \int_0^1 \tilde{P}_n(u)\,x(tu)\,du = \frac{1}{t}\int_0^t \tilde{P}_n(s/t)\,x(s)\,ds. \tag{7.2.14}$$

对 t 求导 (应用莱布尼茨法则):

$$h_n'(t) = -\frac{1}{t^2}\int_0^t \tilde{P}_n(s/t)\,x(s)\,ds + \frac{1}{t}\tilde{P}_n(1)\,x(t) - \frac{1}{t^2}\int_0^t \frac{s}{t}\tilde{P}_n'(s/t)\,x(s)\,ds. \tag{7.2.15}$$

第一项化简为 $-\frac{1}{t}h_n(t)$; 第三项中利用勒让德多项式导数的展开公式:

$$P_n'(x) = \sum_{\substack{k=0 \\ n-k\ \text{奇}}}^{n-1} (2k+1)P_k(x), \tag{7.2.16}$$

因此 $\tilde{P}_n'(u) = \sqrt{2n+1}\sum_{\substack{k<n \\ n-k\ \text{奇}}}(2k+1)P_k(2u-1) = \sqrt{2n+1}\sum_{k<n}\sqrt{2k+1}\tilde{P}_k(u)\cdot[(n-k\ \text{奇})?1:0]$。

将上式代入第三项并整理, 对角项系数汇总为 $-(n+1)/t$, 次对角项系数为 $-\sqrt{(2n+1)(2k+1)}/t$ $(n>k)$, 上三角项为 0。注意 $\tilde{P}_n(1) = \sqrt{2n+1}$(因为 $P_n(1)=1$), 故 $B_n = \sqrt{2n+1}$。

此系统即式 (7.9) 对应的系统，通常写为 $th'(t) = \hat{A}h(t) + \hat{B}x(t)$，并通过时间变换 $\tau = \log t$ 化为标准 LTI 形式。□

数值示例: 对 $N = 4$ 的 LegS 矩阵:

$$A_{\text{LegS}} = -\begin{pmatrix} 1 & 0 & 0 & 0 \\ \sqrt{3} & 2 & 0 & 0 \\ \sqrt{5} & \sqrt{15} & 3 & 0 \\ \sqrt{7} & \sqrt{21} & \sqrt{35} & 4 \end{pmatrix}, \quad B_{\text{LegS}} = \begin{pmatrix} 1 \\ \sqrt{3} \\ \sqrt{5} \\ \sqrt{7} \end{pmatrix}. \tag{7.2.17}$$

矩阵是严格下三角占优的，这保证了在适当归一化后矩阵指数的有界性。

7.2.5 最优投影的精确陈述

定理 7.2 (HiPPO 最优近似保证): 设 $\hat{x}_t(s) = \sum_{n=0}^{N-1} h_n(t)\tilde{P}_n(s/t)$ 为 $x|_{[0,t]}$ 的 N 阶 Legendre 多项式近似。则

$$\hat{x}_t = \arg\min_{p \in \mathcal{P}_{N-1}} \int_0^t \left|p(s) - x(s)\right|^2 \frac{ds}{t}, \tag{7.2.18}$$

其中 $\mathcal{P}_{N-1}$ 是次数不超过 $N-1$ 的多项式空间。即 $h(t)$ 给出了 $x|_{[0,t]}$ 在 $[0,t]$ 上的均方意义下的最优 N 维 Legendre 系数近似。

注记: 不同的测度 $\mu^{(t)}$ 导致不同的”最优性”——LegT 最优近似的是最近的窗口，LegS 最优近似的是全历史的加权 (均匀) 版本。§7.8 将讨论这一点对模型性质的影响。

---

§7.3 S4 的离散化与卷积核

▶这一节在讲什么

为什么需要离散化？连续 ODE 是理论框架，但计算机处理的是离散 token 序列。必须将 $h'(t) = Ah(t) + Bx(t)$ 转换成离散递推 $h_k = \bar{A}h_{k-1} + \bar{B}x_k$，才能在 GPU 上运行。

第一步：ZOH 离散化给出精确的 $\bar{A}, \bar{B}$。零阶保持假设输入在每个步长内为常数，积分闭式可算，得到 $\bar{A} = e^{\Delta A}$。这不是近似，而是直接对 ODE 的较确矩阵和。

第二步：展开递推得到卷积。$y_k = \sum_{j=0}^{k} \bar{C}\bar{A}^{k-j}\bar{B}\,x_j$，这是一个离散卷积运算。卷积核 $\bar{\mathbf{K}} = (\bar{C}\bar{B}, \bar{C}\bar{A}\bar{B}, \ldots)$ 一旦知道，整个序列可以用 FFT 在 $O(L \log L)$ 内完成。

第三步：DPLR 结构把卷积核计算从 $O(N^2L)$ 降至 $\tilde{O}(N+L)$。将 A 写成对角加低秩形式，用 Woodbury 恒等式解析矩阵逆，再把卷积核的乾𝔽函数评估问题转化为 Cauchy 矩阵向量积。

本节要拿走的一件事: S4 的效率来自两个叠加：LTI 保证卷积结构可用 FFT，DPLR 结构保证卷积核可快速计算。去掉任一个，效率就崩塌。

直觉

连续状态空间模型在理论上很优美，但它运行在连续时间内而序列建模处理的是离散 token。离散化步骤就是把连续 ODE 「翻译」成 GPU 可运行的离散递推。

本节出现四个核心概念：**ZOH 离散化、离散卷积核 $\bar{\mathbf{K}}$、DPLR 参数化、Cauchy 矩阵向量积**。它们是 S4 实现高效训练的完整工程路径，看似是算法技巧的堆砌，实际上每一步都有其数学必然性。理解这条链，是理解「S4 为什么既快又准」的关键。

第一环：ZOH 离散化——连续 ODE 到 GPU 递推的精确翻译

连续 ODE $h'(t) = Ah(t) + Bx(t)$ 在实际执行时需要转换为离散递推。最简单的方法是欧拉法（$h_{k+1} \approx h_k + \Delta(Ah_k + Bx_k)$），但这只是一阶近似，步长较大时误差显著。

ZOH（零阶保持）更严格：假设输入 $x(t)$ 在每个时间步 $[k\Delta, (k+1)\Delta]$ 内保持常数 x_k，然后对 ODE 做精确积分，得到：

$$\bar{A} = e^{\Delta A}, \quad \bar{B} = A^{-1}(e^{\Delta A} - I)B \tag{7.3.1}$$

式 (7.3.1) 不是近似——在 ZOH 假设下这是精确解。$\bar{A} = e^{\Delta A}$ 的意义直观：把连续时间步长 Δ 压进矩阵指数，得到的是「以离散步长 Δ 推进的状态转移算子」。Δ 是可学习参数：大 Δ 意味着每步「扫描更多」连续时间；在 Mamba 中，Δ_t 被推广为输入依赖的参数，成为选择性机制的核心（§7.4）。

第二环：展开递推 → 卷积核——为什么可以批量计算

离散递推 $h_k = \bar{A}h_{k-1} + \bar{B}x_k$ 展开后：

$$y_k = \sum_{j=0}^{k} \bar{K}_{k-j} x_j, \quad \bar{K}_m = \bar{C}\bar{A}^m\bar{B} \tag{7.3.2}$$

式 (7.3.2) 是一个**离散卷积**，卷积核 $\bar{\mathbf{K}} = (\bar{C}\bar{B}, \bar{C}\bar{A}\bar{B}, \ldots, \bar{C}\bar{A}^{L-1}\bar{B})$。卷积结构的关键性质：整个序列的输出 $\mathbf{y} = \bar{\mathbf{K}} * \mathbf{x}$ 可以在频域用 FFT 在 $O(L \log L)$ 内完成，而不是逐步递推的 $O(NL)$。这是 S4 训练比 RNN 快得多的根本原因。

前提：只有当 $\bar{A}$ 不依赖输入（LTI 假设）时，卷积核 $\bar{\mathbf{K}}$ 才是固定的，才能预先计算一次、反复使用。一旦 $\bar{A}_t$ 变成输入依赖（如 Mamba），卷积结构消失，需要并行扫描（§7.4）。

第三环：计算 $\bar{\mathbf{K}}$ 的瓶颈——为什么直接算是天文数字

计算卷积核 $\bar{\mathbf{K}}$ 需要 L 个矩阵幂 $\bar{A}^m\bar{B}$。若 $\bar{A}$ 是稠密 $N \times N$ 矩阵，每个 $\bar{A}^m\bar{B}$ 需要 $O(N^2)$，总复杂度 $O(N^2L)$。当 $N = L = 4096$ 时约 7×10^{10} 次运算——不可接受。

DPLR（对角加低秩）参数化解决这个问题：把 A 写成 $A = \Lambda - PQ^*$，其中 Λ 是对角矩阵，$P, Q \in \mathbb{C}^{N \times r}$ 是低秩矩阵（$r \ll N$）。HiPPO-LegS 矩阵恰好可以写成这种形式——正规部分对角化为纯虚特征值，非正规部分为秩 1 修正。这不是巧合，而是 HiPPO 矩阵结构的内在性质，也是 S4 选用 LegS 的另一个原因。

第四环：Cauchy 矩阵向量积——从 $O(N^2L)$ 到 $\tilde{O}(N+L)$

DPLR 结构允许用 Woodbury 恒等式解析矩阵逆，把卷积核的生成函数在 FFT 点处的求值化为 Cauchy 矩阵向量积：

$$\hat{K}(\omega^l) = \sum_{j=1}^{N} \frac{c_j}{\omega^l - \bar{\lambda}_j}, \quad l = 0, 1, \dots, L-1 \tag{7.3.3}$$

式 (7.3.3) 中矩阵元素形如 $1/(\omega^l - \bar{\lambda}_j)$——这正是 Cauchy 矩阵的定义。利用快速多极方法或类 FFT 算法，此计算可在 $\tilde{O}(N+L)$ 时间完成。逆 FFT 还原时域卷积核 $\bar{\mathbf{K}}$，整体从 $O(N^2L)$ 降至 $\tilde{O}(N+L)$，提升约 $NL/\log(N+L)$ 倍。

整条链的回头看

```
连续 ODE h'(t) = Ah(t) + Bx(t)
  ↓ ZOH 离散化（输入在步长内为常数，精确积分）
离散递推 h_k = Āh_{k-1} + Bx_k                ← Ā = e^{ΔA}，精确而非近似
  ↓ LTI → 展开递推 → 离散卷积
卷积核 K = (CB, CĀB, ⋯, CĀ^{L-1}B)          ← y = K * x，可用 FFT O(LlogL)
  ↓ 瓶颈：直接算 K 需要 O(N²L)
DPLR 参数化 A = Λ − PQ*                       ← HiPPO-LegS 矩阵的自然结构
  ↓ Woodbury 解析逆 + 生成函数在 FFT 点求值
Cauchy 矩阵向量积 Õ(N+L)                      ← 复杂度从 O(N²L) → Õ(N+L)
  ↓ 逆 FFT → K → FFT 卷积
训练 O(L log L)，推理 O(N) per token
```

这条链揭示了 S4 的工程本质：**效率来自两个叠加——LTI 保证卷积结构可用 FFT，DPLR 保证卷积核可快速计算。去掉任一个，效率就崩塌**。LTI 被 Mamba 放弃（代价是需要并行扫描），DPLR 被 S4D 简化为纯对角（放弃严格 HiPPO 最优性）——两条路线都在这个框架的不同维度上做权衡。

> **本节的关键骨架**：连续 ODE $h'(t) = Ah(t) + Bx(t)$ → 零阶保持（ZOH）离散化 → 离散递推 $h_k = \bar{A}h_{k-1} + \bar{B}x_k$ → 展开为卷积 $y = \bar{K} * x$ → **关键消元** Cauchy 核 + Vandermonde 矩阵把卷积从 $O(NL)$ 降至 $O((N+L)\log^2(N+L))$。

7.3.1 零阶保持离散化

在实际应用中，序列数据是离散的：时刻 $k = 0, 1, \dots, L-1$，步长 (time step) 为 $\Delta > 0$。最常用的离散化方案是**零阶保持 (Zero-Order Hold, ZOH)**，假设输入在每个时间步内保持常数。

引理 7.3 (ZOH 离散化的精确形式)：在步长 Δ 的 ZOH 假设下，连续 SSM (A, B, C) 离散化为

$$\bar{A} = e^{\Delta A}, \qquad \bar{B} = (\Delta A)^{-1}(e^{\Delta A} - I) \cdot \Delta B, \qquad \bar{C} = C. \tag{7.10}$$

离散递推为 $h_k = \bar{A}h_{k-1} + \bar{B}x_k, y_k = \bar{C}h_k$。

证明：

将时间区间 $[k\Delta, (k+1)\Delta]$ 上的连续解 (??) 应用于 $h((k+1)\Delta)$：

$$h((k+1)\Delta) = e^{A\Delta}h(k\Delta) + \int_{k\Delta}^{(k+1)\Delta} e^{A((k+1)\Delta - s)}Bx(s)\,ds. \tag{7.3.5}$$

ZOH 假设 $x(s) = x_{k+1}$ 为常数, 令 $\tau = (k+1)\Delta - s$, 则

$$\int_{k\Delta}^{(k+1)\Delta} e^{A((k+1)\Delta - s)}Bx_{k+1}\,ds = \left(\int_0^{\Delta} e^{A\tau}\,d\tau\right)Bx_{k+1}. \tag{7.3.6}$$

利用矩阵指数的积分公式:

$$\int_0^{\Delta} e^{A\tau}\,d\tau = A^{-1}(e^{A\Delta} - I), \tag{7.3.7}$$

故

$$\bar{B} = A^{-1}(e^{A\Delta} - I)\cdot B = (\Delta A)^{-1}(e^{\Delta A} - I)\cdot \Delta B. \tag{7.3.8}$$

这正是式 (7.3.1) / (??) 中的 $\bar{B}$。□

与双线性方法的比较:S4 原始论文中也使用双线性 (Tustin/bilinear) 离散化 $\bar{A} = (I - \Delta A/2)^{-1}(I + \Delta A/2)$, 其精度为 $O(\Delta^2)$ 而 ZOH 同阶。两者实践差异较小, 但 Mamba 重新引入 ZOH 以支持更直观的选择机制——因为 ZOH 下 $\bar{A} = e^{\Delta A}$ 具有更清晰的”遗忘率”语义。

7.3.2 离散卷积核

在 LTI 设定下, 离散 SSM 的输入-输出关系展开为卷积。将递推展开:

$$y_k = \bar{C}\bar{A}^k\bar{B}x_0 + \bar{C}\bar{A}^{k-1}\bar{B}x_1 + \cdots + \bar{C}\bar{B}x_k, \tag{7.3.9}$$

即 $y_k = \sum_{j=0}^{k}\bar{K}_j x_{k-j}$, 其中**离散卷积核向量**为

$$\bar{\mathbf{K}} = (\bar{C}\bar{B},\ \bar{C}\bar{A}\bar{B},\ \bar{C}\bar{A}^2\bar{B},\ \ldots,\ \bar{C}\bar{A}^{L-1}\bar{B}). \tag{7.11}$$

由式 (7.3.9) 展开得到式 (??) 的向量 $\bar{\mathbf{K}} \in \mathbb{R}^L$, 称为 SSM 的卷积核 (SSM kernel)。一旦计算出 $\bar{\mathbf{K}}$, 整个序列的输出可由 $\mathbf{y} = \bar{\mathbf{K}} * \mathbf{x}$(离散卷积) 在频域通过 FFT 以 $O(L\log L)$ 计算。

7.3.3 计算复杂度的降低:Cauchy 核与范德蒙（Vandermonde）矩阵

瓶颈: 直接计算 $\bar{\mathbf{K}}$ 需要 L 次矩阵幂 $\bar{A}^k B$, 若 $\bar{A}$ 是稠密 $N\times N$ 矩阵, 则复杂度为 $O(N^2L)$, 在 $N = L = 4096$ 的典型设定下为 $\sim 7\times 10^{10}$ 次运算, 不可接受。

DPLR 参数化:S4 的核心技术是将 A 参数化为**对角加低秩 (Diagonal Plus Low-Rank, DPLR)** 形式:

$$A = \Lambda - PQ^*, \tag{7.12}$$

其中 $\Lambda = \mathrm{diag}(\lambda_1, \dots, \lambda_N)$ 是对角矩阵,$P, Q \in \mathbb{C}^{N\times r}$ 是低秩矩阵 ($r \ll N$)。HiPPO-LegS 矩阵恰好可以写成 DPLR 形式: 其正规部分为斜对称矩阵 $S_{nk} = -\frac{1}{2}[(2n+1)(2k+1)]^{1/2}\mathrm{sgn}(n-k)$, 可对角化为纯虚特征值; 非正规部分为秩 1 修正 $-\frac{1}{2}pp^*$, 其中 $(p)_n = (n+1/2)^{1/2}$。

Woodbury 恒等式: 设 $M = \Lambda - PQ^*$ 为 DPLR 矩阵, 则

$$M^{-1} = \Lambda^{-1} + \Lambda^{-1}P(I - Q^*\Lambda^{-1}P)^{-1}Q^*\Lambda^{-1}. \tag{7.13}$$

由式 (??) (Woodbury 恒等式), Λ^{-1} 是对角矩阵, 右侧所有运算均可以 $O(N + r^3)$ 时间完成。这使得在对角形上计算 $(zI - \bar{A})^{-1}$ 变得可行。

关键算法: 设 Λ 的特征值为 $\{\lambda_j\}_{j=1}^N$, 离散化后 $\bar{\Lambda} = e^{\Delta\lambda}$。卷积核的第 k 个元素可写为

$$\bar{K}_k = \bar{C}\bar{A}^k\bar{B} = \sum_{j=1}^{N} c_j\bar{\lambda}_j^k, \tag{7.3.13}$$

其中系数 $\{c_j\}$ 由伍德伯里（Woodbury）恒等式修正低秩项后得到。将全部 L 个核元素的生成函数求值:

$$\hat{K}(z) = \sum_{k=0}^{L-1}\bar{K}_k z^k \approx C(zI - \bar{A})^{-1}\bar{B} = \sum_{j=1}^{N}\frac{c_j}{z - \bar{\lambda}_j}. \tag{7.14}$$

在 L 个单位根 ω^j(其中 $\omega = e^{2\pi i/L}$) 处求值:

$$\hat{K}(\omega^l) = \sum_{j=1}^{N}\frac{c_j}{\omega^l - \bar{\lambda}_j}, \quad l = 0, 1, \dots, L-1. \tag{7.3.15}$$

这等价于 **Cauchy 矩阵向量积** $\mathrm{diag}(\hat{K}) = M_{\text{Cauchy}}\mathbf{c}$, 其中 $(M_{\text{Cauchy}})_{lj} = 1/(\omega^l - \bar{\lambda}_j)$。利用快速多极方法或 Vandermonde 矩阵的 FFT 类算法, 此计算可在 $\tilde{O}(N + L)$ 时间内完成。最终通过逆 FFT 恢复时域卷积核 $\bar{\mathbf{K}}$。

整体复杂度: 从 $O(N^2L)$ 降至 $\tilde{O}(N + L)$, 提升约 $N \cdot L/\log(N + L)$ 倍。

DPLR 参数化的必要性: 若 A 是一般稠密矩阵, 则其矩阵指数的计算为 $O(N^3)$, 后续矩阵幂亦为 $O(N^2)$ per step, 无法实现 Cauchy 核约化。DPLR 结构保证:(i) 矩阵逆由 Woodbury 恒等式精确解析;(ii) 离散化后 $\bar{A}$ 仍保持 DPLR 结构;(iii) 所有操作在数值稳定的复对角形上进行。S4D(Diagonal S4) 进一步将 DPLR 中 P, Q 简化为零, 直接用纯对角矩阵 $A = \Lambda$, 实现更简洁的实现, 并在实践中与 S4 性能相当。

---

§7.4 Mamba 的选择性扫描

▶这一节在讲什么

为什么 S4 还不够？S4 的 LTI 假设意味着模型对所有位置的相同输入给出相同响应，无法分辨「这个 token 需要记住」还是「这个 token 需要忽略」。Mamba 的解决方案是将 Δ_t, B_t, C_t 设为输入 x_t 的函数。

第一步：Δ_t 控制记忆与遗忘的弹性。$\Delta_t \to 0$ 时 $\bar{A}_t \to I$，当前输入被忽略、历史状态保持；$\Delta_t \to +\infty$ 时 $\bar{A}_t \to 0$，历史全部清除、当前输入优先。模型学会根据内容动态调节这个「分辨率」。

第二步：选择性打破了 LTI，全局卷积核不再有效。$\bar{A}_t$ 现在依赖 t，序列不再是时不变系统，无法预先计算卷积核。但递推中存在二元结合律操作，可用 Blelloch 并行扫描将深度从 $O(L)$ 压到 $O(\log L)$。

第三步：硬件感知实现将状态和参数都放进 SRAM。避免反复读写 HBM，实现训练吞吐量约 5 倍于 Transformer。

本节要拿走的一件事：Mamba = S4 的选择性版本。用输入依赖的 Δ_t, B_t, C_t 换掉静态参数，获得内容感知的记忆，代价是失去全局卷积核、需要并行扫描弥补。

直觉

S4 建立在 LTI 假设上，但这意味着它无法根据内容决定「应该把哪个 token 写入记忆」。Mamba 通过引入输入依赖的参数解决这一问题。

本节出现四个核心概念：**LTI 的内容无关局限**、**输入依赖的 $\Delta_t/B_t/C_t$**、**选择性机制打破卷积**、**Blelloch 并行扫描**。它们是从「固定滤波器」到「内容感知记忆」的完整演进路径。理解这条链，是理解 Mamba 相对于 S4 的本质突破的关键。

第一环：LTI 的内容无关局限——为什么 S4 无法「选择记忆」

S4 的卷积核 $\bar{\mathbf{K}}$ 是固定的——对所有位置和输入都相同。这意味着对于序列中「重要的词」（如命名实体、代词指代对象）和「不重要的词」（如冠词、停顿词），S4 用完全相同的方式处理。

这与人类记忆的工作方式截然不同。人类记忆有明显的「选择性」：重要事件被深刻记住，无关信息快速遗忘。LSTM 的遗忘门正是对这种选择性的早期模拟：$f_t = \sigma(W_f x_t + U_f h_{t-1})$，由输入决定遗忘多少旧状态。

S4 缺乏类似机制：无论当前 token 是「法国大革命」还是「的」，卷积核都不变。这在语言模型中是一个根本缺陷——语义重要性高度依赖内容，LTI 无法捕捉这一点。

第二环：输入依赖的 Δ_t, B_t, C_t——选择机制的数学形式

Mamba 的解法是让 Δ_t, B_t, C_t 成为当前输入 x_t 的函数：

$$\Delta_t = \text{softplus}(W_\Delta x_t + b_\Delta), \quad B_t = W_B x_t, \quad C_t = W_C x_t \tag{7.4.1}$$

式 (7.4.1) 中 $\text{softplus}(x) = \ln(1 + e^x) > 0$ 保证 Δ_t 严格为正。A 保持固定（仅结构参数化），选择性完全通过 Δ_t 来调节。

Δ_t 的物理意义是「离散化分辨率」，控制每一步「扫描」多少连续时间：- $\Delta_t \to 0^+$：$\bar{A}_t = e^{\Delta_t A} \to I$，$\bar{B}_t \to 0$。当前输入被**忽略**，历史状态原样保持。- $\Delta_t \to +\infty$：若 A 有负特征值，$\bar{A}_t \to 0$，历史状态被**清除**，模型聚焦于当前输入。

式 (7.4.1) 的这一机制与 LSTM 的遗忘门在概念上相似，但有更清晰的数学意义：遗忘速率由 $e^{\Delta_t A}$ 的谱范数控制，通过约束 A 的特征值为负实部来保证系统稳定性。

第三环：选择性打破了 LTI——卷积核不再有效

引入输入依赖的 Δ_t 后，$\bar{A}_t = e^{\Delta_t A}$ 现在随时间变化——序列不再是 LTI 系统。这意味着 §7.3 中精心构建的「预先计算卷积核 + FFT 批量推理」的整套技巧**全部失效**：没有固定的卷积核，只有依赖路径的递推 $h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$。

朴素的顺序递推需要 $O(LN)$ 时间，更致命的是它无法在 GPU 上并行化（每步依赖上一步）。Mamba 的关键洞察是：这个递推具有一个满足结合律的二元操作，可以用并行扫描解决。

第四环：Blelloch 并行扫描——从 $O(L)$ 串行到 $O(\log L)$ 并行

定义二元操作 $(a_1, b_1) \otimes (a_2, b_2) = (a_2 a_1,\ a_2 b_1 + b_2)$（即式 (7.4.2)），可以验证它满足结合律。递推 $h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$ 等价于计算前缀积：

$$(h_t, 1) \leftarrow (\bar{A}_t, \bar{B}_t x_t) \otimes (\bar{A}_{t-1}, \bar{B}_{t-1} x_{t-1}) \otimes \cdots \otimes (\bar{A}_1, \bar{B}_1 x_1) \tag{7.4.2}$$

由于 $\otimes$ 满足结合律，可以用 Blelloch（1990）并行前缀扫描在 $O(\log L)$ 层二叉树归并完成所有前缀积，总工作量 $O(L)$，深度 $O(\log L)$。长度 10^6 的序列可以在约 20 层并行操作中完成。

结合**硬件感知实现**（将参数和中间状态放进 SRAM，避免反复读写 HBM），Mamba 的训练吞吐量约 5 倍于同规模 Transformer。

训练时使用并行扫描（并行），推理时切换为顺序递推（$O(N)$ per token，无 KV cache），这是 Mamba 同时具备训练效率和推理效率的关键设计。

整条链的回头看

```
S4 的 LTI 假设 → 卷积核固定 → 内容无关
  ↓ 问题：无法选择性记忆重要 token
引入输入依赖：Δt, Bt, Ct = f(xt)            ← softplus(WΔxt) 控制记忆/遗忘弹性
  ↓ Δt→0：忽略输入保持历史；Δt→∞：清除历史聚焦当前
At = e^{ΔtA} 随 t 变化 → 打破 LTI          ← 卷积核失效，必须逐步递推
  ↓ 递推 ht = Atht-1 + Btxt
二元操作 (a1,b1)(a2,b2) = (a2a1, a2b1+b2)  ← 结合律成立 → 可并行
Blelloch 并行扫描：深度 O(logL)，工作 O(L)  ← 训练并行，推理递推
  ↓ + SRAM 硬件感知（避免 HBM 往返）
吞吐量 ~5× Transformer
```

这条链揭示了 Mamba 的本质：**不是 S4 的简单扩展，而是用内容感知的选择性换掉了全局卷积，代价是失去卷积核，但用结合律并行扫描弥补了训练效率**。选择性和并行性不是鱼与熊掌，而是通过结合律这一代数性质同时得到的。

选择性的代价是失去全局卷积核。因为 $\bar{A}_t$ 现在随时间变化，序列不再是时不变系

统，「预先计算一个卷积核多次使用」的技巧失效了。必须对每条序列逐步递推。但 Mamba 的关键发现是：递推 $h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$ 具有一个可证明满足结合律的二元操作，所有前缀积因此可以用 Blelloch 并行扫描在 $O(\log L)$ 层 GPU 并行内完成。结合硬件感知实现（将状态和参数清退到 SRAM，避免读写 HBM），训练吞吐量约 5 倍于 Transformer。

这里一个常见的误解是：「Mamba 弃用了卷积，所以它就是纯 RNN」。实际上，**Mamba 在训练时使用并行扫描，推理时则以递推模式运行**。训练和推理可以切换模式，这是它同时具备并行训练效率和常数推理时间复杂度的关键。

7.4.1 选择性机制: 从 LTI 到输入依赖

S4 的根本局限在于参数 A, B, C 与输入无关 (LTI)，因此模型在不同位置对相同幅度的输入给出相同响应，无法进行基于内容的选择性记忆。Mamba(Gu & Dao, 2023) 通过使参数依赖于当前输入来打破这一限制。

选择性参数化: 对输入序列 $\{x_t\}_{t=1}^{L}$,Mamba 令

$$\Delta_t = \mathrm{softplus}(W_\Delta x_t + b_\Delta) \in \mathbb{R}^D, \qquad B_t = W_B x_t \in \mathbb{R}^N, \qquad C_t = W_C x_t \in \mathbb{R}^N, \tag{7.15}$$

其中 $W_\Delta \in \mathbb{R}^{D\times D}$,$W_B \in \mathbb{R}^{N\times D}$,$W_C \in \mathbb{R}^{N\times D}$ 是可学习参数,D 是输入维度。步长 Δ_t 决定了当前时刻离散化的”分辨率”，由 $\mathrm{softplus}(x) = \ln(1+e^x) > 0$ 保证正性。

选择性离散化: 对每个时刻 t，独立执行 ZOH 离散化:

$$\bar{A}_t = e^{\Delta_t A}, \qquad \bar{B}_t = (\Delta_t A)^{-1}(e^{\Delta_t A} - I)\Delta_t B_t, \tag{7.16}$$

其中 A 保持固定 (仅结构参数化)。选择性 SSM 的递推为:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \qquad y_t = C_t h_t. \tag{7.17}$$

直觉分析: 当 $\Delta_t \to 0^+$ 时,$\bar{A}_t = e^{\Delta_t A} \to I$ 而 $\bar{B}_t \to 0$，模型忽略当前输入、保持历史状态; 当 $\Delta_t \to +\infty$ 时，若 A 有负特征值，则 $\bar{A}_t \to 0$ 且 $\bar{B}_t \to -A^{-1}B_t$，模型聚焦于当前输入、遗忘所有历史。Δ_t 由输入内容决定，实现了 “token-wise 的记忆/遗忘控制”。

7.4.2 选择性 SSM 的计算挑战

由于 $\bar{A}_t$ 依赖于 t，序列不再是 LTI 系统，**全局卷积核不再有效**。朴素的顺序递推需要 $O(LN)$ 时间但无法在 GPU 上并行化，成为训练的瓶颈。解决方案是**并行扫描算法** (Parallel Scan)。

7.4.3 并行扫描与关联算子

引理 7.4 (选择性 SSM 的关联算子): 定义元素 $(a, b) \in \mathbb{R}^{N\times N} \times \mathbb{R}^N$，其中 a 表示状态转移矩阵,b 表示驱动项。定义二元运算

$$(a_1, b_1) \otimes (a_2, b_2) = (a_2 a_1,\ a_2 b_1 + b_2). \tag{7.18}$$

则此运算满足结合律, 且递推 (??) 可以写成

$$(h_t, 1) \overset{\text{对应}}{\longleftrightarrow} (\bar{A}_t, \bar{B}_t x_t) \otimes (\bar{A}_{t-1}, \bar{B}_{t-1} x_{t-1}) \otimes \cdots \otimes (\bar{A}_1, \bar{B}_1 x_1). \tag{7.19}$$

证明 (结合律):

对三个元素 $(a_1, b_1), (a_2, b_2), (a_3, b_3)$, 验证左右结合相等。

左结合: 先计算 $(a_1, b_1) \otimes (a_2, b_2) = (a_2 a_1, a_2 b_1 + b_2)$, 再与 (a_3, b_3) 结合:

$$(a_2 a_1, a_2 b_1 + b_2) \otimes (a_3, b_3) = (a_3 (a_2 a_1),\ a_3 (a_2 b_1 + b_2) + b_3) = (a_3 a_2 a_1,\ a_3 a_2 b_1 + a_3 b_2 + b_3). \tag{7.4.8}$$

右结合: 先计算 $(a_2, b_2) \otimes (a_3, b_3) = (a_3 a_2, a_3 b_2 + b_3)$, 再与 (a_1, b_1) 结合:

$$(a_1, b_1) \otimes (a_3 a_2, a_3 b_2 + b_3) = ((a_3 a_2) a_1,\ (a_3 a_2) b_1 + (a_3 b_2 + b_3)) = (a_3 a_2 a_1,\ a_3 a_2 b_1 + a_3 b_2 + b_3). \tag{7.4.9}$$

两者相等（比较式 (7.4.8) 和式 (7.4.9)），故结合律成立。□

验证式 (??): 归纳地, 若 $h_0 = 0$, 令 $a_i = \bar{A}_i$,$b_i = \bar{B}_i x_i$, 则

$$h_t = a_t a_{t-1} \cdots a_1 \cdot 0 + a_t \cdots a_2 b_1 + a_t \cdots a_3 b_2 + \cdots + b_t, \tag{7.4.10}$$

恰好由式 (??) 的前缀积给出（即式 (7.4.10)）。此式也可解释为: 隐状态是带指数衰减权重的输入历史加权和, 权重由路径乘积 $\prod_{s=j+1}^{t} \bar{A}_s$ 决定。

7.4.4 Blelloch 并行前缀扫描的复杂度

给定长度为 L 的序列 $\{(a_t, b_t)\}_{t=1}^{L}$, 计算所有前缀积 $\{(a_t, b_t) \otimes \cdots \otimes (a_1, b_1)\}_{t=1}^{L}$:

- **朴素串行**:$O(L)$ 步骤, 无法并行化。
- **Blelloch 并行扫描 (Blelloch, 1990)**: 利用结合律, 将前缀积分为**上扫 (up-sweep)** 和**下扫 (down-sweep)** 两个阶段, 在 $O(\log L)$ 深度的并行计算中完成, 总工作量 (work) 为 $O(L)$。

具体地, 每个 GPU 线程处理 $O(1)$ 个元素, 利用 $O(\log L)$ 层二叉树归并:

$$\text{Work} = O(L), \quad \text{Depth} = O(\log L). \tag{7.4.11}$$

这使得长度 10^6 的序列可以在 $\sim$ 20 层并行操作中完成状态扫描。Mamba 的**硬件感知实现**将参数和中间状态直接存储在 SRAM 中, 避免反复读写 HBM(High Bandwidth Memory), 实现了训练时约 5 倍于 Transformer 的吞吐量提升。

---

§7.5 线性注意力的核函数视角

▶这一节在讲什么

为什么要把注意力近似成线性的？ 标准 Softmax 注意力的 Q, K, V 矩阵乘法需要 $O(L^2)$ 内存和 $O(L^2 d)$ 时间，在长序列上无法扩展。线性注意力的目标是把它压成 $O(Ld)$ 而不丧失太多表达能力。

第一步：用核函数分解代替 Softmax。 把 $\exp(Q_i \cdot K_j)$ 替换为 $\phi(Q_i) \cdot \phi(K_j)$，其中 $\phi : \mathbb{R}^d \to \mathbb{R}^r$ 是特征映射。这让注意力矩阵变成两个低维矩阵的外积，突破了 L^2 的依赖。

第二步：利用矩阵乘法结合律改变计算顺序。 先算 $\mathbf{S} = \sum_j \phi(K_j)^\top V_j$（$O(Lrd)$），再对每个 i 算 $\phi(Q_i)\mathbf{S}$，无需显式构造 $L \times L$ 注意力矩阵。

第三步：推理时状态 $\mathbf{S}_t$ 可在线递推。 $\mathbf{S}_t = \mathbf{S}_{t-1} + \phi(K_t)^\top V_t$，每步 $O(rd)$，内存 $O(rd)$，与序列长度无关。

本节要拿走的一件事：线性注意力不是「注意力的近似」，而是用核技巧把注意力的 $O(L^2)$ 瓶颈变成一个固定大小的递推状态——这把它和 SSM、RNN 统一到了同一框架下（见 §7.6）。

直觉

Softmax 注意力的 $O(L^2)$ 瓶颈来自一个根本原因：要计算每个查询对每个键的相似度，必须构造完整的 $L \times L$ 注意力矩阵。线性注意力的突破思路是：用一个核函数分解把 L^2 的矩阵计算变成两个低维矩阵的乘积。

本节出现四个核心概念：**Softmax 注意力的 $O(L^2)$ 瓶颈、核函数分解、矩阵乘法结合律改变计算顺序、与 Fast Weight Memory 的等价**。它们沿着一条严密的逻辑链一个推出一个。理解这条链，是理解「线性注意力为什么与 SSM 统一到同一框架」的关键。

第一环：Softmax 注意力的 $O(L^2)$ 瓶颈——根本原因是什么

标准 Softmax 注意力：

$$A_{ij} = \frac{\exp(Q_i \cdot K_j / \sqrt{d})}{\sum_{j'} \exp(Q_i \cdot K_{j'} / \sqrt{d})}, \quad O_i = \sum_j A_{ij} V_j \tag{7.5.1}$$

$L \times L$ 的注意力矩阵 A 需要 $O(L^2 d)$ 时间和 $O(L^2)$ 空间。根本原因是：Softmax 的归一化操作要求先把所有 $\exp(Q_i \cdot K_j)$ 全部算出来才能除以分母——这天然要求 $O(L^2)$ 存储。

Softmax 的另一个关键性质是「赢者通吃」的尖锐化能力：当 $\|Q\| \to \infty$ 时，注意力权重趋向于最匹配键的 one-hot 分布，实现精确检索。这个性质在 §7.9 中会成为线性注意力系统性劣势的来源。

第二环：核函数分解——把 L^2 瓶颈打破

线性注意力的核心想法（Katharopoulos et al., 2020）：用特征映射 $\phi : \mathbb{R}^d \to \mathbb{R}^r$ 替换 $\exp(Q_i \cdot K_j) \approx \phi(Q_i) \cdot \phi(K_j)$。这把注意力矩阵写成两个低维矩阵的外积：

$$O_i \approx \frac{\sum_j \phi(Q_i)\cdot\phi(K_j)^\top V_j}{\sum_j \phi(Q_i)\cdot\phi(K_j)^\top} \tag{7.5.2}$$

式 (7.5.2) 中 $\phi(Q_i)$ 是一个 r 维向量（与 j 无关）。利用矩阵乘法的结合律，可以改变计算顺序——先把与 i 无关的部分折叠起来，再对每个 i 做单次查询。

第三环：矩阵乘法结合律——把 $O(L^2)$ 变成 $O(Lr)$

关键代数步骤：提取 $\phi(Q_i)$，令 $\mathbf{S} = \sum_j \phi(K_j)^\top V_j \in \mathbb{R}^{r\times d}$（累积 KV 状态），$\mathbf{z} = \sum_j \phi(K_j)^\top \in \mathbb{R}^r$（归一化项），则：

$$O_i = \frac{\phi(Q_i)\,\mathbf{S}}{\phi(Q_i)\cdot\mathbf{z}} \tag{7.5.3}$$

式 (7.5.3) 无需显式构造 $L\times L$ 矩阵。计算 $\mathbf{S}$ 需要 $O(Lrd)$，对所有 i 查询需要 $O(Lrd)$，总体 $O(Lrd)$。当 $r \ll L$ 时比 $O(L^2 d)$ 显著更快。

推理时的优势更为显著：在自回归生成中，每来一个新 token，状态只需做一次加法更新（式 (7.5.4)）：

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \phi(K_t)^\top V_t, \quad O_t = \frac{\phi(Q_t)\mathbf{S}_t}{\phi(Q_t)\cdot\mathbf{z}_t} \tag{7.5.4}$$

每步计算量 $O(rd)$，内存 $O(rd)$——与序列长度无关。这与 SSM 的递推结构完全同构：式 (7.5.4) 中的 $\mathbf{S}_t$ 就是 SSM 的隐状态 h_t！

第四环：与 Fast Weight Memory 的等价——联想记忆的视角

状态更新 $\mathbf{S}_t = \mathbf{S}_{t-1} + \phi(K_t)^\top V_t$ 可以解读为**快速权重存储器**（fast weight memory）：状态矩阵 $\mathbf{S}_t$ 以外积 $\phi(K_t)^\top V_t$ 的形式存储键值关联，查询时用 $\phi(Q_t)$ 检索。

这与 Hopfield 网络和联想记忆有深刻联系。外积存储的容量约 $O(r)$ 个精确键值对；当待检索的键值对数目超过 r 时，不同键之间产生干扰，检索精度下降。这从机制上解释了线性注意力在精确检索任务上的系统性不足（§7.9）——不是实现缺陷，而是有限秩特征映射的数学必然。

特征映射 ϕ 的选择决定近似质量与效率的权衡：ELU+1 简单但不精确近似 Softmax；FAVOR+ 在理论上无偏但方差较大；RetNet 的固定指数衰减则本质上是对键值贡献按位置加权衰减。每种选择都在「近似质量」与「计算效率」之间做不同的权衡。

整条链的回头看

```
Softmax 注意力 O_i = softmax(QK^T/√d)V
  ↓ 根本问题：L×L 矩阵需要 O(L²) 存储
核函数分解 exp(Q · K)   (Q) · (K)            ← 把注意力矩阵写成外积
  ↓ 矩阵乘法结合律：先折叠 K/V，再查询 Q
S = Σ (K)^T V   R^{r×d}（累积 KV 状态）    ← 固定大小，O(Lr d) 总复杂度
  ↓ 因果设置下在线递推
S_t = S_{t-1} +  (K_t)^T V_t                ← 每步 O(rd)，与序列长度无关
  ↓ 等价解读
```

```
Fast Weight Memory: 外积存储, (Q)检索          ← 容量约 r 个精确键值对
  ↓ 问题: 有限秩 → 无法精确检索 → O(r/L) 上界
→ §7.6 统一视角 (S_t 就是 h_t! )
```

这条链揭示了一个深层事实：**线性注意力不是「注意力的近似」，而是用核技巧把 $O(L^2)$ 瓶颈变成一个固定大小的递推状态——这把它和 SSM、RNN 统一到了同一框架下**。它的局限（有限秩特征映射）和 SSM 的局限（LTI 的内容无关性）来自不同方向，但都是该类架构固有的数学约束。

推理时的优势更为显著。在自回归生成中，每来一个新 token，状态 $\mathbf{S}_t$ 只需做一次加法更新，然后直接用 $\phi(Q_t)$ 检索。每步的时间和内存都是 $O(rd)$，而不是 Softmax 注意力的 $O(td)$——后者随生成长度线性增长。这个「固定大小状态 + 在线递推」的模式与 SSM 完全同构，也是 §7.6 统一视角的切入点。

然而核函数的选择并非无代价。ELU+1 这样的简单特征映射虽然快速，但并不精确近似 Softmax；FAVOR+ 的随机特征理论上无偏但需要更多特征维度；RetNet 的固定指数衰减则本质上是把键值对的贡献按位置加权衰减。每种选择都在「近似质量」与「计算效率」之间做不同的权衡。更根本的限制在于：有限秩的特征映射无法模拟 Softmax 的「赢者通吃」尖锐化能力，这导致线性注意力在需要精确键值检索的任务上存在系统性不足（见 §7.9.3）。

7.5.1 标准归一化指数（Softmax）注意力的复杂度

标准自注意力机制 (Vaswani et al., 2017) 定义为:

$$A_{ij} = \frac{\exp(Q_i \cdot K_j/\sqrt{d})}{\sum_{j'} \exp(Q_i \cdot K_{j'}/\sqrt{d})}, \qquad O_i = \sum_j A_{ij} V_j, \tag{7.20}$$

其中 $Q, K, V \in \mathbb{R}^{L\times d}$ 分别是查询、键、值矩阵。计算 $L \times L$ 注意力矩阵 A 需要 $O(L^2 d)$ 时间和 $O(L^2)$ 空间, 在长序列上形成瓶颈。

7.5.2 核函数分解

线性注意力的核心思想 (Katharopoulos et al., ICML 2020): 用正半定核函数 $k(Q_i, K_j) = \phi(Q_i) \cdot \phi(K_j)^\top$ 替换 $\exp(Q_i \cdot K_j)$, 其中 $\phi : \mathbb{R}^d \to \mathbb{R}^r$ 是特征映射 (feature map)。则

$$O_i = \frac{\sum_j \phi(Q_i) \cdot \phi(K_j)^\top V_j}{\sum_j \phi(Q_i) \cdot \phi(K_j)^\top}. \tag{7.21}$$

关键代数变换: 利用矩阵乘法的结合律

$$O_i = \frac{\phi(Q_i) \left(\sum_j \phi(K_j)^\top V_j\right)}{\phi(Q_i) \left(\sum_j \phi(K_j)^\top\right)}. \tag{7.22}$$

令 $\mathbf{S} = \sum_j \phi(K_j)^\top V_j \in \mathbb{R}^{r\times d}$(累积 KV 状态),$\mathbf{z} = \sum_j \phi(K_j)^\top \in \mathbb{R}^r$(归一化项), 则

$$O_i = \frac{\phi(Q_i)\mathbf{S}}{\phi(Q_i)\cdot\mathbf{z}}. \tag{7.23}$$

在**因果 (causal)** 设定下 (自回归生成),$\mathbf{S}_t = \sum_{j\le t}\phi(K_j)^\top V_j$ 可在线更新：将式 (7.23) 改写为递推式

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \phi(K_t)^\top V_t, \qquad O_t = \frac{\phi(Q_t)\mathbf{S}_t}{\phi(Q_t)\cdot\mathbf{z}_t}. \tag{7.24}$$

这使得推理时每个 token 的计算量为 $O(rd)$(仅依赖固定大小的状态 $\mathbf{S}_t \in \mathbb{R}^{r\times d}$), 而非 $O(td)$。**推理复杂度从 $O(L)$ per token 降至 $O(1)$ per token**。

复杂度分析:

- **训练 (全序列)**: 先计算全局 $\mathbf{S} = \sum_j \phi(K_j)^\top V_j$(复杂度 $O(Lrd)$), 再对所有 i 计算 $O_i = \phi(Q_i)\mathbf{S}/(\phi(Q_i)\cdot\mathbf{z})$(复杂度 $O(Lrd)$), 总体为 $O(Lrd)$。相比 $O(L^2d)$, 当 $r \ll L$ 时有显著优势。
- **推理 (自回归)**: 每步 $O(rd)$, 状态 $\mathbf{S}_t$ 占 $O(rd)$ 内存, 与序列长度无关。

7.5.3 特征映射的选择

不同 ϕ 的选择决定了近似质量与效率的权衡:

1. ELU+1 (Katharopoulos, 2020):

$$\phi(x)_i = \mathrm{elu}(x_i) + 1 = \begin{cases} x_i + 1 & x_i \ge 0, \\ e^{x_i} & x_i < 0. \end{cases} \tag{7.25}$$

此映射保证 $\phi(x)_i > 0$, 使分母 $\phi(Q_i)\cdot\mathbf{z}$ 保持正定, 避免数值不稳定。但此 ϕ 并不对应 softmax 核的精确近似, 故可视为一种启发式替代。

2. FAVOR+ 随机特征 (Performer, Choromanski et al., ICLR 2021):

利用 softmax 核的正值随机特征分解。注意到

$$\exp(Q\cdot K) = \exp\left(\frac{\|Q\|^2 + \|K\|^2}{2}\right)\cdot\exp\left(-\frac{\|Q-K\|^2}{2}\right), \tag{7.5.11}$$

后者是高斯核。利用 Bochner 定理, 高斯核有无偏随机特征展开。FAVOR+ 使用正交正值特征:

$$\phi^+_\omega(x) = \frac{e^{-\|x\|^2/2}}{\sqrt{r}}\left[\exp(\omega_1^\top x), \dots, \exp(\omega_r^\top x)\right], \quad \omega_k \text{ 来自正交矩阵的列}. \tag{7.26}$$

可以证明 $\mathbb{E}_\omega[\phi^+(x)\cdot\phi^+(y)^\top] = \exp(x\cdot y)$, 即无偏估计 softmax 核。使用 r 个随机特征时, 方差为 $O(1/r)$。

3. RetNet 的固定衰减 (Sun et al., 2023):

RetNet 使用固定的指数衰减, 定义保留 (Retention) 机制:

$$A_{ij} = \gamma^{i-j}(Q_i K_j^\top/\sqrt{d}) \cdot \mathbf{1}_{i\geq j}, \quad \gamma \in (0,1), \tag{7.27}$$

等价于 $\phi(Q)_i = Q_i, \phi(K)_j = \gamma^{-j}K_j$，加上按位置的衰减因子 γ^i。多头设定下每个头有不同的 γ（“多尺度保留”），提供不同长度的记忆窗口。

7.5.4 线性注意力与 Fast Weight Memory 的等价性

Schlag et al. (2021) 指出，式 (??)（即式 (7.5.14)）的更新

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \phi(K_t)^\top V_t \tag{7.5.14}$$

可以解释为**快速权重存储器 (fast weight memory)**：状态矩阵 $\mathbf{S}_t$ 以外积 $\phi(K_t)^\top V_t$ 形式存储键值关联，查询时用 $\phi(Q_t)$ 检索。这与 Hopfield 网络和联想记忆 (associative memory) 具有深刻联系，揭示了线性注意力的记忆机制本质。在此视角下，外积存储的容量为 $O(rd)$ 比特，对应约 r 个精确键值对；当待检索的键值对数目超过 r 时，干扰效应导致检索精度下降，这从机制上解释了 §7.8 中讨论的检索任务劣势。

---

§7.6 Mamba、线性注意力（Attention）与 RNN 的统一视角

▶这一节在讲什么

为什么要统一这三类模型？Mamba、线性注意力、门控 RNN 看似来自三条完全不同的研究路线，但它们的推理时递推结构高度相似。找到统一框架能指导新模型的设计，并解释为什么它们有相似的长处和短处。

第一步：所有模型都是同一广义线性递推的特例。$\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t$，区别仅在于门控矩阵 $\mathbf{G}_t$ 的参数化方式：线性注意力取 $\mathbf{G}_t = \mathbf{1}$，RetNet 取 $\mathbf{G}_t = \gamma\mathbf{1}$，GLA 取数据依赖的 sigmoid，Mamba 取 $e^{\Delta_t A}$。

第二步：Mamba-2 的 SSD 对偶性把递推视角和矩阵视角打通。SSM 的输入-输出映射等价于一个「半分离矩阵」乘向量，这将递推模型与结构化注意力精确对应，揭示了它们是同一运算的两个计算视角。

第三步：统一框架指明改进方向。GLA 增加了数据依赖门，Mamba-2 扩大了状态维度，都是在这个框架下对门控 $\mathbf{G}_t$ 做更丰富的参数化。

本节要拿走的一件事：Mamba、线性注意力、门控 RNN 都是「广义线性递推」的实例，不同之处仅在于门控的参数化——从这个角度看，它们在同一条数学流形上，不是三类不同的模型。

直觉

在 §7.4 和 §7.5 各自的语境下，Mamba 和线性注意力看起来是从不同出发点推导出来的不同模型。但两者的推理时递推结构惊人地相似：都是用一个固定大小的状态矩阵 $\mathbf{S}_t$、用新的键值对更新它、用查询向量检索它。

本节出现三个核心概念：**广义线性递推框架、Mamba-2 的 SSD 对偶性、GLA 的数据依赖门控**。它们沿着一条从「三类模型看似不同」到「同一框架的不同参数化」的逻辑链展开。理解这条链，是理解「为什么 Mamba、线性注意力、门控 RNN 是同一条数学流形上的三个点」的关键。

第一环：广义线性递推——三类模型的共同骨架

把三类模型的推理时递推写出来，令人惊讶地相似。把它们统一写成：

$$\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t \tag{7.6.1}$$

式 (7.6.1) 中 $\mathbf{G}_t \in \mathbb{R}^{N \times d}$ 是**门控矩阵**，$\odot$ 是 Hadamard 积。三类模型的区别**仅在于** $\mathbf{G}_t$ 如何参数化：

模型	门控 $\mathbf{G}_t$	物理含义
线性注意力	$\mathbf{1}$	无遗忘，所有键值对均等累积
RetNet	$\gamma \cdot \mathbf{1}$（固定标量）	固定指数衰减，近期更重要
GLA	$\sigma(W_g x_t)$（数据依赖）	内容感知的遗忘
Mamba	$e^{\Delta_t A}$（输入依赖矩阵指数）	结构化、可控的记忆/遗忘

这一统一框架的意义不只是分类学上的整洁。它表明：**三类模型在同一个高维参数空间里，区别只在于门控 $\mathbf{G}_t$ 的参数化约束**。从这个角度看，它们不是三种不同的架构，而是同一类递推的不同实例化——就像 SGD、Adam、RMSProp 都是梯度下降的不同参数化一样。

第二环：为什么统一视角是自然涌现的——而不是事后强行统一

Yang et al.（GLA，ICML 2024）从线性注意力出发引入数据依赖门；Dao & Gu（Mamba-2/SSD，ICML 2024）从 SSM 出发推导结构化对偶性。两条独立研究路线殊途同归地发现了同一类递推框架——这本身就说明框架的自然性。

历史上，LSTM 的遗忘门、GRU 的更新门、RetNet 的保留因子，都是在不同语境下对「该遗忘多少」这一问题的不同回答。广义线性递推框架把这些答案统一到同一个数学语言里：**一切门控都是 $\mathbf{G}_t$ 的参数化，问题简化为「$\mathbf{G}_t$ 应该有多丰富的参数化」**。

从简单到复杂的进化轨迹：$\mathbf{G}_t = \mathbf{1}$（线性注意力）$\to \mathbf{G}_t = \gamma\mathbf{1}$（RetNet）$\to \mathbf{G}_t = \sigma(W_g x_t)$（GLA）$\to \mathbf{G}_t = e^{\Delta_t A}$（Mamba）。每一步都是对门控表达能力的增强，代价是更多的参数和更复杂的计算。

第三环：Mamba-2 的 SSD 对偶性——递推视角和矩阵视角的精确等价

Dao & Gu（SSD）在更严格的数学框架下证明了标量结构 SSM（A 为标量乘以单位矩阵）与「半分离矩阵乘向量」的精确等价：

$$M_{ij} = c_i \left(\prod_{t=j+1}^{i} \alpha_t \right) \beta_j \cdot \mathbf{1}_{i \geq j} \tag{7.6.2}$$

式 (7.6.2) 的对偶性揭示：**SSM 的递推视角和注意力的矩阵视角是同一运算的两个计算视图**——就像 FFT 和 DFT 是同一变换的快速算法和原始定义的关系。

这一对偶性有具体的工程意义：Mamba-2 利用它允许更大的状态维度（dN 而非 Mamba 的 dN/D），相较 Mamba 实现了 2–8 倍的速度提升，同时在语言建模指标上追平或超越了同规模的 Transformer。

第四环：GLA 的数据依赖门控——从统一视角指明改进方向

GLA 在线性注意力框架中引入 $\mathbf{G}_t = \sigma(W_\alpha x_t)$（带低秩约束 $\mathbf{G}_t = \alpha_t \beta_t^\top$）。从统一视角看，这恰好是 Mamba 的 $e^{\Delta_t A}$ 在「去掉矩阵指数结构、换成 sigmoid 激活」后的版本——两者都是对 $\mathbf{G}_t$ 的数据依赖参数化，只是复杂度和表达能力不同。

统一框架的工程价值：知道了所有模型都是 $\mathbf{G}_t$ 的参数化，改进方向就清晰了：- 增加 $\mathbf{G}_t$ 的秩（从标量衰减到矩阵门控）；- 增加 $\mathbf{G}_t$ 对输入的依赖程度（从固定到数据依赖）；- 增加状态维度 N（Mamba-2 的主要改动）。

整条链的回头看

```
Mamba 推理递推：ht = Ā_t h_{t-1} + B_t xt
线性注意力推理递推：S_t = S_{t-1} + (K_t)^T V_t      ← 结构惊人相似
  ↓ 统一为广义线性递推
S_t = G_t  S_{t-1} + k_t^T v_t                        ← 区别仅在 G_t 参数化
  ↓ G_t = 1（线性注意力）→ ·1（RetNet）→ (·)（GLA）→ e^{ΔA}（Mamba）
Mamba-2 SSD 对偶：递推  半分离矩阵乘向量              ← 递推视角 = 矩阵视角
  ↓ 对偶性允许更大状态维度：2-8x 加速
GLA 低秩门控：G_t =  _t  _t^T                         ← Mamba 和线性注意力的桥梁
  ↓ 统一框架指明旋钮：秩、输入依赖程度、状态维度
→ 后 Mamba 时代的架构设计空间
```

这条链揭示了一个深层事实：**Mamba、线性注意力、门控 RNN 不是三种不同的模型，而是同一广义线性递推在门控参数化上的三个不同选择**。统一框架的价值不在于选出最好的一个，而在于指明了哪些维度（门控丰富度、状态维度、秩约束）是可以调节的旋钮——这才是后续架构创新的设计空间。

统一的关键概念是门控矩阵 $\mathbf{G}_t$。把递推写成 $\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t$，三类模型的区别就只在 $\mathbf{G}_t$ 如何参数化：线性注意力不做遗忘（$\mathbf{G}_t = \mathbf{1}$），RetNet 用固定衰减（$\mathbf{G}_t = \gamma\mathbf{1}$），GLA 用数据依赖的 sigmoid，Mamba 用输入依赖的矩阵指数 $e^{\Delta_t A}$。从 Yang et al.（GLA）和 Dao & Gu（Mamba-2/SSD）从两条独立路线殊途同归地发现这一统一，这本身就说明了框架的自然性。

Mamba-2 的 SSD 对偶性更进一步，从代数角度精确描述了递推视角和矩阵视角的等价性：SSM 的输入-输出映射等价于一个「半分离矩阵」乘向量，这与结构化注意力精确对应。这一对偶性不仅是理论上的优美，也有工程意义：Mamba-2 利用它允许更大的状态维度并实现了 2–8 倍的速度提升。**统一框架的价值不在于帮你选择一个最好的模型，而在于指明了哪些维度（门控的参数化、状态维度、秩约束）是可以调节的旋钮。**

7.6.1 通用递推形式

Yang et al. (GLA, ICML 2024) 和 Dao & Gu (Mamba-2/SSD, ICML 2024) 分别从不同方向指出:Mamba、线性 Attention 及经典门控 RNN 可以统一到同一框架下。

定义 7.5 (广义线性递推): 定义隐状态 $\mathbf{S}_t \in \mathbb{R}^{N\times d}$ 的递推

$$\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t, \qquad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t, \tag{7.28}$$

其中 $\mathbf{G}_t \in \mathbb{R}^{N\times d}$ 是门控矩阵,$\odot$ 表示 Hadamard 积,$\mathbf{k}_t, \mathbf{q}_t \in \mathbb{R}^N$,$\mathbf{v}_t \in \mathbb{R}^d$。

三类模型在此框架中的实例化:

模型	门控 $\mathbf{G}_t$ 的参数化
线性注意力 (Katharopoulos)	$\mathbf{G}_t = \mathbf{1}$(恒等, 无遗忘)
RetNet (Sun et al., 2023)	$\mathbf{G}_t = \gamma \cdot \mathbf{1}$(固定标量衰减)
GLA (Yang et al., 2024)	$\mathbf{G}_t = \sigma(W_g x_t)$(数据依赖门)
Mamba (Gu & Dao, 2023)	$\mathbf{G}_t = e^{\Delta_t A}$(输入依赖离散化, 结构化对角)

7.6.2 Mamba-2 的结构化状态空间对偶性

Dao & Gu (SSD, ICML 2024) 在更严格的数学框架下建立了 SSM 与结构化注意力之间的精确对应:

定理 7.6 (结构化状态空间对偶): 考虑标量结构的 SSM(A 为标量乘以单位矩阵), 对每个特征维度:

$$h_t = \alpha_t h_{t-1} + \beta_t x_t, \qquad y_t = c_t h_t, \tag{7.29}$$

其中 $\alpha_t, \beta_t, c_t \in \mathbb{R}$ 为标量。则此模型的输入-输出映射与如下**半分离 (semiseparable) 矩阵**乘向量等价:

$$Y = MX, \quad M_{ij} = c_i \left(\prod_{t=j+1}^{i} \alpha_t \right) \beta_j \cdot \mathbf{1}_{i\geq j}, \tag{7.30}$$

矩阵 M 是 1 阶半分离矩阵。在多头设定下, 矩阵 M 升维为具有更丰富块结构的半分离矩阵族, 统一了 SSM 的递推视角和注意力的矩阵视角。

由式 (7.30) 的对偶性可知:SSM 可以被视为结构化、低秩化的注意力矩阵运算。Mamba-2 利用此对偶性设计了新的 SSD(Structured State Space Duality) 层, 允许更大的状态维度 (dN 而非 Mamba 的 dN), 相较 Mamba 实现了 2-8 倍的速度提升。

7.6.3 GLA 的数据依赖门控

GLA(Gated Linear Attention, Yang et al., ICML 2024) 在线性注意力框架中引入数据依赖的门控:

$$\mathbf{S}_t = (\mathbf{G}_t \odot \mathbf{S}_{t-1}) + \mathbf{k}_t^\top \mathbf{v}_t, \quad \mathbf{G}_t = \sigma(W_\alpha x_t) \in (0,1)^{N\times d}, \tag{7.31}$$

式 (7.31) 中 σ 是 sigmoid 函数。GLA 中对 $\mathbf{G}_t$ 施加了”低秩”结构约束:$\mathbf{G}_t = \alpha_t \beta_t^\top (\alpha_t \in \mathbb{R}^N, \beta_t \in \mathbb{R}^d)$, 减少参数量同时保持表达能力。GLA 可以证明等价于选择性 SSM: 门控

$\mathbf{G}_t$ 对应 Mamba 中的 $\bar{A}_t$，而 $\mathbf{k}_t^\top \mathbf{v}_t$ 对应 $\bar{B}_t x_t \cdot C_t^\top$。这一等价性表明，从线性注意力和 SSM 两条独立路径，研究者们殊途同归地发现了同一类数据依赖线性递推。

---

§7.7 扩散语言模型的数学

▶这一节在讲什么

为什么要把扩散模型用于语言？图像扩散模型在连续像素空间加高斯噪声；语言是离散 token，不能直接加连续噪声。离散扩散的解决方案是用「掩码替换」代替「加噪声」，从而把扩散的数学框架搬到 token 空间。

第一步：连续 DDPM 复习——前向加噪 + 反向去噪。前向过程把 x_0 逐步混入高斯噪声直至变成纯噪声；反向过程学习从噪声恢复 x_0；训练目标简化为「预测加入的噪声 ϵ」，等价于分数匹配。

第二步：离散扩散用吸收态（MASK）替代高斯噪声。前向过程以概率 β_t 将 token 替换为 [MASK]；边际分布有闭式：时刻 t 时 token 要么仍是原值（概率 $\bar{\alpha}_t$），要么已被 MASK（概率 $1-\bar{\alpha}_t$）。

第三步：MDLM 把 ELBO 化简为加权 MLM 损失。对被 MASK 的位置同时预测原始 token，权重系数 $|\alpha_t'|/\alpha_t$ 对不同噪声水平重要性加权——本质上是一个多尺度的遮蔽语言模型目标。

本节要拿走的一件事：离散扩散语言模型的训练目标是「加权 MLM 损失」，生成时可以并行对所有被 MASK 的位置同时预测——这与自回归模型严格左到右的单步生成形成数学上的本质对立。

直觉

自回归语言模型以严格的左到右顺序生成文本：每一步预测一个 token，条件于所有已生成的前缀。这是一个非常特殊的生成方向——为什么不能同时生成所有位置，或者从中间开始生成？扩散语言模型提供了另一条路。

本节出现四个核心概念：**连续 DDPM 的前向-反向框架**、**离散扩散的吸收态转移矩阵**、**MDLM 的加权 MLM 简化目标**、**与自回归的本质差异**。它们沿着一条从「连续扩散如何迁移到离散 token 空间」的逻辑链展开。

第一环：连续 DDPM——前向加噪 + 反向去噪的数学框架

DDPM（Ho et al., 2020）的核心思想：前向过程把数据 x_0 逐步混入高斯噪声，直到变成纯噪声 $x_T \sim \mathcal{N}(0, I)$；反向过程学习从噪声逐步恢复数据。由于高斯分布的性质，边际分布有闭式形式：

$$q(x_t|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)I) \tag{7.7.1}$$

式 (7.7.1) 中 $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$ 随 t 从 1 衰减到 0。这个闭式形式至关重要：它允许在训练时任意采样噪声水平 t，而不需要逐步模拟前向过程。

训练目标简化为「预测加入的噪声 ϵ」：$\mathcal{L} = \mathbb{E}[\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$（连续形式由式

(7.7.1) 导出）。这等价于分数匹配——模型在学习在每个噪声水平 t 处的分数函数 $\nabla_{x_t} \log q_t(x_t)$，指向高密度方向。

第二环：离散扩散的关键困难——为什么不能直接迁移

DDPM 依赖两个连续空间的关键性质：1. 高斯分布可以相加（前向过程）；2. 高斯分布的边际有闭式形式。

token 是离散的整数，不存在「加高斯噪声」这个操作。如果强行把 token 嵌入连续空间再加噪，那是一种间接路线，且不保留 token 的离散结构。

离散扩散的解决方案是用**吸收态**（absorbing state）替代高斯噪声：以概率 β_t 把每个 token 替换为 [MASK]。这是在 token 空间内定义的前向过程，完全离散。关键性质：[MASK] 是吸收态——一旦被 MASK，后续步骤不会恢复原始 token（前向过程单调）。

第三环：吸收态转移矩阵——离散扩散的精确数学

前向转移矩阵 $Q_t = (1-\beta_t)I + \beta_t \mathbf{1}e_M^\top$ 表示：以概率 $1-\beta_t$ 保持不变，以概率 β_t 变为 [MASK]。

令 $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$，边际分布有闭式形式——与连续 DDPM 的式 (7.7.1) 完全对应：

$$q(x_t|x_0) = \bar{\alpha}_t \delta_{x_t,x_0} + (1-\bar{\alpha}_t)\delta_{x_t,M} \tag{7.7.2}$$

即时刻 t 的 token 要么仍是原始值（概率 $\bar{\alpha}_t$），要么已被 MASK（概率 $1-\bar{\alpha}_t$）。式 (7.7.2) 与式 (7.7.1) 形成精确的离散/连续对应：**这个干净的二值边际分布是离散扩散比连续扩散更简洁的地方**——没有连续扩散的随机性污染，每个 token 的状态非黑即白。

后向条件分布也有闭式：若 x_t 未被 MASK，后向分布退化为 δ_{x_0}（确定性）；若 x_t 已被 MASK，后向分布依赖 x_0 的估计（概率性）。这使得 ELBO 的计算非常高效。

第四环：MDLM 简化目标——加权 MLM 的变分推导

MDLM（Sahoo et al., 2024）利用吸收态的特殊结构，将 ELBO 化简为（由式 (7.7.2) 的边际分布导出）：

$$\mathcal{L}_{\text{MDLM}} = -\mathbb{E}_{t,x_0}\left[\frac{|\alpha_t'|}{\alpha_t}\sum_{i:x_t^{(i)}=M}\log p_\theta(x_0^{(i)}|x_t)\right] \tag{7.7.3}$$

式 (7.7.3) 本质上是**加权的 MLM（遮蔽语言模型）损失**：仅对被 MASK 的位置计算预测损失，权重系数 $|\alpha_t'|/\alpha_t$ 对不同噪声水平重要性加权。这个目标形式上与 BERT 的 MLM 非常相似，但有严格的变分推导支撑，权重项来自 ELBO 推导中的噪声调度微分。

与自回归的本质差异：AR 模型因训练时只看 $x_{<i}$，对逆向查询有系统性弱点（「谁发明了 X」vs「X 是谁发明的」的不对称）。扩散模型以全序列为条件进行去噪（由式 (7.7.3) 的目标实现），对所有位置**对称**建模，在逆向任务上有理论优势，这在 LLaDA 实验中已得到验证。代价是生成需要多步迭代（通常 64–256 步），每步都需要完整的前向传播，延迟高于 AR 的每步单 token 生成。

整条链的回头看

```
自回归语言模型：p(x) =  p(x_i|x_{<i})，严格左到右
  ↓ 问题：无法并行生成；逆向查询有系统性弱点
```

```
连续 DDPM：高斯前向加噪 + 分数匹配反向去噪
  ↓ 问题：token 是离散的，无法加高斯噪声
吸收态离散扩散：以概率 _t 将 token → [MASK]    ← 边际分布有闭式：非黑即白
  ↓ 前向边际 q(x_t|x_0) = _t · (x_t,x_0) + (1- _t) · (x_t,M)
后向条件分布闭式 → ELBO 高效可算
  ↓ 吸收态特殊结构 → Rao-Blackwellized 化简
MDLM 目标 = 加权 MLM 损失                          ← | '_t|/ _t 对噪声水平重要性加权
  ↓ 生成时并行预测所有 MASK 位置（T步迭代精化）
消除逆向诅咒，代价是多步推理延迟
```

这条链揭示了一个深层事实：**离散扩散语言模型的训练目标是「加权 MLM 损失」，这不是近似，而是在吸收态转移矩阵结构下 ELBO 的精确推导**。它与自回归的本质对立体现在生成方向的对称性上：AR 以左到右的因果链生成，扩散以全局条件的并行迭代精化生成——两者是对「序列生成」这一问题的两种完全不同的数学解答。

离散扩散的数学形式依然干净。前向过程通过吸收态转移矩阵 Q_t 以概率 β_t 把每个 token 变成 [MASK]；边际分布有闭式形式：时刻 t 的 token 要么还是原始值 x_0（概率 $\bar{\alpha}_t$），要么已被 MASK（概率 $1-\bar{\alpha}_t$）。后向条件分布也有闭式，这使得 ELBO 的计算变得高效。MDLM 进一步将 ELBO 化简为加权 MLM 损失——仅对被 MASK 的位置计算预测损失，权重由噪声调度决定。这个目标形式上与 BERT 的掩码语言模型非常相似，但有严格的变分推导支撑。

自回归与离散扩散之间的数学差异不仅是技术细节，而是根本性的。自回归模型因为训练时只看 $x_{<i}$ 而对「逆向查询」存在系统性弱点（如「谁发明了 X」→「X 是谁发明的」的不对称）。扩散模型以全序列为条件进行去噪，对所有位置对称建模，在逆向任务上有理论优势。代价是生成需要多步迭代（通常 64–256 步），每步都需要完整的前向传播，延迟高于自回归的每步单 token 生成。

7.7.1 连续扩散:DDPM 回顾

去噪扩散概率模型 (DDPM, Ho et al., NeurIPS 2020) 通过马尔可夫链定义数据的渐进加噪过程。给定数据 $x_0 \sim q_{\text{data}}$, 前向过程为:

$$q(x_t|x_{t-1}) = \mathcal{N}\Big(\sqrt{1-\beta_t}\,x_{t-1},\ \beta_t I\Big), \quad t = 1, \dots, T, \tag{7.32}$$

其中 $\{\beta_t\}_{t=1}^T$ 是方差调度 (variance schedule)。令 $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$, 则边际分布有闭式形式——即直觉节式 (7.7.1) 的严格版本:

$$q(x_t|x_0) = \mathcal{N}\Big(\sqrt{\bar{\alpha}_t}\,x_0,\ (1-\bar{\alpha}_t)I\Big). \tag{7.33}$$

反向去噪: 学习参数化的反向过程 $p_\theta(x_{t-1}|x_t)$。通过变分下界 (ELBO) 与**去噪分数匹配**的联系, 最优化目标（ϵ-预测损失）简化为

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t,x_0,\epsilon}\left[\Big\|\epsilon - \epsilon_\theta\Big(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon,\ t\Big)\Big\|^2\right], \tag{7.34}$$

其中 $\epsilon \sim \mathcal{N}(0, I)$。式 (7.34) 等价于在每个噪声水平 t 上的**分数匹配** (score matching)（式 (7.35)）：

$$\nabla_{x_t} \log q_t(x_t) \approx -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}}. \tag{7.35}$$

分数函数的直觉: 在样本 x_t 处, 分数 $\nabla_{x_t} \log q_t(x_t)$ 指向高概率密度方向。去噪网络 ϵ_θ 估计添加的噪声, 等价于估计分数函数的负值, 从而引导采样轨迹朝数据流形靠拢。

7.7.2 离散扩散: 吸收态过渡矩阵

连续扩散对连续向量加高斯噪声, 但语言模型处理的是离散 token。**离散扩散 (Discrete Diffusion)** 通过马尔可夫链在 token 空间 $\{1, \dots, V\}^L$ 上定义噪声过程。

吸收态 (Absorbing State) 扩散: 引入特殊 [MASK] token(索引 M), 定义前向转移矩阵:

$$Q_t = (1-\beta_t)I + \beta_t \mathbf{1} e_M^\top, \tag{7.36}$$

其中 e_M 是 [MASK] 的 one-hot 向量。即以概率 β_t 将当前 token 替换为 MASK, 以概率 $1-\beta_t$ 保持不变。边际分布为

$$q(x_t|x_0) = \bar{\alpha}_t \delta_{x_t, x_0} + (1-\bar{\alpha}_t)\delta_{x_t, M}. \tag{7.37}$$

其中 $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$ (与式 (7.33) 中的 $\bar{\alpha}_t$ 定义相同)。在时刻 t,token 要么仍是原始 token x_0(概率 $\bar{\alpha}_t$), 要么已被 MASK(概率 $1-\bar{\alpha}_t$)——式 (7.37) 是式 (7.33) 的离散版本。

后向条件分布由贝叶斯公式给出:

$$q(x_{t-1}|x_t, x_0) = \begin{cases} \delta_{x_{t-1}, x_0} & \text{若} x_t \neq M, \\ \frac{(1-\bar{\alpha}_{t-1})\delta_{x_{t-1}, M} + (\bar{\alpha}_{t-1} - \bar{\alpha}_t)\delta_{x_{t-1}, x_0}}{1-\bar{\alpha}_t} & \text{若} x_t = M. \end{cases} \tag{7.38}$$

此闭式形式使得训练中的 ELBO 计算非常高效。

7.7.3 MDLM 的简化目标

MDLM(Masked Diffusion Language Model, Sahoo et al., NeurIPS 2024) 推导了吸收态离散扩散的变分下界的 **Rao-Blackwellized 简化形式**。

由式 (7.37) 和式 (7.38) 的特殊结构,ELBO 化简为:

$$\mathcal{L}_{\text{MDLM}} = -\mathbb{E}_{t, x_0}\left[\frac{|\alpha_t'|}{\alpha_t} \sum_{i: x_t^{(i)} = M} \log p_\theta(x_0^{(i)}|x_t)\right], \tag{7.39}$$

其中 $\alpha_t' = d\alpha_t/dt$, 求和仅对被 MASK 的位置 i 进行。式 (7.39) 即式 (7.7.3) 的严格形式, **本质上是加权的 MLM 损失的混合**, 权重系数 $|\alpha_t'|/\alpha_t$ 对不同噪声水平重要性加权——由式 (7.37) 和式 (7.38) 共同决定。

LLaDA(2025) 将此框架扩展至 8B 参数规模, 证明了离散 Mask 扩散与自回归语言模型在同等工程实践下具有可比性能。

7.7.4 与自回归模型的本质差异

自回归 (AR) 语言模型基于因果分解 $p(x) = \prod_i p(x_i|x_{<i})$, 生成时严格左到右顺序进行, 时间复杂度为 $O(L)$ 步。

离散扩散模型则有以下数学差异:

1. **并行解码**: 在任意一步去噪 $p_\theta(x_0|x_t)$ 中, 所有被 MASK 的位置**同时**预测, 不存在左到右的顺序依赖。理论上, 仅需 1 步去噪即可并行生成整个序列, 但质量较差; 实际使用 $T = 64 \sim 256$ 步迭代精化。
2. **迭代精化**: 每步将部分 MASK token 替换为预测值, 逐步精化全序列。理论上, 给定足够的步数 T, 采样质量随 T 单调提升, 等价于更精确地模拟真实反向马尔可夫链。
3. **消除逆向诅咒 (Reversal Curse)**:AR 模型因训练时只看 $x_{<i}$ 而对逆向查询存在系统性弱点。扩散模型以全序列为条件进行去噪, 对所有位置对称建模, 在逆向任务上有理论优势, 这在 LLaDA 实验中已得到验证。

---

§ 7.8 世界模型与预测编码的数学

▶这一节在讲什么

为什么「世界模型」需要数学化? 仅凭「让智能体预测未来」的目标太模糊，不同的形式化选择——预测哪个空间、用什么先验、如何防止 collapse——导致截然不同的模型行为和优化难度。本节拆解四种支撑性数学框架。

第一步：层级预测编码把视觉皮层建模为误差传递网络。每层对下层做「自顶向下预测」，预测误差逐层上传，权重更新仅依赖相邻层信号——这在数学上等价于反向传播，但完全局部。

第二步：变分自由能把感知与行动统一为同一极小化目标。最小化自由能 $F = D_{\mathrm{KL}}(q\|p(z|x)) - \log p(x)$，同时做感知推断（让 q 逼近后验）和「惊奇最小化」(最大化证据)。这与 VAE 的 ELBO 完全等价。

第三步：JEPA 在表示空间预测以规避像素级重建的高方差。能量函数 $E(x, y) = \|s_\phi(y) - P_\psi(s_\phi(x), z)\|^2$，配合 VICReg 三项正则化防止表示 collapse。

本节要拿走的一件事：世界模型的数学骨架不是全新发明，而是把变分推断、Kalman 滤波、对比学习按新的工程目标重新组装——Kalman 增益的误差修正规则与预测编码的局部更新在结构上完全对应。

直觉

「世界模型」这个词在强化学习和认知科学中有不同含义，但数学层面的核心问题是共同的：给定当前观测，如何在**表示空间**中对未来状态进行预测，并用预测误差来驱动学习?

本节出现四个核心概念：**层级预测编码的误差传递规则、变分自由能原理（等价于 ELBO）、JEPA 在表示空间预测 + VICReg 防 collapse、Kalman 滤波作为精确贝叶斯推**

断。它们看似来自神经科学、统计物理、视觉学习、控制理论四个不同领域，实际上沿着一条共同的逻辑链——「预测 → 误差 → 更新」——一个推出一个。

第一环：层级预测编码——误差信号如何在层级之间传播

Rao & Ballard（1999）提出的层级预测编码框架将视觉皮层建模为一个层级生成系统：每一层对下层做「自顶向下预测」$\hat{x}_{l-1} = f_l(x_l)$，预测误差 $\varepsilon_{l-1} = x_{l-1} - \hat{x}_{l-1}$ 向上传播。

表示更新规则（高斯假设下）：

$$\Delta x_l \propto \Pi_l \varepsilon_l - \frac{\partial f_l}{\partial x_l}^{\top} \Pi_{l-1} \varepsilon_{l-1} \tag{7.8.1}$$

右侧第一项是「来自上方的误差」，第二项是「对下方预测残差通过 Jacobian 反传回来的误差」。这一**局部更新规则**完全依赖相邻层之间的信号，无需全局梯度通路。

Whittington & Bogacz（2017）严格证明：在「误差单元与表示单元分离」等条件下，预测编码的权重更新与标准反向传播在数值上收敛到相同结果。这意味着预测编码不是生物学特殊曲解，而是反向传播在局部约束下的一个有效近似。工程意义：世界模型的「预测-误差-更新」规则在生物学层面也有理论支撑。

第二环：变分自由能——把感知与行动统一为同一极小化目标

Friston（2010）的自由能原理把感知、行动、学习都化为「最小化变分自由能 F」：

$$F[q, x] = D_{\mathrm{KL}}(q(z) \,\|\, p(z|x)) - \log p(x) = -\mathrm{ELBO}(q, x) \tag{7.8.2}$$

最小化 F 同时实现两个目标：让 $q(z)$ 逼近真实后验（感知推断），以及最大化边际似然（避免惊奇）。这与 VAE 的训练目标**完全等价**——VAE 的重构损失加 KL 正则项就是 F 的蒙特卡洛近似。

这个等价是深刻的：**世界模型不需要发明新的数学，它就是 VAE 的泛化**。感知 = E 步（更新 q），学习 = M 步（更新生成模型参数），行动 = 通过运动通道抑制感觉预测误差（主动推断）。

主动推断把行动纳入 F 的极小化：$\partial F / \partial a = \sum_l (\partial F / \partial \varepsilon_l)(\partial \varepsilon_l / \partial a)$，感知与行动成为同一极小化问题的两个分支。这在概念上统一了感知与行为，但在高维连续动作空间的扩展上仍有大量未解决问题。

第三环：JEPA——在表示空间预测以规避像素级重建的高方差

JEPA（LeCun, 2022）的洞察：在像素空间预测未来帧需要模型精确重建视觉细节，方差极高、收敛困难。把预测目标移到表示空间，模型只需预测「语义等价的表示」：

$$E(x, y) = \| s_\phi(y) - P_\psi(s_\phi(x), z) \|^2 \tag{7.8.3}$$

式 (7.8.3) 中的能量函数比像素级 MSE 低得多的方差来自一个直觉：表示空间过滤掉了像素级的随机噪声（光照变化、纹理细节），保留了语义结构。

但表示空间预测面临 **collapse 问题**——所有输入映射到同一点就能让误差为零。VICReg 的三项正则化阻止 collapse：不变性项最小化两视图嵌入之差，方差项惩罚任意

维度标准差低于阈值，协方差项惩罚维度间线性相关。从信息论角度看，式 (7.8.12) 中协方差项的极小化是对表示进行近似白化——与去冗余原理一脉相承。

第四环：Kalman 滤波——线性高斯 SSM 的精确贝叶斯推断

SSM 在线性高斯假设下（A, C, Q, R 已知），后验 $p(x_t|y_{1:t})$ 保持高斯，Kalman 滤波可以精确递推：

预测步：$\hat{x}_{t|t-1} = A\hat{x}_{t-1|t-1}$，$P_{t|t-1} = AP_{t-1|t-1}A^\top + Q$

更新步：$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(y_t - C\hat{x}_{t|t-1})$，$K_t = P_{t|t-1}C^\top(CP_{t|t-1}C^\top + R)^{-1}$

Kalman 增益 K_t 的物理意义：**当预测不确定性 $P_{t|t-1}$ 大时，新观测权重高；当观测噪声 R 大时，权重低**。这与预测编码的更新规则 $\Delta x \propto K(y - C\hat{x})$ 在结构上完全对应——$y_t - C\hat{x}_{t|t-1}$ 正是观测层的预测误差，K_t 正是精度矩阵的角色。

Mamba 可视为 Kalman 滤波的**数据依赖非线性推广**：用输入门控的 $\bar{A}(x_t), \bar{B}(x_t)$ 替代固定的 A, B，使噪声协方差的角色由网络动态承担，但「预测 → 误差 → Kalman 增益加权更新」的递推结构保持不变。

整条链的回头看

```
预测-误差-更新的共同骨架
  ↓ 四种数学语言
层级预测编码: Δx  Π·  - J^T·Π·            ← 局部规则   反向传播
  ↓ 等价于
变分自由能 F = KL(q||p(z|x)) - log p(x)    ← F = -ELBO, 与 VAE 完全等价
  ↓ 感知 = E 步, 行动 = 运动误差抑制
JEPA: 在表示空间预测, VICReg 防 collapse    ← 避免像素级高方差; 信息论白化
  ↓ 线性高斯假设
Kalman 滤波: K_t = P C^T(CPC^T+R)^{-1}     ← 精确后验; K_t = 精度加权增益
  ↓ 数据依赖非线性推广
Mamba: Ā(x_t) 替代固定 A                    ← 同一「预测-更新」结构的神经网络版
```

这条链揭示了一个深层事实：**世界模型的数学骨架不是全新发明，而是把变分推断、Kalman 滤波、对比学习这些已有工具按新的工程目标重新组装**。它们在「预测误差驱动更新」这一核心逻辑下天然统一——Kalman 增益的误差修正规则与预测编码的局部更新在结构上完全对应，自由能与 ELBO 的等价让神经科学与机器学习在数学上打通。

变分自由能原理提供了最宏观的统一：感知、行动、学习都是「最小化式 (7.8.8) 的自由能 F」这一目标的不同分支。式 (7.8.9) 表明该框架与 VAE 的训练目标完全等价——VAE 的重构损失加 KL 正则项就是 F 的蒙特卡洛近似。活跃推断（active inference）更进一步把行动也纳入 F 的极小化：感知更新对应 E 步，行动更新对应本体感觉通道的预测误差抑制（参见式 (7.8.10)）。这个框架理论上优美，但计算上仍有大量未解决的扩展问题（如高维连续动作空间）。

JEPA 的洞察则针对世界模型的一个具体痛点：在像素空间预测未来帧需要模型精确重建视觉细节，方差极高、收敛困难。把预测目标移到表示空间，模型只需预测「语义等价的表示」而非「像素级精确重建」，大幅降低了学习难度。但表示空间预测面临 collapse 问题——所有输入映射到同一点就能让误差为零。VICReg 的三项正则化（不变性、方差、协方差）从信息论角度强制表示空间保持「充盈性」，这与白化、去冗

余原理一脉相承。

世界模型的工程目标是让智能体在**表示空间**中对未来状态进行预测，并以此驱动规划与控制。本节从数学角度拆解支撑这一目标的四类形式化：层级预测编码、变分自由能、JEPA 能量函数与防 collapse 正则化、以及 Kalman 滤波作为线性高斯 SSM 是让智能体在**表示空间**中对未来状态进行预测，并以此驱动规划与控制。本节从数学角度拆解支撑这一目标的四类形式化：层级预测编码、变分自由能、JEPA 能量函数与防 collapse 正则化、以及 Kalman 滤波作为线性高斯 SSM 的精确推断。它们共同构成当代世界模型的数学骨架。

7.8.1 预测编码：层级生成模型与误差传递

Rao & Ballard (1999) 提出的层级预测编码框架将视觉皮层建模为一个层级生成系统。设共有 $L+1$ 层，第 l 层持有表示 x_l，第 l 层对第 $l-1$ 层的**自顶向下预测**为

$$\hat{x}_{l-1} = f_l(x_l), \tag{7.8.4}$$

其中 f_l 是可学习的生成映射。预测误差 (prediction error) 定义为

$$\varepsilon_{l-1} = x_{l-1} - \hat{x}_{l-1}. \tag{7.8.5}$$

联合生成模型写作

$$p(x_0, x_1, \dots, x_L) = p(x_L) \prod_{l=0}^{L-1} p(x_l \mid x_{l+1}), \tag{7.8.6}$$

其中每个条件因子在高斯假设下取 $p(x_l \mid x_{l+1}) = \mathcal{N}(x_l;\, f_{l+1}(x_{l+1}),\, \Pi_l^{-1})$，精度矩阵 Π_l 刻画该层预测的置信度。

在此高斯假设下，最大化对数似然等价于最小化**精度加权误差平方和**。对表示 x_l 求负对数似然的梯度，得到表示更新规则：

$$\Delta x_l \;\propto\; \Pi_l\, \varepsilon_l \;-\; \frac{\partial f_l(x_l)}{\partial x_l}^{\top} \Pi_{l-1}\, \varepsilon_{l-1}. \tag{7.8.7}$$

右侧第一项是**来自上方的误差信号**（当前层的预测与实际值的差异），第二项是**来自下方的误差信号**（当前层对下一层的预测残差通过雅可比（Jacobian）反传回来）。式 (7.8.7) 的**局部更新规则**完全依赖相邻层之间的信号，无需全局梯度通路。

Whittington & Bogacz (2017) 严格证明：当网络满足”误差单元与表示单元分离”以及”固定预测权重”等条件时，预测编码的权重更新与标准误差反向传播算法在数值上收敛到相同结果。式 (7.8.7) 的等价性表明预测编码并非生物学的特殊曲解，而是反向传播在局部学习规则约束下的一个有效近似。

7.8.2 自由能原理与变分下界

Friston (2010) 将感知与行动统一为一个极小化目标: **变分自由能** (variational free energy)。设观测为 x, 隐变量为 z, 变分后验为 $q(z)$, 变分自由能定义为

$$F[q, x] \;=\; \mathbb{E}_{q(z)}[\log q(z) - \log p(x, z)]\,. \tag{7.8.8}$$

将联合概率分解为 $\log p(x, z) = \log p(x \mid z) + \log p(z)$ 并重组, 得到与证据下界 (ELBO) 的关系:

$$F[q, x] \;=\; D_{\mathrm{KL}}(q(z) \,\|\, p(z \mid x)) - \log p(x) \;=\; -\,\mathrm{ELBO}(q, x). \tag{7.8.9}$$

由于 $\log p(x)$ 对 q 不可优化, 最小化式 (7.8.8) 中的 F 同时实现两个目标: 其一, 令 $q(z)$ 靠近真实后验 $p(z \mid x)$(感知推断); 其二, 间接最大化边际似然 $\log p(x)$(避免惊奇, 即 surprise minimization)。与变分自编码器 (VAE) 的训练目标完全等价——VAE 的重构损失加 KL 正则项正是式 (7.8.9) 的蒙特卡洛近似。

主动推断 (active inference) 在此框架中把行动 a 纳入自由能极小化: 感知更新通过 $\partial F/\partial z = 0$ 调整隐变量表示, 行动更新通过

$$\frac{\partial F}{\partial a} = \sum_{l} \frac{\partial F}{\partial \varepsilon_l} \cdot \frac{\partial \varepsilon_l}{\partial a} \tag{7.8.10}$$

调整运动输出, 使感觉预测误差在本体感觉通道中得到抑制。感知与行动因此成为式 (7.8.10) 所描述的同一极小化问题的两个分支, 分别对应变分 E 步与 M 步的连续时间版本。

7.8.3 JEPA 的能量函数与防 collapse 机制

LeCun (2022) 提出**联合嵌入预测架构** (Joint-Embedding Predictive Architecture,JEPA), 其核心思想是在**表示空间**而非像素空间中执行预测, 从而规避生成模型在像素级重建上的高方差问题。设 x 为上下文观测,y 为目标观测,s_ϕ 为编码器,P_ψ 为预测器,z 为描述 x 与 y 差异的隐变量, 能量函数定义为

$$E(x, y) \;=\; \big\| s_\phi(y) \;-\; P_\psi\big(s_\phi(x),\, z\big) \big\|^2. \tag{7.8.11}$$

最小化式 (7.8.11) 要求预测器能够从上下文表示与差异变量 z 中复原目标表示。V-JEPA (Bardes et al., 2024) 将此框架应用于视频时序预测, 在屏蔽的时空块上执行表示层预测。

Collapse 问题: 若编码器退化为常数映射 $s_\phi(\cdot) \equiv \mathbf{c}$, 则 $E \equiv 0$ 对任意 x, y 成立, 训练目标被平凡解满足。VICReg (Bardes et al., ICLR 2022) 通过三项显式正则化阻止 collapse:

- **不变性项** (Invariance):$I(Z, Z') = \dfrac{1}{n}\displaystyle\sum_{i=1}^{n} \|Z_i - Z'_i\|^2$, 最小化两视图嵌入之差;

- **方差项** (Variance):$V(Z)=\frac{1}{d}\sum_{j=1}^{d}\max\Big(0,\ \gamma-\sqrt{\mathrm{Var}(Z_{:,j})+\epsilon}\Big)$, 惩罚任意维度标准差低于阈值 γ;
- **协方差项** (Covariance):$C(Z)=\frac{1}{d}\sum_{i\neq j}\big[\mathrm{Cov}(Z)\big]_{ij}^{2}$, 惩罚不同维度之间的线性相关性。

VICReg 目标函数为

$$L_{\text{VICReg}}\ =\ \lambda\, I(Z,Z')\ +\ \mu\, V(Z)\ +\ \nu\, C(Z), \tag{7.8.12}$$

其中 λ,μ,ν 为超参数。式 (7.8.12) 中方差项直接对抗维度坍缩, 协方差项鼓励嵌入维度间信息不冗余, 二者共同维持表示空间的”充盈性”(informativeness)。从信息论角度看,$C(Z)$ 的极小化是对表示进行近似白化, 与去冗余原理 (redundancy reduction) 一脉相承。

7.8.4 Kalman 滤波作为线性高斯 SSM 的精确推断, 以及神经过程视角

§7.1 已将状态空间模型 (SSM) 写为 $x_t=Ax_{t-1}+w_t$、$y_t=Cx_t+v_t$, 其中 $w_t\sim\mathcal{N}(0,Q)$,$v_t\sim\mathcal{N}(0,R)$。当 A,C,Q,R 全部已知且线性高斯时, 后验 $p(x_t\mid y_{1:t})$ 保持高斯, 可用 Kalman (1960) 滤波器在 $\mathcal{O}(n^3)$ 时间内精确递推。

预测步 (predict):

$$\hat{x}_{t|t-1}=A\,\hat{x}_{t-1|t-1},\qquad P_{t|t-1}=A\,P_{t-1|t-1}\,A^{\top}+Q. \tag{7.8.13}$$

更新步 (update):

$$K_t=P_{t|t-1}\,C^{\top}\Big(C\,P_{t|t-1}\,C^{\top}+R\Big)^{-1}, \tag{7.8.14}$$

$$\hat{x}_{t|t}=\hat{x}_{t|t-1}+K_t\Big(y_t-C\,\hat{x}_{t|t-1}\Big),\qquad P_{t|t}=(I-K_tC)\,P_{t|t-1}. \tag{7.8.15}$$

式 (7.8.14) 给出的 Kalman 增益 K_t 是协方差矩阵的函数: 当预测不确定性 $P_{t|t-1}$ 大时, 新观测权重高; 当观测噪声 R 大时, 权重低。式 (7.8.15) 的更新步在结构上与预测编码式 (7.8.7) 的误差修正规则 $\Delta x\propto K(y-C\hat{x})$ 完全对应——$y_t-C\hat{x}_{t|t-1}$ 正是观测层的预测误差。

Mamba(§7.2) 可视为 Kalman 滤波的**数据依赖非线性推广**: 用输入门控的选择矩阵 $\bar{A}(x_t),\bar{B}(x_t)$ 替代固定的 A,B, 使得噪声协方差的角色由网络动态承担, 但预测-更新的递推结构保持不变。

神经过程 (Neural Processes) Garnelo et al., ICML 2018 将贝叶斯滤波的后验近似交给神经网络, 以**摊销推断** (amortized inference) 替代逐步递推: 一个编码器将上下文集合 $\{(x_i,y_i)\}_{i=1}^{n}$ 映射为全局潜在表示 z, 解码器再条件于 z 生成目标预测。这一机制在计算上等价于把 Kalman 增益的迭代计算”压缩”进一次前向传播。

DreamerV3 Hafner et al. (2023) 的循环状态空间模型 (RSSM) 将上述路线推至强化学习世界模型: 隐状态分为**确定性分量** h_t(由 GRU 递推维护, 对应 $\hat{x}_{t|t-1}$) 和**随机分量** $z_t \sim q_\phi(z_t \mid h_t, o_t)$(对应后验更新)。变分目标最小化

$$\mathcal{L}_{\text{RSSM}} = \mathbb{E}_q\left[\sum_t -\log p_\theta(o_t \mid h_t, z_t) + \beta\, D_{\text{KL}}\big(q_\phi(z_t \mid h_t, o_t)\,\big\|\,p_\theta(z_t \mid h_t)\big)\right], \tag{7.8.16}$$

其中先验 $p_\theta(z_t \mid h_t)$ 由确定性递推给出, 与 §7.8.2 的式 (7.8.9) 自由能框架直接对应。智能体在学到的隐空间中展开纯想象 (imagination) 轨迹以进行策略优化, 无需与真实环境交互。

以上四类形式化共同指向一个判断: 世界模型的数学骨架并非全新发明, 而是把变分推断、Kalman 滤波、对比学习这些已有工具按新的工程目标重新组装。下一节给出每种形式化的边界与失效场景。

---

§7.9 反类比与边界

▶这一节在讲什么

为什么要专门讲「边界」? 前几节推导了 HiPPO、S4、Mamba、线性注意力各自的数学保证，但每个保证都在某个假设成立时才有效。本节系统检查：当假设失效时，模型在哪里会垮掉，失败是随机的还是系统性的?

第一步：HiPPO 的「最优性」是测度相对的。LegS 在全历史均匀测度下最优，LegT 在滑动窗口下最优——拿错测度用对方的矩阵就不是最优。实践中 S4 训练后 A 会偏离 LegS 结构，说明任务最优性和理论最优性有差距。

第二步：Mamba 并行扫描有数值稳定性隐患。长前缀积中谱范数大于 1 则指数增长，小于 1 则指数衰减；与 Softmax 注意力的内建归一化不同，状态扫描没有类似机制，极长序列下状态数值范围可能漂移。

第三步：线性注意力在精确检索任务上有数学上界 $O(r/L)$。有限秩的特征映射无法模拟 Softmax 的「赢者通吃」尖锐化，当序列长度 L 增大时，正确键分配到的权重期望不超过 $O(r/L)$，这是系统性不足，不是调参能解决的。

本节要拿走的一件事：每个数学保证背后都有一个假设；把假设和结论分开记，遇到新任务时先检查假设是否成立，再判断方法是否适用。

直觉

本章前面的推导为每类模型建立了数学保证：HiPPO 给出最优压缩，S4 实现高效卷积，Mamba 实现选择性记忆，线性注意力将推理压缩到 $O(1)$。但每个保证都有它的隐含假设，而现实中假设常常不成立。

本节出现三个核心「反类比」：**HiPPO 最优性的测度依赖、Mamba 并行扫描的数值稳定性隐患、线性注意力精确检索的数学上界**。它们不是技术细节，而是三类模型固有

的数学局限，每一个都有其数学根源，都不能通过工程调优解决。理解这些边界，和理解模型的能力同样重要。

第一环：HiPPO 最优性的测度依赖——「最优」相对于什么

§7.2 证明了 HiPPO-LegS 在全历史均匀测度下最优，HiPPO-LegT 在滑动窗口均匀测度下最优。但这里有一个根本的相对性：「**最优」是相对于特定测度定义的，拿错测度就不是最优**。

例如，若任务的真正重要历史是「最近 3 个 token」（如对话中的直接上文），而模型用 LegS（均等对待所有历史），它将大量状态容量用于存储远古历史，对近期的关注反而被稀释。反之，若任务需要长程依赖（如代码中的变量声明），LegT 的滑动窗口会将窗口外的信息完全丢弃。

实践中，S4 训练后 A 矩阵会**显著偏离** LegS 结构——Gu et al.（2023, How to Train Your HiPPO）发现，经过学习的 A 在任务特定意义下比 LegS 更优，但已不再满足 LegS 的正交投影最优性定理。这说明：**HiPPO 的成功部分来自「良好的初始化」，而非全局最优**。后续工作（S4D）放弃了严格的 HiPPO 最优性，直接用可学习的对角矩阵 $A = \Lambda$，在实践中与 S4 性能相当甚至更好——这是一个典型的「理论严格但实践灵活」的权衡。

根本约束：没有一种测度在所有任务上都最优。不同时间分辨率需求（近期 vs 全局历史）对应不同最优测度，而 LegS/LegT 的选择是一个先验假设，不是数据驱动的。

第二环：Mamba 并行扫描的数值稳定性——与 Softmax 注意力的结构性不同

Mamba 的并行扫描在长序列下面临三类数值问题，每一类都有其结构性根源：

问题 1：矩阵乘积的数值范围漂移。在长前缀积 $\prod_{s=j+1}^{t} \bar{A}_s$ 中，若状态转移矩阵的谱范数 $\|\bar{A}_t\|$ 大于 1，乘积指数增长（梯度爆炸）；若远小于 1，乘积指数衰减（梯度消失）。Mamba 通过约束 A 为负实部对角矩阵（从而 $\|\bar{A}_t\| = \|e^{\Delta_t A}\| < 1$）来缓解，但 Δ_t 可以很大，使 $\|\bar{A}_t\|$ 非常接近 0——长序列下状态的有效「半衰期」可能远短于序列长度。

问题 2：缺乏 Softmax 注意力的内建归一化。Softmax 注意力有两层数值保护：(a) 分母归一化保证权重之和为 1；(b) log-sum-exp 技巧（FlashAttention）在数值稳定的对数域计算，避免上溢。状态扫描没有类似机制——隐状态 h_t 的数值范围随序列增长可能无界漂移。Mamba 的实现中有对数域扫描的部分缓解措施，但未被所有实现采用。

问题 3：长序列反向传播的显存。并行扫描需要存储 $O(\log L)$ 层二叉树的中间激活，反向传播时需要 $O(L)$ 显存（尽管深度是 $O(\log L)$）。FlashAttention 用重计算避免 $O(L^2)$ 注意力矩阵存储；类似的 FlashSSM 技术仍在探索中。

这些问题不是实现细节，而是来自「无归一化的状态累积」这一结构性特征——这是 SSM 与 Softmax 注意力在数值稳定性上最根本的差异。

第三环：线性注意力精确检索的数学上界——有限秩的不可逾越限制

实验规律是一致的：在精确键值检索任务（Passkey Retrieval、归纳头、Copy 等）上，线性注意力在序列长度增大时性能系统性下降，而 Softmax 注意力保持稳定。

数学根源：Softmax 注意力通过增大 $\|Q\|$ 可以任意接近 one-hot 权重（「赢者通吃」）：

$$A_{i^*,j^*} = \frac{e^{\|Q^*\|^2/\sqrt{d}}}{\sum_j e^{Q^* \cdot K_j/\sqrt{d}}} \to 1 \quad \text{当} \|Q^*\| \to \infty \tag{7.9.1}$$

而线性注意力分配给正确键的权重受到有限秩的约束——当序列中有 L 个随机键时，命题 7.8 给出期望上界 $O(r/L)$（见式 (7.9.3)）：正确键的权重随序列长度**线性衰减**。

式 (7.9.3) 给出的 $O(r/L)$ 上界的根源是：r 维特征映射最多能「精确区分」约 r 个键；当键的数量超过 r 时，不同键在特征空间里互相干扰，无法精确分离。这不是线性注意力实现得不够好，而是有限秩特征映射的**数学必然**——无论如何调整 ϕ，都无法突破这个界。

这从机制上解释了为什么 GLA、Mamba 等模型在 RULER 等精确检索基准上稳定落后于 Transformer——**不是工程优化的问题，而是架构的数学局限**，需要理论层面的突破（如混合架构、注意力头专门负责检索）。

整条链的回头看

```
HiPPO 最优性保证
    前提：在特定测度下最优
    失效：测度不匹配（近期 vs 全局历史）
    实践：训练后 A 偏离 LegS，S4D 放弃严格最优性更实用

Mamba 并行扫描
    前提：谱范数 < 1 保证数值稳定
    失效：长序列状态范围漂移；无内建归一化
    对比：Softmax 有 log-sum-exp 保护；SSM 无类似机制

线性注意力检索能力
    前提：特征维度 r 足够大
    数学上界：O(r/L)，随序列长度线性衰减
    根源：有限秩映射无法模拟 Softmax「赢者通吃」
    实践：RULER 等检索基准上系统性落后 Transformer
```

这条链揭示了一个深层事实：**每个数学保证背后都有一个假设；把假设和结论分开记，遇到新任务时先检查假设是否成立，再判断方法是否适用**。HiPPO 的测度依赖、Mamba 的数值漂移、线性注意力的检索上界——三个不同的技术问题，但都是同一个模式：理论上优美的保证，依赖于现实中可能不成立的假设。识别这些假设，是在工程决策中正确选择和组合模型的前提。

HiPPO 的最优性依赖于测度选择。LegS 在全历史均匀测度下是最优的，但如果任务真正重要的是近期历史（比如对话的最近几轮），LegS 给所有时刻均等权重可能并不理想。实践中 S4 训练后 A 矩阵会偏离理论的 LegS 结构，这说明「训练后的最优」和「数学推导的理论最优」之间存在差距。后续工作（S4D、How to Train Your HiPPO）探索了更灵活的参数化，放弃了严格的 HiPPO 最优性但在实践中表现更好——这是一个典型的「理论严格但实践灵活」的权衡。

线性注意力的检索上界 $O(r/L)$ 是最直接的例子。当序列中有 L 个随机键时，分配给正确键的权重期望不超过 $O(r/L)$，随序列长度线性衰减。这不是实现缺陷，而是有

限秩特征映射的数学必然：r 个维度最多能「精确区分」约 r 个键，多出来的键之间会互相干扰。这从机制上解释了为什么 GLA、Mamba 等模型在 RULER 等精确检索基准上稳定落后于 Transformer——这是一个需要理论改变才能解决的问题，不是工程调优的范畴。

7.9.1 HiPPO"最优性"的测度依赖

§7.2 的最优性结论具有根本的相对性:

命题 7.7:HiPPO-LegT 和 HiPPO-LegS 给出的 A 矩阵各自在其特定测度下是最优的——LegT 在长度为 θ 的**滑动窗口**均匀测度下最优,LegS 在整个历史 $[0, t]$ 的**均匀测度**下最优。在另一种测度下使用对方的矩阵将不是最优近似。

例如, 若使用**指数衰减**测度 $\mu^{(t)}(ds) = e^{-(t-s)}ds$, 则最优矩阵 A_{LagT} 是 Laguerre 多项式对应的矩阵, 与 LegT、LegS 均不同。实践中,S4 使用 LegS 初始化 (因为它对全局历史最优), 但在学习过程中 A 矩阵会偏离 LegS 结构, 表明任务-特定最优性与理论最优性存在差距。

推论:HiPPO 框架的成功部分来自"良好的初始化", 而非全局最优。后续工作 (How to Train your HIPPO, 2023) 表明,S4D(对角 S4) 等变体虽然放弃了严格的 HiPPO 最优性, 但通过更灵活的参数化在实践中表现更好。此外, 不同测度对应不同"时间分辨率":LegT 在近期历史上有更高精度,LegS 对所有时刻均等对待。没有一种测度在所有任务上都最优。

7.9.2 Mamba 并行扫描的数值稳定性

选择性 SSM 的并行扫描在数值上面临以下挑战:

问题 1: 矩阵乘积的数值范围。在式 (7.19) 的长前缀积中, 若状态转移矩阵 $\bar{A}_k$ 的谱范数大于 1, 则乘积指数增长 (梯度爆炸); 若远小于 1, 则乘积指数衰减 (梯度消失)。Mamba 通过限制 A 为具有负实部特征值的对角矩阵 (从而 $\|\bar{A}_t\| = \|e^{\Delta_t A}\| < 1$) 来缓解此问题。

问题 2: 与 Softmax Attention 的对比。Softmax 注意力的归一化操作保证注意力权重之和为 1, 这是一种内建的数值归一化机制。对数-求和-指数 (log-sum-exp) 技巧 (FlashAttention) 进一步在数值稳定的对数域中计算, 避免上溢。状态扫描没有类似的归一化, 状态 h_t 的数值范围可能随序列长度增长而漂移。Mamba 的对数域扫描技巧是部分解决方案, 但未被所有实现采用。

问题 3: 长序列下的梯度传播。在极长序列 ($L > 10^5$) 上, 即使使用并行扫描, 反向传播时 $O(\log L)$ 层二叉树的激活值存储仍需 $O(L)$ 显存, 可能成为瓶颈。FlashAttention 通过重计算前向激活避免存储 $O(L^2)$ 注意力矩阵,SSM 的类似技巧 (FlashSSM) 也在被探索。

7.9.3 线性注意力在检索任务上的系统性劣势

实验规律: 在需要精确键值检索的任务 (如 Passkey Retrieval、Copy、归纳头（Induction Heads）) 上, 线性注意力一致地输给 softmax 注意力, 差距在长序列下尤为显著。

理论解释:

设查询集中有某个精确匹配的查询 Q^*, 对应键 $K^* = Q^*$。在 softmax 注意力中:

$$A_{i^*,j^*} = \frac{e^{\|Q^*\|^2/\sqrt{d}}}{\sum_j e^{Q^* \cdot K_j/\sqrt{d}}} \to 1 \quad \text{当} \|Q^*\|^2/\sqrt{d} \to \infty, \tag{7.9.2}$$

由式 (7.9.2) 可知, 通过增大 $\|Q^*\|$(由学习控制) 可以任意接近 1, 实现”赢者通吃”的精确检索。

而在线性注意力中, 分配给 j^* 的权重为:

$$w_{j^*} = \frac{\phi(Q^*) \cdot \phi(K^*)}{\sum_j \phi(Q^*) \cdot \phi(K_j)} = \frac{\phi(Q^*) \cdot \phi(K^*)}{\phi(Q^*) \cdot \mathbf{z}}. \tag{7.9.3}$$

命题 7.8 (线性注意力的检索上界): 设特征维度为 r, 序列中有 L 个键均匀随机分布。则在最坏情况下, 线性注意力分配给正确键的权重期望不超过 $O(r/L)$。

证明思路: 由柯西-施瓦茨不等式,$|\phi(Q^*) \cdot \phi(K)| \le \|\phi(Q^*)\|\|\phi(K)\|$。当 L 个键均匀分布时,$\|\mathbf{z}\| = \|\sum_j \phi(K_j)\| \ge \sqrt{L}\|\phi(K^*)\|$(大数定律), 故 $w_{j^*} \le \|\phi(K^*)\|/(\sqrt{L}\|\phi(K^*)\|) = 1/\sqrt{L}$。更精细的分析给出 $O(r/L)$ 的界。□

式 (7.9.3) 从数学上解释了线性注意力在检索任务上的系统性不足: 有限秩的特征映射无法模拟 softmax 的无界尖锐化能力。实践中, 这一限制导致 GLA、Mamba 等模型在 RULER 等需要长距离精确检索的基准上显著落后于 Transformer。

---

§7.10 参考文献

▶这一节在讲什么

为什么列这些文献? 本章涵盖的数学内容跨越控制理论（SSM、Kalman）、逼近论（HiPPO、Legendre）、深度学习（S4、Mamba、线性注意力）、概率生成模型（DDPM、离散扩散）和世界模型（JEPA、DreamerV3）五条研究线索。所列文献是每条线索的原始来源或最具代表性的综述，适合在理解本章正文后深入追溯推导细节和实验验证。

1. **Gu, A., Dao, T., Ermon, S., Rudra, A., & Ré, C.** (2020). HiPPO: Recurrent Memory with Optimal Polynomial Projections. Advances in Neural Information Processing Systems (NeurIPS), 33. https://proceedings.neurips.cc/paper/2020/hash/102f0bb6efb3a6128a3c750dd16729b
Abstract.html
2. **Gu, A., Goel, K., & Ré, C.** (2022). Efficiently Modeling Long Sequences with Structured State Spaces (S4). International Conference on Learning Representations (ICLR). https://arxiv.org/abs/2111.00396
3. **Gu, A., & Dao, T.** (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv preprint arXiv:2312.00752. https://arxiv.org/abs/2312.00752
4. **Dao, T., & Gu, A.** (2024). Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality (Mamba-2/SSD). International Conference on Machine Learning (ICML). https://arxiv.org/abs/2405.21060

5. **Katharopoulos, A., Vyas, A., Pappas, N., & Fleuret, F.** (2020). Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. International Conference on Machine Learning (ICML), PMLR 119. https://proceedings.mlr.press/v119/katharopoulos20a
6. **Choromanski, K., Likhosherstov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., … & Weller, A.** (2021). Rethinking Attention with Performers (FAVOR+). International Conference on Learning Representations (ICLR). https://arxiv.org/abs/2009.14794
7. **Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., …& Wei, F.** (2023). Retentive Network: A Successor to Transformer for Large Language Models. arXiv preprint arXiv:2307.08621. https://www.microsoft.com/en-us/research/publication/retentive-network-a-successor-to-transformer-for-large-language-models/
8. **Yang, S., Wang, B., Shen, Y., Panda, R., & Kim, Y.** (2024). Gated Linear Attention Transformers with Hardware-Efficient Training (GLA). International Conference on Machine Learning (ICML). https://arxiv.org/abs/2312.06635
9. **Sahoo, S. S., Arriola, M., Schiff, Y., Gokaslan, A., Marroquin, E., Chiu, J. T., Rush, A., & Kuleshov, V.** (2024). Simple and Effective Masked Diffusion Language Models (MDLM). Advances in Neural Information Processing Systems (NeurIPS), 38. https://neurips.cc/virtual/2024/poster/95622
10. **Nie, S., Zhu, F., Du, C., Zhang, T., Ou, Z., Muennighoff, N., …& Li, C.** (2025). Large Language Diffusion Models (LLaDA). arXiv preprint arXiv:2502.09992. https://arxiv.org/abs/2502.0999
11. **Ho, J., Jain, A., & Abbeel, P.** (2020). Denoising Diffusion Probabilistic Models (DDPM). Advances in Neural Information Processing Systems (NeurIPS), 33. https://arxiv.org/abs/2006.11239
12. **Blelloch, G. E.** (1990). Prefix Sums and Their Applications. Technical Report CMU-CS-90-190, Carnegie Mellon University. https://www.cs.cmu.edu/~guyb/papers/Ble93.pdf
13. **Schlag, I., Irie, K., & Schmidhuber, J.** (2021). Linear Transformers Are Secretly Fast Weight Programmers. International Conference on Machine Learning (ICML), PMLR 139. https://arxiv.org/abs/2102.11174
14. **Gu, A., Johnson, I., Timalsina, A., Rudra, A., & Ré, C.** (2023). How to Train Your HiPPO: State Space Models with Generalized Orthogonal Basis Projections. International Conference on Learning Representations (ICLR). https://openreview.net/forum?id=klK17OQ3
15. **Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., …& Polosukhin, I.** (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS), 30. https://arxiv.org/abs/1706.03762
16. **Gu, A., Goel, K., Gupta, A., & Ré, C.** (2022). On the Parameterization and Initialization of Diagonal State Space Models (S4D). Advances in Neural Information Processing Systems (NeurIPS), 35. https://arxiv.org/abs/2206.11893
17. **Austin, J., Johnson, D. D., Ho, J., Tarlow, D., & van den Berg, R.** (2021). Structured Denoising Diffusion Models in Discrete State-Spaces (D3PM). Advances in Neural Information Processing Systems (NeurIPS), 34. https://arxiv.org/abs/2107.03006
18. **Voelker, A., Kajić, I., & Eliasmith, C.** (2019). Legendre Memory Units: Continuous-

Time Representation in Recurrent Neural Networks. Advances in Neural Information Processing Systems (NeurIPS), 32. https://proceedings.neurips.cc/paper/2019/hash/952285b9b7e7a1Abstract.html

---

本章完。数学卷至此与主本第 1–7 章逐章镜像，公式与推导补完。

后记

本附册的写作目标是让读者「在主本里看到几何，在附册里看到公式」。如果你读完主本之后愿意翻开这里，并在某一节停下来推一遍——那这本书就达成了它最大的野心。

数学之所以重要，不是因为它能预测一切，而是因为它能让我们看清「哪里是确定的、哪里是猜测的、哪里是隐喻」。第 5 章的渗流相变是隐喻；第 1 章的交叉熵是确定的；第 4 章的 μP 是数学定理但前提需要满足；第 6 章的 I/O 下界是经过 1981 年 Hong-Kung 证明的渐近铁律；第 7 章的 HiPPO 是在「平移 Legendre 测度」这一特定选择下最优——换个测度，矩阵就换。把这些边界讲清楚，是这本附册的本职工作。

如果你在阅读中发现任何公式错误、推导疏漏、或者认为某节应该更深入，欢迎寄信至 fqsx@mail.ustc.edu.cn 指正。

——大队长，2026 年 6 月